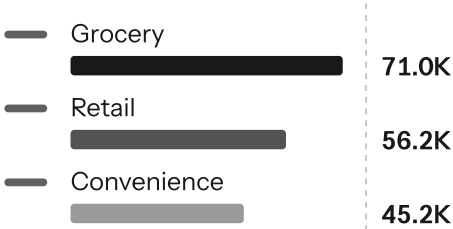


Understand the design. Then build it in Paper.

Fourteen design principles, each paired with exactly how Paper does it: the controls, the shortcuts, the agent workflows and the limits. Written from all 52 videos on Paper's channel and its documentation, so you never have to watch them.

For designers and developers who want to work in Paper with confidence: the canvas, tokens, vectors, shaders, and AI agents over MCP.

Web edition with the live studies: enzosimier.com/projects/paper-guide/



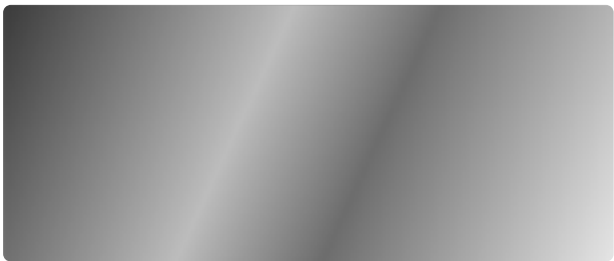
Make comparison effortless.

Chapter 3 · shared lanes and edges

Aa Aa

Give every tone a role.

Chapter 5 · tokens, type and themes



Control the effect.

Chapter 9 · Paper Shaders, live

Contents

Read in order, or jump to a chapter: each one stands on its own and links to the reference when it needs to.

Part 1 · Structure4 chapters

01	Start with what belongs together
	Hierarchy · study, plate, procedures, tutorials
02	Make space follow the content
	Flex · study, plate, procedures, tutorials
03	Use alignment to make comparison effortless
	Alignment · study, plate, procedures, tutorials
04	Design what happens when the frame changes
	Constraints · study, plate, procedures, tutorials

Part 2 · Visual language3 chapters

05	Give the interface a consistent visual language
	Tokens and type · study, plate, procedures, tutorials
06	Make information easy to judge
	Information · study, plate, procedures, tutorials
07	Finish the asset at the size people will see
	Vector · study, plate, procedures, tutorials

Part 3 · Material2 chapters

08	Choose the right starting material
	Starting material · study, plate, procedures, tutorials
09	Shaders: give effects a precise visual purpose
	Shaders · study, plate, procedures, tutorials

Part 4 · Agents and code5 chapters

10	MCP: connect the agent to a specific design context
	MCP · study, plate, procedures, tutorials
11	Parallel AI agents: keep one coherent design
	Parallel agents · study, plate, procedures, tutorials
12	Repeat the rule, then vary the content
	Reuse · study, plate, procedures, tutorials
13	Carry the design’s intent into working code
	Code · study, plate, procedures, tutorials
14	Publish an artifact that survives leaving the canvas
	Export · study, plate, procedures, tutorials

After the chapters3 sections

R	Reference
	Releases, status, shortcuts, troubleshooting, briefs, glossary
B	Seven authors to read next
	The design books behind the principles
S	The 52 videos
	Every source, searchable, with its moments

How every chapter works

Six parts, always in the same order.

01

Principle

The idea, in plain prose, with the chapter's rule in one sentence.

02

Live study

Change the example yourself and watch the relationship hold or break.

03

Plate

A drawn figure that isolates the relationship the chapter is about.

04

In Paper

Procedures, controls and shortcuts: exactly how Paper does it today.

05

Tutorials, written down

What each official video demonstrates, so you never need to watch it.

06

Workshop

Build it in Paper, check it, and avoid the usual mistakes.

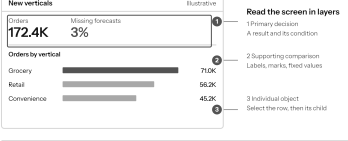
Structure

Layout begins with relationships: what belongs together, how space is shared, which edges line up and what happens when the frame changes. These four chapters cover Paper's frames, selection, Flex and constraints.

4 chapters, 4 live studies
6 source videos, 27 minutes, written down

01 / PAPER IN PRACTICE

A screen is a hierarchy of decisions.
Select the right readouts, changing its appearance.



Reconstructed teaching example: synthetic values, not a Paper screenshot

01 **Start with what belongs together**

02 / PAPER IN PRACTICE

Let the parent do the layout work.
A column's parent's width will replace repeated manual positioning.

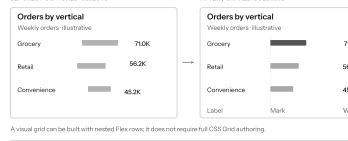


Reconstructed teaching example: synthetic values, not a Paper screenshot

02 **Make space follow the content**

03 / PAPER IN PRACTICE

Give every number a predictable address.
Separate the size, the unit, the unit type, and the value.

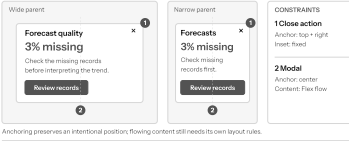


Reconstructed teaching example: synthetic values, not a Paper screenshot

03 **Use alignment to make comparison effortless**

04 / PAPER IN PRACTICE

Anchor the exception. Let content flow.
Base the visual address on the corner, not the middle, not the top.



Reconstructed teaching example: synthetic values, not a Paper screenshot

04 **Design what happens when the frame changes**

Start with what belongs together

A convincing screen begins with relationships: what belongs together, what leads, and what the reader should notice next.

A screen has more than one kind of hierarchy

Visual hierarchy controls attention. Structural hierarchy controls behavior. A large headline may lead the eye, while a quiet parent frame determines how everything beneath it moves. Good interface design makes these hierarchies agree. The heading, explanation and action that form one idea should usually live inside one meaningful container. Otherwise the screen can look ordered while remaining fragile to edit.

Imagine a compact account panel containing a title, a balance and a date. If the title sits near one card and the date sits near another, the reader has to infer their relationship. Before changing color or font size, change the grouping. Put the title and its context together; give the value its own clear position. The composition becomes easier to understand because proximity now describes meaning.

In Paper the structural hierarchy is literal. A frame is what HTML calls a div: the layers inside it are its children, and layers that share a parent are siblings. Moving a frame moves everything inside it, and the code a developer or an agent reads has the same nesting as the layers panel. Grouping is a design decision and an implementation decision at the same time.

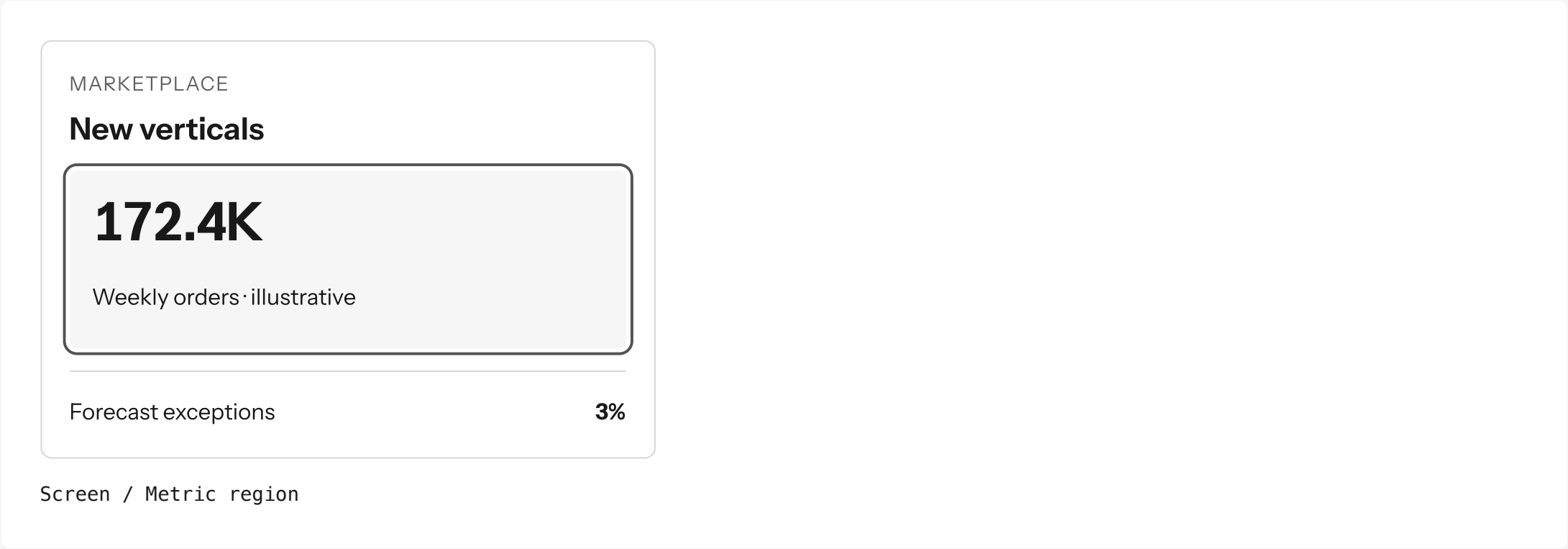
—

A useful frame groups one idea and owns one clear layout responsibility.

The principle of chapter 1

Select the relationship, then group it

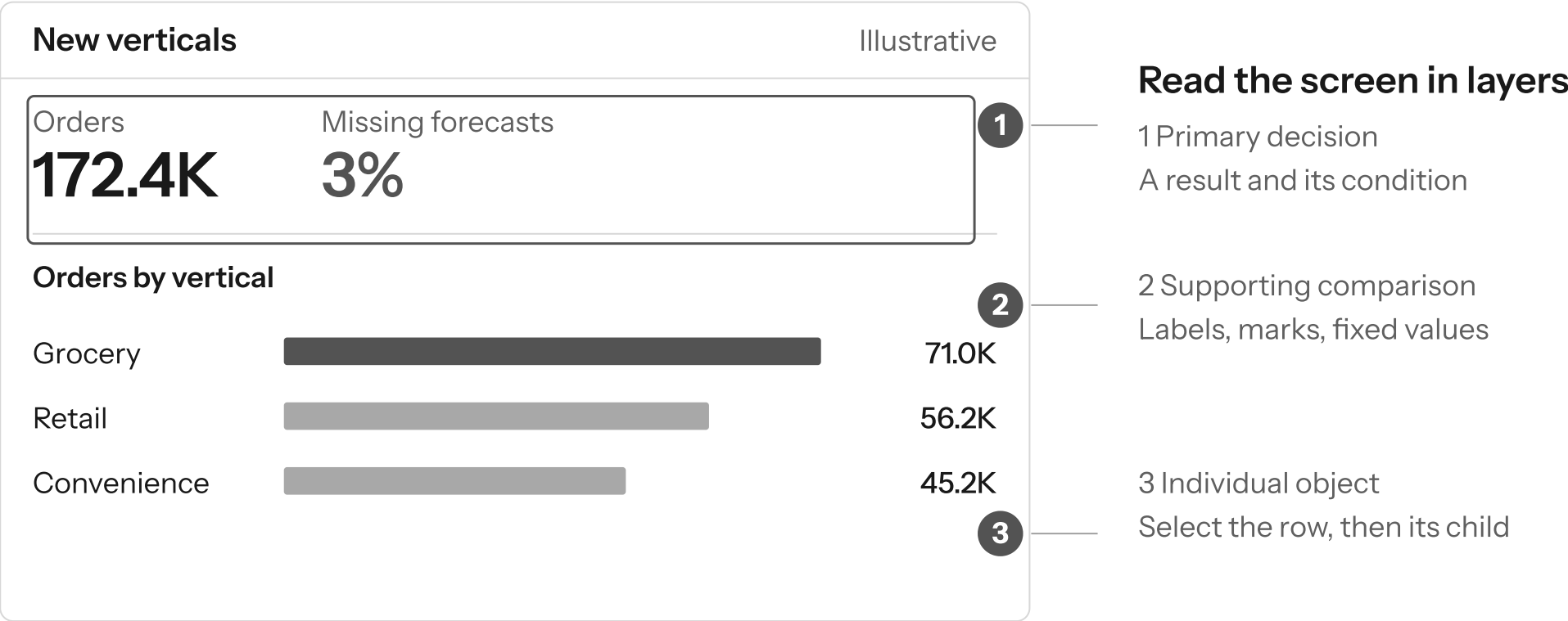
Group the layers, then change the space between them.



The value and context share a parent. Changing its gap moves the relationship as a unit.

A screen is a hierarchy of decisions.

Select the right level before changing its appearance.



Reconstructed teaching example · synthetic values · not a Paper screenshot

Figure 01.1 · Plate

A screen is a hierarchy of decisions.

Start with the decision, then its supporting region, then the individual object. Hover and selection help target the intended layer; the reading hierarchy remains a design choice.

Work at the correct level

Paper’s hover-selection update makes a normally invisible decision visible: which level of the tree receives the edit? Since June 2026, hovering highlights the top-level layer under the pointer, as in Figma. Double-clicking goes deeper, holding ⌘ reaches the innermost layer, Esc selects the parent and Enter selects the children, and ⌘ + right-click lists every layer under the pointer. A spacing change on a parent affects a relationship; a change on a child affects one object. Those are different design decisions.

Build a frame around a complete idea, then build smaller frames around its internal relationships. A page can contain a header, a content region and a footer. The content region can contain a comparison, and the comparison can contain repeated rows. These names are useful because they explain responsibility. Paper calls every new frame “Frame”, so rename it as soon as it exists by double-clicking its name on the canvas or in the layers panel. “Comparison” tells an agent and a future collaborator more than a column of identical “Frame” rows.

The layers panel lets you audit that structure at a glance. The icon beside each name gives the frame’s type: two columns for a horizontal flex frame, stacked rows for a vertical one, and dashed corners for a freeform frame whose children are placed by coordinates. Collapse all (⌘L, or right-click in the layers list) folds the tree back to its top level, and double-clicking a layer’s icon zooms the canvas to that layer. A tree that reads like an outline of the screen is a sign that the structure is sound.

Make the composition readable before decorating it

Read the canvas in three passes. First, notice the largest shapes: is there a clear primary area? Next, read only headings: do they describe a coherent sequence? Finally, inspect the small labels: can the reader tell what each number or action belongs to? These passes expose different failures. A strong overall silhouette does not compensate for a value with an ambiguous label.

Paper supports each pass. ⌘. hides every panel so the first pass sees only the canvas, and ⌘. shrinks the panels to floating pills when you still need the toolbar and the zoom level. The same thinking scales beyond one screen: pages split a file into chapters, and folders in the dashboard nest several levels deep (Marketing › Social Media › Instagram). Once the structural relationships are right, typography and color can reinforce them, and the organization still makes sense after a label grows or another row is added.

Figure 01.2

The selection depth ladder

What you see and the level you edit can differ. Each gesture moves one level.



How you reach each level

Visual	hover and click (default)
Panel	double-click
Receipt	double-click again
Rectangle	⌘ + click, or ⌘ + right-click › Rectangle

Esc	Select parent
Enter	Select children

Deep selection

Hover: tag badge frame, Fit 140 × Fit 34

With ⌘: text layer, 93 × 16

Figure 01.3

Reading the layers panel

The icons tell you how a frame lays out its children, before you select anything.

 horizontal flex frame (row) icon: two vertical bars	 vertical flex frame (column) icon: stacked horizontal bars
 freeform frame icon: dashed corners	 text layer icon: Aa
 image layer icon: picture	 SVG icon: frame with corner handles

How it works in Paper

As of September 2026, structure in Paper is HTML structure: a frame is a div and its layers are its children. Hover selects the top-level layer under the pointer; double-click, ⌘, Esc and Enter move through the tree, and ⌘ + right-click lists every layer under the pointer. ⇧F wraps layers in a freeform frame and ⇧A in a flex frame. The layers panel shows each frame’s type, pages divide a file, and the dashboard organizes files in nested folders.

Checked against the videos and documentation as of September 2026. Paper ships often; the release timeline in the reference shows what changed when.

01

Select at the right depth

1

Hover over a design: Paper highlights the top-level layer under the pointer inside the artboard, not the small object you point at.

2

Click to select that layer, or double-click to go further in (in the demo, the 334 × 300 panel, then the 206 × Fit 194 receipt inside it).

3

Hold ⌘ while hovering or clicking to deep-select the innermost layer directly.

4

Press Esc to select the parent of the current selection, or Enter to select its children.

5

If you prefer hover to pick the frame closest to the pointer, click the file name at the top of the left panel and tick Use deep selection; untick it to return to the default.

Double-click

⌘ Click

Esc

Enter

►

Shown at 0:19

02

Pick a buried layer from the pointer list

1

Place the pointer over the spot where several layers overlap.

2

Hold ⌘ and right-click: a list opens with every layer under the pointer, indented by depth and marked with type icons (in the demo: 02 How it works, Visual, Panel, Receipt, Rectangle).

3

Hover a row to outline that layer on the canvas.

4

Click the row to make that layer the selection.

⌘ Right-click

►

Shown at 1:07

03

Group layers into a named frame

1

To start from an empty frame, draw one with the Frame tool and set its size, or pick a preset in the right panel (Phone, Tablet, Desktop, Presentation, Smartwatch, Paper, Social media); layers you drag into it become its children.

2

To group existing layers, select them on the canvas or in the layers panel (Shift-click adds to the selection).

3

Press ⇧F (Frame selection) for a freeform frame: the layers keep their positions and can be dragged anywhere inside it.

4

Press ⇧A (Wrap in flex) for a flex frame instead. With a freeform frame selected, the same key converts it, and the button in the Layout section reads Add flex ⇧A.

5

Double-click the frame’s name label on the canvas, or its row in the layers panel, and type a name that states its role.

⇧ F

⇧ A

►

Shown at 0:50

04

Navigate a large layer tree

1

Double-click a layer’s icon in the layers list: the canvas zooms to that layer and centers it.

2

Right-click in the layers list and choose Collapse all (⌘L) to fold every group back to the top level.

3

Hover a row to reveal its lock and eye toggles; Lock / unlock is ⇧⌘L and Show / hide is ⇧⌘H.

4

Drag the left panel’s edge to widen it when names are long.

⌘ L

⇧ ⌘ L

⇧ ⌘ H

►

Shown at 0:37

05

Split a file into pages

1

Click + beside Pages at the top of the left panel.

2

The new page appears (“Page 2” in a one-page file) with its name selected; type a name that describes its content.

3

Click a page to open it; a check mark shows the page you are on.

4

Drag pages in the list to reorder them; the pages list can also be resized by dragging its edge.

►

Shown at 2:26

06

Organize files in nested folders

1

In the dashboard, open Files in the sidebar and click + New folder; the folder appears with its name selected (“Untitled project”), ready to type.

2

Right-click a file card and choose Move to › a folder; the folder it is already in is grayed out.

3

Open a folder and click + New folder again to nest another folder inside it; in Move to, folders that contain folders open a further submenu.

4

Move a folder the same way (right-click › Move to); a toast confirms the move, for example “Moved Billboard to Marketing”.

5

Click a level in the breadcrumb (Files / Marketing / Social Media) to go back up. Rename with right-click › Rename, or double-click the current folder’s name in the breadcrumb.

►

Shown at 0:12

Controls you will use

Control	Where	What it does
Layers panel	Left panel, below Pages	Shows the file as a tree of parents and children. The icon beside each name gives the type: two columns for a horizontal flex frame, stacked rows for a vertical flex frame, dashed corners for a freeform frame, Aa for text. Hover a row for its lock and eye toggles; double-click its icon to zoom to the layer.
File-name menu	Top of the left panel (click the file name)	Holds Back to dashboard ⌘D, Show pixel grid ⌘', Snap to pixel ⌘⌘', Layout guides ⌘G, Multiplayer cursors ⌘M, Use deep selection, Open links in desktop app and Nudge settings...; later builds add Theme › Copy theme, Paste theme, Clear theme. The hover-selection video’s narration calls this menu Project Settings.
Use deep selection	File-name menu (unticked by default since June 2026)	When ticked, hover targets the frame closest to the pointer instead of the top-level layer; ⌘ still reaches the innermost layer.
Layers under the pointer	Canvas, ⌘ + right-click	Lists every layer stacked under the pointer, from the artboard down to the smallest shape; hovering a row outlines that layer, clicking selects it.
Select parent / Select children	Right-click menu (shown with Escape and Enter)	Moves the selection one level up to the parent, or down to the children.
Frame selection / Wrap in flex	Right-click menu, and the Layout section of the right panel	Wraps the selection in a freeform frame (⇧F) or a flex frame (⇧A). With a freeform frame selected, the Layout button reads Add flex ⇧A and converts that frame. The right-click menu also lists Remove flex ⌘⇧A and Ungroup ⌘⌘G (seen from June 2026).
Pages	Top of the left panel	+ adds a page (“Page 2” in a one-page file) with its name ready to edit; a check mark shows the current page; drag a page to reorder it.
Files view	Dashboard sidebar › Files (Back to dashboard ⌘⌘D from inside a file)	Shows the team’s folders and files with + New folder, + New file, Search files ⌘F and a grid or list toggle; a breadcrumb such as Files / Marketing / Social Media shows where you are. Since September 2026, search looks through all folders at once.
File and folder menu	Dashboard, right-click a file or folder card	Open, Open in new tab, Copy link, Rename, Move to and Archive. Move to lists folders with nested submenus and grays out the current location. Since August 2026, ⌘-click selects several files and folders to move or archive together, and files can be duplicated from this menu.
Hide or minimize the UI	⌘. and ⇧., or the sidebar icon beside the file name	⌘. hides every panel; ⇧. collapses them into floating pills (file name, toolbar, and avatar with Copy link and zoom). The same keys restore them.

Shortcuts

⌘Right-click	List every layer under the pointer and pick one ▶ 1:07	⌘Click	Deep-select the innermost layer under the pointer ▶ 0:13
Double-click	Go deeper: select a layer inside the hovered or selected one ▶ 0:23	Esc	Select parent ▶ 2:31
Enter	Select children ▶ 2:31	⇧F	Frame selection: wrap the selection in a freeform frame ▶ 0:50
⇧A	Wrap in flex, or Add flex to a selected freeform frame ▶ 1:08	⌘L	Collapse all layers in the layers panel ▶ 2:10
⌘.	Hide or show the UI ▶ 0:18	⇧.	Minimize or restore the UI ▶ 0:23

Limits and what is coming

- 🕒 Permissions per file (choosing who on a team sees which file) were announced as coming soon in September 2026.
- 🕒 Components on the canvas were announced as coming soon in September 2026, so a duplicated section is an independent copy: changing one copy does not update the others.

What the tutorials show

Each official video, written down step by step. The dots mark the moments; they open the video there if you want to see it.

We updated hover selection in Paper based on your feedback

Video · 1 min 45 s · 10 June 2026



Hover selection: parent first, deeper on demand

Since this June 2026 update, Paper’s default hover matches Figma. In a mobile onboarding file, hovering a receipt illustration outlines the top-level block that contains it, not the receipt. Double-clicking went further in, to the 334 × 300 panel and then the 206 × Fit 194 receipt, and holding ⌘ deep-selects straight away. The previous default is still available: click the file name at the top of the left panel and tick Use deep selection (the narration calls it Project Settings). Hover then grabs the frame closest to the pointer, so the “Tagged · Coffee” tag selects its Fit 140 × Fit 34 badge frame, and ⌘ still reaches the 93 × 16 text inside. The update also adds ⌘ + right-click: a list of every layer under the pointer, from the artboard “02 How it works” through Visual, Panel and Receipt to the clicked Rectangle. Hovering a row outlines that layer; clicking selects it.

- 0:19 Hover outlines the top-level block
- 0:23 Double-click reaches the receipt
- 0:38 Use deep selection in the file-name menu
- 0:52 Closest-frame hover on the tag badge
- 1:07 ⌘ + right-click layer list

8 Paper tips and tricks every designer should know

Video · 3 min 46 s · 15 June 2026



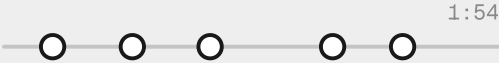
Eight canvas tips

Each tip has an on-screen key overlay. ⌘. hides every panel, and ⇧. shrinks them to floating pills (file name, toolbar, avatar with Copy link and zoom). Double-clicking a layer’s icon in the layers list zooms to and centers it. ⇧F puts selected layers in a freeform frame, where they can be dragged anywhere. ⇧A on a freeform frame adds flex (the button reads Add flex ⇧A); on loose layers it wraps them in a new flex frame. In both demos the gap matched the existing spacing (12, then 22), so nothing moved. Holding ⌘ while dragging an image’s edges crops instead of resizing. Double-clicking an edge or corner, or clicking the size under the file name in Fill (1600×1200), restores the original dimensions, here 800 × 600. Right-click › Extract colors adds a “Magic Swatches” frame of 100 × 60 rectangles named by hex, such as “Color #A65C0D”. With a layer selected, I opens the eyedropper (the same tool as the icon in the color picker), and its click sets that layer’s fill.

- 0:18 ⌘. hides and ⇧. minimizes the UI
- 0:38 Zoom to a layer from its icon
- 0:50 ⇧F wraps in a freeform frame
- 1:08 ⇧A adds flex to a frame
- 2:00 ⌘-drag crops an image
- 2:38 Magic Swatches from Extract colors

How to organize your Paper files with folders and nested folders

Video · 1 min 54 s · 17 June 2026



Folders and nested folders in the dashboard

In the team dashboard, open Files and click + New folder, next to + New file and Search files ⌘F. The folder appears with its name field selected (“Untitled project”); type “Marketing”. To file something, right-click a file card (“Video Thumbnails”) and choose Move to: the submenu lists Files, Design and Marketing, and Files is grayed out because the file is already there. Inside Marketing, + New folder creates “Social Media”, and inside that come “Instagram” and “TikTok”. A third folder, “Billboard”, does not belong there, so right-click › Move to › Marketing moves it up a level; folders that contain folders open a further submenu, and a toast confirms “Moved Billboard to Marketing”. The breadcrumb (Files / Marketing / Social Media) takes you back up. Rename with right-click › Rename, or double-click the current folder’s name in the breadcrumb and type.

- 0:12 New folder, named in place
- 0:30 Move to, current location grayed out
- 0:48 A folder inside Marketing
- 1:16 Move a folder up a level
- 1:32 Rename from the breadcrumb

Build it in Paper

- 01 Select the layers that form one idea (Shift-click adds to the selection) and press ⌘A to wrap them in a flex frame, or ⌘F for a freeform frame. Double-click the new frame’s “Frame” label and name it for its purpose, such as “Header”.
- 02 Repeat at each level: give the header, the main content and the footer their own frames as siblings inside one page-level frame, and nest smaller frames only where items share a relationship.
- 03 Before changing spacing or size, confirm the level: hover shows the top-level layer, double-click goes deeper, Esc selects the parent, Enter selects the children, and ⌘ + right-click lists every layer under the pointer.
- 04 Press ⌘L to collapse the layers panel and read the tree from the top: the names and icons (row, column, freeform) should follow the screen’s reading order.
- 05 Duplicate a section with ⌘D, lengthen one label and check that the grouping still communicates the same hierarchy. Then file the work: in the dashboard, right-click the file and choose Move to › a named folder.

Common mistakes

- × Changing gap or padding on a child because it was the layer under the pointer, when the relationship you meant lives on its parent.

→ Press Esc to select the parent (or ⌘ + right-click and pick it from the list), confirm its name in the layers panel, then edit.
- × Leaving every container named “Frame”, so the layers panel, prompts and handoff notes cannot tell sections apart.

→ Rename each frame when you create it: double-click its name on the canvas or in the layers panel and describe its role.
- × Using ⌘F for a group whose items should reflow, so the children keep fixed coordinates when content changes.

→ Use ⌘A for groups that should stay arranged; to fix an existing freeform frame, select it and press ⌘A (Add flex).
- × Leaving files loose in Files, where old versions sit next to current work.

→ Create folders with + New folder, nest them by product or theme, and move files with right-click › Move to.

Make space follow the content

Flex layout turns a collection of objects into a relationship that can survive longer labels, extra rows and changing dimensions.

From placed objects to a rule

A manually placed icon and label can look perfect at one moment. Change the label, and their relationship has to be repaired. A layout rule solves the more general problem: the icon occupies its lane, the text takes the space it needs, and the distance between them stays intentional. You are designing the response to change as well as the current picture.

Paper’s Flex lesson builds from a menu item to a sidebar. Pressing ⌘A (Wrap in flex) gives a new parent frame responsibility for arranging the selected layers. Direction defines the reading axis; gap separates siblings; padding creates an inner boundary. These controls describe three different relationships. Increasing padding to fix a gap may improve one screenshot while making the surrounding component feel wrong.

Paper calls the feature flex rather than auto layout because its canvas is built on HTML and CSS, and agents and developers already use that vocabulary. A flex frame is a div laid out with CSS flexbox, so the gap, padding and alignment you set in the panel are the same properties a developer or an agent reads in the code.

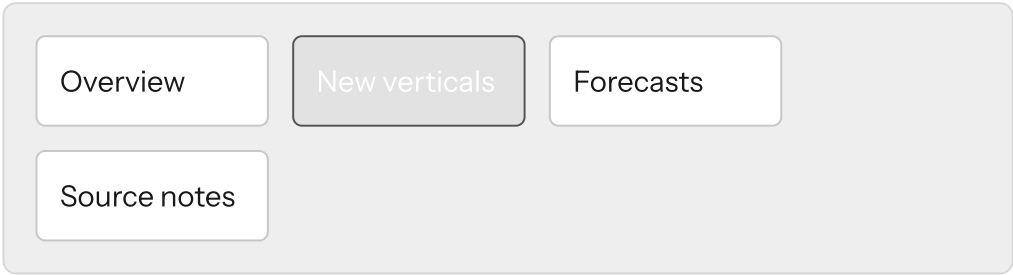
Gap separates siblings, padding protects content, and sizing declares a dependency.

The principle of chapter 2

Study 02 · change it yourself

Build the layout with parent rules

Change the parent rules and watch the children respond.

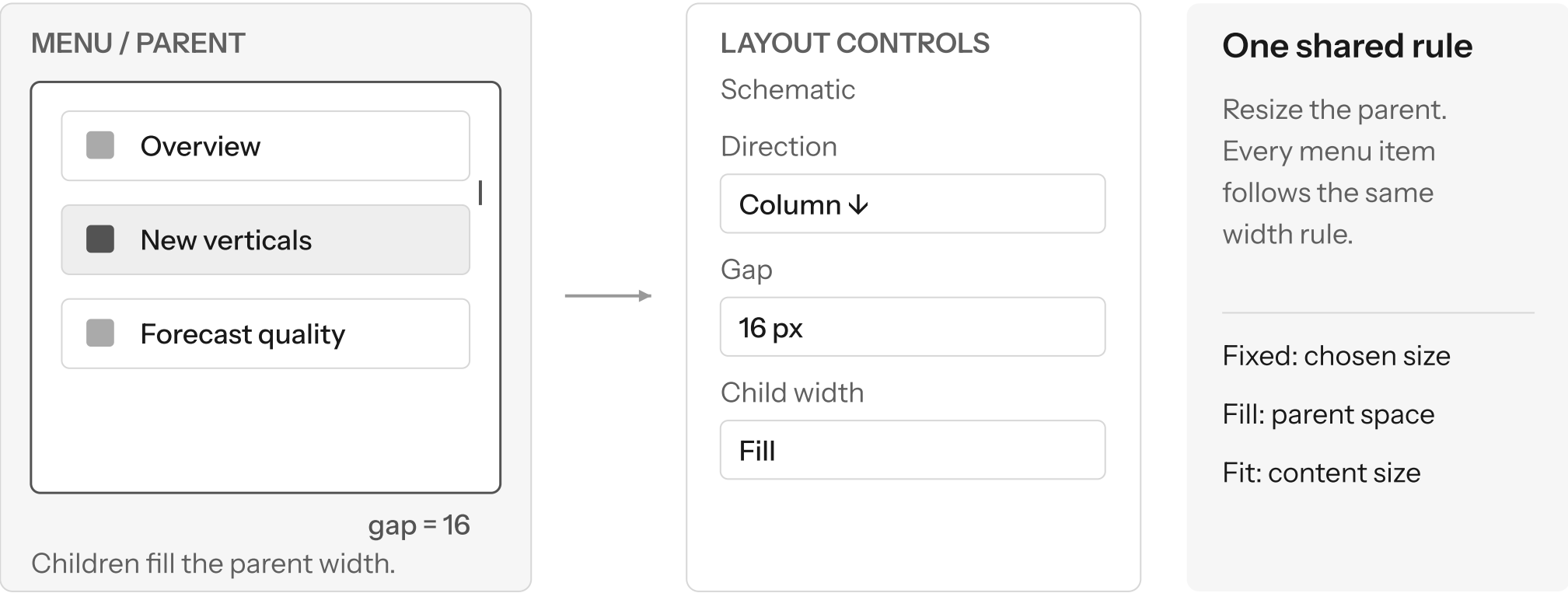


```
display: flex;
flex-direction: row;
gap: 12px; padding: 16px;
flex-wrap: wrap;
```

A row places the items side by side. Each child keeps its 120 px preferred width. The parent and child rules determine the new line breaks.

Let the parent do the layout work.

A column, a gap and a width rule replace repeated manual positioning.



Reconstructed teaching example · synthetic values · not a Paper screenshot

Figure 02.1 · Plate

Let the parent do the layout work.

The menu is a Flex column. Its children fill the parent width and share a 16 px gap. The property panel is a schematic explanation, not a literal interface capture.

Size is a dependency

Fixed, Fit and Fill answer a simple question: what determines this dimension? A Fixed value follows the designer’s decision. Fit follows the content. Fill follows the space left in the parent. Their usefulness comes from combining them deliberately. In a menu row, the icon can keep a stable width while the label receives the remaining room; the row’s height can follow its content. Paper’s instructions for agents make the same point about repeated rows: give icons and trailing actions fixed-width slots that do not shrink, because gap alone will not line up columns from row to row.

A parent that follows its child and a child that follows its parent can create an unclear sizing relationship. When a layout behaves unexpectedly, read the size badge under the selection, which names each axis’s mode (for example “Fill 267 × Fit 317”), before nudging coordinates. Decide where the available space originates. A page establishes a content width; a row fills that width; its icon keeps its own size; its text uses the space left over. The explanation should be easy to say aloud.

Space between distributes available space among siblings. It does not know which siblings form a conceptual group. If an icon, label and trailing value must become two visual groups, nest the icon and label together first with $\hat{A}$. The outer row can then separate the leading group from the value. This is the central move in the tutorial: correct grouping makes an alignment control produce the intended result.

Exceptions should be visible

Some objects do not belong in normal flow. A close button over the corner of a panel should not push the panel’s content aside. Paper’s example ticks Absolute position on such a button: it leaves the flow and sits on top of the content, but it keeps its default Left and Top constraints, so it drifts when the banner is resized until you pin it to the right edge. Treat that as an explicit exception, not a general way to position every item on the screen.

Paper keeps these choices inspectable. The layers panel marks rows, columns and freeform frames with different icons, and since September 2026 a flex layout written by an agent opens in the same Flex panel, ready to edit. A small, meaningful nested frame can remove several arbitrary offsets. When a row can explain where its dimensions come from, the same structure becomes easier to duplicate, adapt to a narrower surface and translate into implementation.

Figure 02.2

Fixed, Fit and Fill, side by side

One Flex row at two parent widths. Only the W setting of the first child changes (Paper's W dropdown also offers Relative).

Fixed

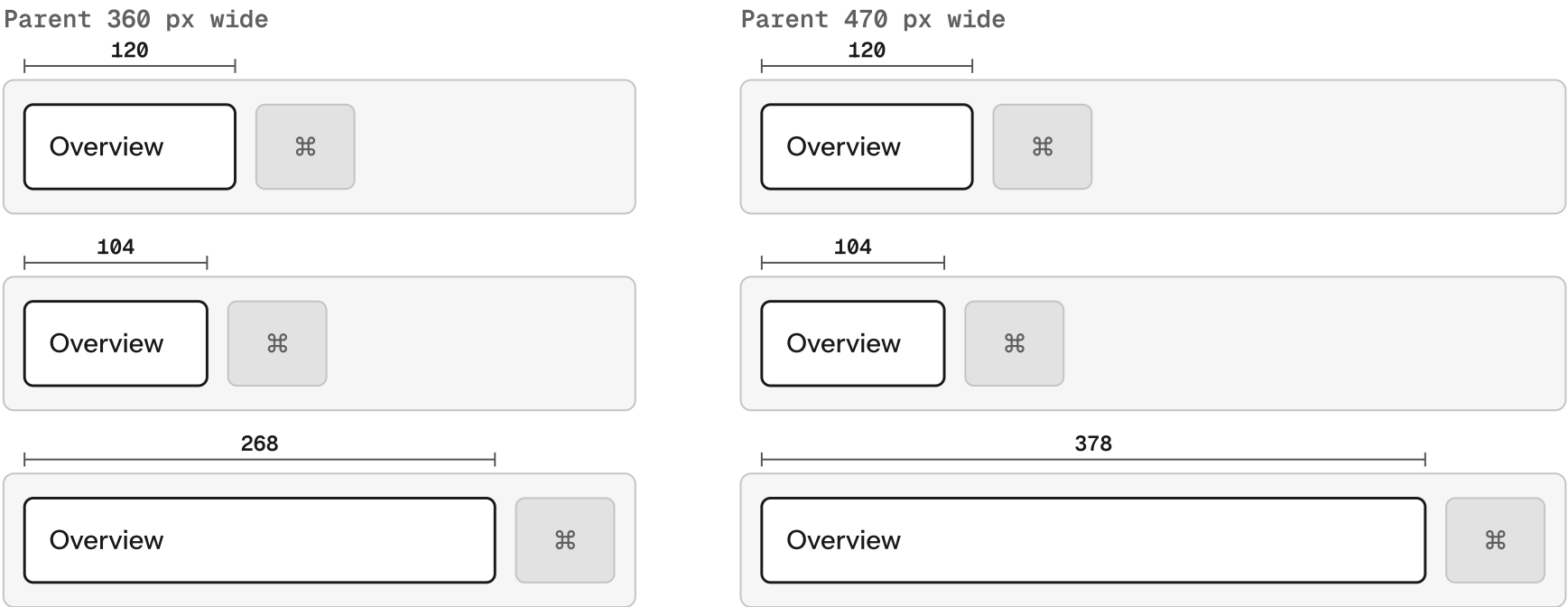
120 px whatever happens

Fit

as wide as its content

Fill

takes the remaining space

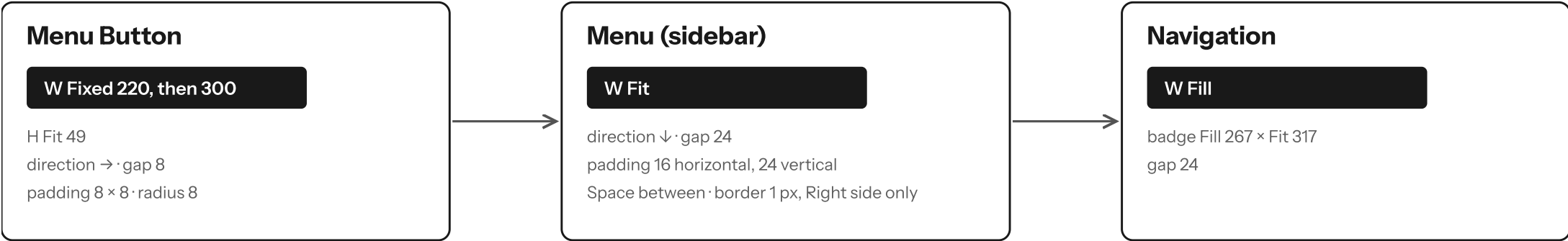


Padding 12 · gap 12 · second child fixed at 56

Figure 02.3

Where each width comes from

Sizing is a chain of dependencies, not three separate numbers.



1. Menu Button width (Fixed) → Menu width (Fit)
2. Menu width → Navigation width (Fill)

Change the button's fixed width and the Fit sidebar follows it; the Fill navigation follows the sidebar.

Figure 02.4

The Flex panel across releases

Where each control moved, so older tutorials still make sense.

July 2026 ▶ 1:46	August 2026 ▶ 4:12	September 2026 ▶ 3:29
Panel rows	Panel rows unchanged; the sliders menu gained Wrap.	Panel rows
Direction → / ↓, then the nine-point grid and the sliders icon to its right	Sliders menu	Nine-point grid on the left; direction ↓ / → and the wrap toggle on the right
Gap field	Align to baseline	Gap field with the sliders icon beside it; a second gap field when wrap is on
Horizontal and vertical padding, individual-padding toggle	Space between	Padding fields, individual-padding toggle
Absolute position (children only), Clip content ⌘C	Wrap	Absolute position, Clip content ⌘C
Sliders menu		Sliders menu
Align to baseline		Space between (with “Vertically” on a column)
Space between		Align to baseline: First baseline, Last baseline

How it works in Paper

Paper’s Flex is CSS flexbox: a flex frame is a div with display: flex. As of September 2026, the Flex section sets direction, wrap, gap (one gap between items and another between lines once wrapped), padding per axis or per side, nine-point alignment, Space between and baseline alignment. Each axis is Fixed, Fit (follows its content, like fit-content) or Fill (takes the parent’s remaining space). ⇧A wraps or converts; Absolute position takes a child out of the flow.

Checked against the videos and documentation as of September 2026. Paper ships often; the release timeline in the reference shows what changed when.

01

Wrap layers in a flex frame

1

Select the layers that should move together; Shift-click adds to the selection.

2

Press ⇧A, or click Wrap in flex ⇧A in the right panel’s Layout section.

3

The new parent is named “Frame”, laid out as a row and sized Fit on both axes, with a starting gap (8 in the demo); the badge under it reads, for example, “Fit 116 × Fit 32”.

4

Rename it by double-clicking its name on the canvas or in the layers panel (“Menu Button” in the demo).

5

To convert an existing freeform frame instead, select it and press ⇧A; the button then reads Add flex. To remove flex, click – in the Flex header or press ⌘⇧A.

⇧A

⌘⇧A

⇧Click

► Shown at 1:37

02

Set gap and padding

1

Select the parent frame: gap and padding belong to it, not to the children.

2

Set the gap (tooltip “Gap between elements”): type a value; or press ↑ or ↓ to change it by 1, and with ⇧ by the nudge amount (8 in the demo, set in Nudge settings... in the file-name menu).

3

Or drag the gap field’s icon left or right, or drag the pink band between two children directly on the canvas; a badge shows the value.

4

Set the horizontal and vertical padding (8 and 8 in the demo).

5

Click the square icon at the end of the padding row to set each side separately; click it again to return to one value per axis.

↑

⇧

↑

► Shown at 2:08

03

Change direction, order and alignment

1

Click ↓ to stack the children in a column, or → for a row; the children keep their order either way.

2

To reorder, drag a child (a blue line marks where it will land) or select it and press the arrow keys.

3

Click one of the nine points to align the children inside the frame; the demo settles on left, vertically centered.

4

For Space between, open the sliders icon in the Flex section and choose Space between, or double-click the nine-point grid.

→

Double-click

► Shown at 4:12

04

Keep related items together under Space between

1

Spot the symptom: with three children, Space between pushes the middle one to the center.

2

Shift-click the items that belong together (avatar and name), press ⇧A and set their gap (8).

3

Name the new frame for its role (“User info”). The parent now has two children, so the group sits at one end and the chevron at the other.

4

To stack text inside the group, select the name and the email, press ⇧A, click ↓ and set the gap to 0.

5

In a vertical sidebar, the same move keeps a top group at the top and a user button at the bottom: set the sidebar to Space between and wrap the brand and the navigation with ⇧A (gap 24).

⇧Click

⇧A

► Shown at 6:38

05

Size with Fixed, Fit and Fill

1

Fixed: drag an edge, type a value in W or H, or press the arrow keys in the field.

2

Fit: choose Fit in the W or H dropdown, or double-click the frame’s top or bottom edge to set the height to Fit.

3

Fill: choose Fill, or hold ⌘ and double-click the left or right edge; the layer then takes the parent’s unused width (badge “Fill 267 × Fit 317” in the demo).

4

To let a child drive its parent, set the parent’s W to Fit: when the fixed-width child changes (220 to 300 in the demo), the parent follows, and siblings set to Fill follow the parent.

Double-click

⌘Double-click

► Shown at 9:52

06

Wrap children onto new lines (September 2026 panel)

1

Select the frame that directly contains the items, such as a row of cards or tags.

2

Click the wrap toggle, the curved-arrow icon beside the direction buttons; items that no longer fit move to a new line.

3

Set the gap between items in the first gap field and the gap between lines in the second field that appears (16 and 20 in the demo).

4

To see which items overflow before wrapping, you can turn off Clip content (⌘C) on the outer frame.

⌘C

► Shown at 3:33

Controls you will use

Control	Where	What it does
Wrap in flex / Add flex	Right panel › Layout (button)	Wraps the selected layers in a new flex frame, or adds flex to a selected freeform frame; shortcut ⇧A. Remove flex with – in the Flex header or ⌘⇧A.
Direction (→ / ↓)	Right panel › Flex	Lays the children out in a row or a column (CSS flex-direction); their order is kept when you switch.
Wrap toggle	Right panel › Flex, curved-arrow icon beside the direction buttons (September 2026 panel)	Lets children break onto new lines instead of overflowing (CSS flex-wrap: wrap) and adds a second gap field for the space between lines. In August 2026, Wrap was an item in the sliders menu.
Gap	Right panel › Flex (tooltip “Gap between elements”)	Sets the fixed space between neighboring children (CSS gap); with wrap on, the gap between items and the gap between lines are set separately. Type it, use ↑ or ↓ (⇧ for the nudge amount), drag the field icon, or drag the pink band on the canvas.
Padding	Right panel › Flex (horizontal and vertical fields, square toggle at the end)	Sets the space between the frame’s edges and its children (CSS padding), per axis or, with the toggle, per side.
Alignment grid	Right panel › Flex (nine points)	Places the children inside the frame along both axes (CSS justify-content and align-items, which swap roles with the direction). Double-click the grid for Space between.
Sliders menu	Right panel › Flex, sliders icon (beside the gap field since September 2026)	Holds Space between (CSS justify-content: space-between) and Align to baseline; the September 2026 panel lists First baseline and Last baseline under it.
W and H sizing	Right panel › Layout, dropdown inside the W and H fields	Fixed keeps a set value; Fit follows the content, like fit-content; Fill takes the parent’s remaining space. A fourth option, Relative, is listed but not explained in the sources. Double-click a top or bottom edge for Fit height; ⌘ + double-click an edge for Fill.
Absolute position	Right panel, checkbox shown for any child of a flex frame	Takes the child out of the flow so it can overlap the others (CSS position: absolute); Left and Top constraint dropdowns appear. Pin the sides it belongs to, or it drifts when the parent resizes.
Size badge	Canvas, label under the selection	Names the sizing mode per axis, for example “Fit 116 × Fit 32” or “Fill 267 × Fit 317”; a number alone means Fixed.

Shortcuts

	Wrap the selection in flex, or add flex to a freeform frame ▶ 1:37		Remove flex ▶ 2:10
	Set the height to Fit (double-click a frame’s top or bottom edge) ▶ 5:36		Set Fill (⌘ + double-click an edge) ▶ 9:48
	Space between (double-click the nine-point alignment grid) ▶ 6:06		In a focused number field, raise the value by 1 (↓ lowers it) ▶ 2:08
	In a focused number field, raise the value by the nudge amount (8 in the demo) ▶ 2:08		Reorder: move the selected child one place later in a row (arrow keys reorder flex children) ▶ 2:55
	Add a layer to the selection ▶ 6:18		Hold ⌘ to show distances from the selection to the parent’s edges ▶ 12:00

Limits and what is coming

- ⦿ Authoring a CSS Grid from the panel was still listed as planned on Paper’s roadmap as of September 2026; the September update only gave existing grid elements alignment and gap controls.
- ⦿ As of September 2026 there was no single-key shortcut for Fit or Fill, according to a Paper team reply on the 8-tips video; the edge double-clicks are the quick route.
- ⦿ The July 2026 tutorial shows the earlier Flex panel. The September 2026 rework moved the wrap toggle beside the direction buttons and the sliders menu beside the gap, so match controls by name rather than by position.

What the tutorials show

Each official video, written down step by step. The dots mark the moments; they open the video there if you want to see it.

Flex layout in Paper

Video · 12 min 29 s · 20 July 2026



From a menu button to a responsive sidebar

Selecting an avatar and a name and pressing ⌘A wraps them in a flex frame (Fit × Fit, gap 8), renamed “Menu Button”. Gap is typed, stepped with arrow keys (⬆ for bigger steps), or dragged from its field icon or the pink band on the canvas. A dropped chevron lands at the blue line; arrow keys reorder it. A “muted” token fill, padding 8 × 8 and radius 8 follow, and ⬇ stacks the children in order. A double-click on the bottom edge sets the height to Fit; the width stays fixed at 220. Space between strands the name mid-row, so avatar and name get their own ⌘A frame, “User info”, and name and email a ⬇ frame with gap 0. A sidebar repeats the move, keeping the button at the bottom. Fill (⌘ + double-click an edge) makes the navigation follow the sidebar; on a Fit sidebar, widening the button to 300 resizes both. Absolute position frees a close button, set 16 from the top and right, that drifts on resize until constraints are added.

- 1:37 Wrap in flex ⌘A
- 2:24 Drag the gap on the canvas
- 5:46 Fit height, fixed width
- 6:38 Nested frame for Space between
- 9:52 Fill and Fit sizing
- 11:54 Absolute position

Build it in Paper

- 01

Select an avatar image and a name and press ⌘A (or click Wrap in flex ⌘A in the Layout section). Double-click the new frame’s name and call it “Menu Button”; leave the direction on → (row).
- 02

With the parent selected, set the gap to 8, horizontal and vertical padding to 8 and the radius to 8, then add a fill, ideally from a color token.
- 03

Drag a chevron icon into the frame (a blue line shows where it will land). Fix the width at 220, then double-click the bottom edge to set the height to Fit.
- 04

Choose Space between from the sliders icon (or double-click the alignment grid). When the name drifts to the middle, Shift-click the avatar and the name, press ⌘A, set that frame’s gap to 8 and name it “User info”.
- 05

Place the button at the bottom of a vertical sidebar set to Space between, wrap the sidebar’s top items with ⌘A, give the navigation group Fill width (⌘ + double-click its side edge) and resize the sidebar to test.

Common mistakes

- ×

Applying Space between to a row of three children and getting the middle one stranded in the center.

→ Group the items that belong together with ⌘A first, so the parent has exactly the children that should be pushed apart.
- ×

Dropping a new item into a top-aligned flex frame and expecting it to stay where you released it.

→ Flex places children by order: watch the blue insertion line, reorder with the arrow keys, or group and use Space between to pin an item to one end.

Check before you move on

- ☐

For every frame, can you say where its width and height come from: Fixed, Fit or Fill?
- ☐

If the longest label doubled, would anything overlap, clip or jump to a new position?
- ☐

Is every space between siblings a gap and every inner margin padding, with no spacer layers or nudged offsets?
- ☐

Does each Space between parent contain only the children that should be pushed apart?
- ☐

Is anything on Absolute position that should be in the flow, and is every overlay pinned to the edges it belongs to?

Use alignment to make comparison effortless

A grid is a set of shared positions. Its value is the reading work it removes, especially when people compare numbers.

An edge can be more useful than a box

When repeated values begin or end at different positions, the eye must search before it can compare. A shared edge removes that search. This is why a calm table often communicates more than a wall of individually polished cards. The reader can travel down a value lane or across a category without reinterpreting the layout at every stop.

Alignment is not the same as centering. A centered value may look balanced inside its own card while being difficult to compare with values elsewhere. Choose the edge that serves the task. Text labels usually need a predictable starting point. Numbers benefit from a consistent ending point and, where the typeface supports them, tabular numerals. Units and qualifiers need positions that remain attached to the values they describe.

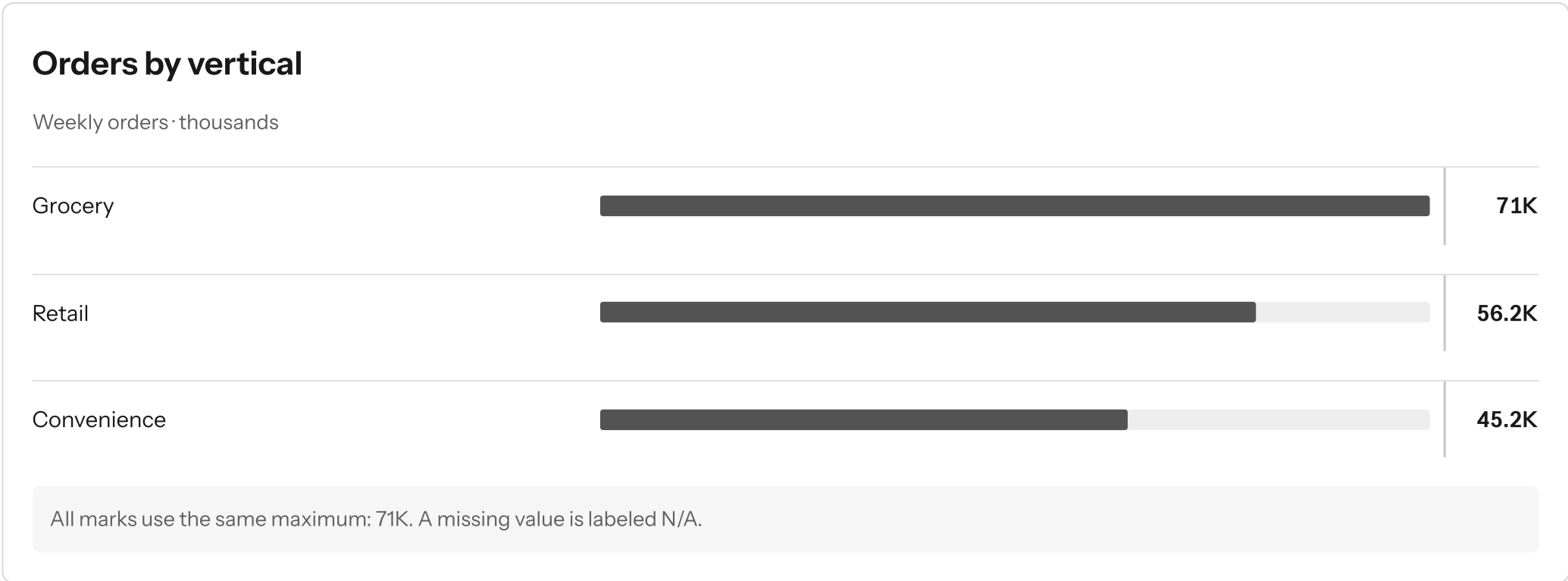
Align the content that people compare, not merely the boxes that contain it.

The principle of chapter 3

Study 03 · change it yourself

Keep label, mark and value in their own lanes

Compare free positioning with shared lanes, then edit the values.

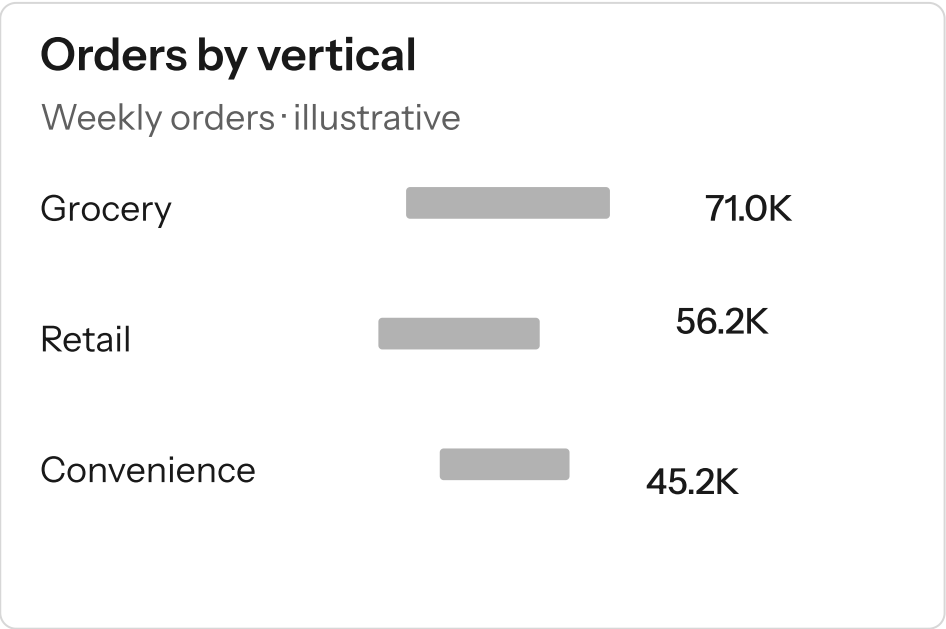


The value column keeps a shared right edge while labels wrap independently. Editing a value changes its mark without moving the value lane.

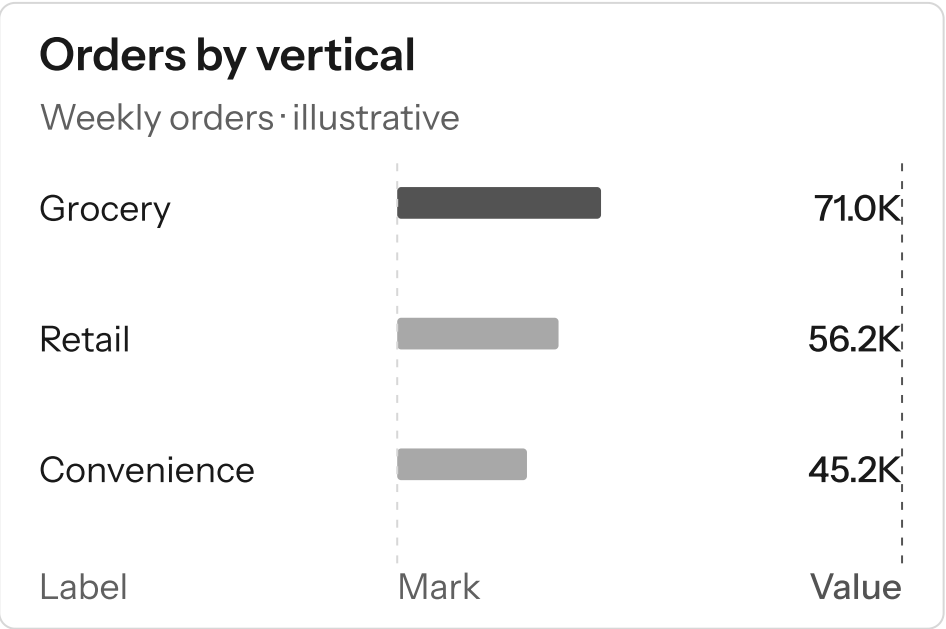
Give every number a predictable address.

Separate the label lane, the mark lane and the value lane.

BEFORE / POSITIONED OBJECTS



AFTER / SHARED COLUMNS



A visual grid can be built with nested Flex rows; it does not require full CSS Grid authoring.

Reconstructed teaching example · synthetic values · not a Paper screenshot

Figure 03.1 · Plate

Give every number a predictable address.

Aligned columns make the comparison quicker to read. Use a common row structure and reserve a fixed numeric lane; a bar’s length should never determine where its value starts.

Build a visual grid with available tools

As of early October 2026, Paper's public roadmap lists native CSS Grid authoring as planned, not shipped. You can still design a precise visual grid with nested Flex frames: a vertical Flex stack of rows, each row a horizontal Flex frame whose children are the lanes. Grid layouts that arrive from code, such as a section captured with Snapshot, still render on the canvas, and the September 2026 release added alignment and gap support for grid elements. The shipped page may use browser Grid, Flex or a semantic table; the design principle is the same shared geometry.

Decide the lanes once, then repeat them. If each row chooses its own label width, the apparent grid drifts as soon as one label grows. Give every value lane the same Fixed width and let the label take the remaining width with Fill. Paper's instructions for agents make the same point in CSS terms: gap alone won't align columns across rows, so repeated rows need fixed-width slots that do not shrink. Use padding to keep values off the edges of their lane, and reserve larger gaps for transitions between genuinely different groups.

Once alignment supplies the order, emphasis can stay selective: a pale band behind one group, or a single accent color for the value that matters. Check the order before you add emphasis. In Paper, hold `⌘` with a layer selected to read its distances to the parent's edges, and read the size badge under the selection, which names each axis's mode, as in "Fill 387.33 × Fit 201.92". The menu under the file name toggles Layout guides (`⇧G`), Show pixel grid (`⌘'`) and Snap to pixel (`⇧⌘'`).

Test the row that makes the layout uncomfortable

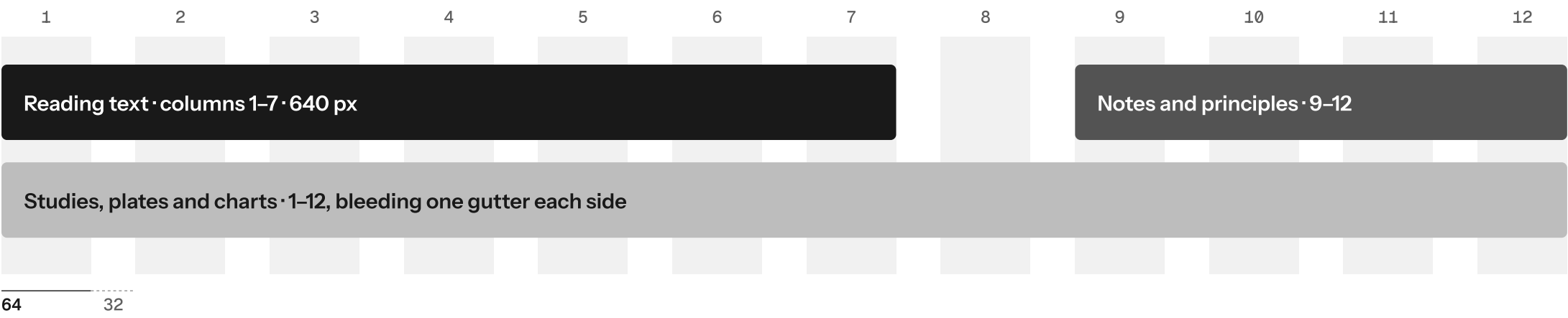
A grid should be judged with a long label, an absent value and a value with an unfamiliar number of digits. These cases reveal whether the apparent order comes from the structure or from conveniently short content. An absent observation should keep its place and show an explicit missing state. Removing the cell destroys the comparison precisely where explanation is most needed.

Paper's Snapshot iteration tutorial shows why that review belongs on the canvas. An agent added a third column to a captured section: it changed the grid from `repeat(2, 1fr)` to three columns, duplicated a column and reported a passing checkpoint review, including that all three columns sat properly on the grid. The new column still had an empty image area, and the media frames kept a fixed height of 324 that now left too much empty space. The presenter fixed each frame by double-clicking its bottom edge to set Fit. Duplication preserves structure, but it also creates new relationships that only a look at the actual rows can confirm.

Figure 03.2

The grid this guide is set on

Twelve columns give every element a role: reading text, notes, or full-width figures.



12 columns × 64 px + 11 gutters × 32 px = 1,120 px. Column 8 stays empty between text and notes.

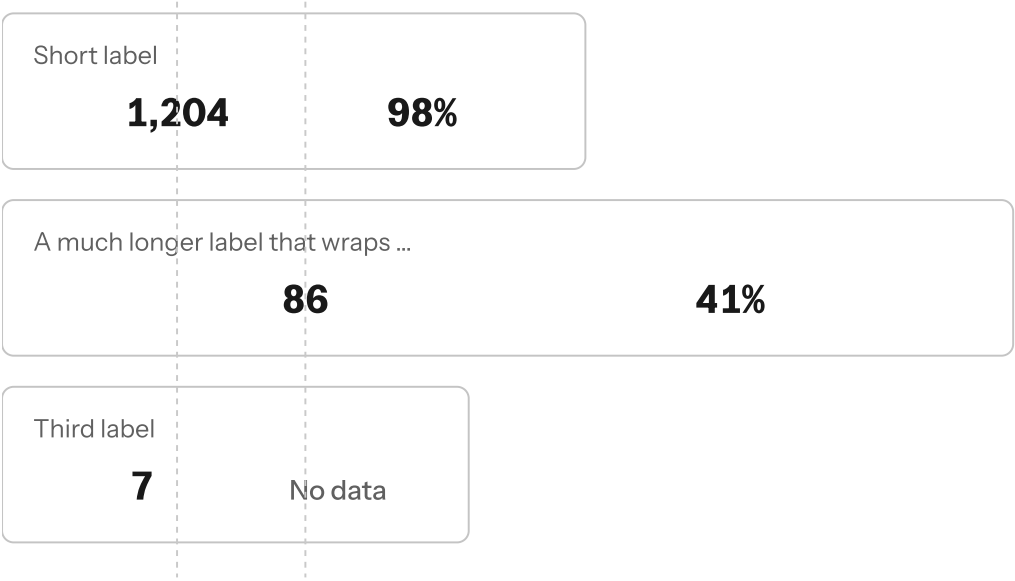
Figure 03.3

Aligned by the box, or by the content

The same rows built two ways. A long label and a missing value show which structure holds its edges.

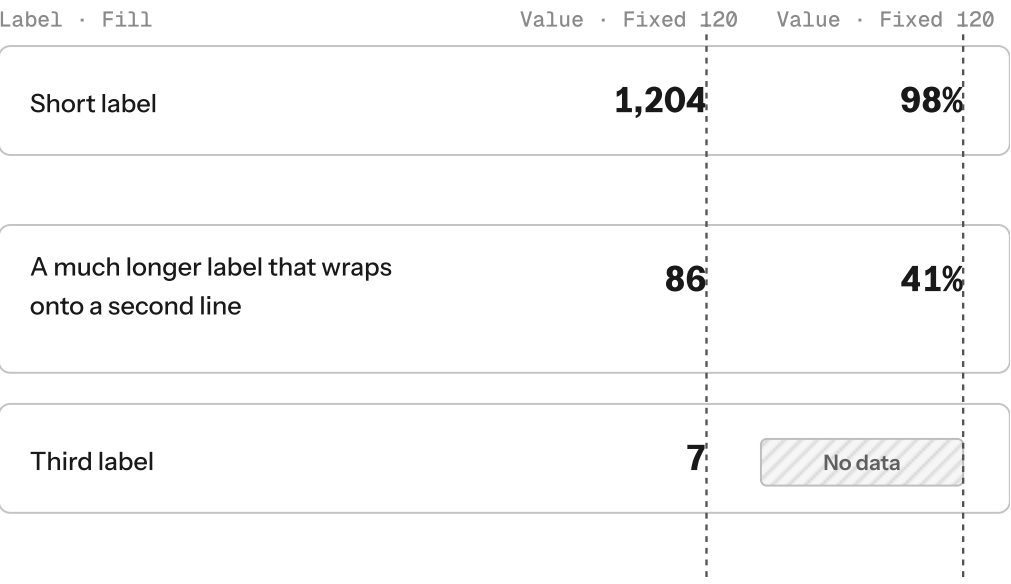
Aligned by the box

Each card centers its own values. The numbers drift.



Aligned by the content

A Fill label lane and two Fixed 120 lanes, right-aligned.

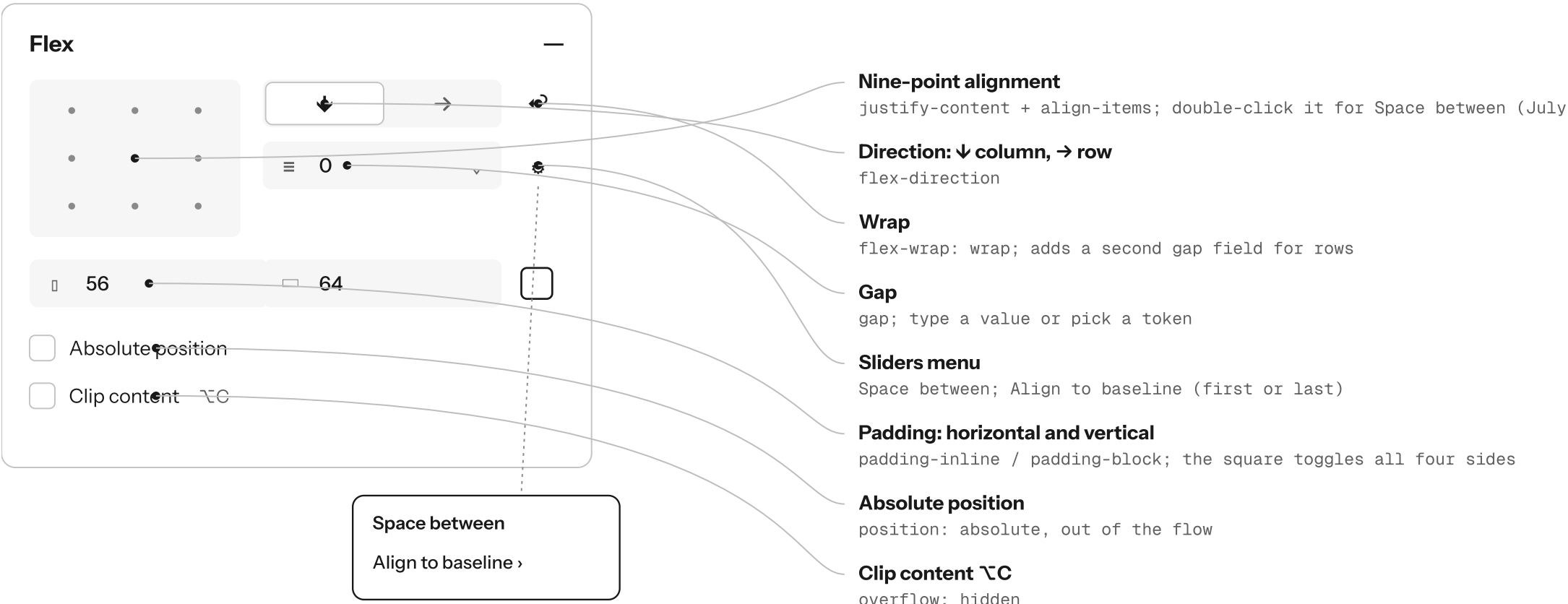


Row gap 12 · padding 16 · a missing value keeps its lane and says so

Figure 03.4

The Flex section, labelled

Where each control sits since the September 2026 rework, and the CSS it writes.



Schematic of the Flex section as shown on 28 September 2026 (values from the demo frame). Not a screenshot.

How it works in Paper

As of September 2026, Paper builds grids from nested Flex frames: a vertical stack of rows, each a horizontal Flex frame whose lanes are Fixed, Fit or Fill. Gap, padding and the nine-point alignment grid map to CSS gap, padding, justify-content and align-items. The September flex-panel rework put Space between beside the gap field, split the wrap gap into separate horizontal and vertical values, and added alignment and gap for grid elements. Native CSS Grid authoring remains planned on the roadmap.

Checked against the videos and documentation as of September 2026. Paper ships often; the release timeline in the reference shows what changed when.

01

Build a row with shared lanes

1

Select the label and value text layers and press ⇧A, or click "Wrap in flex ⇧A" in the right panel's Layout section.

2

In the new frame's Flex section, check the direction: → lays children out in a row, ↓ in a column.

3

Select one value and press ⇧A again to give it a lane frame of its own; repeat for each value.

4

Type the same number into W for every value lane so each one is Fixed.

5

Set the label's W to Fill in the W dropdown (Fixed, Fit, Fill, Relative), or hold ⌘ and double-click its left or right edge.

6

Read the size badge under the selection to confirm each mode, for example "Fill 267 × Fit 317".

⇧A

⌘

Double-click

► Shown at 1:36

02

Stack rows into a column and space them

1

Select the finished row and press ⇧A, then click ↓ in the new frame's Flex section so rows stack.

2

Select the row and press ⌘D for each extra row you need.

3

With the column selected, set the gap: type a value, use ↑ and ↓ (hold ⇧ for the nudge step), drag the field's icon, or drag the pink gap band on the canvas.

4

Set horizontal and vertical padding; the square icon at the end of the padding row switches to four separate sides.

5

When you drag a layer into a Flex frame, a blue outline marks the receiving frame and a blue line marks where the layer will land.

⇧A

⌘D

► Shown at 2:04

03

Align content inside a lane

1

Select the lane frame, not the text inside it.

2

Click a point on the Flex section's nine-point grid. In a row lane, the right-hand middle point ends the number on the lane's right edge and centers it vertically.

3

To push the last child to the far edge, turn on Space between. In the July 2026 panel it sat in the sliders menu, and double-clicking the grid did the same; since September it is in the menu of the button beside the gap field.

4

If Space between scatters three children, select the two that belong together and press ⇧A so the parent has only two children.

Double-click

⇧A

► Shown at 5:59

04

Line up freeform and absolute layers with the align buttons

1

Select a child of a freeform frame, or a child with Absolute position ticked.

2

Use the align buttons in the Layout section header: left, horizontal centers, right, top, vertical centers and bottom, followed by a ... button.

3

Align horizontal centers shows the shortcut ⌘⇧S in its tooltip.

4

The two centering buttons also set that axis's constraint to Center, so the layer stays centered when its parent is resized.

5

Hold ⌘ to read the distances to the parent's edges and confirm the result.

⌘

⇧S

⌘

► Shown at 1:46

05

Have an agent extend a grid, then review the rows yourself

1

Place the new content in a small frame beside the design, so the agent can read it from the canvas instead of from the prompt.

2

Start the prompt with the location, as the tutorial does with "In Paper, add a third column to the frame", then name the frame or text that holds the new content.

3

Approve the write tools when the agent asks; the Claude app offers Allow once (↵) and Always allow for project (local) (⌘↵).

4

Read the agent's review, then inspect the canvas yourself for empty media areas, heights that no longer fit and text that wraps differently.

5

Select each oversized frame and double-click its bottom edge to set its height to Fit.

Double-click

► Shown at 0:37

06

Edit wrapped and grid layouts in the September 2026 flex panel

1

Select a Flex frame. The reworked Flex section shows the nine-point grid on the left, the ↓ and → direction buttons on the right with a wrap toggle beside them, and the gap field below.

2

Click the wrap toggle to let children flow onto new lines; a second gap field appears, so horizontal and vertical gaps are set separately (16 and 20 in the demo).

3

Open the button beside the gap field for Space between, with an axis option (Vertically on a column frame), and Align to baseline, with First baseline and Last baseline.

4

Layouts that an agent builds with Flex now open in this panel too, and grid elements support alignment and gap.

► Shown at 3:33

Controls you will use

Control	Where	What it does
Wrap in flex ⇧A / Add flex ⇧A	Right panel › Layout	Wraps the selected layers in a new Flex frame. With a freeform frame selected, the button reads "Add flex ⇧A" and converts that frame instead. Maps to display: flex. Remove flex is ⌘⇧A in the right-click menu.
Direction (→ / ↓)	Right panel › Flex	Lays children out in a row or a column and keeps their order when you switch; maps to flex-direction.
Nine-point alignment grid	Right panel › Flex	Positions children inside the frame on both axes; maps to justify-content and align-items, whose roles swap with the direction. In the July 2026 panel, double-clicking the grid applied Space between.
Gap	Right panel › Flex	Space between neighboring children; maps to CSS gap. With wrap on, the September 2026 panel shows separate horizontal and vertical gap fields. You can also drag the pink band between children on the canvas.
Space between and Align to baseline	Right panel › Flex, menu of the button beside the gap field (September 2026)	Space between pushes the first and last children to the frame's edges (justify-content: space-between); Align to baseline offers First baseline and Last baseline.
Padding	Right panel › Flex	Horizontal and vertical padding fields; the square icon at the end of the row switches to four separate sides. Maps to CSS padding.
W and H modes	Right panel › Layout	Fixed holds a value, Fit hugs the contents and Fill takes the parent's unused space. Relative is listed but not explained in the tutorials. Double-click a top or bottom edge for Fit height; hold ⌘ and double-click an edge for Fill.
Align buttons	Right panel › Layout section header, for children of freeform frames and absolute children	Aligns the selection to its parent's left, horizontal center, right, top, vertical center or bottom; the centering buttons also set Center constraints. Align horizontal centers is ⌘⇧S.
Layout guides, Show pixel grid, Snap to pixel	Menu under the file name, top left	Toggles for guides (⇧G), a pixel grid (⌘') and pixel snapping (⇧⌘'). Nudge settings... in the same menu sets the step for ⇧ + arrow keys. A Guides section with a + button sits near the bottom of the right panel when a frame is selected.

Shortcuts

	Wrap the selection in a Flex frame, or add Flex to a selected freeform frame ▶ 1:36		Remove flex (listed in the right-click menu) ▶ 3:46
	Put the selection in a new freeform frame (Frame selection) ▶ 0:50		Align horizontal centers (children of freeform frames and absolute children) ▶ 1:46
	Double-click a frame's top or bottom edge to set its height to Fit ▶ 2:53		Hold ⌘ and double-click an edge to set that axis to Fill ▶ 9:40
	Hold to show the distances from the selection to its parent's edges ▶ 12:02		Toggle Layout guides ▶ 0:38
	Duplicate; after an ⌘-drag, it repeats the same offset and rotation ▶ 4:40		With a Flex child selected, the arrow keys move it earlier or later among its siblings ▶ 1:46

Limits and what is coming

- Native CSS Grid authoring was listed as Planned on Paper's public roadmap, not shipped, as of early October 2026. Grid layouts that arrive from code or Snapshot still render, and since September 2026 grid elements support alignment and gap in the flex panel.
- As of October 2026, Paper's MCP instructions tell agents not to write display: grid, margin or HTML tables with write_html; agents build rows from Flex, padding, gap and fixed-width slots that do not shrink.
- There were no keyboard shortcuts for Fit or Fill as of early September 2026, according to a Paper team reply; use the edge double-clicks instead.
- Rulers were not available at the start of October 2026; a Paper team reply said they were in progress and pointed to the Guides section of the right panel.

What the tutorials show

Each official video, written down step by step. The dots mark the moments; they open the video there if you want to see it.

How to use an AI agent to iterate on a design you captured with Paper Snapshot

Video · 3 min 30 s · 2 May 2026



An agent adds a third column; the canvas shows what its review missed

The starting point is a section of Resend's site, captured with Paper Snapshot, beside a small frame holding an icon, the title "Dynamic suppression list" and its description. In the Claude desktop app's Code tab (Sonnet 4.6, Auto accept edits), the presenter starts the prompt with "In Paper, add a third column to the frame" and points to the "Dynamic suppression list" text as the source, because an agent needs location context just as a person does. The agent finds the file, page and frame through the Paper MCP, takes screenshots, reads a tree summary and a column's JSX, notes that the grid uses `repeat(2, 1fr)`, switches it to three columns and duplicates a column. She approves Duplicate Nodes with Always allow for project (local). The agent rewrites the text and icon and posts a passing Checkpoint review. By hand, she finds that the new column has no image and the media frames, fixed at 324, are now too tall; double-clicking each bottom edge sets Fit (201.92).

- 0:37 Prompt starts with "In Paper"
- 1:34 Allow Duplicate Nodes
- 1:52 `repeat(2, 1fr)` becomes three columns
- 2:12 Checkpoint review passes
- 2:34 Missing image noticed
- 2:53 Double-click edge sets Fit

Build it in Paper

- 01

Create one row first: a label text and two value texts. Select them and press ⇧A (Wrap in flex), then check that the new frame's Flex section shows → for a row.
- 02

Select each value and press ⇧A again so it sits in a lane frame of its own. Type the same number into W for both value lanes so they are Fixed, then set the label's W to Fill, or hold ⌘ and double-click its left or right edge.
- 03

Select each value lane and click the right-hand middle point of the nine-point grid so the number ends on the lane's right edge. Set the row's gap and its padding as separate values.
- 04

Select the row, press ⇧A and click ↓ to start a column of rows, then select the row and press ⌘D for each extra row. Replace one label with a long phrase and one value with an explicit missing state.
- 05

Drag the outer frame's edge to resize it, hold ⌘ to read distances, and confirm that every value lane keeps the same right edge in every row.

Common mistakes

- ×

Centering each number inside its own card, so every card looks balanced but the values sit at different positions.

→ Put the values in lanes of one Fixed width and align them to the lane's right edge with the nine-point grid.
- ×

Turning on Space between in a row of three children and getting a middle item that floats to the center.

→ Select the items that belong together and press ⇧A so the parent has two children, then apply Space between.

Check before you move on

- ☐

Does every value end on the same edge in every row, including the row with the longest label?
- ☐

Do all value lanes share one Fixed width, so that no row sets its own geometry?
- ☐

Does a missing value keep its cell and say explicitly that it is missing?
- ☐

Is the padding inside a lane smaller than the gap between groups?
- ☐

After an agent changed the grid, did you check images, heights and wrapping on the canvas itself?

- ×

Relying on gap alone to line up columns across rows.

→ Give each repeated lane a Fixed width; gap only spaces neighbors, it does not create columns.
- ×

Treating an agent's passing self-review as proof that the grid is finished.

→ Inspect the rows on the canvas. In the tutorial the review passed while a column lacked its image and the media frames kept an outdated fixed height.

Design what happens when the frame changes

Responsive design is the preservation of meaning as space changes, not a smaller copy of the desktop composition.

Decide which relationship must survive

A desktop layout has room to separate many things at once. On a phone, that arrangement may no longer fit, but the reader still needs the same essential relationships: a label beside its meaning, an action near the decision it completes, and a route back to the previous state. A responsive design preserves those relationships even when the geometry changes.

Begin with the content's limits. A row that contains a rank, portrait, name and score has several different requirements. The portrait should remain recognizable; the score should remain comparable; the name may wrap. Squeezing every dimension equally ignores these differences. A better small-screen composition gives each element a clear rule, then allows the surrounding structure to adapt.

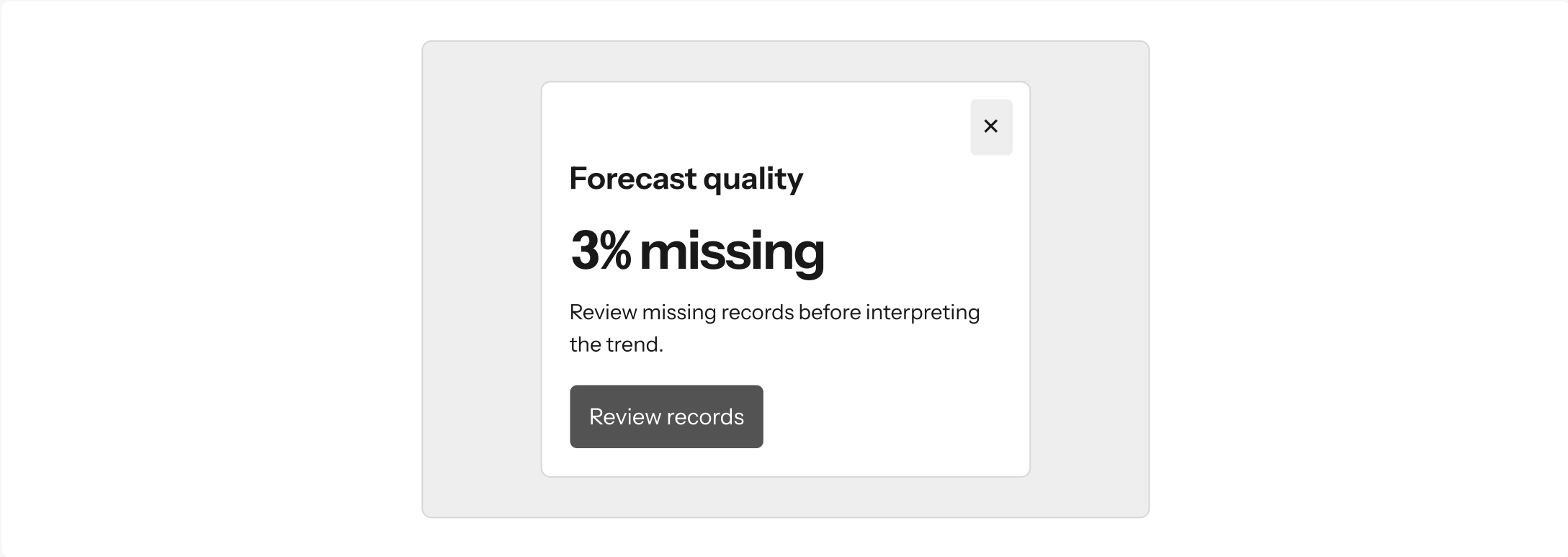
Responsive rules protect the important relationships while allowing the arrangement to change.

The principle of chapter 4

Study 04 · change it yourself

Resize the parent and test the anchor

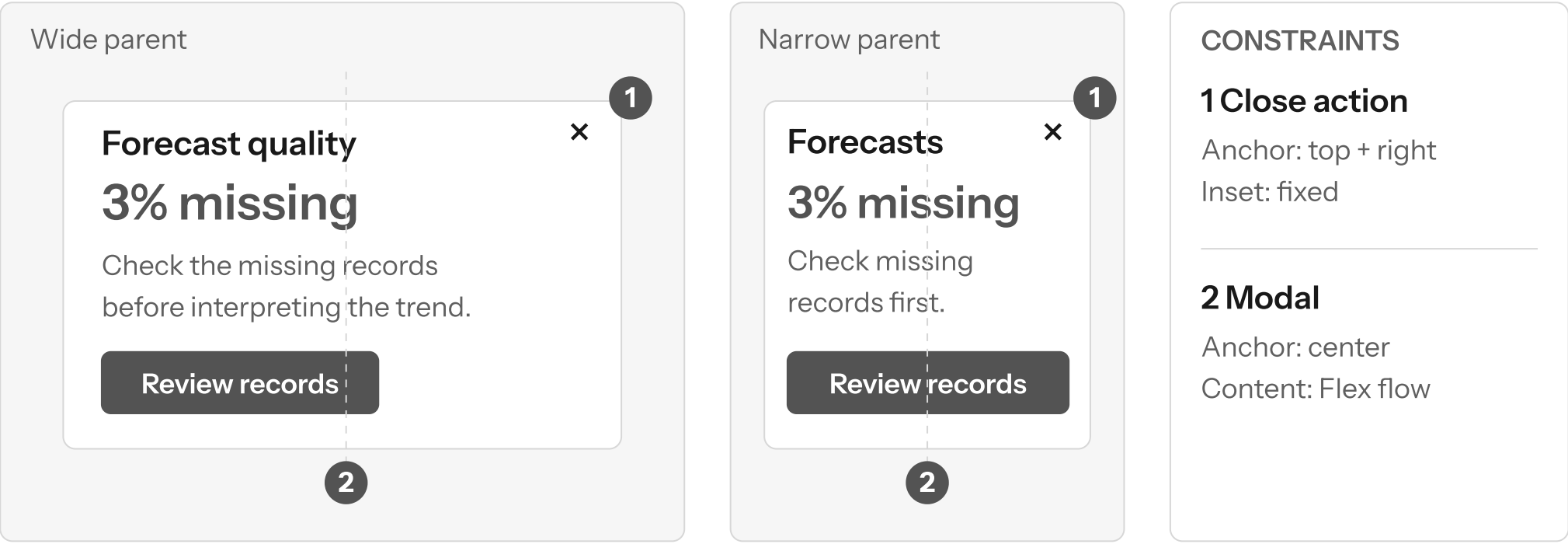
Narrow the parent and test what stays anchored.



The close action keeps its 12 px right inset as the parent changes. The text and main action remain in normal content flow.

Anchor the exception. Let content flow.

Keep the close action in the corner while the modal remains centered.



Anchoring preserves an intentional position; flowing content still needs its own layout rules.

Reconstructed teaching example · synthetic values · not a Paper screenshot

Figure 04.1 · Plate

Anchor the exception. Let content flow.

The corner action uses top/right anchoring; the modal stays centered. These constraints do not replace the Flex layout needed for text and actions inside the modal.

Flow and constraints solve different problems

Flex governs relationships among participating siblings. Constraints govern how a layer outside that flow responds when its parent is resized: a child of a frame without Flex, or a child with Absolute position ticked inside a Flex frame. Paper's constraints tutorial makes the distinction tangible with a corner close button and a centered modal. The close button belongs to an edge relationship; the modal belongs to a center relationship. Neither behavior is described by where it happens to sit in one screenshot.

The tutorial also distinguishes Center, Left + Right and Stretch. Center keeps the child's width and lets the space on each side change. Left + Right keeps the distance to both edges, so the child's width changes instead. Stretch scales the width and the side distances together, and Paper then shows the width as a percentage. These are different promises. For a reading panel, comfortable content dimensions usually matter more than proportional scaling.

A design can require a structural change beyond simple resizing. Two comparison columns may become labeled groups; a toolbar may wrap; a long description may move beneath its title. Paper frames at selected widths can specify those outcomes, and since August 2026 a Flex frame can also wrap its children onto new lines. The frames are design evidence for the implementation, not proof that the running application already switches correctly between them.

Use demanding content at more than one width

Compare the same content under different space conditions. Keep the wording constant while changing width, so the effect of the layout rule remains visible. In Paper, duplicate the frame and drag the copy's edge or type a new W. When an agent creates artboards through the Paper MCP, its default sizes are Desktop 1440 × 900, Tablet 768 × 1024 and Mobile 390 × 844. Then replace one short label with a longer one. If both variables change at once, it becomes difficult to tell whether an improvement came from the structure or from friendlier content.

A narrow layout also changes interaction. A persistent navigation bar occupies valuable height but keeps the user oriented; a floating control may cover the last item unless the content accounts for it. These are part of the responsive composition. The most useful review therefore follows a complete task from its entry point to its result, including the last row and the state after an action.

Figure 04.2

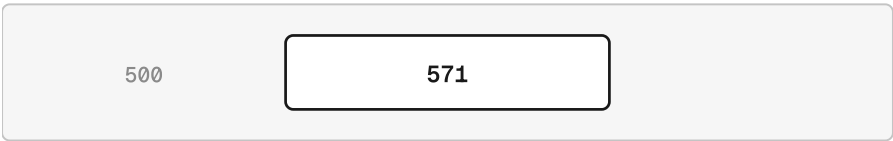
One modal, three constraints

Each rule protects one quantity: the width, the margins or the proportions.

Before: frame 1571

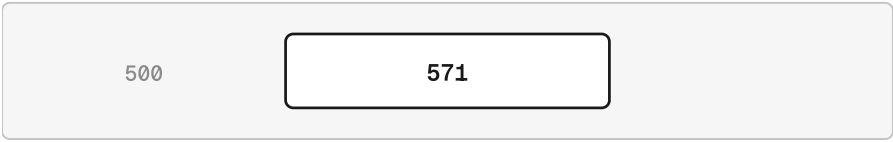
Center

keeps the width



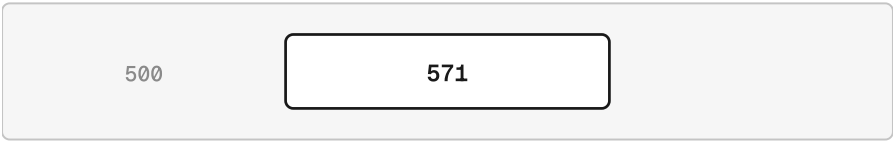
Left + Right

keeps the margins

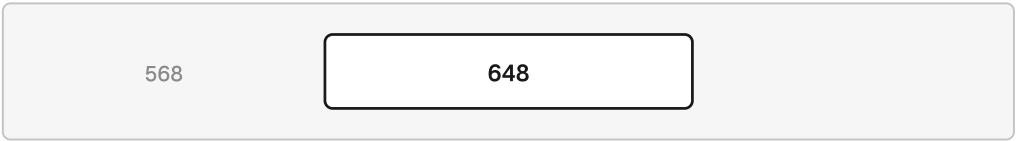
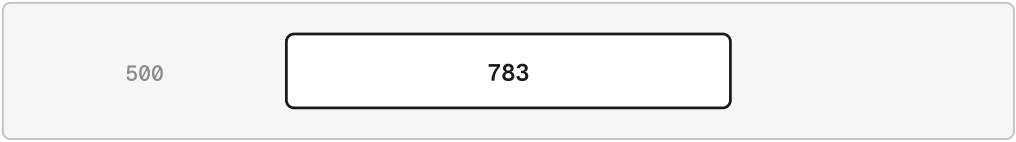
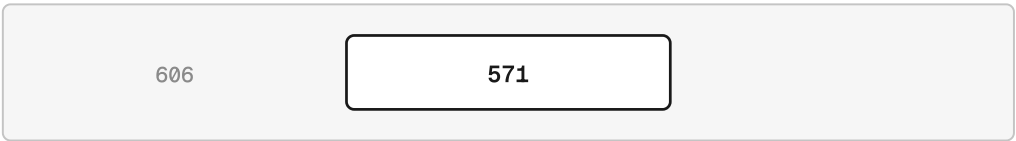


Stretch

keeps the proportions



After widening to about 1783

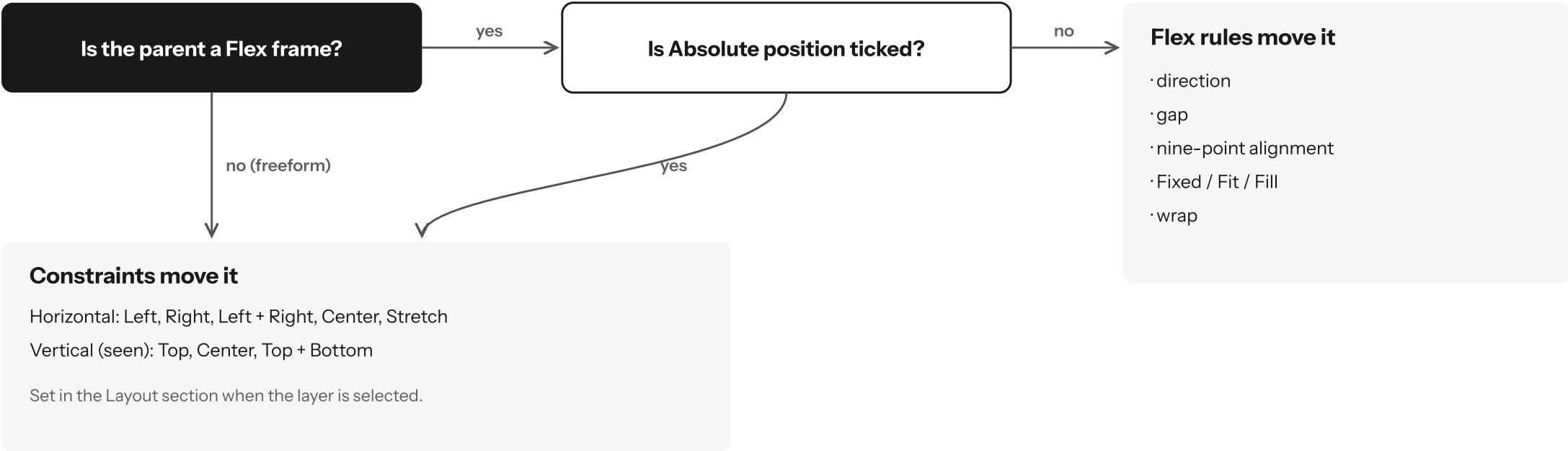


Values from the constraints demo; Stretch showed W as 36% and, after widening, x 568 and width 648.

Figure 04.3

Which rule moves this layer?

Two questions decide whether Flex or constraints position a layer.



How it works in Paper

As of September 2026, Paper handles a resized parent in two ways. Flex children follow the parent's direction, gap and alignment and their own Fixed, Fit or Fill sizes; since August 2026 they can also wrap. Layers outside Flex flow, meaning children of freeform frames and children with Absolute position ticked, follow constraints: Left, Right, Left + Right, Center or Stretch horizontally, a matching vertical choice, and a pin diagram. Stretch expresses the width as a percentage of the parent.

Checked against the videos and documentation as of September 2026. Paper ships often; the release timeline in the reference shows what changed when.

01

Pin an overlay to a corner

1

Select the overlay inside its Flex frame (a 16 × 16 close button in the demo) and tick Absolute position in the right panel; it leaves the flow and floats over the content.

2

Hold ⌘ to read its distances and place it; the Flex tutorial used 16 from the top and 16 from the right.

3

Note the starting constraints: Paper sets Left and Top, so the button drifts away from the corner when the parent widens.

4

Click the right handle of the constraints diagram (its tooltips read like "Pin top"), or open the horizontal dropdown and choose Right. Leave the vertical dropdown on Top.

5

Resize the parent: the button now keeps the same distance from the top and from the right.

⌘

► Shown at 1:05

02

Keep an overlay centered

1

Select the overlay inside a frame without Flex; in the demo, a Transfer modal inside a freeform frame named Modal scrim.

2

Do not rely on dragging: a red guide appears when the modal reaches the middle, but its constraints stay Left and Top.

3

Click Align horizontal centers (⌘⇧S) in the Layout section header; the horizontal constraint changes to Center.

4

Click Align vertical centers; the vertical constraint changes to Center as well.

5

Resize the parent: the overlay keeps its size and stays centered.

⌘⇧S

⌘⇧V

► Shown at 1:46

03

Choose how the width responds

1

Select the overlay and open the horizontal constraint dropdown: Left, Right, Left + Right, Center, Stretch.

2

Center keeps the width and lets the space on each side change.

3

Left + Right keeps both side distances and lets the width change; hold ⌘ to read them (500 on each side in the demo).

4

Stretch changes the width and the side distances in proportion, and the W field then shows a percentage (36% in the demo).

5

Set the vertical dropdown the same way; in the demo, the full-size scrim used Left + Right with Top + Bottom.

⌘

► Shown at 2:30

04

Make Flex children follow their parent

1

Select a child that should grow and set its W to Fill, or hold ⌘ and double-click its left or right edge.

2

Keep items that must hold their size, such as an avatar or a score column, on Fixed.

3

Double-click a frame's top or bottom edge to set its height to Fit, so it follows its content.

4

To let one child set a column's width, set the parent's W to Fit; siblings on Fill then follow the parent.

5

Resize the outer frame and watch which children grow, shrink or keep their size.

⌘

Double-click

Double-click

► Shown at 9:40

05

Let a row wrap

1

If items are cut off, untick Clip content (⌘C) on the outer frame to see the overflow.

2

Select the direct parent of the items; in the August 2026 demo, a row of genre pills on a 390-wide phone screen.

3

Turn on wrap: in the August panel, choose Wrap in the sliders menu; in the September panel, click the wrap toggle beside the direction arrows.

4

Reorder items to change where lines break; an item that does not fit wraps at its own position.

5

In the September panel, set the horizontal and vertical gaps separately while wrap is on.

⌘

C

► Shown at 4:14

06

Test a frame at more than one width

1

Duplicate the frame with ⌘D so the original stays as a reference.

2

Resize the copy by dragging an edge or typing a new W; when you draw a new frame, the right panel lists presets such as Phone, Tablet and Desktop.

3

Check Fill children, wrapped rows and pinned overlays, and hold ⌘ to confirm distances to the edges.

4

Replace one short label with a long one and look at the last row at the narrowest width.

⌘

D

⌘

► Shown at 10:52

Controls you will use

Control	Where	What it does
Absolute position	Right panel, under the Layout and Flex controls	Takes a child out of its Flex frame's flow so it floats over the content; constraints and the align buttons then apply to it.
Horizontal constraint	Right panel › Layout, first dropdown under W and H	Left, Right, Left + Right, Center or Stretch: which horizontal distances to the parent the layer keeps when the parent is resized. Stretch turns W into a percentage of the parent.
Vertical constraint	Right panel › Layout, second dropdown	The vertical counterpart; Top, Center and Top + Bottom appear in the demo.
Constraints diagram	Right panel › Layout, beside the two dropdowns	Click a side handle to pin that side (tooltips such as "Pin top"); the diagram highlights the current pins and changes with the dropdowns.
Align buttons	Right panel › Layout section header, for layers outside Flex flow	Align the layer within its parent; the two centering buttons also set that axis's constraint to Center. Align horizontal centers is ⌘⇧S.
W and H modes	Right panel › Layout	Fixed keeps a size, Fit hugs the content and Fill takes the parent's unused space. Flex children respond to a resize through these modes, not through constraints.
Wrap	Right panel › Flex: the sliders menu in August 2026, a toggle beside the direction arrows since September 2026	Lets a Flex frame's children break onto new lines instead of shrinking or overflowing; maps to flex-wrap: wrap.
Clip content ⌘C	Right panel, under the Layout and Flex controls	Hides whatever extends beyond the frame, comparable to overflow: hidden.
Frame presets	Right panel, while drawing a new frame	Lists size presets for a new frame: Phone, Tablet, Desktop, Presentation, Smartwatch, Paper and Social media.
Shortcuts		
⌘	Hold to show the distances from the selection to its parent's edges ▶ 2:36	⌘⇧SAlign horizontal centers, which also sets the horizontal constraint to Center ▶ 1:46
⌘C	Toggle Clip content ▶ 0:42	⇧AAAdd Flex to a freeform frame, or wrap loose layers in a new Flex frame ▶ 1:08
⇧F	Put the selection in a new freeform frame (Frame selection) ▶ 0:50	Double-clickDouble-click a frame's top or bottom edge to set its height to Fit ▶ 5:30
⌘Double-click	Hold ⌘ and double-click an edge to set that axis to Fill ▶ 9:40	⌘DDuplicate the selection ▶ 4:40
Limits and what is coming		
○ Constraints apply only outside Flex flow. A Flex child ignores them and responds through its Fixed, Fit or Fill size and its parent's Flex settings.		
○ Ticking Absolute position starts a layer at Left and Top, and dragging a layer to the visual center leaves its constraints unchanged; in the July 2026 demo, only the align buttons set Center automatically.		
○ Paper can store breakpoint values as tokens, but as of September 2026 the tutorials show no way to switch one frame between layouts at a breakpoint; you design each width as its own frame, and the implementation decides how one becomes the other.		
○ There was no scale tool as of 28 September 2026 (announced as coming soon), so resizing a frame changes its layout but does not scale its contents. A lock for the aspect ratio of an existing frame was also described as coming soon in a team reply at the start of October 2026.		

What the tutorials show

Each official video, written down step by step. The dots mark the moments; they open the video there if you want to see it.

Constraints in Paper

Video · 3 min 31 s · 20 July 2026



Pin a close button, center a modal, compare Center, Left + Right and Stretch

Part two of Paper's layout series defines constraints as the rules for a layer that is not a Flex item: a child of a frame without Flex, or a child with Absolute position inside a Flex frame. When the banner card from the Flex tutorial widens, its absolute close button stays behind, because Paper pinned it Left and Top. Under W and H sit a horizontal dropdown, a vertical dropdown and a constraints diagram; clicking the diagram's right handle sets Right, and the button holds the top-right corner. Next, a Transfer modal inside a freeform Modal scrim stays Left and Top even after being dragged to the middle. Align horizontal centers, then Align vertical centers, switch both constraints to Center. With Left + Right, holding `⌘` shows 500 on each side, and those margins hold while the modal widens. With Stretch, the margins and the width change together and W reads 36%.

- 0:35 Close button drifts on resize
- 1:00 Dropdowns and pin diagram
- 1:06 Horizontal options; Right chosen
- 1:48 Align buttons set Center/Center
- 2:36 Left + Right keeps 500 each side
- 3:00 Stretch: W becomes 36%

Build it in Paper

- 01 Build the screen from Flex frames and give each child a size mode by its job: Fill for text that should grow, Fixed for a portrait or a score column, Fit for a frame that should hug its content.
- 02 For a close button or badge that floats over content, tick Absolute position, then click the right handle of the constraints diagram (or choose Right in the horizontal dropdown) and keep Top.
- 03 For a modal inside a freeform frame, click Align horizontal centers (`⌘⇧S`) and Align vertical centers in the Layout section header so both constraints read Center; on a copy, try Left + Right and Stretch to compare.
- 04 Duplicate the frame with `⌘D`, drag the copy's edge to a narrow width, and hold `⌘` with an overlay selected to confirm its distances to the edges.
- 05 Swap in a long name, check the last row and any floating control at the narrow width, and turn on wrap for rows of variable-length items.

Common mistakes

- × **Placing a close button in the corner by hand after ticking Absolute position and assuming it will stay there.**

→ Pin Right and Top in the constraints diagram or dropdowns, then resize the parent to check.
- × **Dragging a modal to the middle of its frame and calling it centered.**

→ Use Align horizontal centers and Align vertical centers, which also set both constraints to Center.
- × **Choosing Stretch for a reading panel, so its line length grows with the screen.**

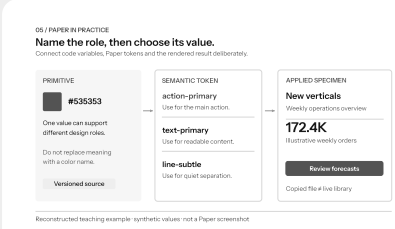
→ Use Center to keep the width, or Left + Right when fixed margins matter more than a fixed width.
- × **Expecting constraints to control a child that is still in Flex flow.**

→ Change the child's Fixed, Fit or Fill mode or the parent's Flex settings, or tick Absolute position if the child should float.

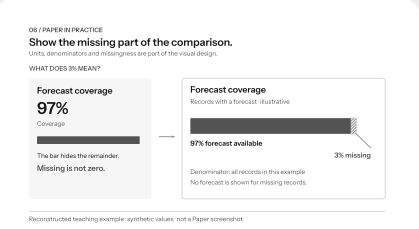
Visual language

Tokens give type, color and spacing their roles; charts make numbers easy to judge; vectors have to work at the size people actually see them.

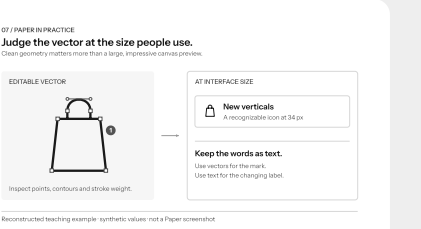
3 chapters, 3 live studies
10 source videos, 36 minutes, written down



05 Give the interface a consistent visual language



06 Make information easy to judge



07 Finish the asset at the size people will see

Give the interface a consistent visual language

Tokens make decisions reusable. Typography makes those decisions visible in the rhythm, emphasis and density of the page.

Name the role before choosing the value

A palette is a collection of colors. A design language explains what those colors mean. A red swatch may be an identity accent, a destructive action or an error signal. If one token carries all three roles, changing the brand can unintentionally change the language of feedback. Semantic names make that conflict visible before it spreads across a product.

Start with a small set of roles: page background, content surface, primary text, secondary text, boundary, action and selected state. These are proposed concepts, not a mandatory token vocabulary. Their value comes from consistency. A boundary token should describe a separator's purpose rather than the particular gray selected on the first day of the project.

Paper supports this split directly. Each token is a CSS custom property such as `--color-action`, and its value can be another token written as `var(--color-blue-500)`, so a semantic role can alias a palette color and a brand change happens in one place. Through the MCP, a token can also carry a description of up to 1,024 characters, a good home for its role. Paper's team showed why roles matter: after the accent and gold tokens became teal, a related token still carried the name gold, and the new teal sat close to the success green.

Study 05 · change it yourself

Tokens preserve decisions; typography gives those decisions a readable hierarchy.

The principle of chapter 5

Change semantic roles and inspect the result

Change a shared role and inspect its effect across the interface.

New verticals

Overview Forecasts

172.4K

Illustrative weekly orders

Review forecasts

Text / surface

17.58:1

White text / action

7.69:1

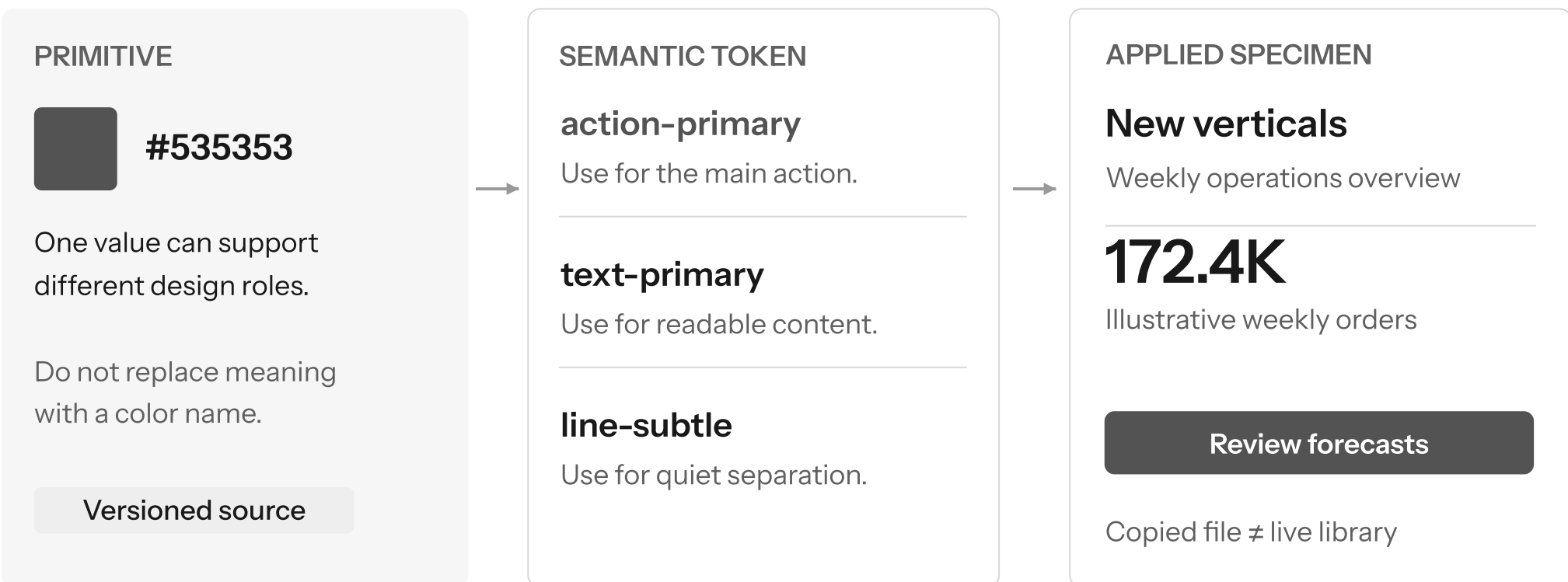
```
--text-primary: #191919;
--surface-panel: #ffffff;
--action-primary: #535353;
```

The same action role updates the selected tab and primary button. Both measured text pairs meet the 4.5:1 normal-text benchmark. This is a contrast calculation, not a full accessibility audit.

05 / PAPER IN PRACTICE

Name the role, then choose its value.

Connect code variables, Paper tokens and the rendered result deliberately.



Reconstructed teaching example · synthetic values · not a Paper screenshot

Figure 05.1 · Plate

Name the role, then choose its value.

A semantic name describes a design role. Map existing code variables into Paper and inspect the result. Copied tokens in another file are an independent snapshot, not an automatically synchronized library.

Typography creates hierarchy through relationships

A useful type system does more than list font sizes. It explains how a page title differs from a section heading, how a label differs from a paragraph, and how a number behaves next to its unit. Weight, line height, width and spacing all participate. Enlarging a title can establish emphasis, but reducing competing emphasis elsewhere can achieve the same clarity with less visual noise.

Compact interfaces need especially deliberate typography. Smaller text does not automatically create useful density; it can simply make weak hierarchy harder to read. Keep the primary values easy to find, give labels sufficient contrast and use restrained differences between secondary levels. Paper’s agent guide (October 2026) asks for higher contrast below 16 px and accepts text of 12 px or smaller only in dense productivity interfaces or as an all-caps effect. Variable fonts help with restraint: a weight of 560 can sit between Medium and Semibold instead of forcing a jump.

Paper’s CSS-variable tutorial imports an existing vocabulary, has an agent draw a style guide board and then checks the bindings on real layers. A specimen is valuable because it shows roles together. An action color that looks fine alone may be too similar to success; a muted label may disappear on a tinted surface. Design the relationships in a visible specimen before applying them throughout a flow.

Figure 05.2

Three layers of tokens

A binding should have an intended scope

Changing a token inside a Paper file updates every layer bound to it, whether you edit it in the Theme tab or an agent calls set_tokens. Copying a theme to another file creates an independent set, so cross-file consistency requires a deliberate synchronization process. This distinction affects how you organize experiments. A separate file can protect an alternative, while a shared token inside one file can intentionally keep several screens moving together.

Detaching is the usual way a binding loses its scope. A detached fill becomes a literal value that later token edits skip, which is exactly what an agent warned about after Paper’s team recolored two elements by hand. Before you change a token, select the frame and read Selection colors in the right panel: bound colors appear as token names, stray ones as hex values, each with a usage count. Typography deserves the same discipline. A font available in Paper must also be available to the implemented product; otherwise line breaks and component heights may change even when the exported sizes are identical.

Primitives hold values, semantic tokens hold roles, layers use roles.

Primitives

Raw values with neutral names

- grey-950 #191919
- grey-700 #535353
- grey-500 #8A8A8A
- grey-200 #DADADA
- grey-50 #F6F6F6
- white #FFFFFF

Semantic tokens

Roles that point at primitives

- text/default
- text/muted
- action/fill
- border/subtle
- surface/page
- surface/raised

Where they land

Layers that use the roles

- Body text
- Secondary label
- Primary button
- Divider
- Page background
- Card

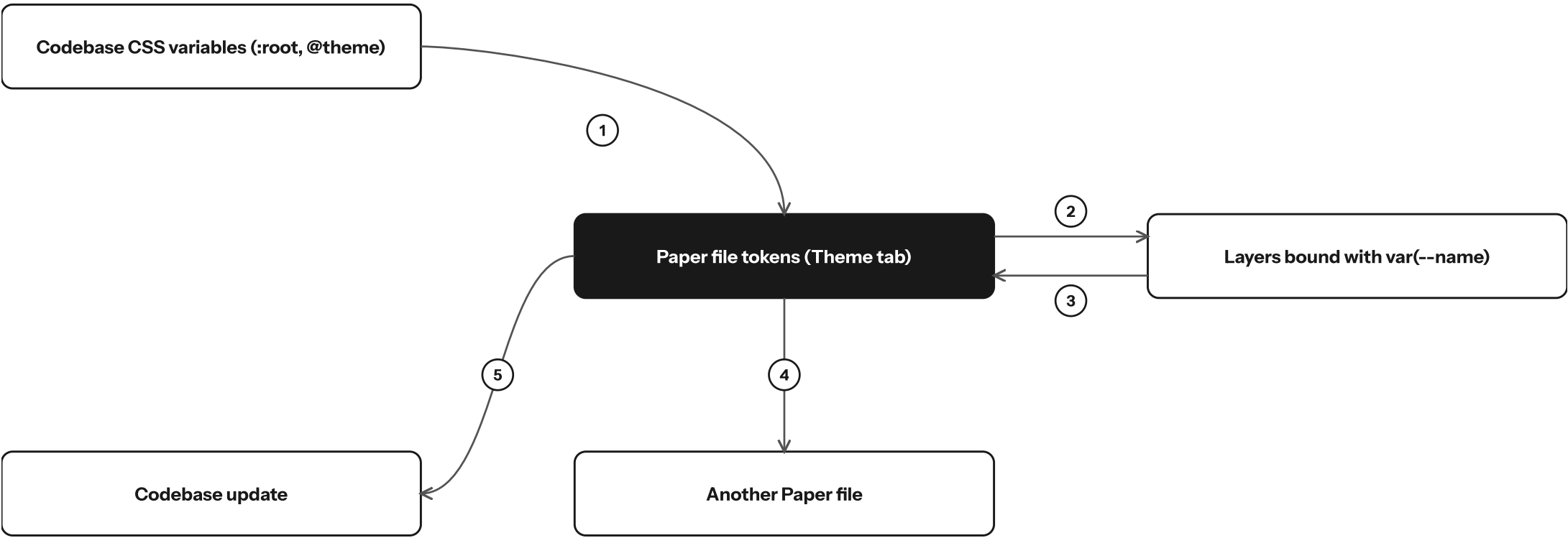
Change one primitive and every role that points at it follows. Swap the mapping for another theme and the layers never change.

Paper keeps tokens in the Theme tab of the right panel (September 2026); agents can read and write them over MCP.

Figure 05.3

The token round trip

Which steps keep links and which make copies.



- 1

Codebase CSS variables → Paper file tokens
create_tokens: palette before aliases; double-dash names rewritten; light-dark() keeps the light value
- 2

Paper file tokens → Layers bound with var(--name)
write_html or update_styles with var(--name); Theme tab edits and set_tokens propagate
- 3

Layers bound with var(--name) → Paper file tokens
Detach and explore, then set_tokens revalues and renames; re-link the detached layers
- 4

Paper file tokens → Another Paper file
Copy theme / Paste theme: independent copy
- 5

Paper file tokens → Codebase update
get_tokens with format css (:root) or tailwind (@theme)

Figure 05.4

Where tokens show up in Paper

Audit a frame field by field: which control takes a token and what a bound value looks like.

Property	Control	What a bound value looks like	Shown
Fill color	Four-circle Color tokens button beside the value	Chip with the token name (blue-3, accent) and a detach icon	▶ 1:17
Fill opacity	Percentage field of the fill	Opacity token such as muted 60%	▶ 2:17
Border color	Border section color field	Chip (border)	▶ 7:36
Font family	Font picker, Tokens tab	Chip (sans, mono, paper)	▶ 3:01
Font size	Size dropdown	Boxed number; list xs 12 to 7xl 82	▶ 4:10
Line height	Line-height dropdown	Detach, Normal, none 100% to relaxed 160%	▶ 4:34
Letter spacing	Letter-spacing dropdown	tighter -4% to wide 8%	▶ 4:48
Width and height	W / H dropdown	Spacing group (2 = 8 to 18 = 72), Container group (prose 560, content 1180, page 1440)	▶ 5:21
All tokens	Theme tab	Grouped list with count, Search tokens and +	▶ 2:26

How it works in Paper

As of September 2026, Paper tokens are named CSS custom properties stored per file, in eleven types that follow Tailwind v4: color, opacity, font family, font weight, font size, line height, letter spacing, spacing, radius, container and breakpoint. You create and edit them in the Theme tab, from a fill’s token menu, or through the MCP (create_tokens, set_tokens, get_tokens). Layers reference them as var(--name), so edits propagate within the file. Copy theme moves a snapshot to another file; modes, theme classes and libraries had not shipped.

Checked against the videos and documentation as of September 2026. Paper ships often; the release timeline in the reference shows what changed when.

01

Create tokens with an agent, from code or from a frame

1

Open the target file in Paper Desktop. In June 2026 the agent simply worked on the open file; since tabs arrived in August, name the file in the prompt, as in “In Paper, in the finance app file, ...”.

2

From the repository, prompt: “Create tokens in the open Paper file based on the CSS variables in this codebase.” To start from a design instead, point at a frame: “create color tokens based on the color swatch frame” (August 2026 demo).

3

Expect Get Guide, Get Basic Info and Get Tokens first, then Create Tokens calls batched by category. Approve them if your agent asks; Cursor showed Allowlist MCP Tool and Run.

4

Each create_tokens entry needs type, name and value, plus an optional description: {"type": "color", "name": "--color-action", "value": "var(--color-blue-500)"}. Create palette tokens before the aliases that point at them.

5

Read the summary. The June demo reported 96 tokens (65 colors, 16 typography, 10 radius, 5 spacing and layout) with their source files; light-dark() values kept only the light value, and Paper rewrote --text-xs--line-height as --text-xs-line-height.

6

Spot-check on the canvas: draw a rectangle, click the four-circle Color tokens button beside the Fill color and pick a token.

► Shown at 0:18

02

Create and edit tokens in the Theme tab

1

In the left panel, switch from Design to Theme. The tab shows the token count, a Search tokens field and +.

2

Click + and pick a type: Color, Opacity, Radius, Spacing, Container, Breakpoint, Font family, Font weight, Font size, Line height or Letter spacing. Opacity joined the list in September 2026.

3

The new Untitled token opens in an editor with a preview, Name and Value. Type a name; for a font family, Value opens a picker with Fonts, Favorites and Tokens tabs, and starred fonts appear under Favorites.

4

To edit a token, click it. Value can be another token, which makes an alias, or a color from the picker; compare Previous and New, then close the editor to save. Every bound layer in the file updates.

5

To capture a color already on the canvas (September 2026), select a layer with a raw fill, click the token button beside the fill value and press + at the top of the token menu, then name the new Untitled token in the Theme tab.

%

F

► Shown at 2:26

03

Apply, swap and detach tokens on a layer

1

Select the layer and click the four-circle button beside the Fill color (tooltip Color tokens). Pick a token; the field becomes a chip such as accent, with a broken-link detach icon beside it.

2

Reopen the menu to swap tokens, or click the detach icon to get a plain hex value you can edit in the color picker.

3

For text, open the size or line-height dropdown: tokens are listed with values (xs 12 to 7xl 82; none 100%, tight 105%, snug 126%, normal 150%, relaxed 160%). On a bound field Detach is the first item, and hovering the field shows the token name, such as text-lg.

4

For dimensions, the W and H dropdowns list Spacing and Container tokens below Fixed, Fit, Fill and Relative. Gap, padding, radius, letter spacing and border color take tokens too.

5

To dim a color without another color token, click the fill’s percentage field and choose an opacity token; the September demo paired ink with muted at 60%.

► Shown at 2:13

- 04 **Update and rename tokens from hand edits (set_tokens)**
- 1

Before editing, ask the agent which layers use a token: find_nodes accepts a token name as its style value. In an August 2026 demo a search for #000000 also matched 0-valued properties, and the agent read the icon subtrees separately to catch SVG stroke colors.
- 2

Detach the elements you want to explore and try values in the color picker (sRGB or Display P3 tabs, with L C H, H S L and R G B rows).
- 3

Prompt with the frame and the tokens: look at the first onboarding frame, grab the new colors for accent and gold label, update those tokens, and rename them.
- 4

The agent read #89F0C5 and #319F77 and called set_tokens once: --color-accent became --color-teal and --color-gold became --color-teal-deep, and var() references followed the rename. Entry shape: {"name": "--color-accent", "newName": "--color-teal", "value": "#89F0C5"}; "delete": true removes a token.
- 5

Re-link the elements you detached: they kept literal values and will not follow the next edit.
- 6

Review neighbors: --color-gold-deep kept a stale name, and the new teal sat close to the success green.
- 7

To carry the result back to code, ask for get_tokens with format tailwind (an @theme block) or css (a :root block), filtered with namePattern such as --color-* if needed.

► Shown at 7:31

- 05 **Copy a file’s tokens to another file**
- 1

In the source file, click the Paper logo at the top left, hover Theme and choose Copy theme.
- 2

Open or create the destination file. In a file without tokens, Copy theme and Clear theme are grayed out and the Fill row has no token button.
- 3

Choose Paper logo › Theme › Paste theme. A Theme pasted toast confirms, and the token button now lists the same tokens.
- 4

Treat the copy as independent: copy again after changing the source, or ask an agent to compare get_tokens output across the two files and create or update what differs. Clear theme, and pasting into a file that already has tokens, were not demonstrated.

► Shown at 0:10

- 06 **Set an exact variable font weight**
- 1

Select a text layer set in a font with variable axes (DM Sans in the demo).
- 2

Click the more button (sliders icon) at the top right of the Text section, then the Variable font tab; the other tabs are Formatting and Features.
- 3

Drag Weight or type a value: 400 is Regular, 100 is thin, 800 is extra bold, and values between the stops, such as 364 or 560, work too. The weight dropdown’s label follows the value.
- 4

Leave Automatic optical sizing checked unless you need a fixed Optical size; the Optical size field stays grayed while it is on.
- 5

Record any custom weight, for example as a font weight token, so the implemented product uses the same number.

► Shown at 0:12

Controls you will use

Control	Where	What it does
Theme tab	Left panel, beside Design	Lists the file’s tokens grouped by type, with a count, a Search tokens field and + to create one; clicking a token opens an editor with Name and Value. A Value can point at another token, which makes an alias.
Color tokens button (four circles)	Right panel › Fill, right of the color value; also on Page and other color fields	Opens the file’s color tokens. On a raw fill its menu header has + to save that color as a new token (September 2026). The same header switches between list, small grid and large grid views.
Token chip	Inside a bound field	Shows the token instead of a value: a name for colors (accent, ink) and a small box around the number for type tokens. Hovering shows the token name, such as text-lg.
Detach	Broken-link icon right of a bound color; first item of a bound size or line-height dropdown	Replaces the token with its current literal value, which later token edits no longer change.
Size, line height and letter spacing dropdowns	Right panel › Text	List type tokens with their values (xs 12 to 7xl 82; none 100% to relaxed 160%; tighter -4% to wide 8%); line height also offers Normal. Letter-spacing tokens defined in em display as percentages: -0.04em reads -4%.

Control	Where	What it does
W and H dropdowns	Right panel › Layout	Offer Fixed, Fit, Fill and Relative, then the file's Spacing and Container tokens (prose 560, content 1180 and page 1440 in one demo).
Opacity slot	Fill row, percentage field	Accepts an opacity token, so a color token can render at 60% (ink with muted) without a second color token.
Selection colors	Right panel, below Filters	Lists every color in the selection as a token name or hex value, with opacity and a usage count; Show all N colors expands the list. Hex values here are the bindings a token edit will miss.
Paper logo menu › Theme	Top left of a file	Holds Copy theme, Paste theme and Clear theme for moving a file's token set; a Theme pasted toast confirms a paste.
Variable font tab	Right panel › Text › more button (sliders icon)	A Weight slider and number field for any value the font supports, an Optical size slider and field, and an Automatic optical sizing checkbox (checked in the demo).

Shortcuts

	Open the eyedropper, for example to pick up a magic swatch color ▶ 3:05		Paste an image copied in the browser onto the canvas ▶ 0:14
	On an image: context menu with Vectorize image, Extract colors and Remove background ▶ 0:16		Search tokens in the Theme tab (hint shown in the field) ▶ 2:33
	Copy a link to the selected frame, to paste into an agent prompt ▶ 2:01		

Limits and what is coming

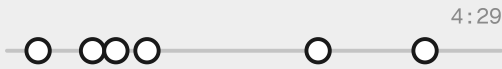
- Modes (light and dark, possibly contrast) and theme classes (bundled styles such as an H1, or a card's shadows, borders and radius) were announced but not shipped as of September 2026. light-dark() values import as their light value, so a dark variant needs dark tokens applied to a duplicated frame.
- Tokens belong to one file. Copy theme and Paste theme make an independent copy; linked libraries were still in progress as of September 2026.
- On token-bound text written by agents, the font weight field has read Unavailable (a June 2026 viewer report and a September 2026 demo). Check weights after an agent pass.
- In a June 2026 demo, line-height tokens came out as percentages (100% to 160%) although the brief asked for px. Paper's agent guide (October 2026) prefers px for --leading-* tokens and em for --tracking-* tokens.

What the tutorials show

Each official video, written down step by step. The dots mark the moments; they open the video there if you want to see it.

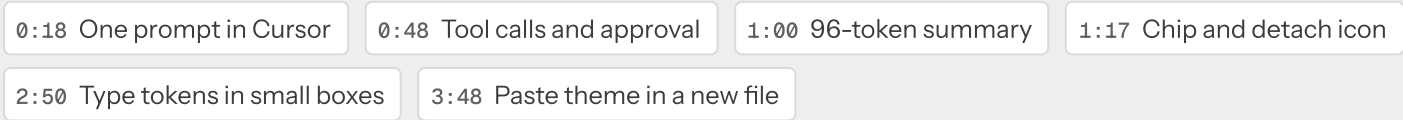
From CSS variables in your codebase to Paper design tokens with MCP

Video · 4 min 29 s · 24 June 2026



CSS variables to tokens in one prompt, then a style guide and a copied theme

With Paper's codebase in Cursor and a file open in Paper Desktop, the presenter asks the agent to create tokens in the open Paper file from the codebase's CSS variables. It reads colors.css and typography.css, runs Get Guide, Get Basic Info and Get Tokens, and waits for approval on Create Tokens. Its summary reports 96 tokens: 65 colors, 16 typography, 10 radius and 5 spacing and layout. light-dark() values kept their light value, and aliases such as --color-popup point at base tokens through var(). On a rectangle, the four-circle Color tokens button beside the Fill lists them; choosing one makes the field a chip with a detach icon. A second prompt produces a style guide board through repeated Write Html calls. On its text, size 13 and line height 16 sit in small boxes, hovering shows text-lg, and the dropdown starts with Detach. Copy theme and Paste theme move every token into a new file. In June 2026 the MCP was the only way to create tokens.



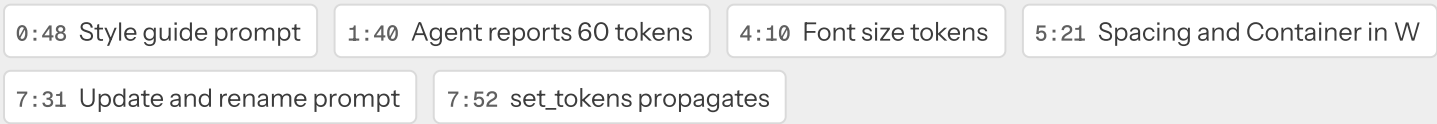
Explore Paper's MCP-generated tokens with the Paper team

Video · 10 min 16 s · 22 June 2026



Tokens from an existing design, then one update that recolors everything

Two members of Paper’s team ask Claude (Opus 4.8, Claude desktop app with the Paper plugin) for a style guide based on a landing page frame: type sizes, line heights, letter spacing, colors, spacing, radius, and Tailwind-default breakpoints. Unasked, the agent also creates 60 tokens in Tailwind v4 namespaces and binds the board through `var()`. On the next turn it runs `get` tokens instead of duplicating them, and flags that line heights came out as percentages, not the px its brief named. They tour the controls: the color token list, detach to hex, the size dropdown (xs 12 to 7xl 82), line height (none 100% to relaxed 160%), letter spacing (tighter -4% to wide 8%) and the Spacing and Container groups in the W dropdown. After an onboarding flow is built on them, they detach two elements, recolor them teal and ask the agent to update and rename those tokens. It calls `set_tokens`, and every bound element turns teal, style guide included; the two detached elements stay literal.



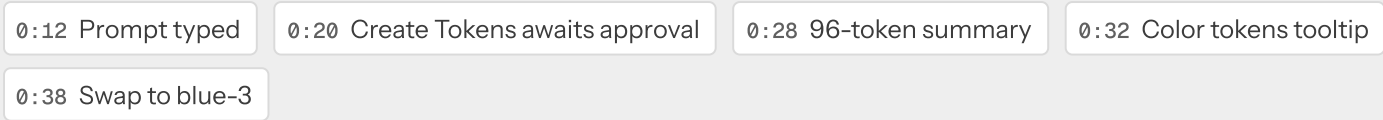
Create Design Tokens From Your Code Base

Short · 52 s · 26 June 2026



One prompt from a codebase to 96 tokens

A vertical cut of the CSS-variables tutorial. In Cursor, with Paper’s codebase open, the prompt asks the agent to create tokens in the open Paper file from the codebase’s CSS variables. The log shows Get Guide, Get Basic Info and Get Tokens, then a Create Tokens call waiting for approval, and a summary of 96 tokens across colors, typography, radius, and spacing and layout. In Paper, hovering the four-circle button beside the Fill color shows the tooltip Color tokens; picking blue-1, then blue-3, swaps the chip in the field.



Copy and paste tokens between Paper files

Short · 25 s · 26 June 2026 0:25



Copy a theme into another file

While shared libraries are in progress, tokens move between files by copy and paste. In the source file, click the Paper logo at the top left, hover Theme and choose Copy theme. Create a new file from the dashboard, open the same Paper logo menu, hover Theme and choose Paste theme; every token is available at once. In the empty file, Copy theme and Clear theme are grayed out. The narration adds that a theme can also be copied into a codebase, without showing the format.

- 0:02 Theme submenu
- 0:06 Paper logo menu
- 0:10 Copy theme
- 0:12 New file
- 0:14 Paste theme
- 0:16 Tokens in the new file

Create magic swatches in Paper

Short · 34 s · 21 July 2026 0:34



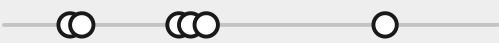
Magic swatches: a palette from a photo

The presenter copies a photo in the browser (orange and red stairs under a blue sky, from Unsplash) and pastes it into Paper with ⌘V. Right-clicking the image opens a long context menu; near the bottom, between Vectorize image and Remove background, is Extract colors. Choosing it places a frame named Magic Swatches (740 × 100) under the image, holding six swatch layers named after their values, such as Color #9A0D2C and Color #E7790A. The colors can then be used anywhere in the design. They are plain layers, not tokens, so save the ones that earn a role.

- 0:12 Copy the image in the browser
- 0:14 Paste with ⌘V
- 0:17 Extract colors
- 0:19 Swatches appear
- 0:22 Layers named by hex

If your font supports variable weights, your design tool should too!

Short · 1 min 27 s · 9 June 2026 1:27



Exact weights with the Variable font tab

Answering a request for weights beyond Regular, Medium and Semi-bold, the presenter selects a DM Sans text layer, skips the weight dropdown and clicks the more button (sliders icon) at the top right of the Text section. The popover has Formatting, Features and Variable font tabs. Variable font shows a weight preview, a Weight slider with dot stops and a number field, an Optical size slider and field (16, grayed) and a checked Automatic optical sizing box. Regular reads 400; dragging gives 100 (thin) and 800 (extra bold), and values between the stops work, such as 364 or 560. She lists three benefits: exact control, smooth weight animation (800 to 400 passes through every value) and one font file instead of one per weight, which she says reduces file size.

- 0:12 More button in the Text section
- 0:14 Variable font tab
- 0:31 Weight 100
- 0:33 Weight 800
- 0:36 Custom weight 364
- 1:07 One file, smaller size

Build it in Paper

- 01

Open the target file in Paper Desktop and ask your agent: “Create tokens in the open Paper file based on the CSS variables in this codebase.” Accept the import when its summary lists counts per category, source files and how light-dark() values and aliases were handled. Without a codebase, create the roles by hand in the Theme tab with +.
- 02

Build a specimen frame with page background, surface, primary and secondary text, boundary, action and selected state. Apply colors with the four-circle Color tokens button beside each Fill, sizes and line heights from the Text section’s dropdowns, and spacing or radius from their fields.
- 03

Select a specimen text layer and confirm that size and line height sit in small boxes, which marks a bound token; hover a box to read its name, such as text-lg, and check Selection colors for stray hex values.
- 04

Change one token in the Theme tab: click it, edit Value, compare Previous and New in the color picker, then close the editor to save. Inspect the specimen and one representative screen for the expected changes and nothing else.
- 05

If another file needs the set, choose Paper logo › Theme › Copy theme, then Paste theme in the destination, and record which file is the source, because the copy does not stay linked.

Common mistakes

- ×

Detaching a token to try a color, then leaving the literal value in place.

→ Once you decide, update the token in the Theme tab or with set_tokens and re-link the detached layers; Selection colors shows any hex values left behind.
- ×

Picking dark-mode colors by eye, or recoloring the light frame itself.

→ Duplicate the frame and ask the agent to apply the file’s dark tokens or the codebase’s dark-mode variables to the copy, as two 2026 demos did, leaving the light frame unchanged.

Check before you move on

- ☐

Does every color, size, radius and spacing value on the frame come from a token, or does Selection colors still list stray hex values?
- ☐

Does each token name describe a role (surface, action, danger) rather than a hue, and would a rename leave a related token with a stale name?
- ☐

If you edit the action token now, which layers should change, and is anything detached that will not follow?
- ☐

Do the type tokens give clear steps between title, heading, body and label at the sizes you actually use?
- ☐

If other files use this system, which file is the source, and when was its theme last copied?

Make information easy to judge

A useful chart or dashboard helps the reader answer a specific question without hiding the source, unit or uncertainty behind its visual polish.

Start from the question and the content

A number becomes useful when the reader knows what it measures, which period it covers and what to compare it with. “Higher” has no clear meaning without a reference. Before choosing a chart, write the decision in an ordinary sentence: compare a scenario with its baseline, find the item leading a ranking, or identify an item below its target. The sentence reveals the relationships the display must preserve.

Realistic content changes design choices. Long category names need room. Different units need separation. Missing observations need an honest state. Paper’s Notion demonstration is relevant because an agent reads a named page through an MCP connection and lays it out as editable layers before any refinement. That is a transfer of material for a design, not evidence that the canvas has become a continuously updating database view: when the source changes, you ask again.

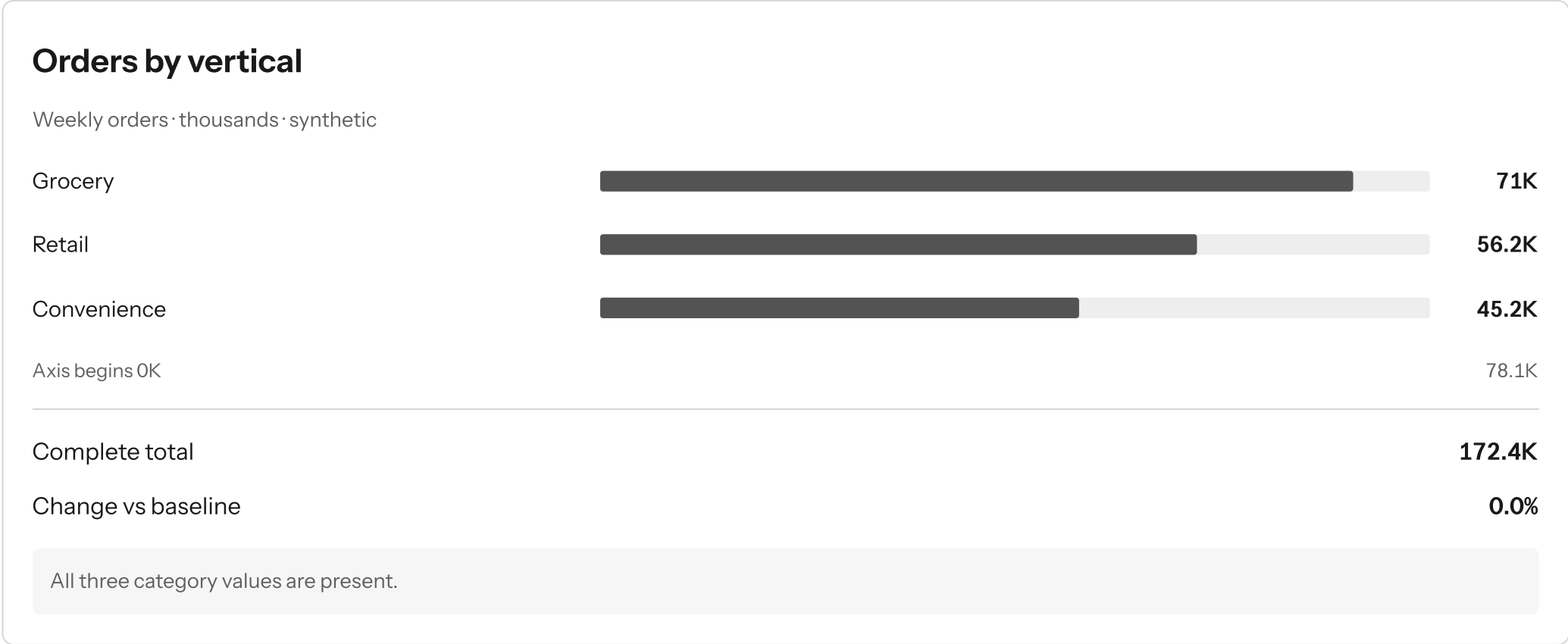
The chart should reveal a relationship, while its labels preserve the exact meaning.

The principle of chapter 6

Study 06 · change it yourself

Make the scale and missing values explicit

Change the scale or remove a value. Inspect what remains comparable.

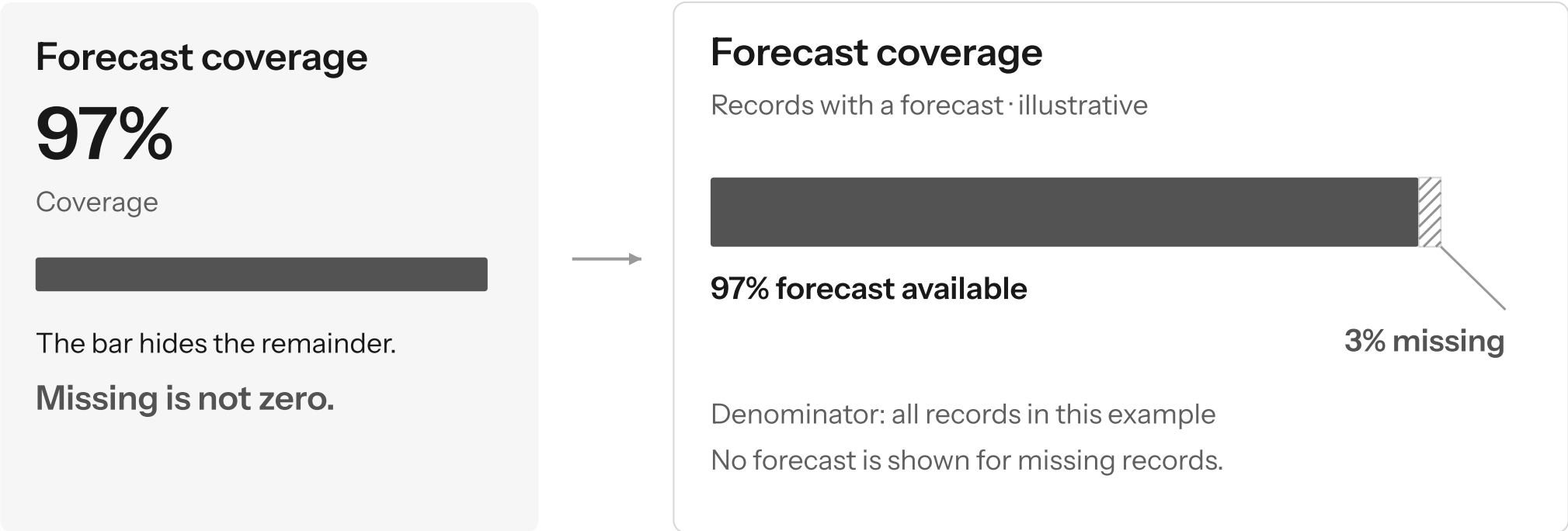


The bar lengths share a zero baseline. The complete total changes by 0.0% against 172.4K. Labels preserve the exact values even when the scale is poor.

Show the missing part of the comparison.

Units, denominators and missingness are part of the visual design.

WHAT DOES 3% MEAN?



Reconstructed teaching example · synthetic values · not a Paper screenshot

Figure 06.1 · Plate

Show the missing part of the comparison.

This is original chart-design guidance applied to illustrative records, not a Paper analytics feature. A complete scale and a distinct missing segment reveal the denominator and prevent missing values from appearing to be zero.

Choose an encoding that supports comparison

For values that must be compared precisely, position on a shared scale is a strong starting point. An aligned bar or dot can reveal an ordering faster than differently sized cards containing large numbers. The number remains useful as a label; the chart supplies the relationship. In Paper a bar is an ordinary frame or SVG shape, so the scale is something you set deliberately: one ratio from value to width, applied to every row.

Use a consistent scale when neighboring panels invite direct comparison. If one panel silently changes its range, identical bar lengths can represent different values. A target marker should look different from an observed amount. An uncertainty interval should look different from a decorative background band. The reader should be able to distinguish measurement, reference and qualification without memorizing an elaborate legend.

Color should carry a limited, stable responsibility. It can distinguish a scenario from a baseline or mark the selected series, but adding a unique color to every label often creates more decoding work. A mostly neutral field makes selective emphasis easier to read. Paper’s agent guide (October 2026) makes the same argument: one intense color moment is stronger than five, and secondary colors are meant for cases such as categories, complex data visualization and success, warning or error states. Store series colors as tokens so every chart keeps the same meanings.

Keep the caveat next to the claim

A synthetic example can teach a chart without pretending to be a result. Label it where the values are read, not only in a distant footer. Likewise, a conditional estimate needs its condition near the conclusion. The visual design should make the limitation part of the reading experience rather than a small-print correction to an overconfident headline.

Review the labels with the chart hidden, then the chart with its explanatory paragraph hidden; in Paper, Show / hide (⇧⌘H) on each group makes this a quick test. Each should contribute something useful. When an agent builds the chart, ask for the review Paper already expects of agents: a screenshot after each section with a verdict on spacing, typography, contrast and alignment. Add two questions of your own: is the scale shared, and does every caveat sit next to its number?

Figure 06.2

Choose the chart by the question

Start from what the reader needs to judge, then pick the encoding.

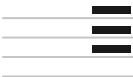
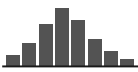
The reader's question		Use	Avoid
Which is bigger, and by how much?		Bars on a shared zero baseline, sorted unless the order means something	Truncated axes; area or angle for exact comparisons
How did it change over time?		A line, or columns, on an evenly spaced time axis	Unordered categories on the x-axis; uneven intervals
What share of the whole?		One stacked bar with few parts, labelled directly	Pies with many slices; shares that do not add up
What is the exact value?		A table with right-aligned, tabular numerals and units in the header	Charts without labels when people need to look values up
Do two measures move together?		A scatter plot, one dot per item, both axes labelled	Two lines on two y-axes
How are values spread?		A histogram or a strip of dots	An average on its own

Figure 06.3

A chart built from Flex rows

Alignment comes from fixed lanes; one scale sets every bar.

One row of a comparison chart

One factor, for example 4 px per unit, shared by every row



Label	W Fixed 160	<code>width: 160px; flex-shrink: 0</code>
Track	W Fill	<code>flex: 1</code>
Bar	W Fixed = value × scale	<code>width: value × scale px</code>
Target mark	Absolute position, x = target × scale	<code>position: absolute; left: target × scale px</code>
Value	W Fixed 64, right-aligned	<code>width: 64px; flex-shrink: 0; text-align: right</code>

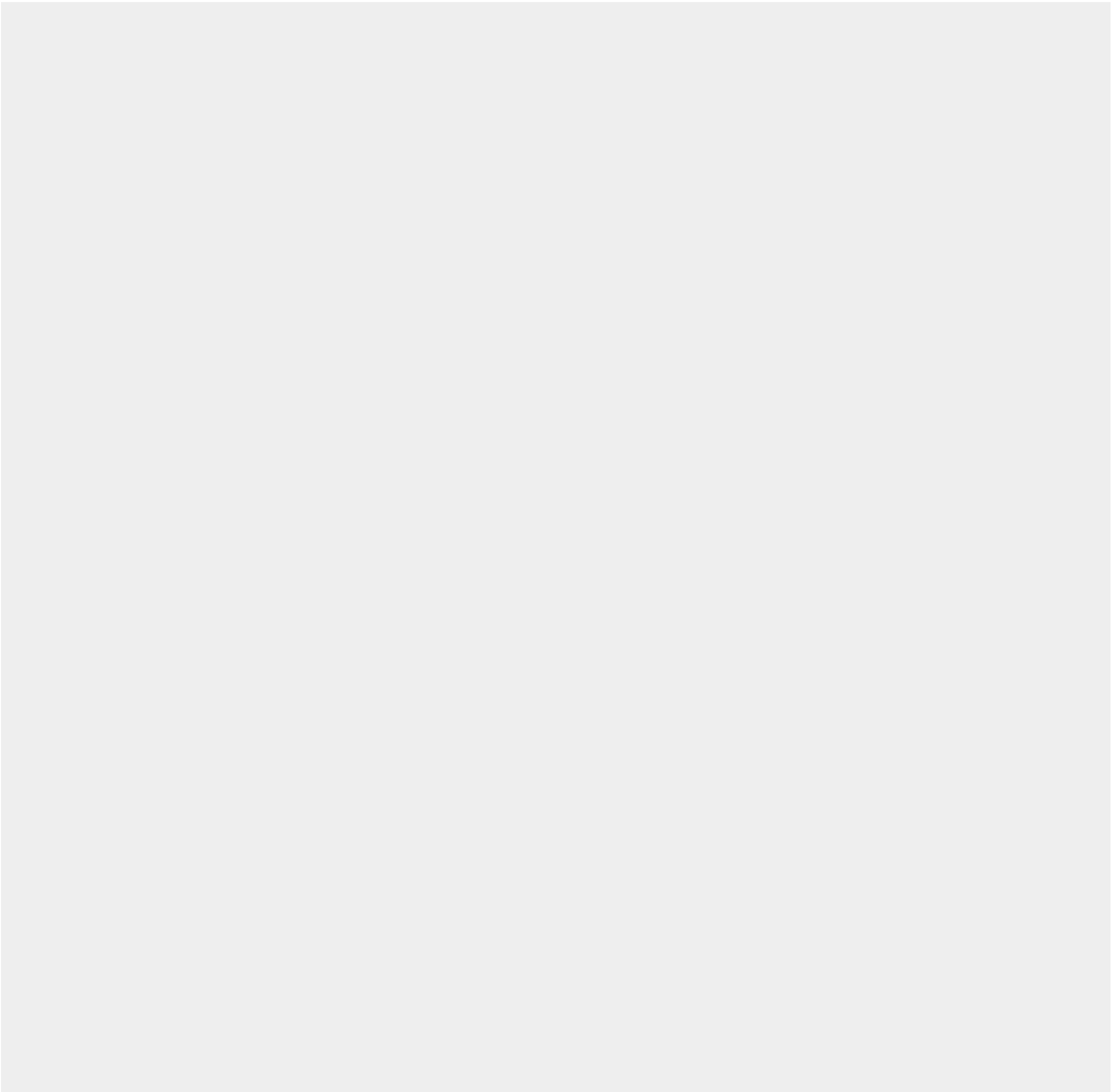
Missing value: Row kept, value slot reads No data, no bar drawn

Figure 06.4

From a Notion page to a designed artboard

The agent's sequence, stage by stage, with the moment each one is shown.

▶ 0:41	Add Notion's MCP server with the command from Notion's docs
▶ 0:55	/mcp: Notion connected, authenticated, 12 tools
▶ 1:12	Prompt names the page and the goal
▶ 1:26	notion-search and get_basic_info in parallel; permission prompt
▶ 1:44	notion-fetch, then get_font_family_info
▶ 2:01	Design brief, then create_artboard 1440 × 2600
▶ 2:30	Checkpoint screenshot with verdicts on spacing, typography, contrast, alignment, repetition
▶ 3:10	Named sections: Header, Core Values Section, Tactics Section, Reading Section, Footer



How it works in Paper

As of September 2026, no Paper video or doc shows a chart component or live data binding: a chart or table is ordinary frames, text and SVG, which map to HTML with flexbox. Real content arrives through an agent that reads another source (an MCP server such as Notion, a repository’s API, a web search) and writes layers with write_html as a one-time copy, or through Paper Snapshot captures. Paper’s agent rules exclude HTML tables and CSS grid, require fixed-width lanes, and allow tokens in SVG fills and strokes.

Checked against the videos and documentation as of September 2026. Paper ships often; the release timeline in the reference shows what changed when.

01

Pull a named page from another MCP source and design it

1

Add the source’s MCP server to your agent. The demo pasted the command from Notion’s docs into the terminal, `claude mcp add --transport http notion https://mcp.notion.com/mcp`, then launched claude; the Paper MCP was already installed.

2

Type `/mcp` to check both servers. Notion showed Status connected, Auth authenticated and 12 tools; a first connection goes through Notion’s OAuth flow in the browser.

3

Name the content and the goal in one prompt: “grab our core Values page from Notion and create a nice design in Paper to highlight the content.” Approve tool permissions when asked.

4

Expect parallel reads (`notion-search` next to Paper’s `get_basic_info`), then `notion-fetch`, `get_font_family_info`, a written design brief, `create_artboard` and a series of `write_html` calls with named layers.

5

Refine by hand. The layers are a copy of the page at that moment, so rerun the prompt when the source changes.

►

Shown at 0:41

02

Fill a layout with real records from a repository or a live page

1

Run the agent inside the repository that owns the data, with the Paper file open.

2

Name the frame and the source together: “In Paper, please build out this ‘for you’ section ... use the data from the API in this repo.” The demo agent duplicated an existing card and filled the copies with real playlist names.

3

Watch the canvas: the agent’s avatar joins yours, and dotted markers show which frames it is reading.

4

To see the result in context, capture the live page with Paper Snapshot (`⌘Return` grabs the whole page), paste it, and paste the new section inside it; untick Absolute position so the section joins the page’s flex flow.

5

Ask for alternatives on a duplicate (“duplicate this frame and show me ...”) so the original stays intact.

⌘

Return

►

Shown at 2:16

03

Build a table or bar chart from flex rows

1

Select a label and a bar rectangle and press `⇧A` (Wrap in flex, in the Layout section) to make one row; set the gap in the Flex section.

2

Give each column a fixed-width slot: set the label’s W to Fixed so every row shares the same lane. Paper’s agent guide asks agents for width plus `flexShrink: 0`, even where a row leaves the slot empty, and warns that gap alone does not align columns.

3

Set the track that holds the bar to Fill (`⌘+double-click` its edge) and the row’s height to Fit (double-click its top or bottom edge).

4

Duplicate the row for each item, then select the rows and press `⇧A` again to wrap them in a flex frame; set its direction to vertical in the Flex section if needed.

5

With an agent, ask for flex rows with inline styles and inline SVG marks colored with `var()` tokens; the `write_html` rules exclude `<table>`, `display: grid`, `margin` and `display: inline`.

⇧

A

⌘

Double-click

Double-click

►

Shown at 1:36

- 04 **Check scales, lanes and overflow**
- 1

Select a bar and read W in the Layout section; every bar should equal its value times the same scale factor.
- 2

Hold ⌘ with a layer selected to see its distances to the surrounding edges, and confirm that every bar starts at the same zero point.
- 3

Turn off Clip content (⌘C) on the parent to reveal labels or legend items that overflow; Wrap in the flex settings lets a legend row break onto new lines.
- 4

After three or more rows, take a screenshot and trace vertical lines through labels, values and icons.
- 5

For exact numbers, ask the agent for get_computed_styles rather than reading a screenshot; Paper’s guide keeps screenshots for verification.



► Shown at 12:00

- 05 **Brief an agent from reference charts and real constraints**
- 1

Paste reference images next to the work (⌘V) and prompt: “look at the reference images & list best practices.” Edit the list before relying on it.
- 2

Name the question the chart answers, the real values or their source, the units, and any caveat that must sit next to a number.
- 3

Ask for the file’s tokens through var() for series colors and for layers named by meaning, like Header, Tactics Section and Footer in the Notion demo.
- 4

Ask for a checkpoint screenshot after each section with a one–line verdict on spacing, typography, contrast and alignment, as the Notion demo’s agent produced.
- 5

Copy a link to the target frame (⌘L) and paste it into follow-up prompts so the agent edits the right frame.



► Shown at 0:24

Controls you will use

Control	Where	What it does
Wrap in flex	Right panel › Layout (button), or ⌘A	Puts the selection in a frame with flex layout, CSS display: flex; rows of a table or chart, and the column that stacks them, are flex frames.
Gap	Right panel › Flex	Space between children along the flex direction; maps to CSS gap. Arrow keys change it by 1 and ⌘ + arrow by the nudge setting; double-clicking the nine–point alignment grid sets space between.
W and H sizing	Right panel › Layout dropdowns	Fixed keeps a set size (label and value lanes), Fit hugs the content, Fill takes the parent’s free space (the bar track); Relative is also listed. The canvas badge reads like Fill 1328 × Fit 318.
Clip content	Right panel, checkbox below the Flex settings (⌘C)	Hides children that overflow the frame. Turn it off to see labels or legend items that do not fit.
Wrap	Flex section › more properties (sliders icon); the September 2026 panel was reworked	Lets children break onto new lines instead of shrinking or overflowing, useful for legends and tag rows (August 2026). Order matters: a long item wraps at its own position.
Absolute position	Right panel, checkbox in the Layout or Flex area	Takes a layer out of the flex flow so it can overlay others, for a target line or annotation. Pasted frames can arrive with it ticked; untick it to rejoin the flow.
Layer names	Layers panel; agents set them with the layer-name attribute	Semantic names (Header, Tactics Section, Footer) make review, find_nodes searches and code export easier than a stack of Frame layers.
Selection colors	Right panel, below Filters	Lists every color in the selection with token names or hex values and usage counts; Show all N colors expands it. A quick audit of series colors.
Other styles	Right panel, near the bottom	Shows raw CSS on a layer that has no panel control, such as max-width or white-space-collapse, each with a checkbox. Look here when agent HTML or a capture carries unexpected CSS.

Shortcuts

⌘A	Wrap the selection in a flex frame, or add flex to a frame ► 1:36	Double-click	On a frame’s top or bottom edge: set its height to Fit ► 5:24
⌘ Double-click	On an edge: set that dimension to Fill the parent ► 9:37		

Build it in Paper

- 01

Bring real content in first: connect the source through its MCP server, a repository’s API or a capture of the live page, and ask the agent to place the actual labels and values on a new artboard. Mark any invented values as such inside the frame.
- 02

Build one comparison row: select a label and a bar, press ⌘A (Wrap in flex), give the label a Fixed width so every row shares the same lane, and set the bar’s track to Fill with ⌘ + double-click on its edge.
- 03

Duplicate the row (⌘D) for each item and set each bar’s W from one shared scale. Put units in the labels, and draw the target as a distinct mark, such as a thin line with Absolute position checked.
- 04

Keep an unavailable observation in its lane with an explicit “No data” label. Hold ⌘ to check distances, turn off Clip content (⌘C) to reveal overflowing labels, and take a screenshot after three rows.
- 05

Hide the chart group with Show / hide (⌘⌘H) and read the labels alone, then show it again and hide the explanatory text.

Common mistakes

- ×

Designing the chart with placeholder labels and round numbers, then finding that real names wrap and real values crowd the scale.

→ Bring real content in first through an MCP source, a repository’s API or a capture, and label any invented values inside the frame.
- ×

Asking an agent for an HTML table or a CSS grid.

→ Ask for flex rows with inline styles and inline SVG marks; Paper’s write_html rules exclude <table> and display: grid.

Check before you move on

- ☐

Can you state the question the chart answers in one sentence, and does the layout answer it first?
- ☐

Do all comparable marks share one scale and one zero point, and does the target look different from an observed value?
- ☐

Are the real labels readable at their longest with Clip content off, and does a missing value keep its lane?
- ☐

Is every unit, source and caveat next to the number it qualifies rather than in a footer?
- ☐

Does a vertical line traced through labels, values and icons stay straight across every row?

- ×

Aligning table columns with gap alone, so values drift when a label or icon changes width.

→ Give every column a fixed-width slot (W Fixed; width plus flexShrink: 0 in agent HTML), even where a row has no content, and screenshot after three rows.
- ×

Treating the canvas as a live view of the source.

→ Write the source and the retrieval date on the frame, and rerun the prompt when the data changes; as of September 2026 Paper does not refresh it.

Finish the asset at the size people will see

An editable vector is a starting material. Its quality depends on silhouette, stroke, bounds and the way it participates in the composition.

An illustration has a job

A useful asset contributes something the surrounding layout cannot say as efficiently. An icon can identify a familiar action; a diagram can reveal a relationship; an illustration can establish a tone. Decide that job before choosing how to make the asset. A decorative asset placed beside every paragraph can increase visual activity without increasing understanding.

For an interface icon, recognition at the final small size matters more than impressive detail when enlarged. For a cover illustration, composition and negative space may matter more than exact geometry. These different jobs imply different finishing decisions. The same intricate drawing that works on a chapter opener may become an indistinct mark beside a compact label.

An asset is finished when it serves the composition at its actual display size.

The principle of chapter 7

Study 07 · change it yourself

Shape the curve, then inspect the icon

Edit the handles and compare the curve at two display sizes.



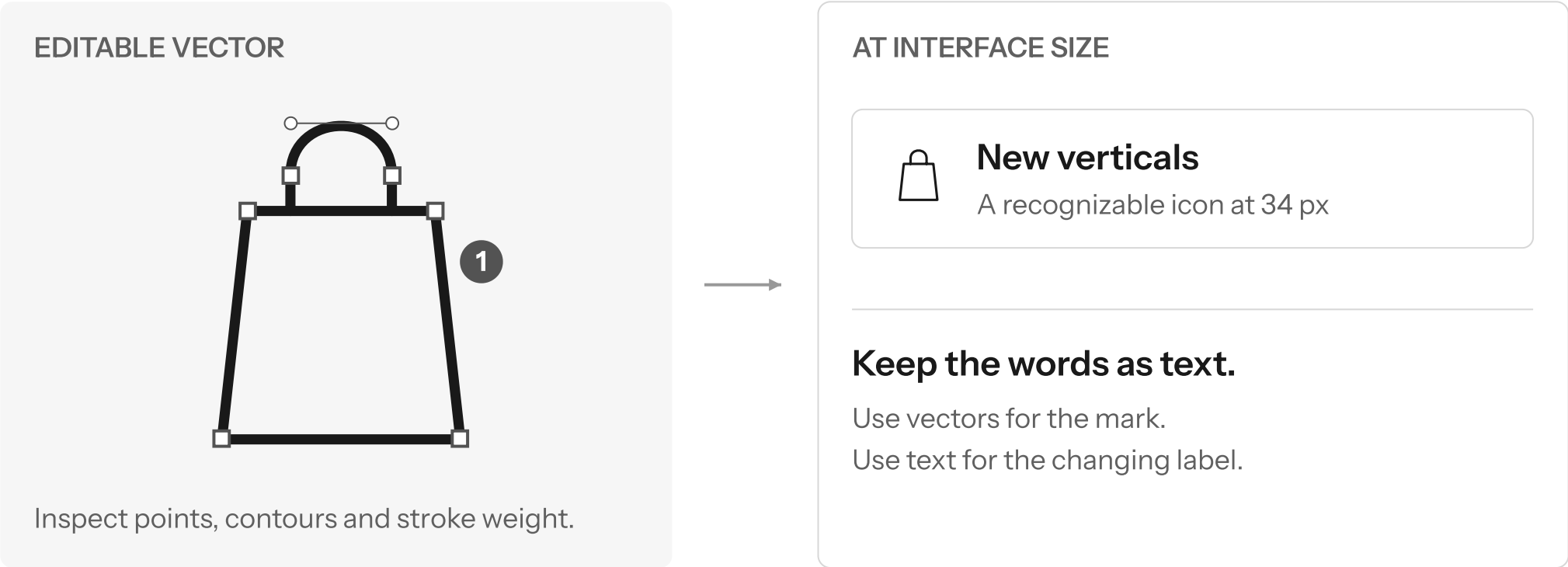
 New verticals
24 px in a navigation row

 40 px

At 24 px, this 4-unit stroke is approximately 0.30 px thick. A thin stroke may lose definition at this size.

Judge the vector at the size people use.

Clean geometry matters more than a large, impressive canvas preview.



Reconstructed teaching example · synthetic values · not a Paper screenshot

Figure 07.1 · Plate

Judge the vector at the size people use.

The original shopping-bag icon demonstrates editable contours and a small-size check. Keep the label as text and inspect the exported vector; generated geometry still needs review.

Treat generated geometry as editable material

Paper’s Quiver tutorial demonstrates a complete idea: describe the subject, style, colors and restraint, generate an SVG, then refine it by hand. The presenter hides three unwanted polygons, sets two mismatched stroke widths (0.28 and 0.99) to 1, deletes a stray path, trims sections of another and skips about five more minutes of editing. The useful distinction is between requesting a plausible starting image and accepting a finished asset. Generation solves the first problem; design judgment still solves the second.

An SVG contains shapes that can be inspected independently. In Paper, an SVG layer is a container that behaves like a frame and clips what it holds; the Path layers inside it are the editable geometry, each with its own fill and stroke. A click selects the container, a double-click selects one path, and a second double-click shows that path’s nodes. Moving a point is not the same as moving the asset. After an edit, inspect the container’s bounds: a node dragged past them is cut off at the edge.

Look at the silhouette before the details. A good small asset should remain recognizable when its internal decoration is temporarily ignored. Then inspect stroke consistency and the spaces between shapes. A cramped gap may close when the image is reduced; a thin stroke may disappear against the intended background. Generated output makes this check necessary: the tutorial’s illustration arrived as 3 polygons and about 20 separate paths with uneven strokes. The final composition, rather than the enlarged editing view, is the meaningful test.

Keep the asset and the layout independent

Use a vector for geometry that benefits from remaining editable, such as a logo or an icon. Keep ordinary interface text as text and structural regions as frames. Flattening the entire screen into an image can preserve its current appearance while discarding the relationships needed for later edits, accessibility and implementation. Choose the representation according to what must remain changeable.

In Paper, the container is the asset’s boundary. Containers that Paper creates, whether you draw with the pen or generate with Create SVG, start at 256 × 256 with Clip content on, which is rarely the size the asset will be seen at. With a path selected, dragging the container’s edges changes only the bounds, so you can tighten them around the artwork or give a clipped node room; the paths keep their size. As of September 2026, true crop, scale and resize for SVGs, boolean operations and snapping to other geometry were still roadmap items, so judge placement by eye at the real size, on the real background. The best edit often removes a distraction instead of adding another effect.

Figure 07.2

The double-click ladder

Three levels, three gestures. The edge of the container is also a crop.

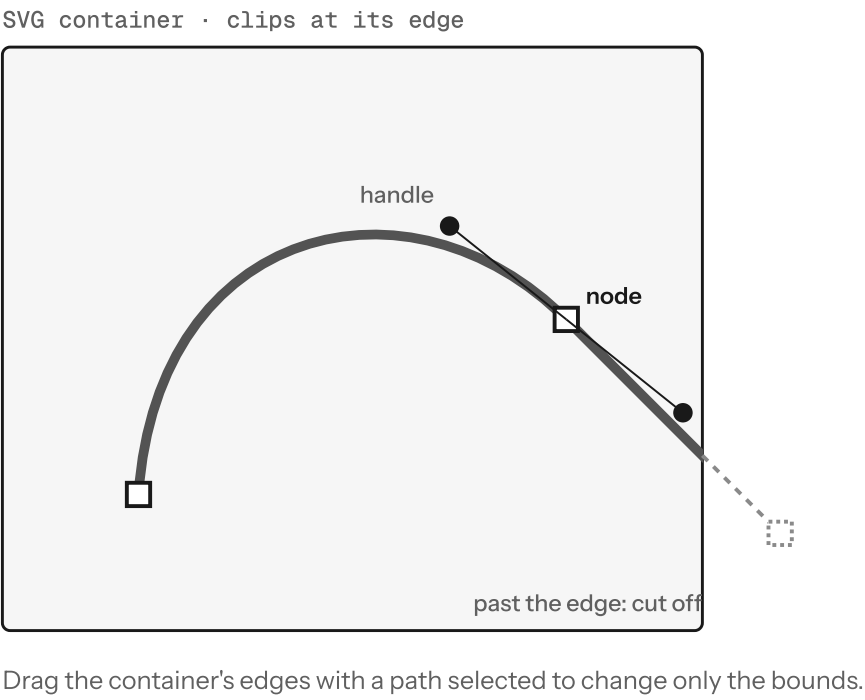
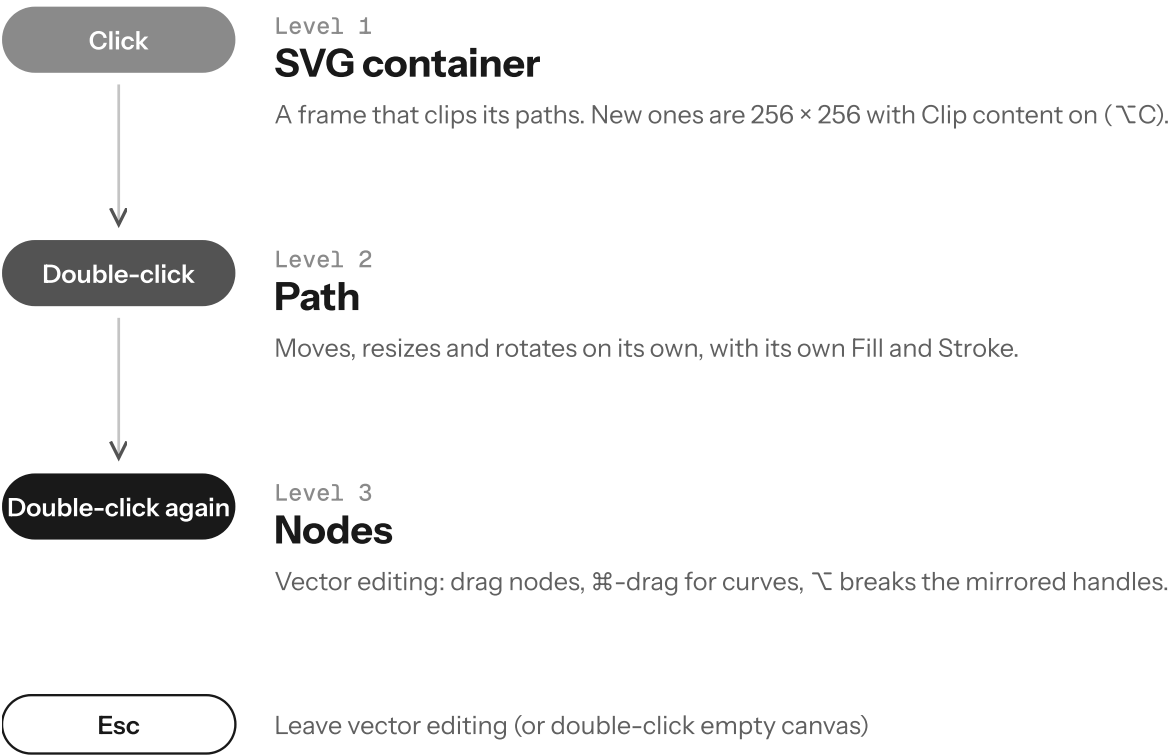


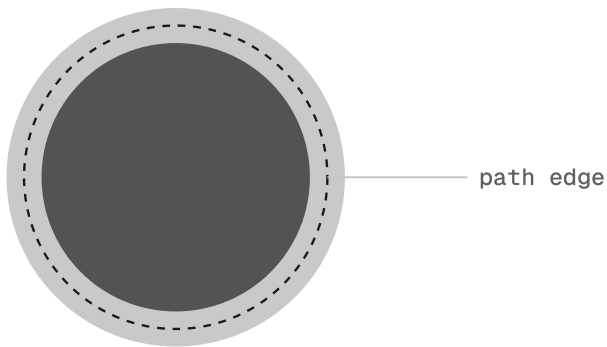
Figure 07.3

Above fill or Below fill

Where the stroke sits decides what survives when the asset gets small.

Above fill

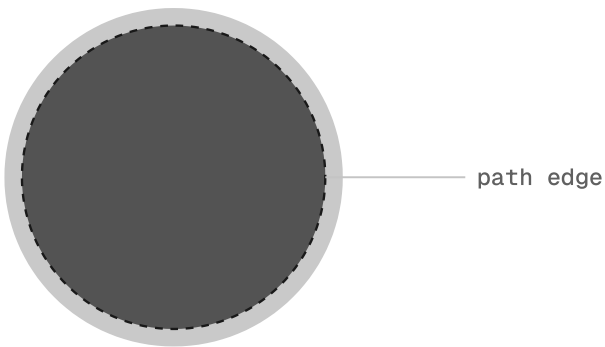
The stroke straddles the edge: half outside, half over the fill.



visible stroke: 9 = 4.5 out + 4.5 in

Below fill

The fill covers the inner half: only an outside outline shows.



visible stroke: 4.5 outside only

At small sizes

A thin bar with a thick stroke at icon size.



Above fill: the bar disappears



Below fill: the bar keeps its weight

Stroke width 9 in the tutorial; drawn here at d

How it works in Paper

As of September 2026, Paper edits SVGs on the canvas, a beta since June 2026. An SVG layer is a clipping container, 256 × 256 when Paper creates it, holding Path layers with their own Fill and Stroke (width, Above fill or Below fill). A double-click reaches a path and a second one its nodes; the pen tool (P) draws new paths. Create SVG (⇧⌘J) generates editable vectors with QuiverAI models, and right-click › Vectorize image converts a raster. Booleans, snapping to other geometry and true SVG scaling remain on the roadmap.

Checked against the videos and documentation as of September 2026. Paper ships often; the release timeline in the reference shows what changed when.

01

Edit a path inside an SVG

1

Click the SVG on the canvas to select its container.

2

Double-click to select one of its paths, or pick the Path in the layers list. Drag its handles to move, resize or rotate that path alone; the other paths stay put.

3

Double-click the selected path to enter vector editing mode, then drag its nodes. Paper's vector documentation also lists Enter for this step.

4

Recolor the path in the right panel's Fill row. Click + beside Stroke to add a stroke, then set its color and width.

5

Leave vector editing with Escape or a double-click on empty canvas.

Double-click

Esc

►

Shown at 2:04

02

Draw a shape with the pen tool

1

Press P, or click the pen button in the toolbar, just below the rectangle tool.

2

Click to place points. Points snap to the nearest half pixel; hold Shift while clicking to place the next point at a 45° angle.

3

Click the first point to close the shape.

4

A new pen path has no fill and a 1 px stroke in 000000, Above fill. Click + beside Fill to add a fill, and – beside Stroke to remove the stroke if you want a solid shape.

P

⇧

Click

►

Shown at 3:11

03

Shape curves and add points

1

Enter vector editing with the move tool active (the arrow at the top of the toolbar), not the pen, so clicks move nodes instead of adding them.

2

Hold ⌘ and drag from a node to pull out curve handles. Once the segment curves you can release ⌘; the two handles stay mirrored.

3

Hold ⌥ while dragging a handle to move it on its own. Hold ⌘ again to mirror the pair once more.

4

Hold ⌘ and drag from the middle of a straight segment to bow it into a curve, with handles at both ends.

5

To insert a point, switch to the pen tool, hover anywhere on the edge and click.

⌘

Drag

⌥

Drag

►

Shown at 4:02

04 **Add paths to the right SVG**

- 1 To extend the path you are editing, draw with the pen while it is in vector editing mode; the new segments join that same Path layer.
- 2 To add a separate path, press Escape to leave vector editing, keep the SVG or one of its paths selected, and draw with the pen inside the container. A new Path appears in that SVG with its own fill and stroke, and moves independently.
- 3 Drawing outside the container’s bounding box creates a new SVG container around the new path.
- 4 To bring that path into the original SVG, select it and press ⌘C, select the original SVG and press ⌘V. The copy lands inside the original; the stray container remains until you delete it.



► Shown at 5:14

05 **Generate an SVG with Create SVG**

- 1 Click Create SVG in the toolbar (a frame icon with a sparkle, below Create image) or press ⇧⌘J. A prompt panel opens at the bottom of the canvas.
- 2 Pick a model. In June 2026 the menu offered QuiverAI Arrow 1.1 (“Balanced SVG quality and speed”, the default) and QuiverAI Arrow 1.1 Max (“For detailed SVGs, takes more time”).
- 3 Write the prompt: the subject, the drawing style (line quality, and whether it is an illustration, an icon or a logo), specific colors, and restraint such as minimal with lots of negative space.
- 4 Click Create SVG. A placeholder layer named “Generating with QuiverAI...” appears and the drawing builds live on the canvas.
- 5 The result is an SVG layer named from the prompt, 256 × 256 with Clip content on. With it selected, the panel shows a row with its thumbnail and prompt. Run a cleanup pass before using it.



► Shown at 0:11

06 **Clean up a generated SVG**

- 1 Expand the SVG in the layers list and take inventory; the tutorial’s result held 3 Polygon layers and about 20 Path layers.
- 2 Hide unwanted shapes with the eye toggle before deciding to delete them.
- 3 Select paths one at a time and read their Stroke width. Set lines that should match to one value; in the tutorial a 0.28 stroke and a 0.99 stroke both became 1.
- 4 Select a stray path and delete it. Each stroke is its own path, so the rest of the drawing holds together.
- 5 For a path that is only partly wrong, enter vector editing, drag the doubtful piece aside to see what it is, then select its nodes and press Delete. ⌘Z restores anything removed by mistake.
- 6 Judge the result at display size before calling it finished.



► Shown at 2:26

Controls you will use

Control	Where	What it does
SVG container	Layers list (the SVG layer); its Layout section in the right panel when selected	The outer layer of every SVG. It behaves like a frame, holds Path layers and hides anything beyond its edges. New containers measure 256 × 256. With a path selected, drag the container’s edges to change its bounds; the paths keep their size.
Path	Inside an SVG in the layers list	One editable shape. Each path moves, resizes and rotates on its own; a selected path shows only Fill and Stroke in the right panel.
Fill	Right panel › Fill, with a path selected	The path’s fill as hex and opacity; + adds a fill, – removes it, the eye hides it. The picker has sRGB and Display P3 tabs, an eyedropper, L C H, H S L and R G B fields, and Previous and New swatches.
Stroke	Right panel › Stroke, with a path selected	A width field (1 by default), a position menu, a visibility eye and a color row; click + beside Stroke to add one. Strokes that should read as one line weight need the same width value.
Above fill / Below fill	Right panel › Stroke › position menu	Above fill centers the stroke on the path’s edge, half inside and half outside the fill. Below fill lets the fill cover the inner half, so the stroke reads as an outside outline. Use Below fill when a thick outline would eat into a thin shape.
Clip content ⌘C	Right panel › Layout, with the SVG container selected	Ticked by default on SVG containers, so geometry dragged past the container’s edges is cut off.
Pen tool	Toolbar, below the rectangle tool; P	Click to place points, hold Shift for 45° angles, click the first point to close. In vector editing, clicking an edge with the pen adds a point. Points snap to the nearest half pixel. Paper’s vector documentation says this snapping is toggled from the zoom menu or with ⇧⌘J’.
Vector editing mode	Double-click a selected path; Escape or a double-click on empty canvas leaves it	Shows the path’s nodes and curve handles. With the move tool you drag nodes; with the pen you add points or extend the path.
Create SVG	Toolbar, frame icon with a sparkle; ⇧⌘J	Opens the generation panel with a prompt field, a model menu and a Create SVG button. The result is an editable SVG of Path and Polygon layers, not a picture.

Control	Where	What it does
Vectorize image		
	Right-click an image	Runs an AI model that turns a raster image into an SVG, which then scales without losing quality. Flat images convert with little loss of fidelity.
Shortcuts		
P	Pen tool ▶ 3:11	Double-click Select a path inside the selected SVG; double-click the path again to edit its nodes ▶ 0:09
Esc	Leave vector editing mode (a double-click on empty canvas also works) ▶ 2:15	⇧ Click Place the next pen point at a 45° angle ▶ 3:33
⌘ Drag	Pull curve handles from a node, or bow a straight segment from its middle ▶ 4:02	⌥ Drag Move one curve handle without its mirrored partner ▶ 4:21
⌥ C	Toggle Clip content on the selected container ▶ 1:33	⇧ ⌘ J Create SVG (AI vector generation) ▶ 0:12
Delete	Delete the selected nodes or path ▶ 3:48	⌘ Z Undo, the safety net while trimming generated paths ▶ 3:54
Limits and what is coming		
○ SVG editing launched as a beta on 3 June 2026. As of September 2026, Paper’s vector documentation still listed improved snapping and guides, crop, scale and resize for SVGs, converting shapes to paths, and boolean operations (unite, subtract) as near-term roadmap items. ○ Mirroring, line, grid and radial repeaters, anchoring objects to a path, vector networks, an anneal brush and a shape builder appeared only as June 2026 prototypes or longer-term ideas; none was available as of September 2026. A scale tool for resizing several layers at the same ratio was announced as coming soon on 28 September 2026. ○ Deleting nodes removes that section of the path. In June 2026 Paper replied to a viewer that deleting a point while keeping the path joined was on the roadmap. ○ Model names change: the June 2026 Create SVG menu offered QuiverAI Arrow 1.1 and Arrow 1.1 Max, while Paper’s agent guide in early October 2026 lists QuiverAI Arrow 2 and Arrow 2 Telos.		
What the tutorials show		
Each official video, written down step by step. The dots mark the moments; they open the video there if you want to see it.		
SVG editing in Paper: pen tool, path editing, and what’s coming next Video · 9 min 17 s · 3 June 2026 ○ ○ ⌘ ○ ○ 9:17		
SVG editing tour: paths, strokes, the pen tool and the roadmap Harcourt opens on a roadmap board: editing paths, fill and stroke, importing and generating SVGs, and moving, resizing and rotating layers are available; booleans, better snapping, path guides, shape-to-path conversion and SVG crop, scale and resize are near-term. A pasted SVG is a container, Clip content ⌥C ticked, holding two Path layers. She transforms one path alone, double-clicks it to drag its nodes and leaves with Escape. She fills it navy, adds a 9 px stroke and switches Above fill, which straddles the edge, to Below fill, which reads as an outside outline. With the pen (P) she places half-pixel-snapped points, holds Shift for 45°, closes the shape and swaps the default 1 px black stroke for a fill. ⌘-drag pulls mirrored handles, ⌥ breaks the mirror, ⌘-drag bows an edge, and the pen adds points on edges. A shape drawn outside the container gets its own 256 × 256 SVG; ⌘C and ⌘V copy it back. Prototypes close the video: mirror and radial repeater modifiers, vector networks, anchoring. 0:36 Roadmap board 2:04 Vector editing mode 2:56 Above fill vs Below fill 3:11 Pen tool 4:02 ⌘ and ⌥ on curve handles 6:08 Container bounds as a crop		

SVG editing just launched in Paper

Short · 48 s · 5 June 2026



0:48

The double-click ladder in 48 seconds

With an SVG selected, a double-click selects one of its two paths, which then resizes, moves and rotates on its own while the other stays put. A second double-click shows that path’s nodes for dragging. A node dragged above the container is cut off at its edge; raising the container’s top edge reveals more of the shape. The short ends by changing the fill from 1D4D93 to 1A498D, adding a stroke (1, Above fill, 000000), coloring it 81ACEC and editing its width.

- 0:09 Double-click selects a path
- 0:13 Path transformed alone
- 0:18 Double-click again for nodes
- 0:25 Node clipped at the edge
- 0:32 Fill and stroke

Generate and refine SVG illustrations with Quiver AI inside Paper

Video · 4 min 36 s · 18 June 2026



4:36

Generate a line illustration with Quiver, then clean it up

Harcourt opens Create SVG (⇧⌘J) and picks QuiverAI Arrow 1.1 Max over the default Arrow 1.1, accepting a longer wait for detail. Her prompt names the subject (one hand making an OK sign), the style (a single continuous wobbly black brush line, loose modern line art), the colors (cream, black, lavender) and the restraint (minimal, lots of negative space, flat three-tone color). After a golden-ratio loading animation the SVG draws itself live and lands as a 256 × 256 layer named “Wobbly black hand”; the lavender ground visible during the build is gone from the result. In the layers list she hides the three Polygon layers, finds one stroke at 0.28 and another at 0.99 and sets both to 1, deletes a stray fingertip path, and in vector editing deletes the nodes of a section she does not need, with ⌘Z as the safety net. About five minutes of further editing are skipped. Generation supplied the starting point; path edits supplied the finish.

- 0:24 Arrow 1.1 vs Arrow 1.1 Max
- 0:56 Prompt anatomy
- 1:58 Live build
- 2:26 Hide the polygons
- 2:38 Match stroke widths
- 3:47 Trim part of a path

Build it in Paper

- 01

Create or import an SVG with a defined role and display size: paste one (⌘V), draw with the pen tool (P) or generate one with Create SVG (⇧⌘J). Duplicate it (⌘D) and keep the untouched copy beside the working version.
- 02

Expand the SVG in the layers list and take inventory of its Path and Polygon layers; hide doubtful parts with the eye toggle before deleting anything.
- 03

Double-click into a path and again into its nodes. Match stroke widths in the Stroke row, choose Above fill or Below fill deliberately, delete stray paths, and press Escape to leave vector editing.
- 04

With a path selected, drag the container’s edges until the bounds fit the artwork and nothing important is clipped.
- 05

Place the asset at its real size on its real background, view it at 100% zoom (the zoom level shows at the top of the right panel), and compare its silhouette with the untouched copy.

Common mistakes

- ×

Dragging a node past the SVG’s edge and concluding the shape is broken when it is only clipped.

→ Select a path and drag the container’s edge outward until the shape shows, or keep the node inside; Clip content is on by default.
- ×

Entering vector editing with the pen tool active and adding points by accident.

→ Switch to the move tool before you double-click into the path; use the pen only when you mean to add points.

Check before you move on

- ☐

Does the asset still read at its display size when you ignore interior detail and look only at the silhouette?
- ☐

Do strokes meant to match share one width value, and is each stroke’s position (Above fill or Below fill) a deliberate choice?
- ☐

Is anything cut off by the SVG container, or is empty container space throwing off alignment with its neighbors?
- ☐

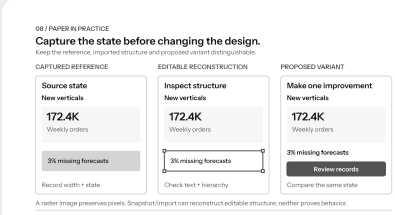
Has every hidden or stray path been either restored for a reason or deleted?
- ☐

Is interface text still live text, with only icons, logos and illustrations drawn as vectors?

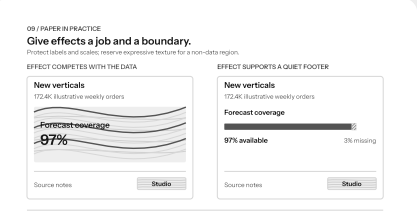
Material

Where a design starts (a live site, a Figma file, generated images) and how effects such as shaders earn their place.

2 chapters, 2 live studies
12 source videos, 34 minutes, written down



08 Choose the right starting material



09 Shaders: give effects a precise visual purpose

Choose the right starting material

Snapshot, Figma paste, HTML import and image generation begin with different kinds of evidence and preserve different kinds of editability.

Capture the product you intend to change

Starting from the actual interface gives a redesign useful resistance. Its current content, spacing and awkward cases are already present, so the proposal must respond to a real situation. Record the viewport and the state being captured. A populated results screen and an empty state are different design problems even if they share the same route.

Paper Snapshot is a Chrome extension that captures the part of a page you choose: hover until a blue box surrounds it, fine-tune the target with the arrow keys, press Return (or ⌘Return for the whole page), then paste into Paper with ⌘V. Because the page and Paper’s canvas are both HTML and CSS, the capture arrives as editable layers rather than a screenshot: text layers, flex frames with their gaps and Fill or Fit sizing, per-side borders with alpha, radii, and every color listed in Selection colors. It is a visual and structural starting point, fixed at the size and state you captured. It does not carry the application’s authentication, persistence or live data behavior into the canvas; those responsibilities stay in the running product.

Capture preserves a starting point; import translates structure; generation creates new material.

The principle of chapter 8

Keep the structure, or flatten the pixels

Edit the layers, freeze a PNG, then compare reflow with scaling.

Title

Value

Detail

Action

Forecast availability

97%

Records with a forecast · illustrative

Review 3 missing records

Four independent text layers. Narrow the frame or lengthen the title: text wraps while the structure keeps its spacing.

Capture the state before changing the design.

Keep the reference, imported structure and proposed variant distinguishable.

CAPTURED REFERENCE

Source state

New verticals

172.4K

Weekly orders

3% missing forecasts

Record width + state

EDITABLE RECONSTRUCTION

Inspect structure

New verticals

172.4K

Weekly orders

3% missing forecasts

Check text + hierarchy

PROPOSED VARIANT

Make one improvement

New verticals

172.4K

Weekly orders

3% missing forecasts

Review records

Compare the same state

A raster image preserves pixels. Snapshot/import can reconstruct editable structure; neither proves behavior.

Reconstructed teaching example · synthetic values · not a Paper screenshot

Figure 08.1 · Plate

Capture the state before changing the design.

Snapshot and structured imports can provide editable starting points. Inspect their hierarchy against the captured reference. A raster image preserves appearance, while behavior and data logic still need implementation and testing.

Distinguish import from invention

Normal copy and paste moves a Figma selection into Paper: ⌘C in Figma, ⌘V in Paper. Frames keep their names, sizes and structure, and images follow once you connect a Figma account that can open the file. Components, instances and variables arrive detached, masks are hidden, and effects with no CSS equivalent, such as noise, glass or gradient strokes, do not come across. HTML paste has a different boundary: only inline styles transfer, and class names and stylesheet rules are discarded. The important question is therefore not whether an import succeeded, but which decisions survived in an editable form and which now need to be restated.

An image has a different role. A screenshot is useful evidence of appearance, but its pixels do not explain the original layout hierarchy. A generated image is new material produced from a brief, rather than a record of the existing product. Place these materials on the canvas with clear purposes: reference, editable baseline, or proposed asset. Confusing those purposes makes a visually convincing file harder to trust.

Image generation benefits from a concrete composition brief: subject, materials, surface, light, palette, aspect ratio and the space reserved for nearby copy. In Paper, Create image (⇧⌘I) sends that brief to the model you pick, and selecting the result makes it context, so the next prompt changes one thing while the rest of the scene holds. This is a production method rather than a promise about a particular model’s accuracy: in Paper’s own

Inspect the transfer before redesigning it

A missing image can make a captured layout look cleaner while actually making it incomplete. A substituted font can change every line break. Compare the imported baseline with the source before you start improving it, so translation errors do not become mistaken design decisions. When a capture from a local development server arrives without its images, the usual cause is cross-origin blocking. Paper’s troubleshooting page says to allow the origin https://app.paper.design in the server’s CORS settings and, on Vite 6 and later, which blocks /@fs/ paths regardless of CORS, to serve assets from public/.

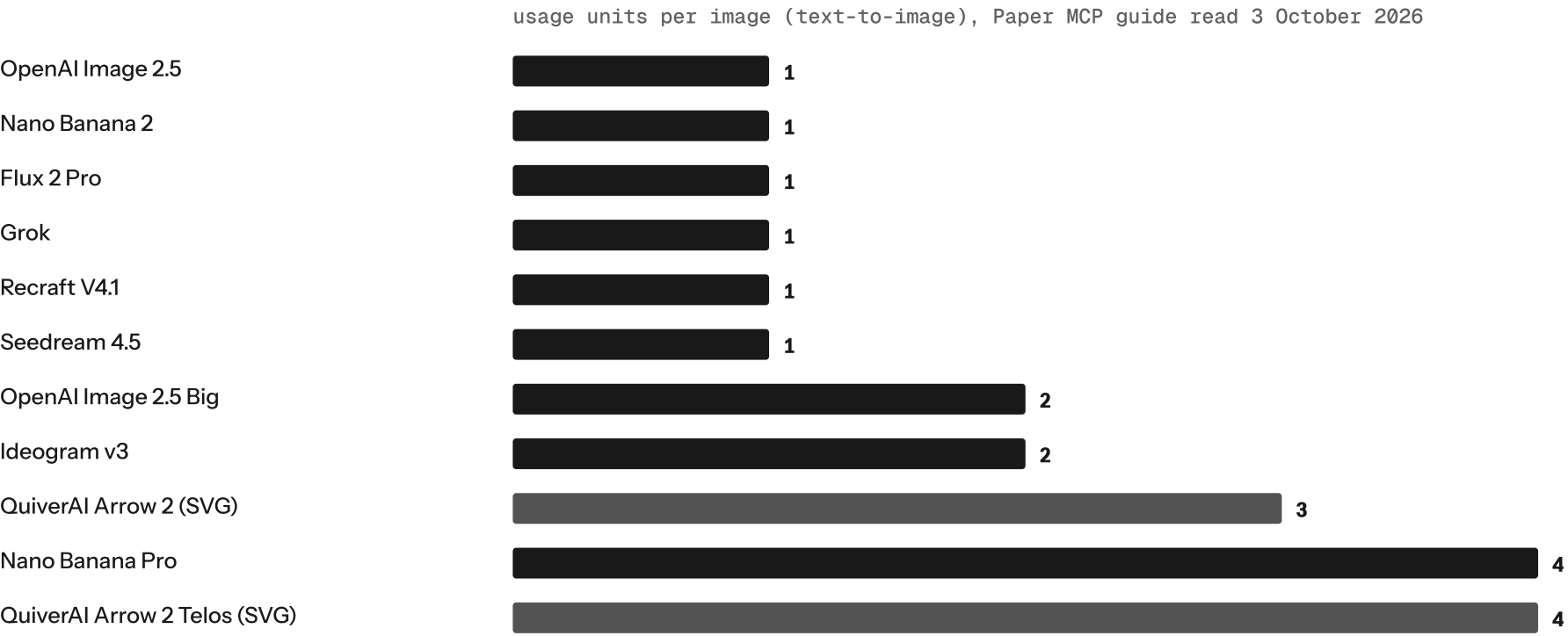
Keep the three kinds of material apart on the canvas. Leave the original capture untouched and duplicate it (⌘D) for the working version. Rename generic layers: Snapshot names its frames Frame, and Figma pastes keep names such as Property 1=Variant4. Generated images are named from their prompt, and their Image fill records the model and the pixel size, which tells you whether the resolution suits the display size. Then explain each intentional difference. That creates a clear line from the product you observed to the proposal you want implemented.

demo, a lighting change also removed a wall. If the result must describe a real place, person or product, verify it against appropriate evidence instead of accepting plausible detail as fact.

Figure 08.2

Image models and what they cost

Usage units per image in Paper's agent guide, October 2026.



In-app Create image menu on 26 May 2026: Variety pack (OpenAI Image 2, Grok, Imagen 4, Seedream 4.5), OpenAI Image 2, Imagen 4, Flux 2 Pro, Grok, Recraft V3, Ideogram v3, Seedream 4.5, Nano Banana Pro.

Figure 08.3

What each starting route brings in

Choose the route before you start: what arrives editable, what to check, what is lost.

Route	How	Arrives editable	Check	Lost or not supported
Paper Snapshot	Chrome extension: hover, ⬆, Return or ⌘Return; ⌘V in Paper	Frames, text layers, flex with gap and Fill/Fit sizing, per-side borders with alpha, radii, colors; icons as SVG	Images from local dev servers (CORS for https://app.paper.design); generic Frame names; fixed size	Behavior, live data, authentication
Figma paste	⌘C in Figma, ⌘V in Paper	Frames with Figma names and sizes, colors, fonts, shapes, structure, SVGs; images once Figma is connected	Images need an account that can open the file; API rate limits	Component, instance and variable links; masks; noise, texture, glass, repeat, symmetry; gradient strokes; rich text
HTML paste	⌘V with HTML on the clipboard	Elements as layers with their inline styles; images with public URLs	Inputs become frames with text; mixed inline text flattens	Class and stylesheet rules
Create image	Toolbar or ⌘⌘I; model and aspect ratio	A new image layer named from the prompt; Image fill shows model and pixel size	Unrequested changes between versions; weekly limit	Not evidence of the existing product
Pasted image	Copy an image in the browser, ⌘V in Paper	A raster image layer; Remove background, Vectorize image, Extract colors on right-click	Resolution at display size	Layout structure

How it works in Paper

As of September 2026, Paper takes starting material five ways. Paper Snapshot, a Chrome extension, rebuilds a page region or a whole page as editable HTML/CSS layers. Figma selections paste with ⌘C and ⌘V, components detached. HTML pastes with inline styles only. Create image (⇧⌘I) generates images from a choice of models, and since September agents can generate through the MCP. Plain images paste too, and right-click offers Remove background, Vectorize image and Extract colors. Captured flex, borders and radii arrive as real CSS properties, not pixels.

Checked against the videos and documentation as of September 2026. Paper ships often; the release timeline in the reference shows what changed when.

01

Install and start Paper Snapshot

- Open the Paper Snapshot page in the Chrome Web Store (linked from Paper’s site) and click Add to Chrome.
- Pin it: open Chrome’s Extensions menu, find Paper Snapshot and click its pin so the icon stays in the toolbar.
- On the page you want, click the Paper Snapshot icon. If Chrome’s Extensions menu lists it under Access requested, click it there to give it access to that site.
- A hint bar appears at the top of the page: Click or ⇧ to capture, ⇧⇩ to fine-tune target, Esc to cancel. In the June and September 2026 recordings it also shows ⌘⇧ to capture page.
- Paper’s quick start page adds ⇧⌘P to switch the extension on; you can remap it at <chrome://extensions/shortcuts>.

►

Shown at 0:34

02

Capture a region or a whole page

- Hover the page. A blue box marks the element under the pointer; everything inside it is the selection.
- Press ⇧ to widen the target to an enclosing element. The hint bar pairs it with ⇩ for the other direction.
- Click or press Return to capture the target, or press ⌘Return to capture the entire page. A Capturing selection... badge shows progress, then a toast confirms “Copied to clipboard. Ready to paste into Paper.”
- Switch to Paper, on the web or in the desktop app, and press ⌘V on the canvas.
- Press Esc instead to cancel without capturing.

⇧

Return

⌘Return

⌘V

Esc

►

Shown at 1:02

03

Paste a Figma selection

- In Figma, select frames or layers (⇧A selects everything on the page) and press ⌘C.
- In Paper, press ⌘V. A toast reads “Pasting from Figma. Images will be fetched.”
- If the selection holds images, the Images from Figma dialog appears the first time. Choose Connect and continue, approve Paper on the Figma page that opens in your browser, then return to Paper. Continue without images pastes the layers without them.
- Check the result against Figma: frames keep their names and sizes (two 1000 × 700 frames named Property 1=Variant4 and Property 1=Variant5 in the demo), nested frames carry flex settings with Fill and Fit sizing, and components arrive detached.
- If images are missing, connect an account that can open the Figma file; Figma’s API rate limits can also hold them back.

⌘C

⌘V

►

Shown at 0:39

- 04 **Generate and iterate an image**
- 1 Click Create image in the toolbar (a picture icon with a sparkle) or press ⇧⌘I. A prompt panel opens at the bottom of the canvas.

2 Pick a model. The default, Variety pack, sends one prompt to four models and returns four images.

3 Pick an aspect ratio: Square 1:1 (the default); Landscape 3:2, 4:3, 16:9 or 2:1; Portrait 2:3, 3:4, 9:16 or 1:2.

4 Describe subject, materials, surface, light, mood and photographic register, then click Create image. A placeholder layer (Generating with Grok, in the demo) becomes an image layer named from the prompt.

5 To iterate, select the image. The panel shows it as context, with its thumbnail and original prompt, and hides the ratio menu. Ask for one change and keep the same model.

6 Each version lands beside the source at the same size. Reselect the original before the next change, and click away to drop the context.



► Shown at 0:08

- 05 **Cut out a subject**
- 1 Right-click the image on the canvas.

2 Choose Remove background, in the menu's image section beside Vectorize image, Extract colors and Open image in new tab.

3 Wait a couple of seconds. The subject now sits on transparency, and the layer keeps its original size (1731 × 2181 in the demo).

4 Place the cutout in its target frame, which crops it at its edges. To trim the transparent margin, hold ⌘ while dragging a resize handle; that crops the image instead of scaling it.



► Shown at 0:06

- 06 **Generate images through an agent**
- 1 Connect the agent once: with nothing selected, click Connect your agent in the right panel's MCP section, choose the harness and click its Add to button (for example Add to Cursor), or open Or, install manually.

2 Ask in plain words and say where the images go. One of the dialog's test prompts is “Generate an image of a beautiful sunset in Paper”.

3 In the September 2026 demo, asking for the variety pack of models produced a board titled with the subject and four images captioned by model: OpenAI Image 2.5, Nano Banana 2, Flux 2 Pro and Recraft V4.1.

4 Behind the scenes the agent writes paper-gen:// image URLs naming a model, a prompt and an optional aspect ratio. Each image costs 1 to 4 usage units depending on the model, and a call that authors more than 10 images is declined as a whole.

5 Paper's agent guide tells agents to generate only when you ask, and to make variants as new images that reference the original.

► Shown at 1:31

Controls you will use

Control	Where	What it does
Paper Snapshot hint bar	Top of the web page after you start the extension	Lists the capture keys: Click or ⌘↵ to capture, ⌘↵ to capture page, ⌘↕↴ to fine-tune target, Esc to cancel. After a capture, a Capturing selection... badge shows progress (as a percentage in later builds) and a toast confirms “Copied to clipboard. Ready to paste into Paper.” The quick start page lists ⇧⌘P to start Snapshot; remap it at chrome://extensions/shortcuts.
Selection colors	Right panel, near the bottom	Lists every color in the selection with its alpha and how many layers use it, so you can check a capture against the live site. A Snapshot of a dark section listed its card border color as DDF3FF at 18.3%.
Images from Figma	Dialog on the first Figma paste that contains images	Connect and continue links a Figma account so Paper can read the file's images; Continue without images pastes the layers without them. The connected account must be able to open the file.
HTML paste	Canvas, ⌘V with HTML on the clipboard	Turns HTML into editable layers. Only inline styles transfer; class names and selectors are discarded, and every element gets box-sizing: border-box. Images from img elements and background-image need public URLs, form inputs become frames with text, and the layer-name, data-paper-locked and hidden attributes set a layer's name, lock and visibility. Documented rather than shown in the videos; listed in the May 2026 build log.
Create image	Toolbar, picture icon with a sparkle; ⇧⌘I	Opens the image panel: prompt, model menu, aspect-ratio menu and Create image button. While an image is selected, the panel shows it as context and hides the ratio menu.
Variety pack	Create image › model menu, the default	Sends one prompt to four models and returns four images. On 26 May 2026 the four were OpenAI Image 2, Grok, Imagen 4 and Seedream 4.5.
Image fill label	Right panel › Fill › Image, with a generated image selected	Names the model and the source pixel size, for example Grok 1184×888 on a 592 × 444 layer, so you can see how much resolution the display size has.
Remove background	Right-click an image	Cuts the subject out onto transparency in place; the layer keeps its original size.

Control	Where	What it does
		No MCP tool performs it as of October 2026; it is a manual canvas step.
Vectorize image and Extract colors	Right-click an image	Vectorize image converts a raster into an SVG with an AI model. Extract colors adds swatches, labeled with hex values, taken from the image.
Connect your agent	Right panel › MCP section, with nothing selected	One-click Paper MCP setup for Cursor, Claude, Claude Code, GitHub Copilot, VS Code, OpenCode, Codex, Codex CLI, Antigravity or other agents; one of its test prompts asks the agent to generate an image.

Shortcuts

Return	Capture the highlighted element in Paper Snapshot (a click also captures) ▶ 1:18	⌘ Return	Capture the entire page in Paper Snapshot ▶ 7:26
↑	Widen the Snapshot target to an enclosing element (↓ fine-tunes it the other way) ▶ 1:13	Esc	Cancel a Snapshot capture ▶ 1:06
⌘ V	Paste a capture, Figma layers or a copied image onto the canvas ▶ 1:30	⌘ C	Copy a selection in Figma before pasting it into Paper ▶ 0:39
⇧ ⌘ I	Create image ▶ 0:11	Right-click	Image actions: Remove background, Vectorize image, Extract colors, Open image in new tab ▶ 0:06
⌘ D	Duplicate, to keep an untouched copy of a capture ▶ 0:06	⌘ Drag	Crop an image by holding ⌘ while dragging a resize handle ▶ 1:54

Limits and what is coming

- Paper Snapshot is a Chrome extension; no other browser appears in the sources (Chrome Web Store version 0.4.4, updated 27 September 2026). A capture is a static copy at the width and state you captured, with generic layer names and none of the page’s behavior or live data.
- Figma paste, in beta at its June 2026 launch, detaches components, instances and variables, hides masks, turns diamond gradients into radial ones, and does not support noise, texture, glass, repeat or symmetry effects, gradient strokes, arbitrary dash lengths, rich text or pass-through blending of backgrounds (Paper’s Figma paste documentation, October 2026).
- HTML paste reads inline styles only and has no rich text: a block whose children are all inline becomes a single text layer, so mixed styling inside it is lost.
- Image generation is metered. In May 2026 the panel warned “You are approaching your rate limit. New images are available each Saturday.” with an Upgrade button; Paper’s pricing lists limited generation on Free and 100 times more per week on Pro, and the MCP charges 1 to 4 usage units per image depending on the model.

What the tutorials show

Each official video, written down step by step. The dots mark the moments; they open the video there if you want to see it.

Design using your live site as a starting point with Paper Snapshot

Video · 2 min 30 s · 2 May 2026

2:30



Capture a live section with Paper Snapshot

Harcourt installs Paper Snapshot from the Chrome Web Store, pins it, and opens Resend’s homepage to rework its “Go beyond editing” section. Clicking the extension icon shows the hint bar; hovering draws a blue box around the element under the pointer, ↑ widens the box from one card to the whole section, and a click or Return captures it; a toast confirms “Copied to clipboard. Ready to paste into Paper.” In the Paper web app, ⌘V drops a 1280 × 900 frame named Frame. The heading and paragraph are text layers, and a card is a flex column with a 16 gap, Fill width, Fit height, a 24 radius and 1 px Top, Left and Right borders in DDF3FF at 18.3%. Selection colors lists every captured color with its alpha. Pressing → on a selected card swaps it with its neighbor, and a double-click edits the heading in place. Her point: layout changes happen visually, on real structure, before anything goes back to code.

- 0:41 Add to Chrome, then pin
- 1:02 Hover shows the blue box
- 1:13 ↑ widens the target
- 1:30 ⌘V in Paper
- 1:44 Flex, borders and radius kept
- 1:54 Edit text in place

Copy designs from Figma and paste them into Paper

Video · 4 min 01 s · 3 June 2026

4:01



Copy Figma frames into Paper

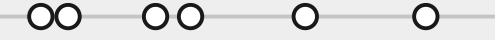
Harcourt selects two 1000 × 700 frames in a Figma brand-exploration file, presses ⌘C, and presses ⌘V in Paper. A toast reads “Pasting from Figma. Images will be fetched.” Because one frame holds an image, the first-time Images from Figma dialog offers Connect and continue or Continue without images; she connects through Figma’s authorization page in the browser and returns to Paper. Colors, fonts, symbols, shapes and structure match, and the frames keep their Figma names, Property 1=Variant4 and Property 1=Variant5. She then asks a Claude agent (Sonnet 4.6) to duplicate the frame named in the prompt and translate its copy to Spanish through the Paper MCP; the Spanish title wraps to two lines, a design decision the canvas exposes. Meanwhile she right-clicks a section and chooses Copy as › Copy as React CSS. Components paste detached, and the feature was in beta.

- 0:39 ⌘C in Figma, ⌘V in Paper
- 0:49 Images from Figma
- 1:05 One-to-one result
- 1:39 Agent duplicates and translates
- 2:06 Copy as React CSS
- 3:18 Components arrive detached

Generate images in Paper using the xAI model

Video · 3 min 37 s · 26 May 2026

3:37



Generate an image with Grok, then change one thing at a time

Harcourt opens Create image (⇧⌘I), switches the model from the default Variety pack to Grok (xAI image generation) and the ratio from 1:1 to 4:3. Her prompt names the medium, the subject, the planter, the surface, the light, the mood and an interior-photo register. A Generating with Grok placeholder becomes, in about two seconds, a layer named Monstera plant planter: 592 × 444 on the canvas, with an Image fill labeled Grok 1184×888. Selecting it makes it context; the panel shows its thumbnail and prompt and hides the ratio menu. She asks for 7 pm sunset light, then reselects the original for a terracotta pot, then for a healthier plant without fallen leaves. Each version lands beside the source with the rest of the scene intact, except that the sunset edit removed a side wall. A banner then warns that the weekly rate limit is near, with an Upgrade button.

- 0:19 Variety pack and model list
- 0:31 Aspect ratio 4:3
- 1:09 Result in about two seconds
- 1:24 Selected image as context
- 2:14 An unrequested change
- 3:07 Four versions, rate-limit banner

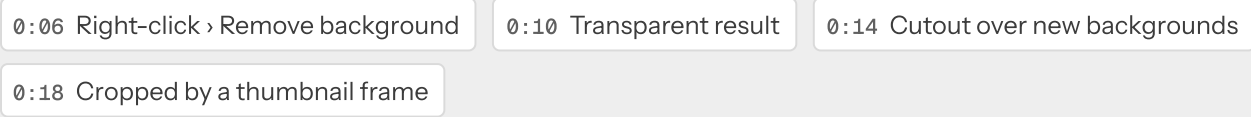
Remove image backgrounds in Paper

Short · 26 s · 29 August 2026



Remove a background in two clicks

Right-clicking a portrait and choosing Remove background leaves the subject on transparency after a couple of seconds, in the same 1731 × 2181 layer. The short then shows the cutout over other backgrounds, without showing how those were made, and dropped into a YouTube-thumbnail frame that crops it at its edges. The captured menu also lists Vectorize image, Extract colors and Open image in new tab, plus Duplicate ⌘D and Paste to replace ⌘R.



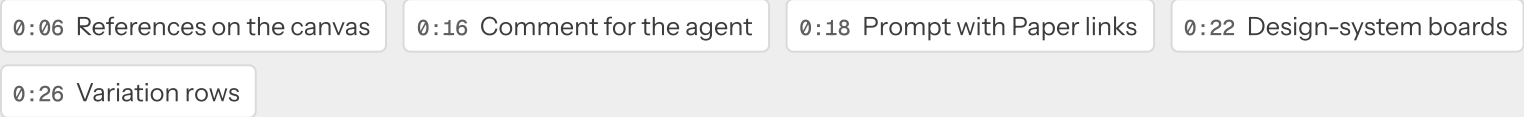
Use Mobbin and Paper to design flows from proven patterns

Short · 42 s · 23 September 2026



Start a flow from references, steered by comments

An agent pulls add-to-cart and add-to-bag flows from five products through Mobbin’s MCP and writes them onto the canvas as boards named Ref · <product> — <flow>, each with numbered, captioned screens. The designer keeps only the references that suit the brand and pins comments, sent with Send, on the screens she likes. Her prompt in the Claude app (Opus 5, High) links the reference frames and the design-system boards, 01 · Foundations and 02 · Components, and asks for five variations; the agent writes rows such as V1 · Drawer, V2 · Toast and V3 · Popover. She repeats the comment loop, then finishes by hand. How Mobbin’s MCP was connected is not shown.



Build it in Paper

- 01

Open the live page at a known width and state, click the Paper Snapshot icon in Chrome’s toolbar, hover the region, refine it with ↑ and ↓, and press Return; press ⌘Return instead to capture the whole page.
- 02

In Paper, press ⌘V. Compare text, images, fonts and spacing with the live page, rename the generic Frame layers, and duplicate the capture (⌘D) before editing.
- 03

For Figma material, copy in Figma (⌘C), paste in Paper (⌘V) and choose Connect and continue when Images from Figma appears, then look for detached components. For HTML, inline the styles before you copy.
- 04

Generate new imagery with Create image (⇧⌘I): pick a model and an aspect ratio, select the result to change one thing per prompt, and use right-click › Remove background for cutouts.
- 05

Group captured references, editable baselines and generated assets in separately labeled areas so no one mistakes one for another.

Common mistakes

- ×

Redesigning directly on the only capture of the live page.

→ Duplicate it with ⌘D and keep the original untouched beside the working copy, so every change can be compared with what shipped.
- ×

Asking an image model for several changes at once and losing the parts that worked.

→ Select the original as context, request one change per prompt with the same model, and compare each version with the source.

Check before you move on

- ☐

Did you record the URL, viewport width and state you captured, and does the pasted frame match the live page’s text, images and fonts?
- ☐

Have captured and pasted layers been renamed from Frame or Figma variant names to names that say what they are?
- ☐

Which parts are reference, which are editable baseline and which are generated proposals, and is each labeled that way on the canvas?
- ☐

Did any image fail to transfer, or any component arrive detached, and have you restated those decisions?
- ☐

For each generated image, is the source resolution in its Image fill enough for its display size, and did any edit change something you did not ask for?

Shaders: give effects a precise visual purpose

A shader can create material, light or motion. It becomes useful design when its role is clear and the surrounding content remains readable.

Choose the role before the preset

An effect can establish atmosphere, make a brand mark distinctive or create a focal point. Those are different jobs from explaining a value or completing an action. Begin by naming the job. If a static texture supplies enough character, movement may add distraction without adding meaning, and in Paper every animated shader becomes a still at Speed 0%. The strongest effect is often the one whose intensity has been reduced to fit the composition.

Paper’s Shaders panel sorts its shaders into three families, and the family matters more than any slider. Image filters, such as Fluted Glass, Halftone CMYK and Lens Distortion, transform an image you feed them. Logo animations (Heatmap, Liquid Metal and Gem Smoke) react to the edges of a shape you supply. Effects, from Mesh Gradient to God Rays, generate a field from parameters alone. As of September 2026 that is 7 image filters, 3 logo animations and 20 effects. Choosing between these families is more consequential than moving several sliders on an unsuitable preset.

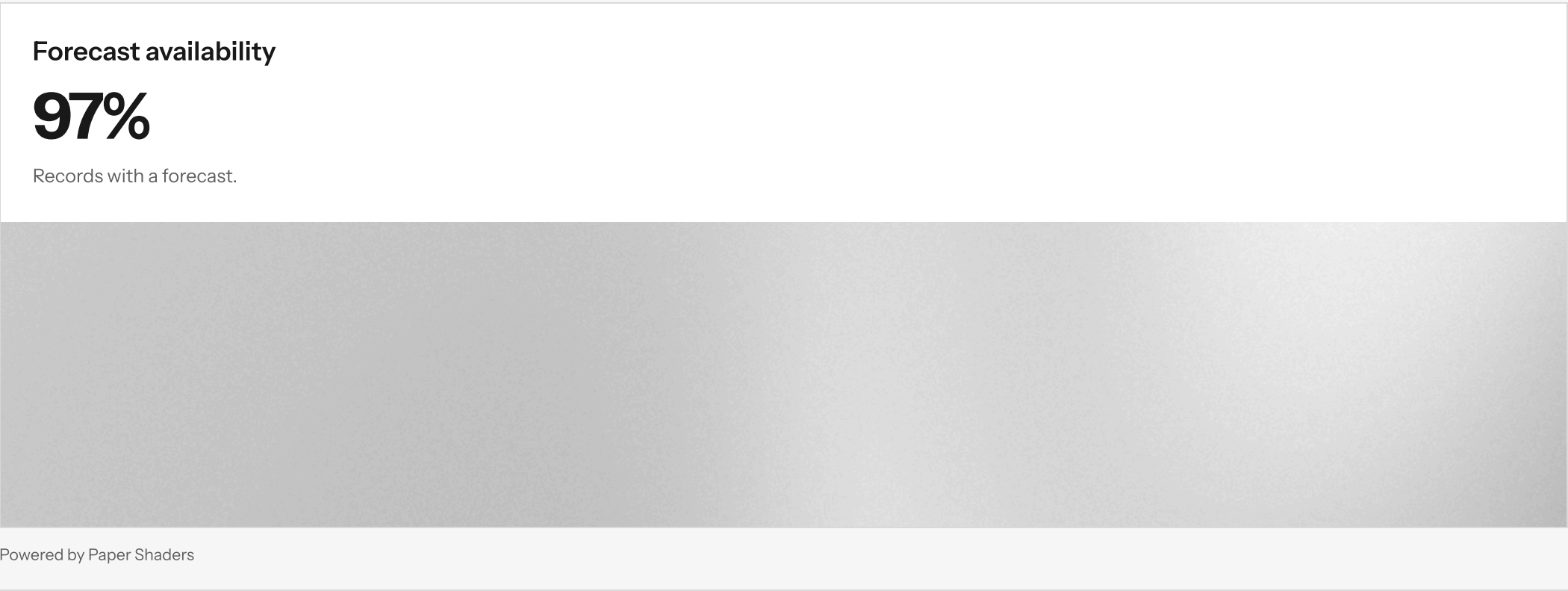
—
An effect should support one visual purpose without borrowing attention from the task.

The principle of chapter 9

Study 09 · change it yourself

Give the effect a boundary

Keep text still while you shape the texture and inspect its motion.

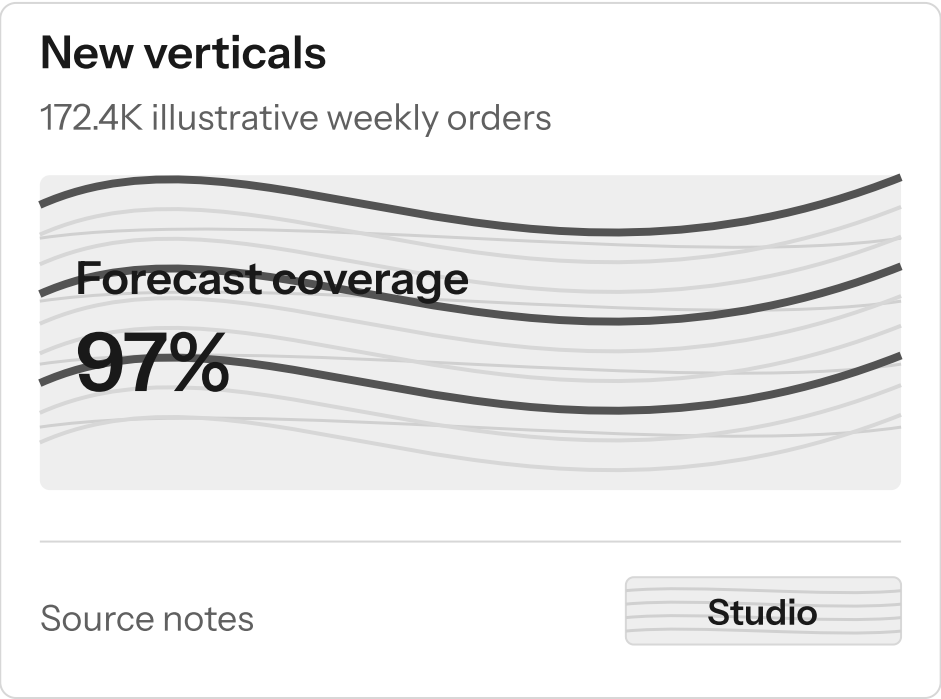


Animation paused while the lab is out of view.

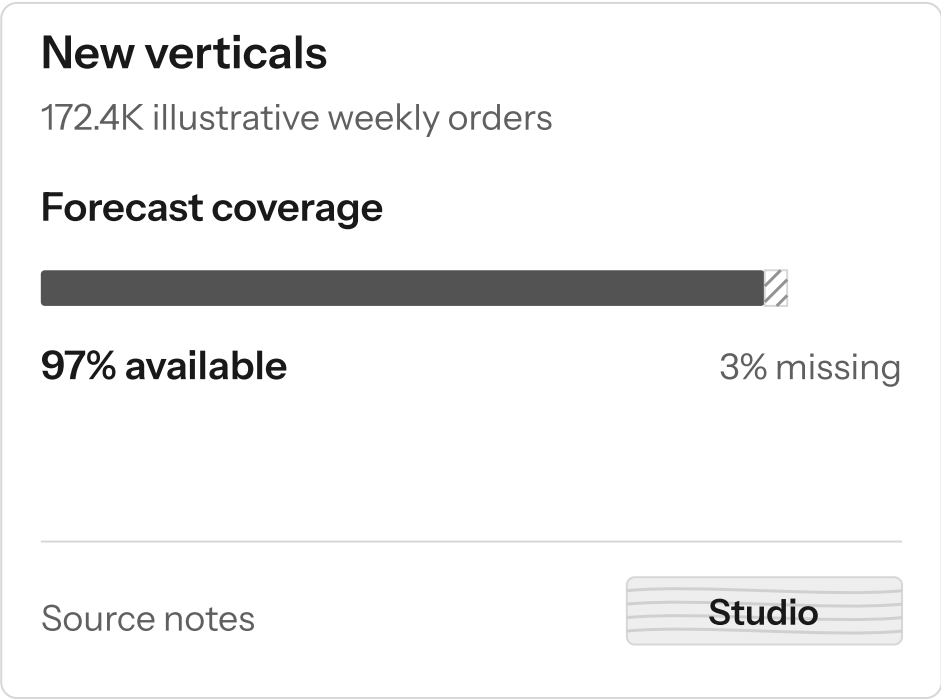
Give effects a job and a boundary.

Protect labels and scales; reserve expressive texture for a non-data region.

EFFECT COMPETES WITH THE DATA



EFFECT SUPPORTS A QUIET FOOTER



Reconstructed teaching example · synthetic values · not a Paper screenshot

Figure 09.1 · Plate

Give effects a job and a boundary.

The left reconstruction deliberately lets texture interfere with the reading task. The right confines a static shader-like treatment to a non-data footer. This drawing does not reproduce a specific shader; production should include a static or reduced-motion fallback.

Tune the relationship, not the isolated preview

Paper’s logo-animation tutorial captures a real site footer with Paper Snapshot, places a Gem Smoke version of the brand’s wordmark inside it and tunes the smoke beside the footer’s links. That is the decisive move. A dramatic mark in an empty preview can become distracting on a real page. Assess the amount of motion, the brightness of the busiest region and the contrast of nearby text together, because they compete for the same attention. In that demo the fixes were concrete: Outer glow at 0% so no smoke crossed the links, less Inner glow, and Clip content on the frame so the smoke stopped at its edge.

Change one parameter family at a time: first the input and its fit, then the palette, then the form (shape, proportion, distortion, contour), and only then motion and surface detail such as Speed and grain. Paper’s own rebuild of a reference follows this order: nearest preset, then colors sampled from the reference, then Speed, Scale, Softness and swirl. Each change has a visible cause. If several controls move together, an attractive result is hard to reproduce or refine. A habit from the same tutorials helps: push a slider to its extreme to see its range, then settle.

Some controls depend on others. The Lens Distortion reference makes Spread the master control of its color split: at 0, the other color controls do nothing. Waves’ frequency needs some amplitude, and Halftone Dots loses the image entirely at zero Contrast. An apparently inactive slider may therefore be working as specified. Keep the selected effect’s parameter reference

Separate visual atmosphere from evidence

Names such as Heatmap, Waves or Voronoi describe aesthetic shader families, not data bindings. A heat-like color treatment does not become a measured intensity map because the name sounds scientific, and Voronoi cells are not real spatial partitions. Keep decorative effects outside charts and evidence-bearing text. Give the expressive material one bounded region, such as a cover, hero or footer band, and keep the information on a stable, opaque surface beside it.

Motion also needs a quiet version in the implemented product. The shader package pauses animation when a shader leaves the viewport or its tab is hidden, but it does not react to a reduced-motion preference on its own, so keep an accepted still (speed 0 at a chosen frame) and switch to it explicitly. Record the package version with the preset: in version 0.0.81 Paper Texture was rebuilt, its old folds became crumples, its old crumples became wrinkles, contrast and fade were removed, and old seeds no longer match visually. Recheck the composition after an upgrade, not only whether the code runs.

nearby, and save the settings with the input image and final dimensions. A preset without its context is only a partial description of the result.

Thirty shaders, live

Every effect in the official shader library (version 0.0.81), running its default preset.

Effects 20

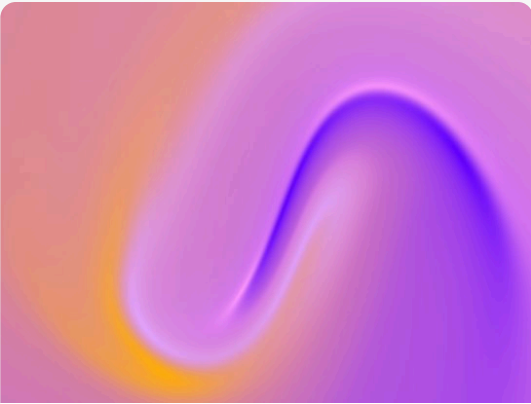


Animated · no input

Mesh Gradient

Up to ten color spots drift along their own paths and blend, bent by organic noise and a swirl around the center.

distortion, swirl, grainMixer, grainOverlay · 3 more presets

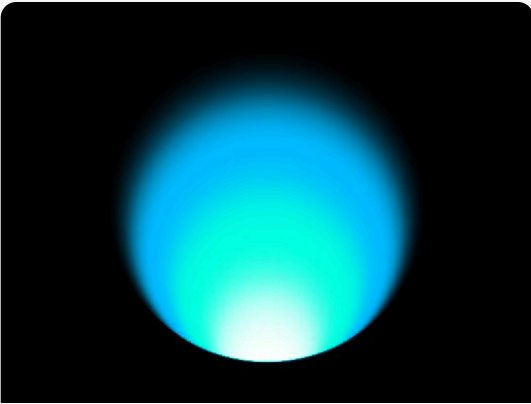


Static · no input

Static Mesh Gradient

A still gradient of up to ten color spots, warped by sine waves along X and Y, with adjustable blend sharpness and grain.

positions, waveX, waveXShift, waveY, waveYShift · 3 more presets

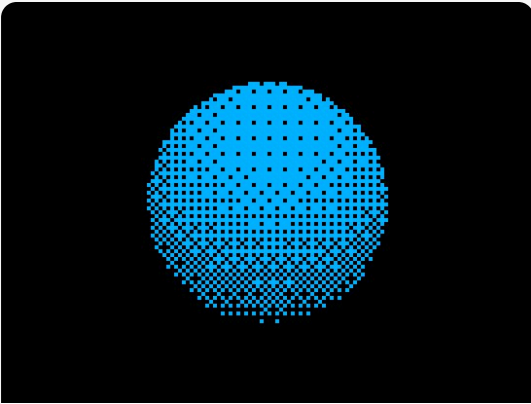


Static · no input

Static Radial Gradient

A still radial gradient of up to ten colors with a movable focal point, falloff, radial distortion and grain.

radius, focalDistance, focalAngle, falloff, mixing · 3 more presets

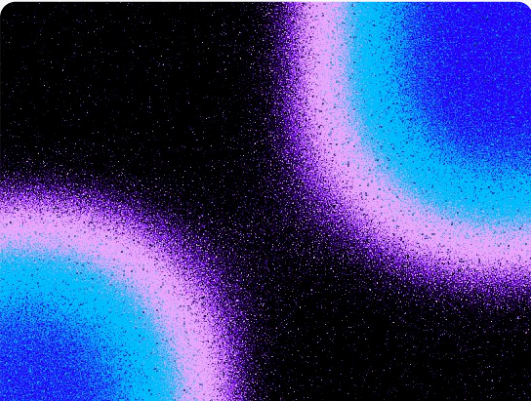


Animated · no input

Dithering

Two-color animated dithering laid over a generated source shape: simplex, warp, dots, wave, ripple, swirl or sphere.

shape, type, size · 5 more presets



Animated · no input

Grain Gradient

A multi-color gradient with grainy, noise-textured edges in seven animated forms.

softness, intensity, noise, shape · 5 more presets

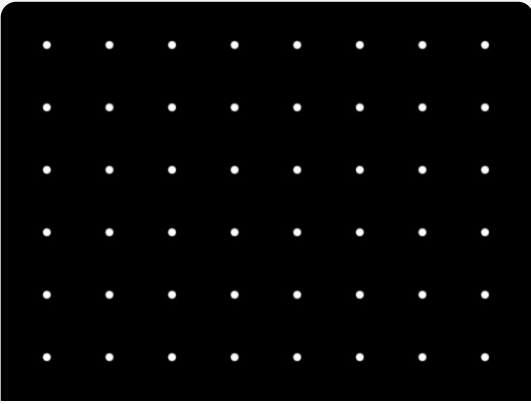


Animated · no input

Dot Orbit

A grid of colored dots, each circling its own cell center, with in-between palette steps and size variation.

size, sizeRange, spreading, stepsPerColor · 3 more presets

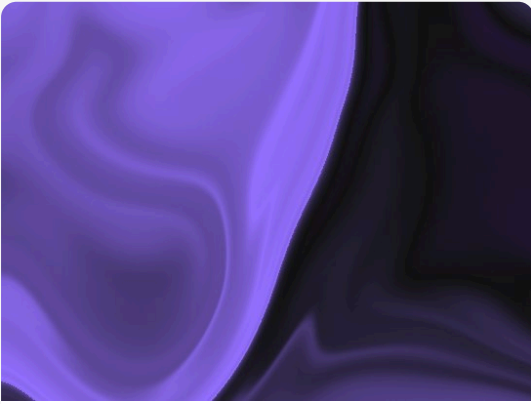


Static · no input

Dot Grid

A still, evenly spaced grid of circles, diamonds, squares or triangles with optional stroke and random size or opacity.

size, gapX, gapY, strokeWidth, sizeRange · 3 more presets



Animated · no input

Warp

Color fields laid over checks, stripes or a split edge, then warped by noise and layered swirls into fluid, smoky or marbled shapes.

proportion, softness, distortion, swirl, swirlIterations · 5 more presets

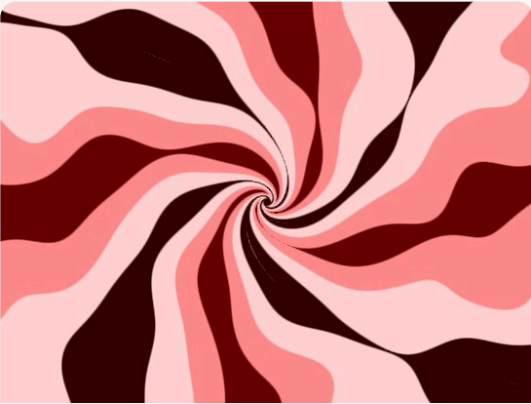


Animated · no input

Spiral

A single-color animated spiral that ranges from crisp thin lines to whirlpools and wavy rings.

density, distortion, strokeWidth, strokeTaper, strokeCap · 3 more presets



Animated · no input

Swirl

Bands of color twisting around a center into spirals, arcs or concentric ripples.

bandCount, twist, center, proportion, softness · 3 more presets

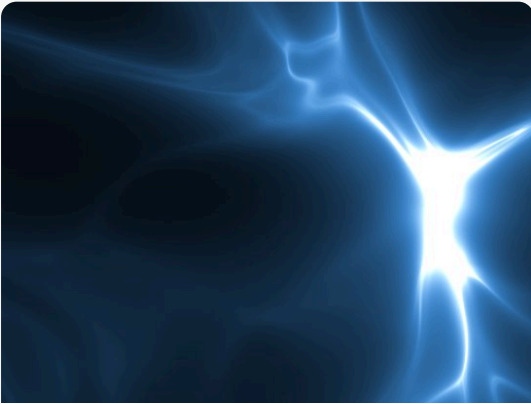


Static · no input

Waves

A still pattern of parallel lines whose shape morphs from zigzag to sine to irregular waves.

shape, frequency, amplitude, spacing, proportion · 3 more presets

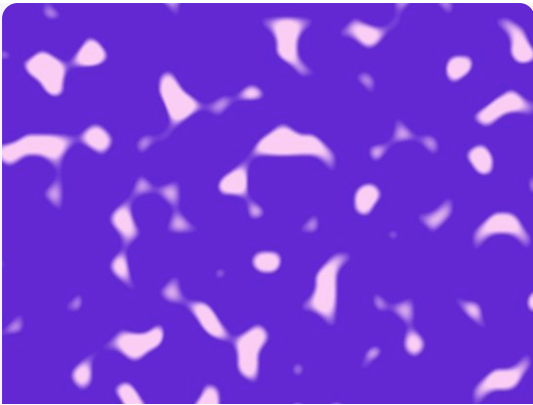


Animated · no input

Neuro Noise

A glowing web of fluid lines with bright crossing points, in three colors.

brightness, contrast · 3 more presets

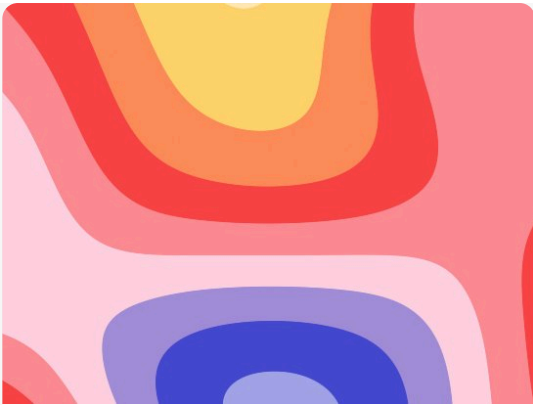


Animated · no input

Perlin Noise

Classic animated 3D Perlin noise in two colors, with octave, persistence and lacunarity controls.

proportion, softness, octaveCount, persistence, lacunarity · 3 more presets

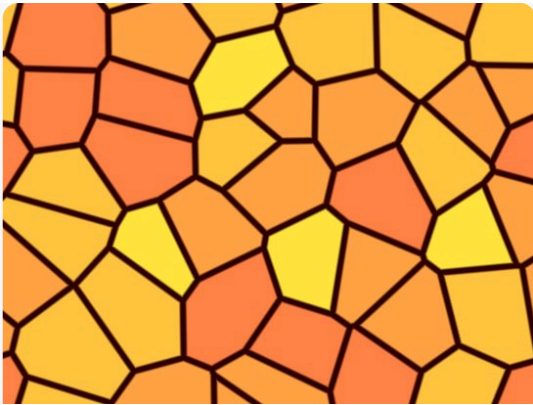


Animated · no input

Simplex Noise

Smooth animated curves from two combined simplex noises, mapped onto a multi-color palette.

stepsPerColor, softness · 3 more presets

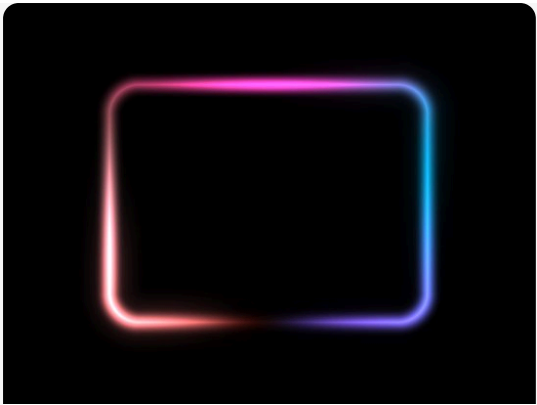


Animated · no input

Voronoi

Animated cells with drifting centers, anti-aliased gaps and an optional inner glow.

stepsPerColor, distortion, gap, glow · 3 more presets

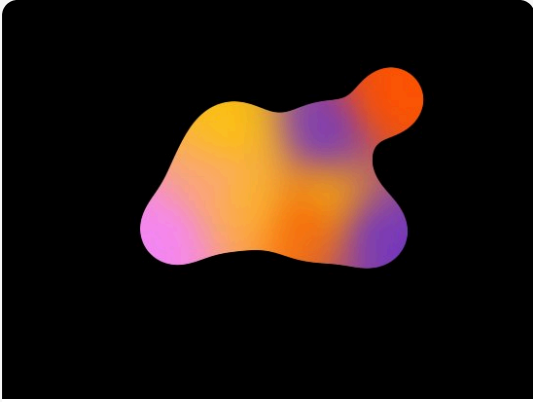


Animated · no input

Pulsing Border

Colored light trails running along a rounded border, with bloom, pulse and optional smoke.

roundness, thickness, aspectRatio, softness, intensity · 3 more presets

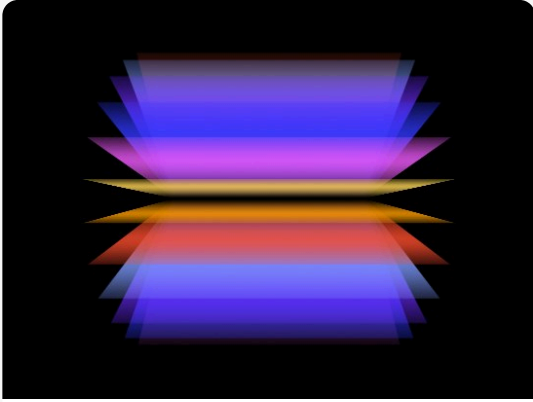


Animated · no input

Metaballs

Up to twenty colored balls circling the center and merging into gooey shapes.

count, size · 3 more presets

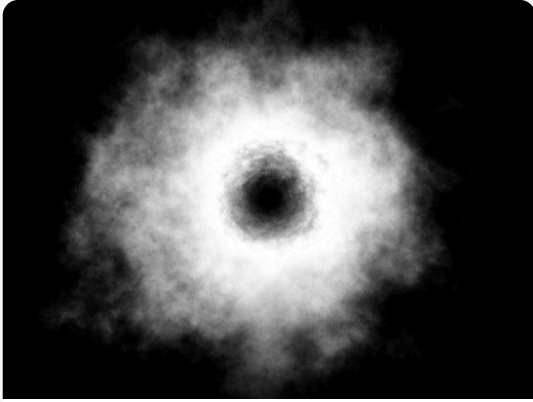


Animated · no input

Color Panels

Translucent, glowing panels rotating around a central axis in 3D.

angle1, angle2, length, edges, blur · 3 more presets

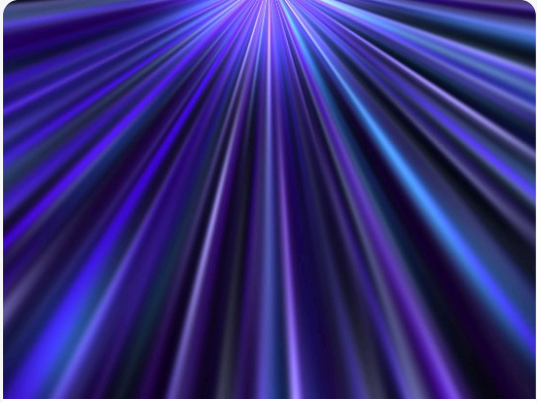


Animated · no input

Smoke Ring

A ring of color shaped by layered noise into a soft, smoky band.

noiseScale, noiseIterations, radius, thickness, innerShape · 3 more presets



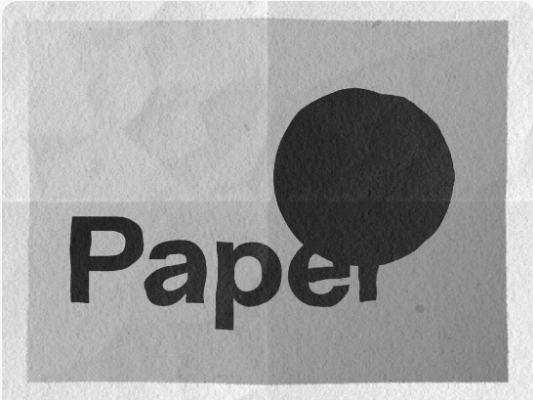
Animated · no input

God Rays

Rays of light radiating from the center in up to five colors, with bloom and a central glow.

density, spotty, midIntensity, midSize, intensity · 3 more presets

Image filters 7

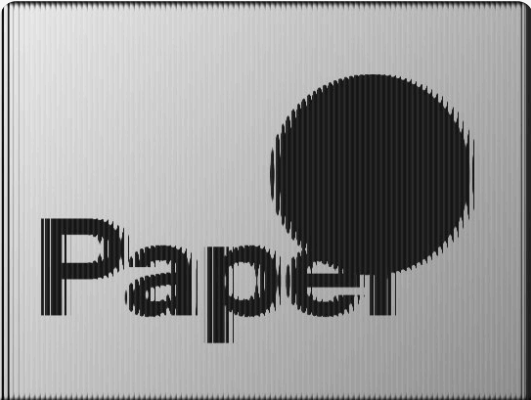


Static · image optional

Paper Texture

Builds a paper sheet with fibers, grain, folds, wrinkles, crumples and speckles, on its own or over an image that blends into and bends with the paper.

blending, distortion, clip, angle, seed · 3 more presets



Static · image

Fluted Glass

Refracts an image through a grid of ribbed glass stripes, with control over the stripe pattern, the distortion profile, blur, edge softness, shadows and highlights.

shadows, size, angle, distortionShape, highlights · 3 more presets



Animated · image optional

Water

Ripples an image with animated caustic light and wave distortion, or renders the caustic pattern on its own.

highlights, layering, edges, waves, caustic · 3 more presets

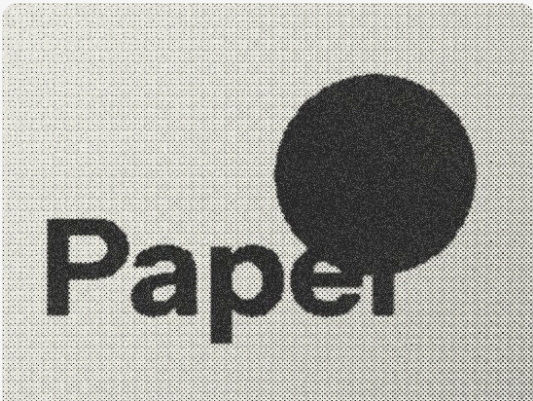


Static · image

Image Dithering

Redraws an image as a dithered pixel pattern (random or 2×2, 4×4, 8×8 Bayer) in one to seven color steps, from a custom palette or the image's own colors.

type, size, originalColors, inverted · 3 more presets

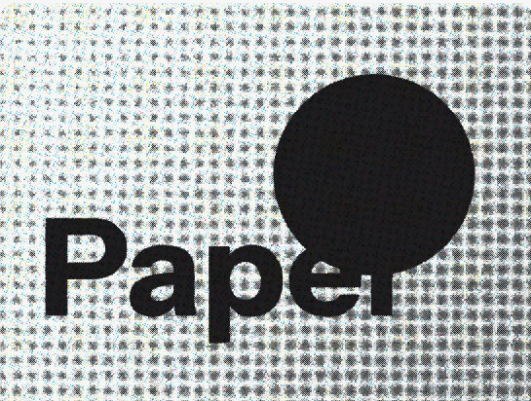


Static · image

Halftone Dots

Turns an image into a grid of dots whose size follows its tones, in classic, gooey, holes or soft styles on a square or hex grid.

size, radius, contrast, originalColors, inverted · 3 more presets



Static · image

Halftone CMYK

Simulates four-ink printing: the image is separated into cyan, magenta, yellow and black dot layers with editable ink colors, per-ink gain and flood, grid jitter and grain.

size, contrast, softness, grainSize, grainMixer · 3 more presets



Static · image

Lens Distortion

Bends an image like a lens, bulging like a fisheye or pinching like a pincushion, and fans it into color-shifted copies for chromatic aberration.

spread, bias, angle, perspective, count · 3 more presets

Logo animations 3

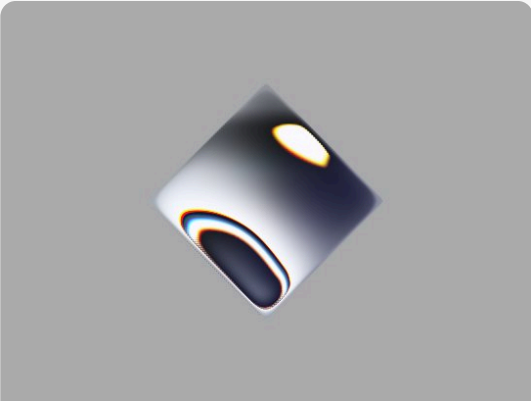


Animated · shape image

Heatmap

Sends an animated, heat-colored glow through and around a shape taken from an image, with separate glow inside and outside the shape.

contour, angle, noise, innerGlow, outerGlow · 1 more presets



Animated · shape image or built-in shape

Liquid Metal

Fills a logo or built-in shape with animated, chrome-like stripes that bend at its edges, with red and blue channel shifts.

distortion, repetition, shiftRed, shiftBlue, contour · 3 more presets



Animated · shape image or built-in shape

Gem Smoke

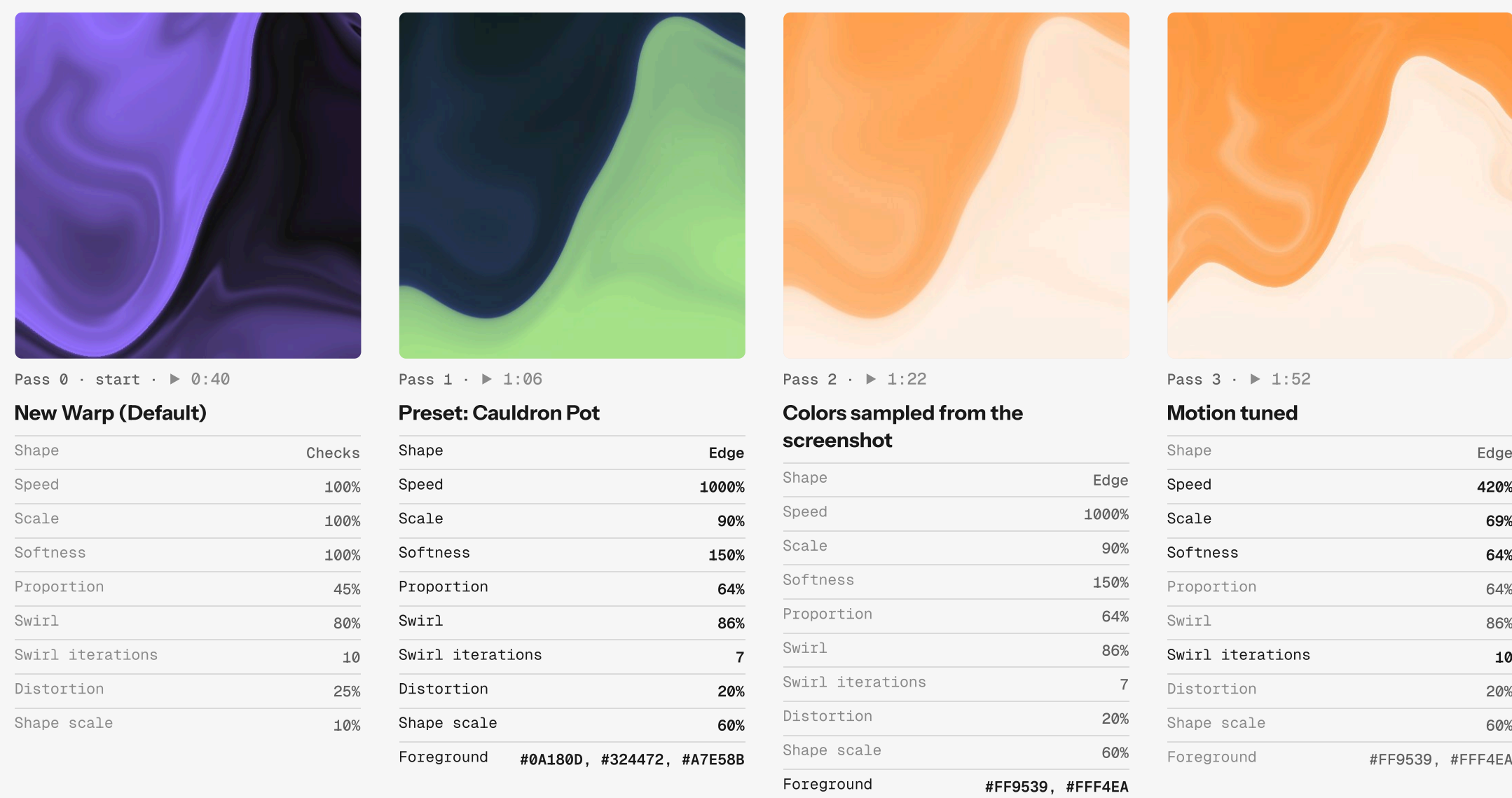
Animates colored smoke inside and around a logo or built-in shape, with separate distortion and glow inside and outside it.

outerGlow, innerGlow, innerDistortion, outerDistortion, offset · 3 more presets

Figure 09.3

Three passes to match a reference

Rebuilding a loading animation from a screenshot: preset first, then colors, then motion.



Live renders of each state with the official Warp shader. Changed values are marked in black; percentages are converted for the runtime (100% = 1).

How it works in Paper

As of September 2026, Paper’s Shaders panel (S) holds 30 shaders in three sections: 7 Image filters, 3 Logo animations and 20 Effects. Each shader is a layer with the usual Layout, Radius, Fill and Export panels, plus Presets, an Image or Shape input, Parameters and color lists. Animated shaders freeze at Speed 0%. Copy as React CSS gives a component from the open-source @paper-design/shaders-react package; @paper-design/shaders mounts the same shaders without React, as a canvas that fills its parent element.

Checked against the videos and documentation as of September 2026. Paper ships often; the release timeline in the reference shows what changed when.

01

Add a shader and tune it in order

1

Press S, or click the last button in the left toolbar (a diamond in a square; builds before late May 2026 showed a cube icon).

2

In the Shaders panel, click a tile under Image filters, Logo animations or Effects. The shader lands as its own layer named after the effect, usually at 800 × 600; resize it to its slot.

3

Click through Presets and keep the closest. A preset rewrites every parameter, so choose it before tuning.

4

Set colors next: click a swatch to open the picker (sRGB and Display P3 tabs, an eyedropper, OKLCH, HSL, RGB and hex fields) and remove extra colors with the minus button.

5

Adjust Parameters one family at a time. Dragging a slider to its maximum shows the range before you settle.

6

Set Speed last: lower it for calm motion, or set 0% for a still texture.

S

►

Shown at 0:32

02

Feed an image filter from the canvas and chain filters

1

Add an image filter such as Fluted Glass. Its Image section shows a sample (flowers.webp, 1600×1200) with an eyedropper and a trash icon.

2

Click the eyedropper (“Capture image from another element”), then click any layer: a photo, a generated image, a text layer or another shader. The input becomes a .webp named after that layer.

3

Try the presets again on the new input, then lower Distortion or Size if the effect should be subtle.

4

To chain, add a second filter, for example Halftone CMYK with the Vintage preset, and eyedrop the first shader. Its output arrives as “Fluted Glass.webp”.

►

Shown at 2:12

03

Animate a logo

1

Press S and pick Heatmap, Liquid Metal or Gem Smoke under Logo animations. The default shape is a diamond.

2

In Shape, click the eyedropper and then your logo layer, or click Choose file to upload one. Liquid Metal and Gem Smoke also have a dropdown of built-in shapes (Liquid Metal lists None, Circle, Daisy, Diamond and Metaballs).

3

Set the shader’s Fill to the ground the logo sits on (black in the demos), or remove it when the page behind already provides that ground.

4

Set Contour: 0% acts as a plain mask, low values react only at the rim, 100% fills the shape with edge-following bands.

5

Lower the glows and Noise until the mark reads, then lower Speed. Next to text, set Outer glow to 0%.

S

►

Shown at 1:06

- 04 Put a shader inside a button or section
- 1 Duplicate the component (⌘D) to keep a before and after. With nothing selected, set Page in the right panel to the product’s background so you judge the glow in context.

2 Add the shader (Pulsing Border in the demo), remove its Fill and use the color list’s eyedropper (“Pick colors from another element”) on the original.

3 Match the shape: Scale 100%, Roundness 100% for a pill, and Radius Full on the shader layer.

4 Remove the component’s own fill, then drag the shader into the outer frame rather than the content frame.

5 Because the frame uses flex, check Absolute position in Layout (under Wrap in flex), set W and H to Fill and X and Y to 0.

6 For a hard inner edge, drag the shader above the opaque content frame in the layers list so it renders behind it. On a section, turn on Clip content (⌘C) so smoke or glow stops at the edge.
- ⌘ D

⌘ C

► Shown at 2:56

- 05 Freeze, copy or export a shader
- 1 For a still, duplicate the shader and set Speed to 0% on the copy.

2 Right-click and choose Copy as React CSS (⌘R), Copy as Tailwind (⌘T) or Copy as PNG. Since September 2026 the same options also sit behind the copy icon next to Export, where Copy as PNG shows ⇧⌘C.

3 Paste the React code: it starts with the package and version (/** @paper-design/shaders-react@0.0.71 */ in March 2026), imports the component, links back to the Paper file and node, and passes each parameter as a prop.

4 Read the props as the panel’s percentages divided by 100: Speed 350% becomes speed={3.5} and Scale 247% becomes scale={2.47}.

5 For motion files, use the Video section (+) in the right panel; video export is a paid-plan feature.



► Shown at 1:15

- 06 Use the shader in code
- 1 Install the exact version named in the header, for example npm i @paper-design/shaders-react@0.0.81. Releases under 0.0.x can break, so pin it.

2 Paste the component and give it a size with width and height or CSS; it renders a canvas that fills that box.

3 Add a reduced-motion branch that renders the same props with speed 0 and a chosen frame. The package already pauses offscreen and in hidden tabs, but not for reduced motion.

4 Without React, install @paper-design/shaders and call new ShaderMount(element, warpFragmentShader, uniforms, undefined, speed) with u_-prefixed uniforms; shaders that sample noise also need getShaderNoiseTexture().

5 For large backgrounds, lower minPixelRatio (default 2) or maxPixelCount to cut GPU work; at speed 0 the animation loop stops, so a still costs nothing after its first frame.

► Shown at 1:24

Controls you will use

Control	Where	What it does
Shaders panel	Left toolbar › last button (tooltip “Shaders S”)	Opens a floating panel titled Shaders with three sections of thumbnails: Image filters, Logo animations and Effects. Click a tile to add that shader to the canvas. Paper’s CMYK halftone article also describes dragging a tile onto the canvas.
Presets	Right panel › Presets, below Blending	Named starting points; clicking one rewrites every parameter. Warp offers Default, Cauldron Pot, Live Ink, Kelp, Nectar and Passion. Choose the preset before tuning, or try presets on a duplicate.
Image / Shape	Right panel › Image (image filters) or Shape (logo animations)	Shows the input’s thumbnail, file name and pixel size, an eyedropper that captures any canvas layer (“Capture image from another element”) and a trash icon. Shape adds Choose file for uploads. Name the source layer first: the capture is saved as a .webp under that name.
Speed	Right panel › Parameters (animated shaders)	Animation rate in percent; 100% is speed 1 in code and 0% stops the animation, leaving a still texture. Static shaders (most image filters, the two static gradients, Dot Grid and Waves) have no Speed.
Scale	Right panel › Parameters	Zooms the graphic inside the layer without resizing the layer; 100% is scale 1 in code (documented range 0.01 to 4). On Heatmap, lowering Scale reveals more of the outer glow.
Contour	Right panel › Parameters (Heatmap, Liquid Metal)	Sets how far inside the shape pixels react to its edge: 0% acts as a plain mask, about 18% hugs the rim, 100% fills the shape with edge-following bands.
Inner glow / Outer glow	Right panel › Parameters (Heatmap, Gem Smoke)	Inner glow sizes the effect inside the shape’s bounds; Outer glow sizes it outside them. Set Outer glow to 0% when text sits next to the mark.
Color list (Foreground, Colors, Tint, Inner)	Right panel, below Parameters	The shader’s palette. Click a swatch to edit it, + to add a color, minus to remove one; the list’s eyedropper (“Pick colors from another element”) samples colors from any layer, including images and other shaders.

Control	Where	What it does
		The label varies: Warp and Heatmap say Foreground, Gem Smoke says Colors and Inner, Liquid Metal says Tint.
Absolute position	Right panel › Layout, under Wrap in flex (children of a flex frame)	Takes the shader out of the flex flow (CSS position: absolute) so it can sit behind content; Left and Top constraint menus and alignment buttons appear. Then set W and H to Fill and X and Y to O.
Clip content	Right panel › the parent frame’s layout options (⌘C)	Hides whatever extends past the frame, such as smoke or glow from a shader larger than its container.

Shortcuts

S	Open the Shaders panel ▶ 0:05	⌘R	Copy as React CSS (a shader becomes a @paper-design/shaders-react component) ▶ 1:15
⌘T	Copy as Tailwind ▶ 1:15	⇧⌘C	Copy as PNG ▶ 3:54
⌘D	Duplicate a shader or component, for a before and after or a still copy ▶ 1:15	⌘C	Toggle Clip content on the selected frame ▶ 7:01
⇧A	Wrap the selection, such as a pasted capture, in a flex frame ▶ 6:06	⌘V	Paste a Paper Snapshot capture onto the canvas ▶ 0:20
Return	Capture the hovered element with Paper Snapshot in Chrome (⇧ and ⇩ refine the target first) ▶ 0:20		

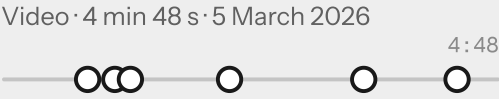
Limits and what is coming

- As of September 2026 the Paper MCP has no tool that inserts or edits a shader; presets, inputs and parameters are set by hand in the right panel.
- Video export of shaders and frames is limited to paid (Pro) plans as of September 2026, according to Paper’s support shortcuts and build log.
- Eyedropper captures are stored as .webp snapshots named after the source layer. No video shows them updating when the source changes, so re-capture after editing a source layer.
- The shader library is Paper’s own: 30 shaders as of September 2026, extended through a monthly “shader of the month”. The roadmap lists more motion effects, image filters and post-processing effects as in progress, and particle systems, Three.js islands and Lottie or Rive embeds as planned, not shipped.

What the tutorials show

Each official video, written down step by step. The dots mark the moments; they open the video there if you want to see it.

How to use Paper Shaders to elevate your designs



A tour of shaders: presets, stills, React code and chained filters

Stephen starts from a Warp shader whose panel shows Presets (Default, Cauldron Pot, Live Ink, Kelp, Nectar, Passion), Parameters (Shape: Checks, Stripes or Edge; Speed; Scale; Softness; Proportion; Swirl; Swirl iterations; Distortion; Shape scale) and a Foreground color list. Clicking a preset rewrites every value, so presets are starting points. With Speed and Softness at 0% the shader becomes a hard-edged still texture. Right-click › Copy as React CSS (⌘R) pastes a component headed `/** @paper-design/shaders-react@0.0.71 */` whose props are the panel values as decimals (Speed 350% is `speed={3.5}`); he calls it the same code that renders on the canvas. He then generates four sunset images at once (Variety pack), pulls one into Fluted Glass with the Image eyedropper, tries the Waves and Folds presets, and chains Halftone CMYK (Vintage preset) by eyedropping the glass layer, which arrives as Fluted Glass.webp. He ends with right-click › Vectorize on a flat image: an SVG he enlarges to 3558 × 3558 without visible loss.

- 0:50 Presets rewrite the values

1:06 Speed 0%: a still texture

1:15 Copy as React CSS ⌘R
- 2:12 Capture image from another element

3:30 Halftone CMYK on Fluted Glass.webp

4:24 Vectorized at 3558 × 3558

Recreate Cloudflare's loading shader from scratch with Paper Shaders

Video · 2 min 11 s · 2 May 2026

2:11

Match a reference: preset, then color, then motion

Harcourt rebuilds the loading shader Cloudflare teased on X. She pastes a screenshot of the teaser next to her work, opens the shader list (S or the toolbar button), picks Warp and makes the new 800 × 600 layer square (600 × 600). Presets come first: Default and Live Ink miss, and Cauldron Pot is closest (Shape Edge, Speed 1000%, Scale 90%, Softness 150%, Proportion 64%, Swirl 86%, Swirl iterations 7). Colors come second: she opens a Foreground swatch, uses the picker’s eyedropper on the darkest orange and the lightest cream in the screenshot (FF9539 and FFF4EA) and deletes the leftover blue. Motion comes last: Speed down to 420%, Scale 69%, Softness 64%; she notes that higher Softness blends more and hides the light cream. Her “more swirl” shows on screen as Swirl iterations going from 7 to 10 while Swirl stays at 86%. She calls the match done “in a matter of seconds”; it is a close approximation, not a pixel copy.

0:32 S opens the shader list

0:42 Make the layer square

1:06 Cauldron Pot values

1:13 Sample colors from the screenshot

1:38 Softness hides the light color

1:52 Final match

Remix Resend's action button with Paper Snapshot and Paper Shaders

Video · 4 min 30 s · 28 May 2026

4:30

A pulsing border inside a captured button

Harcourt captures Resend’s “Announcing Resend Forward” pill with Paper Snapshot (hover, refine with the arrow keys, press Return) and pastes it with ⌘V: a button frame, a Rectangle holding the hover state (she hides it) and a Content frame with the text and an SVG chevron. She duplicates it for a before and after, adds Pulsing Border from Effects, sets the page color to black and removes the shader’s Fill. The Foreground eyedropper (“Pick colors from another element”) pulls the original border colors; she deletes the black and one blue. Scale 100%, Roundness 100% and Radius Full match the pill; by eye she sets Thickness 38%, Intensity 33% and Softness 54%. She removes the button’s fill, drags the shader into the outer frame, checks Absolute position, sets W and H to Fill and X and Y to 0. For a thinner ring she either lowers Thickness or drags the shader above Content in the layers list, which renders it behind Content’s black fill with hard inner and outer edges.

0:20 Paste the Snapshot capture

1:14 Page color to black

1:35 Pick colors from another element

2:16 Radius Full

3:17 Absolute, Fill × Fill, X and Y 0

3:46 Shader behind Content

How to use logo animation shaders in Paper

Video · 9 min 05 s · 7 July 2026

9:05

Logo animations, tuned in a real footer

After announcing that Paper Shaders are now open source and free to use anywhere, Harcourt covers the three Logo animations. In Heatmap she eyedrops the Paper logo into Shape (it becomes Paper Logo.webp), adds Noise for grain, sets Angle 45°, trims the colors to grays and white on black and lowers Scale to reveal the outer glow, ending at Noise 27%, Inner glow 28%, Outer glow 27%, Speed 49% and Scale 53%. Liquid Metal on the Vercel logo explains Contour (she keeps 40%), then gets a black Fill, Repetition back at 200% after 1000% overwhelms, Distortion 7%, Softness 47%, Angle 180° and a darker Tint (5E5E5E). Gem Smoke goes into Resend’s footer, captured with Snapshot, pasted (⌘V), wrapped in flex (⌘A) and given a black fill and side padding; she drags the shader in (Absolute position), removes its Fill and turns on Clip content (⌘C). Light gray smoke, a dark Inner color, Outer glow 0%, Inner glow 42%, Size 94%, Offset -20% and Angle 115° keep the links legible.

0:02 Open source and free

1:22 Paper Logo.webp as the shape

3:01 Heatmap settled

5:46 Liquid Metal values

7:01 Clip content on the footer

8:20 A faint Gem Smoke

Understanding contour in Paper's logo animation shaders

Short · 56 s · 9 July 2026

0:56

Contour at 18%, 0% and 100%

A recut of the Liquid Metal segment, on the Vercel triangle. Every pixel of a logo animation knows its distance to the shape’s edge, and Contour sets how far in that distance matters. At 18% only pixels near the rim react. At 0% nothing reacts to the edge, which looks like an ordinary shader masked by the shape. At 100% pixels deep inside follow the outline too, giving the strongest edge-following liquid bands. The full tutorial settles on 40%.

0:12 Distance to the edge

0:24 Contour 18%

0:32 Contour 0%: a plain mask

0:46 Contour 100%

Apply a Halftone Shader to Text

Short · 55 s · 20 July 2026

0:55

Halftone Dots on a single letter

She clicks the Shaders button (tooltip “Shaders S”), adds Halftone Dots and, with it selected, uses the eyedropper in its Image section to grab a large black S from the canvas in place of the sample flowers. She keeps Type on Gooley (Classic, Gooley, Holes and Soft are the options), raises Scale from 88% to 100% so the letter fits, then explores Size on the Hex grid: at 99–100% the dots merge and the S melts into a blob. She settles on Size 88%, Radius 112% and Contrast 37% for a mostly filled letter, then recolors Foreground from 2B2B2B to ECB437 and the Fill to olive.

0:05 Shaders S tooltip

0:10 Eyedrop the S

0:14 Type: Gooley

0:26 Size 100%: the S melts

0:40 Size 88%, Radius 112%, Contrast 37%

0:46 Yellow on olive

Paper shader of the month: Lens Distortion

Short · 35 s · 21 August 2026 0:35

Lens Distortion on a photo and on headlines

August 2026’s shader of the month is Lens Distortion, a new image filter. Added from Image filters, it shows Presets (Default, Fisheye, Duotone, Smoke), an Image input with the sample flowers and parameters starting with Fit, Spread 60% and Bias 100%. As with other filters, the eyedropper pulls in the content to distort. On a sky photo she moves Focus edges, Swirl (35% to 0%), Lens bulge into negative values (about -66% to -74%) and Scale (140%); below sit Lens circle, Position X, Position Y and a Grain section. Five copies of one headline then carry different settings, showing that any layer, text included, can be the input; the strongest settings smear the words past reading.

- 0:04 Image filters: Lens Distortion
- 0:06 Presets and Image input
- 0:12 Focus, Swirl and Noise
- 0:16 Lens bulge -74%, Scale 140%
- 0:20 Five headline variants

Build it in Paper

- 01 Place the image or logo you will treat beside an unchanged copy. Press S, pick the family that fits (an image filter for a picture, a logo animation for a mark, an effect for a generated field) and use the eyedropper in its Image or Shape section to capture your layer.
- 02 Click the nearest preset first, then set colors with the color list’s eyedropper or typed values, then adjust one parameter family at a time, leaving Speed for last.
- 03 Drag the shader into its frame, check Absolute position, set W and H to Fill and X and Y to 0, remove the shader’s Fill if the page should show through, and turn on Clip content (⌘C) on the parent.
- 04 Watch the nearby text through a full cycle and lower Outer glow, Intensity or Speed until it stays legible. Duplicate the shader (⌘D) and set Speed to 0% on the copy for the still and reduced-motion state.
- 05 Copy as React CSS (⌘R) and keep the header’s package version, the preset, the input image and the final size next to the shader.

Common mistakes

- × **Tuning a shader, then clicking another preset to compare and losing every value.**
 - Presets rewrite all parameters. Duplicate the shader first (⌘D) and try presets on the copy, or choose the preset before you tune.
- × **Dropping a shader into a flex button or section, where it lines up beside the content instead of behind it.**
 - Select the shader, check Absolute position in Layout, set W and H to Fill and X and Y to 0, then order it behind the content frame in the layers list.
- × **Deciding a slider is broken because moving it changes nothing.**
 - Check its prerequisite in the effect’s reference: Lens Distortion’s color controls need Spread above 0, Waves’ frequency needs amplitude above 0, and Paper Texture’s fold sizes need Folds above 0.
- × **Upgrading @paper-design/shaders and shipping a preset that now looks different.**
 - Pin the version from the copied header, read the changelog and migration guide (Paper Texture changed in 0.0.81), and re-approve the still before release.

Check before you move on

- ☐ Can you name the one job this effect does, and would a still at Speed 0% do it as well?
- ☐ Does every nearby line of text stay legible through a full animation cycle, at the size people will see it?
- ☐ Is the shader absolutely positioned at Fill × Fill inside its frame, with Clip content on wherever smoke or glow could spill?
- ☐ Have you saved the shader’s name, preset, parameter values, colors, input image, size and the package version from the Copy as React CSS header?
- ☐ Is there a defined still frame for reduced motion, and does no decorative effect sit under a chart, number or form field?

5 chapters, 5 live studies
21 source videos, 75 minutes, written down

10	MCP: connect the agent to a specific design context	11	Parallel AI agents: keep one coherent design	12	Repeat the rule, then vary the content	13	Carry the design's intent into working code	14	Publish an artifact that survives leaving the canvas
----	---	----	--	----	--	----	---	----	--

MCP: connect the agent to a specific design context

MCP makes Paper accessible to an agent. Reliable work comes from giving that connection an exact file, a bounded change and an observable result.

The connection is the beginning of the workflow

MCP (Model Context Protocol) is the connection through which an agent calls Paper’s design operations. In Paper, the server runs locally inside Paper Desktop: opening the app starts it, and it reads and writes the files open there, including files in background tabs. Paper in a browser does not provide this local server, and the desktop dashboard alone is not enough; the agent needs Paper Desktop running with a design file open. The connection does not replace design judgment or decide which file should change. An agent still needs to understand the material, locate the relevant frame and distinguish a requested edit from surrounding content that must remain stable.

As of September 2026, the quickest setup starts inside Paper: with nothing selected, the right panel’s MCP section has a Connect your agent button that adds Paper to a chosen client in one click. Paper’s documentation names the Paper CLI as the current route for most clients: the client launches it with the argument `mcp` , and it relays to the desktop app. Spring 2026 tutorials register a local HTTP address instead, `http://127.0.0.1:29979/mcp` , which Paper Desktop still serves while a file is open. Use one route per client rather than mixing configuration fragments from different versions, and restart the agent session after any change, since a running session does not load new tools. A successful connection establishes access; it does not establish that the agent has identified the correct design context.

Connection grants access; an exact target and a reviewable change make the work reliable.

The principle of chapter 10

Target, read, change, verify

Choose a target, inspect it, apply one change, then verify.

SIMULATION · NO PAPER CONNECTION OR EDITS

Align the plan values

Target: Pricing site → Comparisons → Plan values. Keep labels and values unchanged.

Requested target

Pricing site / Comparisons / Plan values

BEFORE · VALUES FOLLOW LABEL LENGTH

Starter plan 24.00

Team plan 37.50

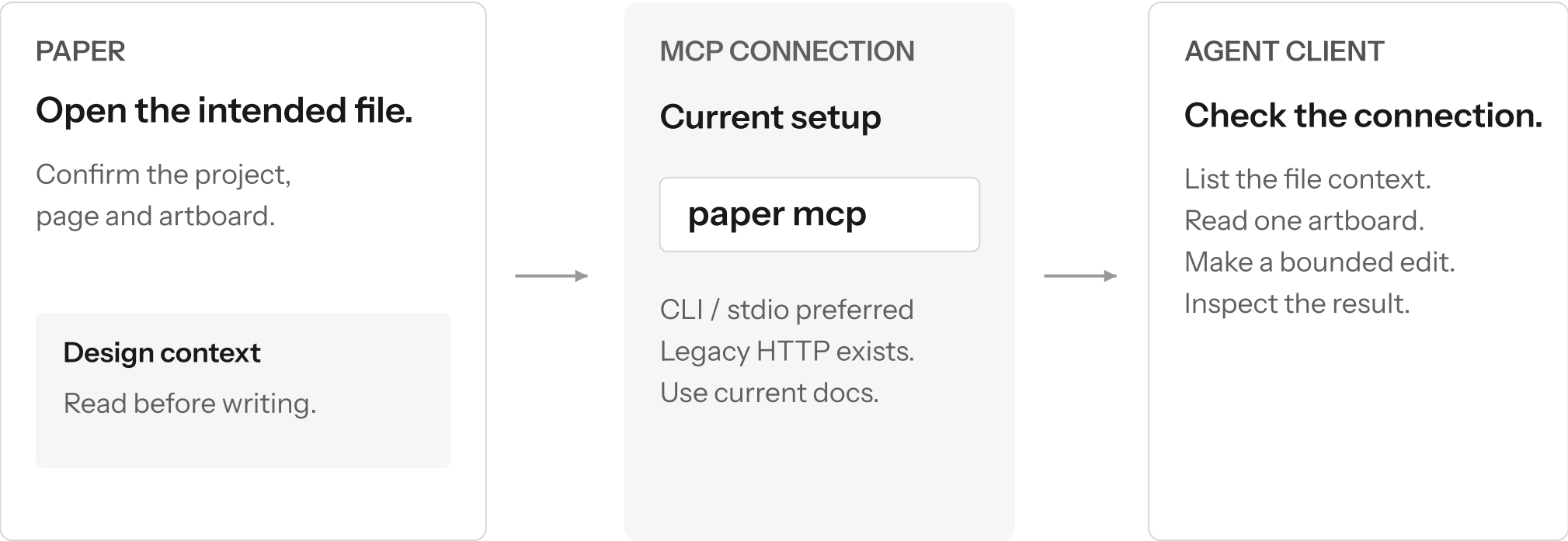
Business plan 52.00

The change belongs to each row: let the label occupy available space and reserve the same width for every value. The data stays unchanged.

10 / PAPER IN PRACTICE

Connect the tool, then confirm the context.

Use current setup instructions; historical videos can show a different transport.



Connection setup enables tool access. It does not establish correct scope, complete design or release.

Reconstructed teaching example · synthetic values · not a Paper screenshot

Figure 10.1 · Plate

Connect the tool, then confirm the context.

Current official Paper setup prefers the CLI stdio connection, while older videos demonstrate local HTTP. Read the intended file and artboard context before a bounded edit, then inspect its rendered result.

Read before writing

Make the result inspectable

A useful first request asks the agent to identify the target file and describe the selected frame’s structure without editing it. One of Paper’s tutorials opens with a question as plain as “Are we connected to the Paper MCP?”, and the agent answers with the open file, page, artboard and its size, node count and fonts, read through tools such as `get_basic_info` and `get_selection`. That small read establishes shared context. If the description refers to a different page or misses the intended component, correct the target before requesting a redesign.

Since August 2026, Paper Desktop opens files as tabs, and agents can work in any open file, including tabs in the background. That makes explicit targeting more important. Name the file (“In the Learn Section file in Paper, ...”) or paste its link from Copy link (⌘L); select a frame before pressing ⌘L to link that frame instead. “The active Paper file” means the tab in front when the agent starts, so stay on it until the agent is working. A good task names the file, page or frame and the outcome in one sentence: “In the comparison frame of this file, align the existing values without changing the labels.” That sentence is a briefing pattern, not a special command syntax.

Comments make a target even more precise. Press C, click a layer and write the request; the comment stays pinned to that layer, so a short prompt such as “I left comments on this frame. Check it out and implement those changes.” gives the agent both the list and the locations. The agent can list and read comment threads and mark them resolved, but it cannot write comments; that part stays with you.

After reading, the agent can make a bounded structural or style change. Good write scope describes the smallest coherent unit that owns the problem: adjusting a row’s layout is clearer than requesting a whole-screen redesign to repair one column. For bigger transformations, Paper’s tutorials duplicate the frame first and have the agent change the copy. When the agent brings in content or values from elsewhere, such as a document, a Slack canvas or the codebase’s CSS variables, name the source and say whether the values are real or placeholders.

Ask for the resulting frame to be inspected after the change. A tool response reports that an operation happened; the rendered frame shows whether it produced the intended design. In Paper’s Claude Code tutorial, the agent finishes by taking a screenshot of its artboard and running a review checkpoint on spacing, typography, contrast, alignment, clipping and repetition before it summarizes. Ask for the same, plus an account of anything missing or unresolved, rather than treating successful execution as proof of visual quality.

Paper shows where an agent is working: animated dots replace the file icon on its tab, a dotted pattern runs above the artboard it is editing, and its avatar joins the presence row at the top of the right panel. The agent removes the artboard marker when it finishes (the `finish_working_on_nodes` tool). Check the frame yourself, then close the loop: have the agent resolve the comment threads it fully addressed and leave the rest open with a reason.

The agent does not track edits you make by hand. Paper’s advice is to have it re-read the canvas before the next request. A useful handoff names the target, the changed properties and the evidence used to judge them, which gives the next collaborator, or the next session, a clear starting point. The loop also works away from the desk: with Remote Control in Claude Code, the screenshots the agent sends back are how you review from a phone.

Figure 10.2

The 36 tools an agent can call

Grouped by job. Read before you write; take a screenshot to verify.

Files and pages7

Find, open and organize the document before touching it.

list_resourcesopen_filecreate_file

get_basic_infocreate_pagerename_pages

rename_resource

Read the design10

Inspect structure, styles, text, images and fonts; take a screenshot to check.

get_selectionget_tree_summary

get_childrenget_node_infofind_nodes

get_jsxget_computed_styles

get_fill_imageget_font_family_info

get_screenshot

Write and edit9

Add HTML as editable layers, then adjust styles, text and order.

create_artboardwrite_htmlupdate_styles

set_text_contentduplicate_nodes

move_nodesrename_nodedelete_nodes

finish_working_on_nodes

Tokens3

Read the file's tokens, create new ones, or change their values.

get_tokenscreate_tokensset_tokens

Comments4

Read threads left on the canvas and resolve them when the work is done.

list_comment_threadsget_comment_thread

list_comment_thread_authors

set_comment_thread_status

Export and guidance3

Export frames, combine PDFs, and load the server's own instructions.

exportexport_combined_pdfget_guide

Figure 10.3

Target, read, change, verify, hand off

The routine every agent pass in the tutorials follows, with the tools behind each step.

01

Target

You

Name the file, paste a Copy link (⌘L) link, or select the frame

Agent calls

open_filelist_resources

You see

The agent opens the linked file first

→

02

Read

Agent calls

get_guideget_basic_infoget_selectionget_tree_summaryget_jsxlist_comment_threads

You see

The agent states file, page, artboard and size, node count and fonts

→

03

Plan

You see

A design brief: palette with hex values, type scale, spacing, direction

→

04

Change

Agent calls

write_htmlupdate_stylesset_text_contentduplicate_nodes

You see

Dots on the tab and above the artboard; the agent's avatar in the presence row

→

05

Verify

Agent calls

get_screenshot

It checks

Spacing, Typography, Contrast, Alignment, Clipping, Repetition

→

06

Hand off

Agent calls

set_comment_thread_statusfinish_working_on_nodes

You see

A summary of what changed and what is unresolved; resolved threads; the artboard marker disappears

🔗 Hand edits by you → Read. The agent does not track manual edits (Paper's tip shown in ▶ 1:48)

Figure 10.4

Setup routes by client

Pick one route per client; do not mix configurations from different dates.

Client	Route	Command or place	Verify	Source
Any listed client	One click from Paper (September 2026)	Right panel › MCP › Connect your agent › choose Cursor, Claude, Claude Code, GitHub Copilot, VS Code, OpenCode, Codex, Codex CLI, Antigravity or Other agents › click the step 2 button	Restart the client, run a test prompt	► 0:56
Claude Code	Plugin (docs)	claude plugin marketplace add paper-design/agent-plugins && claude plugin install paper-desktop@paper	Restart, then /mcp	paper.design/docs/mcp
Claude Code	HTTP (May 2026 video)	claude mcp add paper --transport http http://127.0.0.1:29979/mcp --scope user	Restart, then /mcp or claude mcp list	► 1:05
Cursor	Plugin	Cursor Marketplace › Paper › Get, or /add-plugin paper-desktop in an agent chat	Cursor Settings › Tools & MCP; reload or toggle if stale	► 0:12
Claude (desktop app)	Extension (docs)	Paper's .mcpb extension, installed from Settings › Extensions › Advanced settings › Install extension	Paper tools available in a chat	paper.design/docs/mcp
VS Code	Manual stdio (docs)	.vscode/mcp.json: {"servers":{"paper":{"type":"stdio","command":"/path/to/paper","args":["mcp"]}}}	Server listed in VS Code	paper.design/docs/mcp
Codex CLI	Command (docs)	codex mcp add paper -- /path/to/paper mcp; or codex plugin marketplace add paper-design/agent-plugins, then codex plugin install paper-desktop@paper	Server listed in Codex	paper.design/docs/mcp
Codex in the ChatGPT app	Manual (docs)	Settings › Plugins › Add › Add MCP server: name Paper, command = Paper CLI path, arguments mcp	Paper listed under plugins	paper.design/docs/mcp
GitHub Copilot	One click or manual	Add to GitHub Copilot in Paper's dialog, or Customize › Add › MCP Server (command = Paper CLI path, args mcp); then switch Paper on under Customize › MCP › Installed	Paper toggle on	► 1:12
OpenCode	Manual (docs)	opencode.json: {"mcp":{"paper":{"type":"local","command":["/path/to/paper","mcp"],"enabled":true}}}	opencode mcp list	paper.design/docs/mcp
Antigravity and other stdio clients	Manual (docs)	{"mcpServers":{"paper":{"type":"stdio","command":"/path/to/paper","args":["mcp"]}}}	Server listed in the client	paper.design/docs/mcp

/path/to/paper stands for the Paper CLI path shown in Paper Desktop. Docs read in early October 2026.

How it works in Paper

As of September 2026, Paper’s MCP server runs locally inside Paper Desktop and works on the files open there, including background tabs. You connect a client with the one-click Connect your agent dialog, the Paper CLI, or the older local HTTP address. Its tools (36 in an installed client, October 2026) read files, pages, selections, structure, styles, tokens and comments, write HTML and styles, export, and capture screenshots. Comments pinned to layers become instructions, and Remote Control in Claude Code lets you steer a desktop session from a phone.

Checked against the videos and documentation as of September 2026. Paper ships often; the release timeline in the reference shows what changed when.

01

Connect an agent from inside Paper

- Open Paper Desktop on a design file. With nothing selected, click Connect your agent in the right panel’s MCP section.
- In the dialog’s left column, choose your client: Cursor, Claude, Claude Code, GitHub Copilot, VS Code, OpenCode, Codex, Codex CLI, Antigravity or Other agents.
- Step 1, Install Paper Desktop, is already checked. In step 2, click the client’s button, for example Add to GitHub Copilot (“Add Paper to GitHub Copilot as an MCP server”) or Add to Cursor (“Add through the Cursor Marketplace”).
- A toast confirms it (“Added to GitHub Copilot”) and tells you to restart the client. Restart it.
- Check that Paper is switched on in the client. In GitHub Copilot, open Customize › MCP and turn on Paper under Installed.
- Test with one of the dialog’s prompts: “Create a basic Hello World frame in Paper”, “Create 3 color tokens and make a swatch in Paper” or “Generate an image of a beautiful sunset in Paper”.
- For a manual setup, expand Or, install manually. Paper’s docs describe the generic setup as a stdio server whose command is the Paper CLI path from Paper Desktop, with the single argument `mcp`.

►

Shown at 0:56

02

Connect Claude Code from the terminal

- Install Claude Code if needed (the video uses `curl -fsSL https://claude.ai/install.sh | bash`), run `claude` and accept the folder trust check.
- Add Paper. Paper’s docs (early October 2026) use the plugin: `claude plugin marketplace add paper-design/agent-plugins`, then `claude plugin install paper-desktop@paper`.
- The May 2026 video registers the HTTP server instead, `claude mcp add paper --transport http http://127.0.0.1:29979/mcp --scope user`, which Claude Code saves to the user config in `~/.claude.json`.
- Restart Claude Code. A server added inside a running session does not load its tools; Claude says so and suggests checking with `claude mcp list`.
- Open Paper Desktop on a design file (not Paper in a browser, and not only the dashboard), then run `/mcp` and look for paper marked connected.
- Start prompts with “In Paper” so the agent builds on the canvas, for example “In Paper, design a basic social media home page”.

►

Shown at 1:05

03

Install the Paper plugin in Cursor

- Open Cursor Settings (⇧⌘J), go to the plugin marketplace, search for Paper and click Get.
- Or paste `/add-plugin paper-desktop` into a New Agent chat and pick Paper in the Add Plugin popover.
- On the card (Paper · Verified by Cursor), choose Add for Myself or Add to Project, then click Add Plugin.
- Check that the paper server is listed in Cursor Settings › Tools & MCP. If the agent cannot see it, reload the Cursor window or toggle the server off and on.
- Keep Paper Desktop running with a design file open whenever the agent works.

⌘

↑

J

►

Shown at 0:24

04 **Point each agent at the right file, page and frame**

- 1 Open every file the agents need as a tab; the + at the end of the tab bar opens a new or existing file.
- 2 Start each prompt with its file: “In the Learn Section file in Paper, create 5 variations ...”.
- 3 Or paste the file’s link instead of its name: click Copy link (⌘L) at the top of the right panel and paste it into the prompt. Given a link, the agent loads the Paper guide and opens that file (`open_file`) before anything else.
- 4 To point at one frame, select it, press ⌘L and paste the link, or say “the selected frame”, since the agent can read your selection.
- 5 Write “in the active Paper file” only if you stay on that tab until the agent has started; after that it keeps its file and you can switch tabs.
- 6 Name the page when the work spans pages. In the demo, one agent read the token frames on a Tokens page and applied them to a frame on the Design page.
- 7 Watch the tab bar: a tab with an agent working in it shows animated dots instead of its file icon, even in the background.



► Shown at 0:42

05 **Run a read, change, verify pass from comments**

- 1 Press C (or click the comment button in the left toolbar), click the exact layer to change, type the request in Leave a comment and click Send. Keep one request per comment.
- 2 Tell the agent where the comments are, with the frame’s link if several files are open: “I left comments on this frame. Check it out and implement those changes.”
- 3 Ask it to read first: list the open threads on that frame (`list_comment_threads` , `get_comment_thread`) and describe the frame’s structure before editing.
- 4 Have it change only the commented layers, editing the existing nodes rather than rebuilding the frame.
- 5 Ask for a screenshot of the changed frame and one line per thread: what changed, or why it could not be done.
- 6 Have it resolve only the threads it fully addressed (`set_comment_thread_status`), then open the comments list in the right panel and use the resolved toggle to see what is still open.



► Shown at 0:08

06 **Continue a Claude Code session from your phone**

- 1 On the computer, open Paper Desktop on the file and start Claude Code in a terminal. In July 2026, Remote Control had to start from the terminal, not from the Claude desktop app.
- 2 Inside the session, type `/remote-control` or `/rc` . Your phone shows a notification that the session is ready to control.
- 3 Keep the computer awake: in System Settings › Lock Screen, set Turn display off on battery when inactive and Turn display off on power adapter when inactive to For 3 hours, as in the video.
- 4 In the Claude mobile app, open the Code tab and tap the session marked Connected under Sessions; the whole conversation is there.
- 5 Type or dictate prompts that name the frame, the source of any content and where the result goes, such as a duplicated frame.
- 6 Review the screenshots the agent sends back in the chat; tap one to see it full screen.

► Shown at 0:11

Controls you will use

Control	Where	What it does
Connect your agent	Right panel › MCP (shown when nothing is selected)	Opens the setup dialog: pick a client, add Paper with that client’s button or expand Or, install manually, then test with one of three example prompts and click Done. Added in September 2026.
Copy link (⌘L)	Top of the right panel; tab and layer right-click menus	Copies a link to the file, or to the selected frame. Pasted into a prompt, it tells the agent exactly which file and frame to open. A toast confirms “Link copied to clipboard”.
Tabs	Top of the Paper Desktop window	One tab per open file after the fixed Dashboard tab; + opens a new or existing file. Agents can work in any open tab, including background ones. Double-click a tab to rename it; right-click for Reload, Copy Link, Rename, Open in Browser, Move to New Window, Close Tab, Close Other Tabs, Close Tabs to the Right and Close All Tabs.
Agent working indicators	Tab bar, canvas and the presence row at the top of the right panel	Animated dots replace a tab’s file icon while an agent works in that file, a dotted pattern runs above the artboard being edited, and the agent’s avatar joins the people in the presence row. The agent clears the artboard marker by calling <code>finish_working_on_nodes</code> when it is done.
Comment (C)	Left toolbar	Enters comment mode. Click any layer to pin a comment (Leave a comment, Send); threads take replies, emoji reactions and a resolve check at the top.
Comments list	Right panel, in comment mode only	Lists every comment in the file with Search, a sort menu (Newest first, Oldest first, By page), Anyone and Any page filters, and a toggle between unresolved and resolved threads.
MCP list in your client	In the agent app: <code>/mcp</code> in Claude Code, Cursor Settings › Tools & MCP, Customize › MCP in GitHub Copilot	Shows whether the paper server is installed, connected and switched on. If the tools are missing, restart the client or toggle the server off and on.

Shortcuts

C	Enter comment mode, then click a layer to pin a comment ▶ 0:06	⌘ L	Copy a link to the file, or to the selected frame, to paste into a prompt ▶ 2:00
Double-click	Rename a tab (double-click the tab) ▶ 2:21	Right-click	Open a tab's menu: Reload, Copy Link, Rename, Open in Browser, Move to New Window and the Close commands ▶ 2:27
⌘ Drag	Duplicate a frame, so an agent can transform the copy and leave the original intact ▶ 4:40		

Limits and what is coming

- As of September 2026, the MCP server runs only inside Paper Desktop with the target file open. Paper in a browser has no local server (its WebMCP support, added in September 2026, is for in-browser agents on partner sites). Remote MCP, for cloud agents without the desktop app, was announced as coming next on 28 September 2026.
- The tool set (36 tools in an installed client, October 2026) can list and read comment threads and set them to resolved or open, but it cannot create comments, delete pages or files, or add shaders.
- Remote Control, as shown in July 2026, starts only from a Claude Code session in a terminal and needs the computer awake with Paper open for the whole session. There is no Paper mobile app.
- Paper's pricing page in early October 2026 lists 100 MCP tool calls per week on the Free plan and 1 million on Pro. Image generation through the MCP also draws on your image allowance.

What the tutorials show

Each official video, written down step by step. The dots mark the moments; they open the video there if you want to see it.

Agent set up: How to connect Claude Code to Paper using the Paper MCP server

Video · 4 min 50 s · 2 May 2026

4:50

Connect Claude Code and let it design a first screen

As of May 2026. Starting from an empty Paper Desktop file, Paper's MCP docs and a terminal, the presenter installs Claude Code, runs `claude` and accepts the folder trust check. She pastes the docs' command, `claude mcp add paper --transport http http://127.0.0.1:29979/mcp --scope user`, into Claude Code, which saves the server to `~/ .claude.json`. Her first prompt, "In Paper, design a basic social media home page", fails because tools do not load in the session that added the server; Claude suggests restarting and checking with `claude mcp list`. In a new session the prompt works. The agent reads the file, posts a design brief (off-white #F7F5F2, terracotta accent #C4623A, Inter 700/500/400, 24/16/8 px gaps), creates a "Social Home" artboard and builds it in sections while a dot pattern marks it. It then screenshots the artboard, checks spacing, typography, contrast, alignment, clipping and repetition, and summarizes. Paper's docs now favor the plugin or the Paper CLI over this HTTP command.

- 0:24 Install Claude Code
- 1:12 Add the paper server
- 2:04 Restart needed
- 2:34 Design brief
- 2:48 Dots mark the artboard
- 3:48 Screenshot review

Agent set up: How to connect Cursor to Paper using the Paper MCP server

Video · 1 min 39 s · 11 May 2026

1:39

Add the Paper plugin to Cursor, two ways

As of May 2026. The Connect to Cursor section of Paper’s docs offers one command, `/add-plugin paper-desktop`, plus a link to the Cursor Marketplace. Route one: open Cursor Settings (⇧⌘J), browse the plugin marketplace, search for Paper and click Get. Route two: paste the command into a New Agent chat, pick Paper in the Add Plugin popover, then on the card (Paper · Verified by Cursor) choose Add for Myself rather than Add to Project and confirm Add Plugin. Cursor’s agent then lists what the plugin added at the time: the `paper` MCP server at `http://127.0.0.1:29979/mcp`, two skills (`code-to-design` and `design-to-code`) and an auto-applied rule, `ensure-paper-started`; it also checks that the endpoint responds. The docs add that the server appears in Cursor Settings › Tools & MCP and that you may need to reload the window or toggle the server. Cursor and the presenter both repeat the one requirement: Paper Desktop open with a design file.

0:12 Connect to Cursor in the docs

0:26 Cursor Settings, ⇧⌘J

0:42 Paste /add-plugin paper-desktop

0:50 Add for Myself, Add Plugin

1:04 What the plugin adds

1:08 Keep Paper Desktop open

Using tabs to run agents across multiple Paper files at once

Video · 2 min 57 s · 10 August 2026

2:57

Three agents, three tabs, three ways to target

As of August 2026. With Learn Section, Instagram Grid and Finance open as tabs, the presenter runs a Conductor agent in each. The first prompt names the file: “in the Learn Section file in Paper, create 5 variations ...”. Animated dots replace that tab’s icon and stay while she works elsewhere, and a robot avatar joins the presence row. The second prompt carries the file’s link from Copy link (⌘L); that agent loads the Paper guide and calls `open_file` first. For the third, she duplicates “Finance UI — Dashboard” as “Finance UI — Dashboard Dark” and asks to apply the dark design tokens to it “in the active Paper file”, staying on the Finance tab until it starts. That agent reads the token frames on the Tokens page and applies them on the Design page; it notices that a `#000000` search matched zero values and that its color searches missed SVG icon strokes, so it reads the sidebar and top bar with `get_jsx`. Double-click renames a tab, and right-click opens its menu.

0:42 Name the file in the prompt

0:54 Dots on the working tab

1:14 Copy link, then paste it

1:36 “In the active Paper file”

2:13 Agent checks its color search

2:27 Tab menu

Tabs, now in Paper

Short · 33 s · 11 August 2026

0:33

Tabs announced: agents in many files at once

As of 11 August 2026. A 33-second animation with captions and music, without narration. It shows the Paper Desktop tab bar: a fixed Dashboard tab, one tab per open file and a + at the end to open another. While an agent works in a file, its tab swaps the file icon for animated dots, and many tabs can show dots at once. A Claude Code panel appears beside wireframe cards that carry dotted working markers, presence avatars pair people with agents, and frames such as Landing A and B or Dashboard A and B are shown mid-build. The captions state the intent: get more done, let your agents work across different files at the same time, stay in flow and keep exploring.

0:02 Tab bar

0:06 + opens another tab

0:08 Dots on a working tab

0:10 Agents across files

0:20 People and agents present

0:24 Dashboard A and B

Use Claude's Remote Control to design in Paper from your phone

Video · 3 min 00 s · 31 July 2026

3:00

Keep designing from a phone with Remote Control

As of July 2026. Paper has no mobile app, so the presenter steers a desktop session instead. In a Claude Code session that already uses the Paper MCP (it had to be the terminal, not the Claude desktop app), she types `/remote-control` (or `/rc`), and her phone shows that the session is ready. She sets System Settings > Lock Screen > Turn display off on battery when inactive, and on power adapter, to For 3 hours. In the Claude mobile app’s Code tab she opens the session marked Connected; the whole conversation is there, with model and permission chips and a microphone. From a car she dictates: in Paper, duplicate the final email frame, fill it with the copy from a private Slack canvas, and place the images already in the file whose names match. The agent replies with screenshots of the new frame. A second request asks for dark mode on a duplicated frame using the codebase’s CSS variables, not invented colors; the result matches the code’s colors.

- 0:24 Type /remote-control
- 0:40 Lock Screen timers
- 0:49 Open the Connected session
- 1:02 Dictate the prompt
- 1:48 Review returned screenshots
- 2:03 Dark mode from CSS variables

Leave comments for your agent in Paper

Short · 27 s · 19 August 2026

0:27

Comments as an agent’s to-do list

As of August 2026. After a montage of teammates’ comments and three agent logos (Claude, Cursor, OpenAI), the presenter types one line into an agent: she left comments on this frame, check it out and implement those changes. On a finance dashboard, a comment pinned to the “Research Agent” card asks to add another card to the quick preview. The result shows “Your cards” with Personal, Research Agent and a new “Ops Agent” card in the same style, and the narration says the other commented problems were handled in the same pass. In the August release video the result sits in a separate frame named “Finance UI — Cards · Feedback pass”. The Short does not show whether the threads were resolved, so ask for that explicitly.

- 0:04 Works with any agent
- 0:08 One-line prompt
- 0:12 Comment pinned to a card
- 0:16 New Ops Agent card
- 0:20 Other comments handled

Comment, reply, and resolve in Paper

Short · 54 s · 20 August 2026

0:54

Comment, reply, resolve and filter

As of August 2026. Click the comment button in the left toolbar (tooltip “Comment C”) or press C, click a layer, type in Leave a comment and click Send. Click a teammate’s pin to open the thread, type in Reply and click Send, or add an emoji reaction from the picker. When the point is handled, click the check icon at the top of the thread to resolve it. In comment mode only, the right panel lists every comment in the file, with Search, a sort menu (Newest first, Oldest first, By page), Anyone and Any page filters, and a toggle between unresolved and resolved threads. The narration presents comments as a channel to your team and to your agent.

- 0:06 Comment C
- 0:10 Leave a comment
- 0:18 Reply
- 0:28 Resolve
- 0:34 Comments list
- 0:42 Sort and filters

Build it in Paper

- 01

Open Paper Desktop on the file you want to change. With nothing selected, click Connect your agent in the right panel’s MCP section, pick your client, click the button in step 2 (for example Add to Cursor), restart the client and confirm that Paper is listed (`/mcp` in Claude Code).
- 02

Select the target frame, press ⌘L and paste the link into a first prompt: “In Paper, describe the structure of this frame without changing anything.” If the answer names another file or page, correct the target.
- 03

Press C and pin one comment on each layer that needs work, then ask for one bounded change: “Implement the open comments on this frame by editing the existing layers, and keep every label unchanged.”
- 04

Ask the agent to screenshot the frame, review spacing, typography, contrast, alignment and clipping, and list what it changed and what it could not do.
- 05

Check the result yourself, have the agent resolve only the threads it fully addressed, and after any hand edit tell it to re-read the canvas before the next request.

Common mistakes

- ×

Adding the Paper server inside a running agent session and prompting straight away; the tools are not loaded yet.

→

Restart the client after any setup change, then confirm Paper is listed: `/mcp` in Claude Code, Cursor Settings › Tools & MCP, or Customize › MCP in GitHub Copilot.
- ×

Opening the design in a browser, or leaving only the desktop dashboard open, and expecting the agent to reach it.

→

Keep Paper Desktop running with the target file open in a tab for the whole session.
- ×

Writing “this file” or “the active Paper file” and then switching tabs before the agent starts, so it works in whichever tab is in front.

→

Name the file or paste its link (⌘L) in the prompt. If you rely on the active file, stay on that tab until its dots appear.
- ×

Reworking a frame by hand and then asking for the next change; the agent still works from what it read before.

→

Begin the next prompt by asking the agent to re-read the canvas, or that frame, before it edits.

Check before you move on

- ☐

Did the agent’s first answer name the file, page and frame you meant, before it changed anything?
- ☐

Did the change stay inside the layers you named or commented on, with every label and value you asked to keep unchanged?
- ☐

Has the agent shown a screenshot of the changed frame and said what it changed and what it could not do?
- ☐

Are the threads it addressed resolved, and is every other thread still open?
- ☐

If you edited the frame by hand, did the agent re-read it before its next change?

Parallel AI agents: keep one coherent design

Parallel work is useful when its parts are independent. A shared canvas needs clear ownership, a common reference and a deliberate moment of integration.

Parallelize alternatives, not uncertainty

Several agents can produce more material quickly, but more material is not automatically a better design. Give them different questions to answer. One can explore a denser hierarchy, another a clearer comparison and another a more explicit empty state. Each direction should preserve the same required content so the final comparison evaluates design choices rather than accidental omissions.

The Paper and Conductor demonstration shows the bounded version. The brief names its references (a captured hero, a changelog and a roadmap page), fixes the count at three iterations and ties one of them to the changelog. The agent returns three labeled directions on separate artboards, and the presenters choose among them before any code is written. Sending three agents the same vague instruction merely multiplies the ambiguous outcomes you must inspect.

Parallel does not always mean several agents. One agent can spread a brief across many artboards: asked for as many good meditation home screens as possible in five minutes, Opus 5.5 named six mood directions, created six artboards and built them breadth-first, headers on all six before any hero, so every variant reached the same stage at the same time.

Independent ownership creates useful parallelism; explicit review turns the alternatives into one coherent result.

The principle of chapter 11

Own the destination before running the agents

Compare separate ownership with conflicting writes.

SIMULATION · NO AGENTS ARE LAUNCHED

Three jobs, one design system

Agent A

Clarify KPI hierarchy

Destination: Overview artboard

Ready

Agent B

Explain missing forecasts

Destination: Forecast artboard

Ready

Agent C

Improve review action

Destination: Review artboard

Ready

SHARED SPACING TOKEN

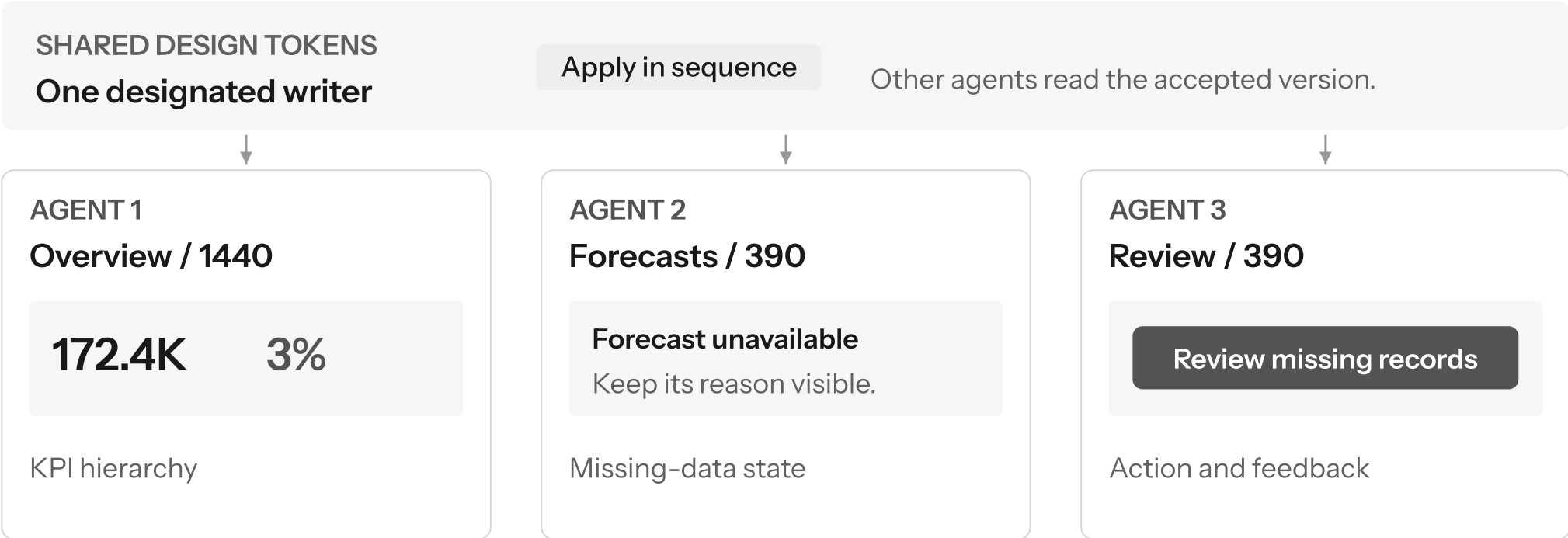
16px

Coordinator accepts 16px once; all agents read the same decision.

The scopes are independent and the shared input has one owner. Parallel execution can now produce alternatives that a designer can compare and deliberately integrate.

Parallelize ownership, not collisions.

Three agents can work independently when their canvas boundaries are explicit.



Parallel ownership of separate artboards → one visual review → an accepted direction

Reconstructed teaching example · synthetic values · not a Paper screenshot

Figure 11.1 · Plate

Parallelize ownership, not collisions.

Assign each agent a separate artboard and keep shared-token changes serialized through one writer. Review the rendered result and the missing-action or missing-data states before accepting the direction.

Assign ownership at the level of possible conflict

Give each design agent a named destination file, page or artboard. Separate artboards help keep alternative layouts independent, but they do not isolate every resource. A token edited in a file changes every layer bound to it in that file. A renamed frame or an edited reference can alter the context another agent is reading. The ownership rule must therefore cover shared inputs as well as visible output regions.

Paper shows where agents work: a robot avatar joins the presence row, and animated dots mark the file's tab and the artboard being edited. It does not reserve anything. In the Conductor session a second agent restyled the frames while the first was still writing them. It worked there, with a prompt that named the frames and the exact change, but steering the first agent would have been simpler. A practical arrangement is one owner for shared tokens and one agent per artboard or file, with the others proposing token changes rather than applying them.

Code work needs its own isolation. In Conductor, each new workspace is an isolated copy of the repository on its own branch, so coding agents do not collide, while a new tab inside a workspace shares that copy. Neither separates Paper: every agent writes to the same open files through Paper Desktop. When several files are open as tabs, name the file in every prompt or paste its link, so that parallel means independent work rather than simultaneous writes to the same material.

Integrate through a design decision

Compare alternatives against the same question and the same required actions. A layout can be visually attractive while dropping a necessary control: in one Paper demonstration a variant lost its add button although the agent's own checkpoint review passed it. Review the complete flow and identify the particular traits worth retaining: one hierarchy, one navigation treatment, one useful explanation. Ask for a composed revision with those named traits instead of the ambiguous instruction to combine everything good.

That review is the point where parallel work becomes one design. Keep the alternatives on the canvas long enough to explain the decision, then mark the accepted frame by name. An implementation agent should receive that exact name, not choose among unresolved drafts. In the Conductor session the presenters pasted the title “Conductor Roadmap — 02 Network Map” into the prompt; the agent read the frame with `get_screenshot` and `get_jsx` and compared a headless screenshot of its page with the Paper frame. Later feedback stays located: comments on the canvas in Paper, or comments on diff lines in the code tool.

Figure 11.2

One session, four agents, two Paper files

Code isolation (workspaces) and design destinations (files and artboards) are separate axes.

Who	Isolated code copy	Where it designed	What it produced
Agent 1, design Opus 5, High	conductor-roadmap-component (project paper-website)	auto-named “Young frost”, renamed “conductor roadmap”	- Conductor Roadmap — 01 Trunk Line - Conductor Roadmap — 02 Network Map - Conductor Roadmap — 03 Vertical Rail
Agent 2, restyle Opus 5, Medium	conductor-roadmap-font-dark-mode (project paper-website)	same file as agent 1	same three frames: Matter to Geist, light to dark
Agent 3, musical theme Opus 5, Medium	new workspace (project paper-website)	new file “Musical Conductor Roadmap”, created and named by the agent	Musical Roadmap — 01 The Score
Agent 4, code port Opus 5, Medium	new workspace in the empty conductor-roadmap repository, branched from origin/main	reads 02 Network Map only	index.html (421 lines), served with python3 -m http.server 4321

Agent 1’s brief: Matter for display and UI, IBM Plex Mono for labels; corrected to Geist with IBM Plex Mono.

Line 1, Shipped

#2E7D6B

Line 2, Building

#B0502F

Line 3, Next

#3B5BA9

Line 4, Exploring

#9A7B32

Figure 11.3

A fair model comparison

Hold everything constant but the model, then read the results for what they can and cannot say.

Held constant

- = prompt text, except the requested artboard title
- = effort: High
- = app: Claude desktop, Code tab
- = Paper file: “Opus 4.8 vs. Fable 5”, Page 1
- = start time, within about five seconds

The one variable

≠ model: Fable 5 vs Opus 4.8

Fable 5

\$24.50 hero, 72px Archivo Black, yellow underline
chips Food, Transit, Home, Fun
its own numeric keypad
one accent #FFD400 in three places
summary describes each choice

Opus 4.8

\$42.50 hero, 84px Inter Tight Black, yellow caret
chips Dining, Transit, Shop
Merchant and Note rows, Add Expense bar
checklist: contrast, 24px icon and 92px label lanes, artboard fit, color restraint
finished significantly faster; offers a next variant

What the presenters concluded. Fable 5 may do better, more independent work but is slower and significantly more expensive; no winner named; a multi-screen test suggested

How it works in Paper

As of September 2026, several agents can work in Paper at once: on separate artboards of one file, or across files open as desktop tabs (since August 2026), including tabs in the background. Each agent appears as an avatar in the presence row and as animated dots on its tab and above the artboard it is editing. Paper does not lock nodes or isolate code: prompts set each agent's destination, and the harness, such as Conductor's workspaces, isolates the repository. File links, exact frame names and comments coordinate the work.

Checked against the videos and documentation as of September 2026. Paper ships often; the release timeline in the reference shows what changed when.

01

Explore directions on separate artboards

1

With the file open in Paper Desktop, ask “Are we connected to the Paper MCP?”. The agent should report the file, page, artboard and size, node count and fonts.

2

Name the frame and the count: “In Paper, design two iterations of the Social Home frame. Explore different colors and layouts.”

3

Expect the agent to read the frame's structure and name its directions (here “Ink” and “Slate”) before it writes.

4

Each direction appears as a new artboard beside the untouched original, named after it, such as “Social Home — Ink (Dark)”, with working dots above it while the agent builds.

5

Check every variant against the original's controls yourself; the agent's review passed a variant that had lost its add button.

6

Combine by naming traits and their sources: navigation from Ink, palette and side bar from Slate, a dark purple accent. The agent should state a plan with hex values before it builds.

►

Shown at 1:08

02

Give each agent its own file

1

Open every file the agents will use as a tab in Paper Desktop. Agents can work in background tabs, so the tab in front does not matter once they have started.

2

Start each prompt with its file: “In the Learn Section file in Paper, ...”, or paste a link copied with Copy link (⌘L).

3

Write “in the active Paper file” only if that tab stays in front until the agent has started; the agent settles on its file at the start.

4

To give an agent a fresh file, ask it to create and name one through the MCP, for example “a new file called Musical Conductor Roadmap”. It opens as a new tab.

5

Rename auto-named files before prompting: double-click the tab, or right-click it and choose Rename.

⌘

L

►

Shown at 1:04

03

Isolate code work in Conductor

1

Press ⌘N for a new workspace. Each workspace is an isolated copy of the repository on its own branch, so coding agents in different workspaces do not collide.

2

Open a new tab inside a workspace instead when the job only changes Paper; tabs share the workspace's copy of the code.

3

Set model and effort per job in the composer. The session used Opus 5 at High for the first designs and Medium for the restyle and the code port.

4

To correct a running agent, type and press Enter to steer it; press ⌘Enter to queue the message until its turn ends.

5

Press ^O to expand all tool calls, then click a call such as write_html to see its target node, mode and HTML.

⌘

N

Enter

⌘

Enter

^

O

►

Shown at 2:28

04

Correct brand drift while agents run

- Select a heading on each agent's artboard and read Right panel › Text. The first roadmap heading read Matter, Medium, 52, not the brand's Geist.
- Fix a single instance by hand: click the font family field, type “gei” in the Fonts popover's search and choose Geist.
- For every frame, steer the agent that owns them, or give one agent a precise edit: use the Paper MCP to edit the roadmap frames, change the font from Matter to Geist and change them from light mode to dark mode.
- If a second agent edits frames another agent is still building, it can only follow behind; check the frames again when both have finished.
- Confirm each frame in the Text section and in Selection colors before anyone ports the design.

►

Shown at 7:08

05

Compare two models fairly

- Open two agent sessions against the same blank Paper file, one model each, at the same effort (High in the test).
- Paste the same prompt into both and change only the requested artboard title, such as “Fable 5 Version” and “Opus 4.8 Version”.
- Send both as close together as you can and note the gap; the presenter allowed about five seconds for the later start.
- Watch both artboards build side by side and record which run finishes first.
- Judge each screen against the brief, including invented content such as a \$42.50 coffee, then read both summaries.
- Treat one screen as one sample: repeat with several screens or tasks before you settle on a model.

►

Shown at 0:40

06

Hand the accepted frame to an implementation agent

- Double-click the accepted artboard's title on the canvas and copy its exact name, such as “Conductor Roadmap — 02 Network Map”.
- In a fresh code workspace, prompt: “Look at the <name> frame in Paper and recreate that and run it locally.”
- Expect reads before writes: get_guide, list_files, get_basic_info, get_screenshot and get_jsx.
- Review the diff and the running page. In an empty repository the agent wrote a standalone index.html, served it on port 4321 and checked a headless screenshot against the Paper frame.
- Leave feedback where it belongs. In Conductor, select diff lines, click + in the gutter, type the note and choose Comment; it lands in the composer for the agent. In Paper, press C, click the frame, type in “Leave a comment”, choose Send, then tell the agent to read your comments.

C

►

Shown at 11:42

Controls you will use

Control	Where	What it does
Tab bar	Top of the Paper Desktop window	Holds one tab per open file after the Dashboard tab; “+” opens an existing file or creates one. Agents can work in any open tab, including tabs in the background. While an agent works in a file, animated dots replace that tab's file icon.
Tab menu	Right-click a file tab	Reload, Copy Link (⌘L), Rename, Open in Browser, Move to New Window, Close Tab, Close Other Tabs, Close Tabs to the Right, Close All Tabs. Rename auto-named files so every prompt can name its file unambiguously.
Copy link	Right panel, below the presence avatars (also in the tab menu)	Copies a link to the file or to the selected frame. Pasted into a prompt, it pins the agent to that file, which the agent opens before anything else.
Presence avatars	Top row of the right panel, above Copy link	Show everyone in the file. An agent appears as a robot avatar next to the people.
Working indicator	Canvas, beside the title of the artboard an agent is reading or editing	Animated dots show where each agent is busy. They clear when the agent calls finish_working_on_nodes. With parallel agents, several artboards and tabs show dots at once.
Text section	Right panel › Text	Shows the selected text's font family, weight, size, line height and letter spacing. On the first roadmap heading it read Matter, Medium, 52, 56, -2.5%.
Fonts popover	Right panel › Text › font family field	Lists available fonts with a search box; typing “gei” narrowed it to Geist and Geist Mono. Use it for one-off fixes; ask one agent for changes across many frames.
Selection colors	Right panel, below Filters, with a frame selected	Lists every color used inside the selection with its usage count, showing token names where colors are bound. A quick check for off-brand values in an agent's output.
Comment tool	Left toolbar comment button, or C	Click anywhere, type in “Leave a comment” and choose Send. In comment mode the right panel lists the file's comments with Search, sorting and filters. Agents can read and resolve comment threads but cannot write them.

Shortcuts

⌘L	Copy a link to the file or selected frame, to paste into an agent's prompt ▶ 0:32	C	Comment mode: click the canvas to leave a comment for a teammate or an agent ▶ 6:50
⌘Enter	Paper Snapshot (Chrome extension): capture the whole page instead of one element ▶ 1:36	⌘V	Paste a Snapshot capture into Paper as editable layers ▶ 1:46
⌘N	Conductor: new workspace, an isolated copy of the repository ▶ 2:28	Enter	Conductor: send the typed message to steer the running agent ▶ 10:14
⌘Enter	Conductor: queue the typed message until the agent's current turn ends ▶ 10:14	^0	Conductor: expand all tool calls to see which Paper tools the agent calls ▶ 5:16

Limits and what is coming

- As of September 2026 the Paper MCP has no tool to reserve or lock nodes. The working dots show where an agent is busy, but two agents can edit the same frames, so ownership lives in your prompts.
- Agents can list, read and resolve comment threads through the MCP but cannot create comments (tool list as of October 2026); people write the comments.
- Remote MCP, for cloud agents with no Paper Desktop open, and per-file permissions were announced as coming next on 28 September 2026, not shipped.
- MCP tool calls count against a weekly plan allowance: 100 on Free and 1M on Pro (pricing as of October 2026). Several agents running at once use it up faster.

What the tutorials show

Each official video, written down step by step. The dots mark the moments; they open the video there if you want to see it.

Paper and Conductor: design to code with parallel AI agents

Video · 13 min 53 s · 19 August 2026

13:53

Three directions, a drift fix and a port, with parallel agents

Three references sit in one Paper file: Conductor's changelog and hero, captured with Paper Snapshot, and Paper's own roadmap page. In Conductor, an Opus 5 agent at High effort is asked to create a Conductor Roadmap component in Conductor's design language, styled like a subway map: “Create three iterations,” one inspired by the changelog. It reads the file before writing, then builds 01 Trunk Line, 02 Network Map and 03 Vertical Rail with write_html. The drift: headings in Matter, Paper's font, on light frames. The presenter fixes one heading through the Fonts popover (search “gei”, choose Geist), then an agent in a second workspace converts every frame from Matter to Geist and from light to dark while the first is still writing. A third agent creates and names a new file, Musical Conductor Roadmap. A fresh workspace finally recreates 02 Network Map from its copied name and serves it locally. The hosts note that steering the first agent would have fixed the font without a second one.

- 2:28 New workspace: an isolated repo copy
- 3:56 Prompt: three subway-map iterations
- 7:08 Fonts popover: Matter to Geist
- 7:46 Second agent: Geist and dark mode
- 9:44 Third agent creates a new file
- 11:42 Port the 02 Network Map frame

Use an AI agent to explore different design directions in Paper

Video · 6 min 15 s · 2 May 2026

Named directions beside the original, then a combination by name

In the Claude desktop app's Code tab (Sonnet 4.6, Medium), the presenter first asks “Are we connected to the Paper MCP?”; the answer names the file, page, artboard “Social Home” (1440×900), 189 nodes and the font, Inter. The prompt: “In Paper, design two iterations of the Social Home frame. Explore different colors and layouts.” The agent reads the frame, proposes “Ink” (warm near-black, amber gold, icon-only nav) and “Slate” (cool, top nav instead of a sidebar, DM Sans, indigo accent), builds each as a new artboard beside the untouched original and screenshots each one. Its checkpoint review passes both, yet Slate has lost the add button; the presenter catches it. Her follow-up combines by name: Ink's navigation, Slate's palette and side bar, a dark purple accent. The agent answers with a plan (“Violet”, #5B21B6 as the single accent) before building, reviews the result and calls finish_working_on_nodes. She then lightens the nav rail herself in the Fill color picker instead of prompting again.

- 0:44 Pre-flight: connected to the Paper MCP?
- 1:08 Prompt: two iterations
- 1:48 Named directions: Ink and Slate
- 3:38 Slate lost the add button
- 4:50 Plan for Violet with hex values
- 5:50 Manual fix in the Fill picker

Claude's Fable 5 vs Opus 4.8: which model designs better in Paper?

Video · 4 min 22 s · 10 June 2026

A two-model test in one file

Two Claude desktop sessions, Fable 5 and Opus 4.8, both at High effort, get the same prompt: explore a design for an expense tracking app in Paper, one screen showing an expense entry, sans-serif type and bright color, clean and minimal. Only the requested title differs (“Fable 5 Version”, “Opus 4.8 Version”), and both write into one file, “Opus 4.8 vs. Fable 5”, where the two artboards fill in side by side with separate working dots. Opus starts a few seconds later; the presenter plans to subtract about five seconds. Opus 4.8 finishes significantly faster. Her conclusion is hedged: Fable 5 may do better, more independent work but is slower and significantly more expensive, and a multi-screen task would test more. She names no winner. The outputs differ in content (Fable draws its own keypad; Opus prices a flat white at \$42.50) and in reporting: Fable explains its choices, while Opus prints a self-check list and offers a next variant.

- 0:23 Model menu: Fable 5 and Opus 4.8
- 0:40 Same prompt, title per model
- 0:55 Second session: Opus 4.8 at High
- 1:26 Two artboards building in one file
- 2:36 Both done: Opus 4.8 finished first
- 3:42 Opus 4.8 checklist and variant offer

What can Opus 5.5 design in five minutes?

Short · 55 s · 24 September 2026

One agent builds six directions breadth-first

After a benchmark table comparing Opus 5.5 with Fable 5.1, the presenter dictates one prompt in the Claude desktop app, with Opus 5.5 at Extra effort: explore as many good variations of a meditation app's mobile home page as possible on the Paper canvas in five minutes. The agent names six mood directions (Tidal, Candlelit, Alpine Swiss, Chapel, Editorial pop, Mineral), creates six artboards titled like “01 Tidal — Device (402×874)”, then adds headers and greetings to all six before any hero. One step reports “Used 18 tools (6 failed)”. All six finish in just under six minutes. It is a demonstration, not a controlled comparison.

- 0:02 Benchmark table
- 0:14 Dictated prompt, Opus 5.5 at Extra
- 0:28 Six mood directions; 6 of 18 calls failed
- 0:34 Six 402×874 artboards fill in
- 0:40 All six done in under six minutes

Paper is the design tool for teams building with agents

Short · 1 min 12 s · 18 September 2026 1:12



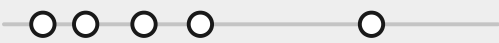
What reading and writing the canvas means

The presenter defines the two halves of agent work in Paper, whose canvas is rendered with code. Reading: the canvas is visual context beside the prompt. The agent checks what is selected, and the on-screen prompts ask it to list best practices from reference images, check a frame pasted from Figma, and apply the presenter's manual changes to the rest of a flow. Writing: first drafts (“make a login screen using my style guide” yields “Login explore / 01 — Centred, email-first”), many variations (“explore 10 variations of my selection”) and boilerplate, shown as generic Frame layers renamed to Main Content, Sections, Section Header and Curator Row. The designer keeps every design decision.

- 0:18 Agent reads the file and the selection
- 0:24 Read: best practices from references
- 0:28 Read: apply my changes to the flow
- 0:38 Write: login screen from a style guide
- 0:40 Write: 10 variations of my selection
- 0:46 Boilerplate: semantic layer names

Paper: the connected canvas for teams shipping with agents

Short · 35 s · 26 August 2026 0:35



Why agents work well on an HTML and CSS canvas

A Paper team member explains that Paper uses HTML and CSS as its rendering engine, which agents already understand; tools with custom rendering engines, he says, mean higher token spend, slower runs and lower accuracy. The animated b-roll shows the pattern this supports: read the selection and a code component, fetch real playlist data from an API, create a desktop artboard, write one track row per write_html call and set copy with set_text_content. After “now generate the mobile screens”, the agent creates “Mobile / Home”, “Mobile / Playlist” and “Mobile / Library” at 390×844. The team is about twelve designers and engineers. The comparison with other tools is a claim, not a measured benchmark.

- 0:03 Agent log: selection, code, live data
- 0:06 One write_html call per row
- 0:10 Three 390×844 mobile screens
- 0:14 Custom engines: more tokens, slower
- 0:26 About twelve designers and engineers

How the Skydive team ships with their agents in Paper

Short · 2 min 09 s · 21 September 2026 2:09



A design agent and a build agent share one Paper design

Two Skydive designers create an agent in Skydive (name Tallulah, purpose “UX Design Expert”) and train it on their design system by giving it the file's link from Copy link (⌘L) in Paper's right panel. In Slack they ask it to read two Paper files and create a new artboard for a Skydive and Paper collaboration page; it drafts “Skydive × Paper · Collaboration Landing”, then a dark copy on request. The designers finish the page by hand for pixel accuracy. In a Slack group message with a second agent, Moon, they ask Tallulah to share the new design with Moon; the agents coordinate while the people supervise, and Moon publishes the page to Webflow, which the team describes as identical to the Paper design, animation included. How Skydive's agents connect to Paper is not shown.

- 0:26 Create an agent: UX Design Expert
- 0:32 Copy link ⌘L for the design system
- 0:46 Slack: read two Paper files, draft
- 1:06 Manual pixel pass in Paper
- 1:32 Group message: share with Moon
- 1:46 Live page matches the design

Build it in Paper

- 01

Gather the shared references on one page of a Paper file and name each frame. Capture live pages with Paper Snapshot (⌘Enter takes the whole page) and paste them with ⌘V.
- 02

Give each agent one question and one destination: a named artboard for an alternative in the same file, or its own file open as a tab. Start each prompt with the file name or a link from Copy link (⌘L), then the fonts, the theme and the content every version must keep.
- 03

Run the agents and follow the working dots on tabs and artboards. Correct drift in the agent that owns the frames (in Conductor, Enter steers a running agent and ⌘Enter queues a message) instead of sending a second agent onto the same frames.
- 04

Review every alternative against one checklist: required actions present, the brand font in the Text section, only brand values in Selection colors. Fix one-off issues by hand, for example a font through the Fonts popover's search.
- 05

Name the accepted frame, paste its exact title into a fresh code workspace's prompt, and have the agent compare its build with a screenshot of that frame before you review the diff.

Common mistakes

- ×

Sending several agents the same vague prompt and letting them choose where to write.

→ Give each agent one question and one named artboard or file, state how many directions you want and list what must stay fixed.
- ×

Trusting an agent's self-review. The Slate variant passed its checkpoint review without its add button.

→ Check each variant against a list of the original's controls before you combine or accept anything.

Check before you move on

- ☐

Does every agent's prompt name its file and destination artboard, or carry a Copy link?
- ☐

Do all alternatives keep the source frame's required content and actions, counted one by one?
- ☐

Are the brand fonts and theme stated in every brief, and do the Text section and Selection colors confirm them on each frame?
- ☐

Did compared runs share the prompt, effort, file and start time, with only one thing changed?
- ☐

Is exactly one frame marked as accepted, and does the implementation prompt quote its exact name?

- ×

Leaving the typeface and theme to the agent. In the Conductor session it set headings in Matter, Paper's font, on light frames, for a team that wanted Geist and dark mode.

→ Name fonts, theme and palette in the brief. If drift appears, steer the agent that owns the frames or give one agent an explicit from-and-to edit, then recheck the Text section.
- ×

Assuming separate Conductor workspaces keep agents apart in Paper.

→ Workspaces isolate code only. Separate Paper work by artboard or file and name it in each prompt; use a tab rather than a workspace for canvas-only edits.

Repeat the rule, then vary the content

A reusable pattern preserves a relationship across different content and states. Visual similarity alone does not establish that relationship.

Find the repeated decision

Two cards can share a border radius while behaving quite differently. That is a shared style, not necessarily a shared component. A reusable pattern begins with a repeated responsibility: a track row relates a title to its artist, length and play state; a metric row relates a label to several values; a notice connects a condition to an action. The responsibility explains which parts should stay stable.

Describe the pattern through its inputs and states. A row might accept a name, a secondary label, a value and a selection state. A missing value is then a defined state rather than a reason to redraw the row. A long name is ordinary input, not an exceptional design. This way of thinking lets the layout accommodate variation without losing its visual identity.

Write that contract where the pattern lives. In Paper’s September 2026 components tutorial, the agent that rebuilt a music app’s components set a one-line spec beside each one without being asked. The cover card states its art radius, its title and byline sizes and its two widths, 148 on a shelf and 112 in a grid. The track row fixes its lanes (number 20, art 34, a title that fills, album 120, time 40, save 24, more 24) at a height of 52, and a playing row gains an amber dot while nothing else changes. Ask for that line whenever you build a pattern.

Study 12 · change it yourself

A useful pattern repeats a responsibility while making legitimate variation explicit.

The principle of chapter 12

Change the rule; preserve the variation

Compare linked rows with copies. Change the rule, then test the exceptions.

One row, explicit states

01	Camille Bernard	42%
02	Noor El Mansouri Selected	31%
03	Alexandra de Saint-Pierre Awaiting a source	Unavailable

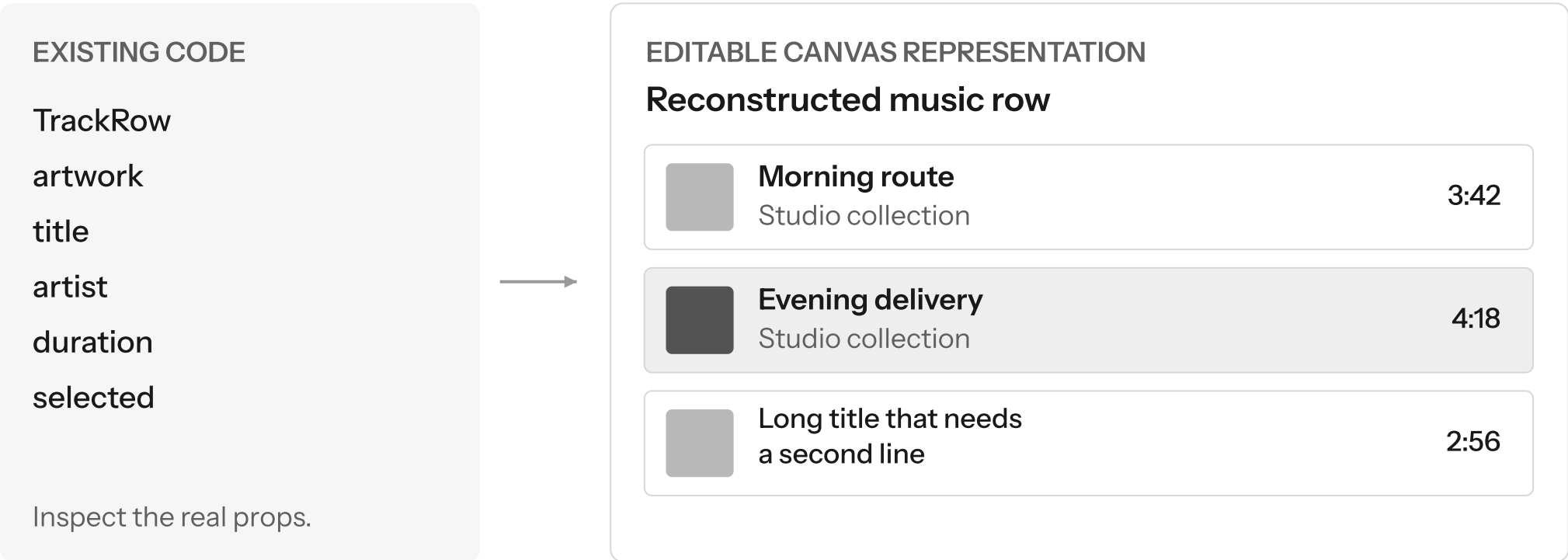
Requested width 460px; capped to the available stage. Values retain a fixed 80px lane.

One spacing rule updates all three instances. The second row’s label override and each row’s state remain independent content inputs.

► Notes

Reuse the contract behind the row.

Bring a real component into the canvas, then test its important states.



A reused structure is not proof of native linked components or automatic code–canvas synchronization.

Reconstructed teaching example · synthetic values · not a Paper screenshot

Figure 12.1 · Plate

Reuse the contract behind the row.

A repository component can be reconstructed as an editable canvas representation. Preserve its content contract and test selected and long-label states; do not infer native linked Paper components or automatic synchronization.

Ground the pattern in an existing product

The repository-to-canvas tutorial brings a music app’s existing components and tokens into Paper, explores a new section with real content and returns the chosen result to the codebase. The valuable starting point is the existing product vocabulary. Reusing it preserves familiar behavior: when the new section reached the code, its cards already opened their playlist pages, because they were the app’s own card component.

Be exact about what Paper links today. As of September 2026 the canvas has no native components: Paper announced components on the canvas as coming soon on 28 September, and its public roadmap lists components with slots as coming soon and the use of code components on the canvas as in progress. A duplicate made with ⌘D or ⌘-drag is an independent copy. What stays linked is tokens: change a token in the Theme tab and every layer bound to it in the file updates. A component rebuilt by an agent, or captured with Paper Snapshot, is a set of editable frames, not an instance.

You can still define a reliable pattern today with a canonical frame, named tokens, documented inputs and repeated specimens. Keep the canonical specimen next to demanding variants: the ordinary state, the selected state, long text and unavailable information. These variants are more useful than many nearly identical copies because they reveal the pattern’s limits. If one case needs a different structure, decide whether it is

Preserve meaning when you repeat

Consistency should make exceptions easier to see. A selected row must remain related to its siblings while being clearly selected. An unavailable value should preserve the comparison lane while explaining its absence. When every state adds another badge, color or panel, the pattern can become harder to read despite being technically comprehensive.

Vary one rule at a time, on a copy. In the tutorial, the agent duplicated the finished section, named the copy after its single change (For you section · Grid 112), swapped the 148-pixel covers for 112-pixel ones and reported the cost: nine covers now fit in the row, but longer bylines clip at that size. The original stayed untouched, so the comparison was fair, and the designer kept the larger cards.

Reuse the important rule, not every dimension or sentence. On the code side, the same contract becomes a component’s properties and states; on the canvas, it keeps the design explanation stable until Paper’s native components arrive. Say clearly which relationships the tool enforces (token bindings) and which the workflow maintains (copies, specimens and agent updates).

a legitimate variant or a different responsibility that deserves its own component.

Figure 12.2

What stays linked in Paper

Which relationships the tool enforces, and which your workflow has to maintain.

Mechanism	How you make it	Linked to its source	What propagates	Source
Duplicate	⌘D, ⌘-drag or right-click › Duplicate	No	Nothing; each copy is edited separately	▶ 4:27
Token	Theme tab › +, or + in a fill’s token list	Yes, within the file	Every bound layer updates when the value changes	▶ 3:08; ▶ 2:19
Agent-rebuilt code component	Prompt an agent connected to the repository	No (a design representation)	Only its token bindings; structure does not follow the code	▶ 0:28
Paper Snapshot capture	Chrome extension: click an element, ⌘Return for the page	No	Nothing; no tokens, generic Frame names	▶ 1:33
Native component with slots	Not available yet	Announced	Coming soon as of 28 September 2026	▶ 4:32; paper.design/roadmap

linked announced not linked

Figure 12.3

One row, its contract and its specimens

A pattern defined by inputs and states, tested by the cases that stress it.

The contract

Track row · height 52 px · lanes in px; the title fills what is left

amber

art

title

album

time

save

more

20

34

fills

120

40

24

24

States

- idle
- playing (amber dot; nothing else changes)
- compact (no album copy)

Four specimens that stress it

Long title	01	A title long enough to need two lines, or to b...	Album name	3 : 41	♡	-
Missing album	02	Short title	—	2 : 58	♡	-
Selected	03	Short title	Album name	4 : 05	♡	-
Playing	04	● Short title	Album name	3 : 12	♡	-

How it works in Paper

As of September 2026 Paper has no native components. Duplicates (⌘D, ⌘-drag or Duplicate in the right-click menu) are independent copies with no link back. What stays linked is tokens: change one in the Theme tab and every bound layer in the file updates, as with a CSS custom property. For a code-based system, an agent can rebuild repository components as editable, token-bound layers, and Paper Snapshot can copy them from a live site. Native components with slots were announced as coming soon.

Checked against the videos and documentation as of September 2026. Paper ships often; the release timeline in the reference shows what changed when.

01

Rebuild code components on the canvas with an agent

1

Open a new Paper file and make sure your agent is connected; with nothing selected, the right panel’s MCP section has a Connect your agent button for the setup.

2

In an agent session inside your repository, name the flow you are about to design and the components it will need (the demo listed cover cards, artist cards and track cards), and ask the agent to build them in the open Paper file before any design work.

3

Watch the canvas: the agent’s avatar joins yours at the top of the right panel, and a dotted pattern above a frame marks the one it is reading or writing.

4

Review the sheet. The demo’s, For You · Components, had numbered sections (01 Cover card, 02 Person, 03 Track row, 04 Section header), a dark and a light version of each, semantic layer names such as Cover · Bloom and a one-line spec per section.

5

Select a text layer and read the right panel: Fill showed the token text, the font family sans, and size and line height showed 14 and 20 as token chips, because the agent had imported and bound the codebase’s tokens on its own.

► Shown at 0:28

02

Copy a live component or page with Paper Snapshot, then fit it into a layout

1

Install the Paper Snapshot extension in Chrome and open the live page. The first time on a site, Chrome’s Extensions menu lists Paper Snapshot under Access requested; click it to grant access.

2

Turn Snapshot on, hover the element and click to copy it. Press ⌘Return instead to copy the whole page.

3

Paste in Paper. The layers are editable and match the coded component, but they are named Frame, keep raw CSS under Other styles (margins, max-width, font-synthesis) and have no token bindings.

4

To place a section inside a captured page, copy the section, select the page’s content column and paste. The frame arrives with Absolute position ticked and floats over the layout.

5

Untick Absolute position (right panel, under Flex) so the section joins the column’s flex flow, then drag it to its place in the order.



► Shown at 3:15

03

Try a variant on a duplicate

1

Decide on the single change you want to test and keep the current section as the original.

2

Ask the agent to duplicate the section frame and apply that change to the copy (the demo asked to see the section with the smaller card components instead of the larger ones).

3

The agent leaves the original untouched, places the copy beside it and names it after the change: For you section · Grid 112.

4

Read its report of the tradeoff: 112-pixel covers instead of 148, nine fit in the row, bylines shortened to curator names, and longer lines clip at that size.

5

Compare the two side by side and keep the better one; the demo kept the larger cards.

► Shown at 3:56

04

Send the chosen frame back to code

1

Give the winning frame a name nobody can mistake: the demo double-clicks the frame's name on the canvas and types FINAL.

2

In the agent, point to that frame and to the destination, for example by asking it to take the For You section from the final frame in Paper and add it to the Explore page of the codebase.

3

The agent reads the frame and edits existing files rather than creating parallel ones; the demo reported four files, among them the Explore page component and its stylesheet, and checked the page with browser actions before reporting.

4

Open the page locally and use it. The coded section matched the frame, and clicking a card opened its playlist page, because the cards were the app's existing component.

►

Shown at 4:55

05

Duplicate specimens with a repeated offset

1

Select the canonical layer or frame.

2

Hold `⌘` and drag to place a copy; set a rotation on the copy if the pattern needs one (the demo typed 30°).

3

Press `⌘D`: each new duplicate repeats the same offset and rotation (since August 2026), which in the demo turned one bar into a ring of bars.

4

Change each copy's content for the state it represents and rename it after that state.

5

Remember that the copies are independent: a later fix to the original has to be repeated on each copy, by hand or by an agent.

`⌘`

Drag

`⌘`

D

►

Shown at 4:42

06

Change a shared value once with a token

1

Switch the left panel from Design to Theme. It lists the file's tokens by type, with a count, a search field and + to create one.

2

Click a token to edit its Name and Value. For a color, pick a new value or point it at another token; the picker shows Previous and New side by side.

3

Every layer bound to that token updates at once across the file. Close the picker to keep the new value.

4

To turn a raw fill into a token, select the layer, click the token icon beside the Fill value, press + at the top of the token list, then name and save the new token.

5

Check bindings in Selection colors at the bottom of the right panel: bound colors appear by token name, with the number of layers that use them.

►

Shown at 3:08

Controls you will use

Control	Where	What it does
Duplicate	Right-click menu (<code>⌘D</code>)	Deep-copies the selection; after an <code>⌘</code> -drag it repeats that offset and rotation. The copy is independent of the original.
Copy styles / Paste styles	Right-click menu (<code>⌘C</code> / <code>⌘V</code>)	Transfers one layer's styling to another in a single step; the result is a one-time copy, not a link. Listed in the menu; not demonstrated in the videos.
Theme tab	Left panel, beside Design	Lists the file's tokens by type with a search field and + to create one; changing a token's value updates every layer bound to it in the file.
Token chip	Right panel fields such as Fill and Text	Shows a field bound to a token by the token's name (text, sans) instead of a raw value. The token icon beside the field opens the token list; the broken-link icon detaches the field and keeps its current value.
Selection colors	Right panel, near the bottom	Lists every color in the selection, by token name when bound, with the number of layers using it; editing a row changes all of them.
Absolute position	Right panel, under Flex	Takes a layer out of its parent's flex flow, like CSS position: absolute. Pasted frames arrive with it ticked. Untick it so a pasted section flows with its siblings.
Other styles	Right panel, near the bottom	Lists raw CSS declarations that have no dedicated control, each with a checkbox; Snapshot captures carry margins, max-width and font-synthesis here.
Connect your agent	Right panel › MCP, with nothing selected	Installs the Paper MCP in one click for Cursor, Claude, Claude Code, GitHub Copilot, VS Code, OpenCode, Codex, Codex CLI, Antigravity or other agents.
Paper Snapshot	Chrome extension	Copies a hovered element (click) or the whole page (<code>⌘Return</code>) from a live site, ready to paste into Paper as editable layers.

Shortcuts

<code>⌘</code><code>D</code>	Duplicate the selection (after an <code>⌘</code> -drag, repeats its offset and rotation) ► 4:52	<code>⌘</code><code>Drag</code>	Duplicate while dragging ► 4:42
<code>⌘</code><code>⌘</code><code>C</code>	Copy styles ► 3:46	<code>⌘</code><code>⌘</code><code>V</code>	Paste styles ► 3:46

Limits and what is coming

- As of 28 September 2026 Paper had no native components: components on the canvas were announced as coming soon, and the public roadmap listed components with slots as coming soon and code components on the canvas as in progress. Until they ship, a fix to an original must be repeated on every copy.
- A component an agent rebuilds from code is a design representation, not a linked instance: when the code changes, the canvas copy changes only if you ask the agent to rebuild it.
- Tokens link layers within one file; tokens copied to another file form an independent set that needs explicit updates (Paper’s token documentation, September 2026).
- Snapshot captures carry no token bindings and use generic Frame names, and Figma components pasted into Paper arrive detached.

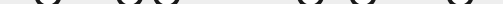
What the tutorials show

Each official video, written down step by step. The dots mark the moments; they open the video there if you want to see it.

How to bring your components from code onto the Paper canvas

Video · 6 min 09 s · 18 September 2026

6:09

A horizontal progress bar with five circular markers. The first marker is filled, and the second is partially filled. The rest are empty. The bar is positioned at the bottom of the page, below the video player controls.

Code components onto the canvas, a variant, and back to code

A demo music app's live site is built from reusable code components: a search bar, playlist cards, track cards. In an empty Paper file, the presenter asks her agent in Cursor to bring in the components a new For You section might need. It builds a labeled sheet (cover card, person, track row, section header, each dark and light) and, unasked, imports the repository's tokens and binds them to the layers. Paper Snapshot is shown as a second route, with no agent and no tokens: hover the live section header, click, paste. The agent then composes the section from its sheet, duplicating one cover card and filling the copies with real playlists from the repository's API. She pastes the result into a ⌘Return capture of the whole page and unticks Absolute position so it flows. A smaller-card variant, built on a duplicate, loses to the original, which she renames FINAL. In code, the new Explore section matches the frame, and its cards already open their playlists.

0:28 Prompt: bring in the relevant components

1:28 Token chips on a rebuilt text layer

1:56 Snapshot: grant access, then click to copy

3:42 Absolute position off, section flows

4:22 Variant on a duplicate: Grid 112

5:34 Coded card opens its playlist page

Build it in Paper

- 01

With your agent connected (right panel › MCP › Connect your agent), ask it to rebuild one repeated unit from your codebase on its own sheet, bound to the codebase’s tokens, with a one-line spec per component: sizes, radius, type and states.
- 02

Select a text layer in the rebuilt unit and check that Fill and Text show token chips rather than raw values; turn any raw fill into a token from the token icon beside it (+ at the top of the token list).
- 03

Place specimens beside the canonical frame with ⌘-drag and ⌘D: ordinary, selected, long-content and unavailable-data states, each renamed after its state.
- 04

Try a variant on a duplicated section named after its one change, and keep the untouched original next to it for comparison.
- 05

Rename the chosen frame (double-click its name on the canvas), ask the agent to build it into the existing page with the same code components, and click through the result in the browser.

Common mistakes

- ×

Treating duplicates as instances: fixing the original and assuming its copies follow.

→ As of September 2026, copies are independent. Put shared values in tokens so they propagate, and when the structure changes, ask the agent to apply the change to every copy, then check each one.
- ×

Reusing a Snapshot capture as it was pasted.

→ Have an agent rename the generic Frame layers and bind colors and type to your tokens, and review the raw CSS under Other styles, before anyone builds on it.

Check before you move on

- ☐

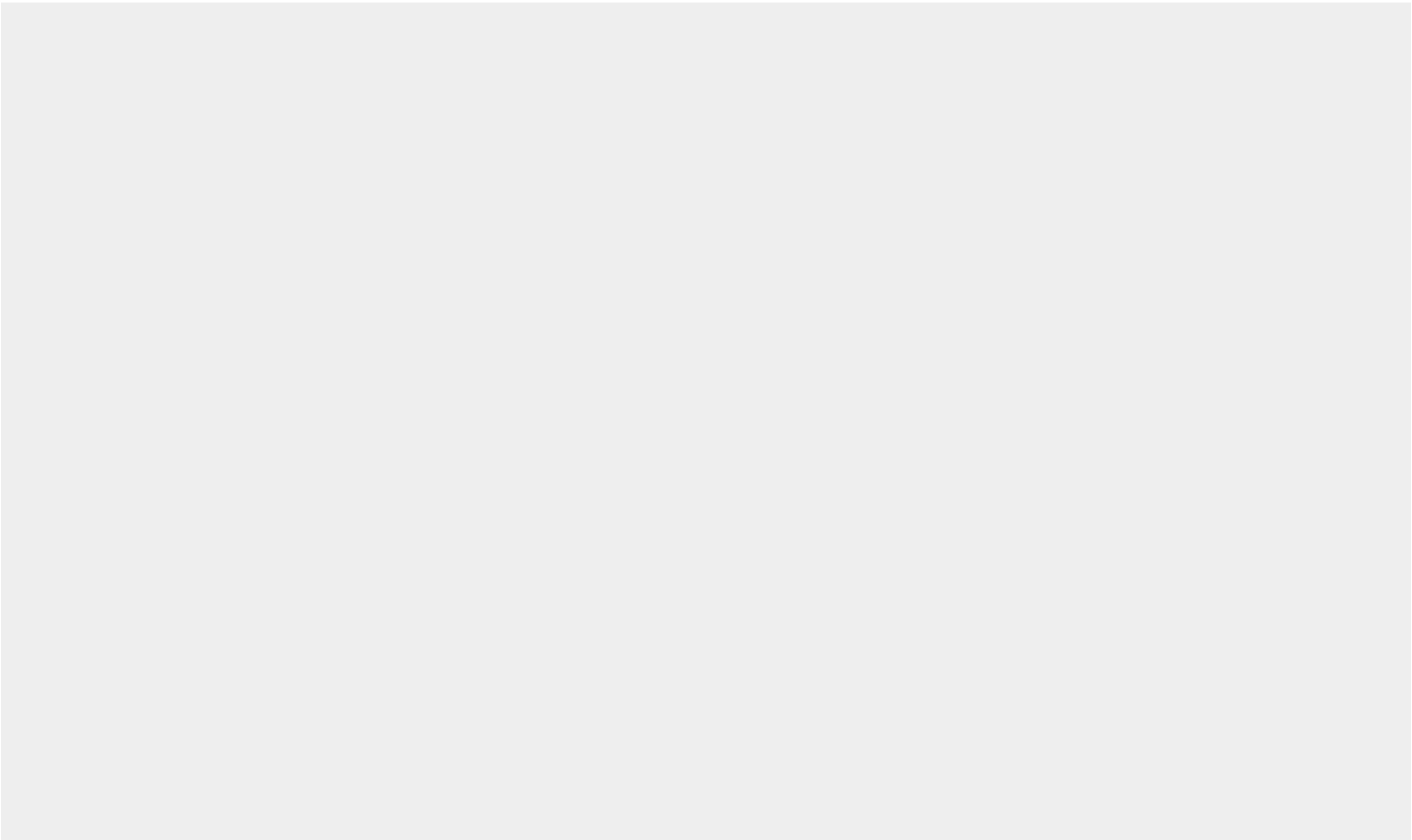
Can you state in one line the responsibility this pattern repeats, and the inputs and states it accepts?
- ☐

Does every shared value in the canonical frame show a token chip rather than a raw value?
- ☐

Are the ordinary, selected, long-content and unavailable-data specimens on the canvas beside the canonical frame?
- ☐

If you changed the original now, do you know which copies would need the same change?
- ☐

Does the frame you will send to code use the codebase’s own components, so their behavior comes with them?



Carry the design’s intent into working code

The handoff is complete when the implemented product preserves the accepted relationships, content and behavior at the sizes people use.

A frame specifies appearance and relationships

A canvas can describe where content sits, how it is grouped and which visual states matter. It cannot answer every behavior question through appearance alone. A search field needs matching rules, an empty result needs wording, and a scrolled header needs a defined response. The implementation brief should make those decisions explicit so the developer or agent does not have to invent them from a still image.

Paper’s push-to-code tutorial shows how quickly those questions surface. The redesign showed one category selected, so before writing code the agent asked three questions, starting with what the All item should show (every category stacked under its heading, or one flat grid) and whether choosing a category should change the URL. Draw the states you care about as their own artboards, as the presenter did with a scrolled state, and answer the rest when the agent asks.

Start from the accepted frame, not an old screenshot of an earlier draft. Ask the agent to read its structure, styles, token references and assets through Paper’s MCP, and treat the screenshot only as a visual check. Pixels mislead: in Paper’s email tutorial, an agent judging a manual edit from a screenshot called the new footer color #455574, then read the computed styles and reported the actual #415377.

Exact design extraction provides the starting point; the running task establishes completion.

The principle of chapter 13

Inspect the rules that become code

Change the component and inspect the HTML and CSS it produces.

Scenario comparison

Example values

Baseline	Illustrative scenario	Difference
100.0	112.0	+12.0%

► Read the basis

HTML fragment

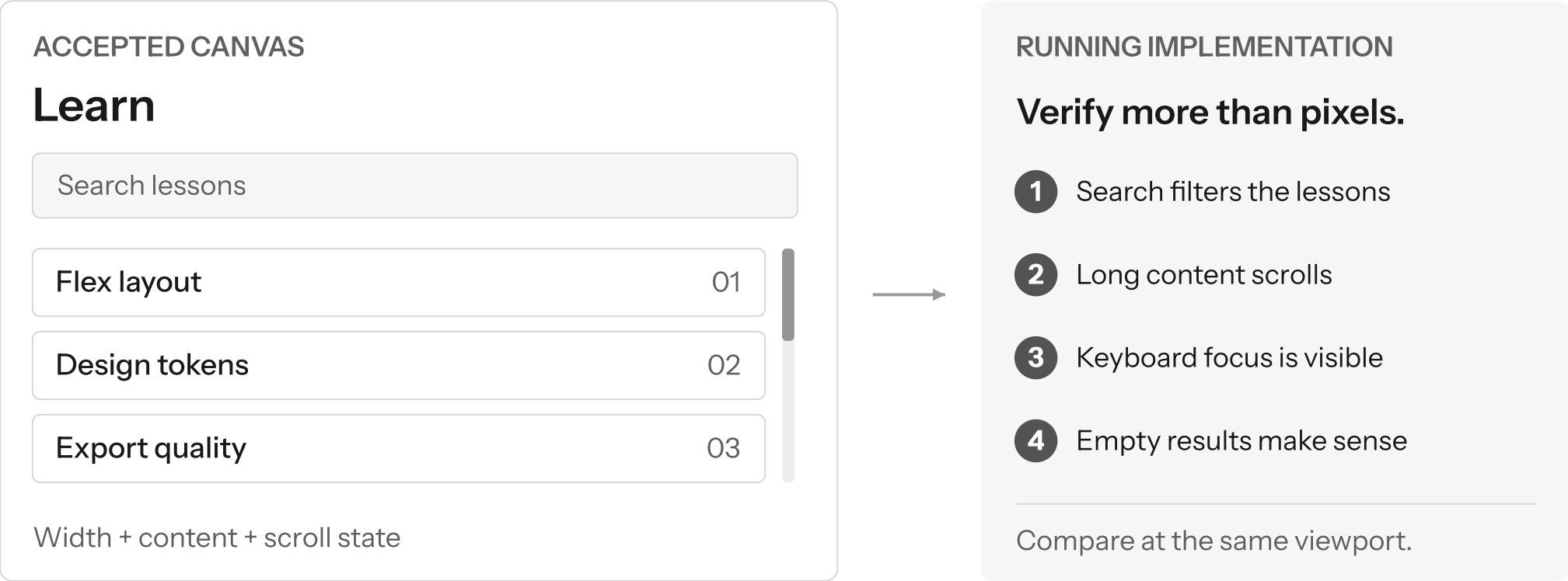
CSS rules

```
<section class="paper-fixture" aria-label="Illustrative metric summary">
  <h2>Scenario comparison</h2>
  <p class="basis">Example values</p>
  <div class="metrics">
    <div class="metric"><span>Baseline</span><strong>100.0</strong></div>
    <div class="metric"><span>Illustrative scenario</span><strong>112.0</strong></div>
    <div class="metric"><span>Difference</span><strong>+12.0%</strong></div>
  </div>
  <details><summary>Read the basis</summary><p>This static example compares the illustrative values 100 and 112. The difference is (112 - 100) / 100 = 12%.</p></details>
</section>
```

13 / PAPER IN PRACTICE

Port the intent, then test the behavior.

A matching screenshot is one checkpoint in a working product.



Reconstructed teaching example · synthetic values · not a Paper screenshot

Figure 13.1 · Plate

Port the intent, then test the behavior.

The reconstructed Learn page makes search, scrolling, focus and an empty result concrete acceptance checks. Compare the running page with the approved canvas at the same width and data state, then exercise the interactions.

Reuse the product’s working behavior

The Learn-page tutorial gives the agent a redesign and a scrolled state, names the existing components to reuse, and then tests the running result. This is a stronger handoff than requesting a visual clone of one screen. It acknowledges that the design is entering a product with established navigation, data and interaction rules: the agent copied the app’s existing sticky-header pattern and used its existing search field, so search worked with no new logic.

The exported or generated representation still needs an application context. A visual button must connect to the intended action. A selected row must have a meaningful state. A chart must receive validated values rather than the convenient sample shown on the canvas. Motion is handled the same way: Paper’s own website was designed statically in Paper, shaders included, and its animations were prompted on top of the file while an agent built the Next.js site.

For a manual handoff, Paper copies a selection as Tailwind (⌘T), as React with inline styles (Copy as React CSS, ⌘R) or as a PNG (⇧⌘C), from the right-click Copy as... menu and, since September 2026, from a copy icon in the Export section. Select a coherent nested unit, ideally one that is already a code component, rather than a whole page. Treat this as a present capability with its own scope: native Tailwind integration was still listed as in progress on Paper’s roadmap.

Figure 13.2

Two routes from frame to code

Compare the result at the level of the task

The first visual comparison should use the same viewport, content and state as the accepted frame. Differences then have a clear meaning. If the fonts, data and width all change together, it becomes difficult to identify whether a mismatch comes from implementation or from a different test condition. Resolve the controlled comparison before evaluating broader variation.

Then use the feature. Search for something absent, move through the navigation, resize the page and inspect the last item. The tutorial’s review of categories, scrolling and search illustrates this shift from appearance to behavior. An observed problem can belong to the frame, the extraction, the implementation or the review, so trace it to its stage before patching the code.

Review the change itself as carefully as the page. Read the agent’s summary of files changed and patterns reused, then the diff. In Paper’s token round trip, the agent changed only the five colors in the selection and listed two other variables that still used the old hue instead of silently changing them; that report is a decision handed back to you.

Choose by scope: a page with behavior, or one component's markup and styles.

Route	Scope	How	Output	Carries behavior	Source
Agent over the MCP	A frame or page, with behavior	Prompt: frame name, file to update, behavior to keep, reuse existing components	Edits to existing files, new routes and data files	Yes, when it reuses existing components	▶ 1:04
Copy as Tailwind	One layer, ideally a code component	⌘-click, right-click › Copy as... (⌘T), or the Export copy icon	Markup with Tailwind classes	No	▶ 3:05; ▶ 3:41
Copy as React CSS	One layer	Right-click › Copy as... (⌘R)	A React function with inline style objects, headed by the Paper link and date	No	▶ 3:18; ▶ 2:12
Write-back of edited values	Selected layers’ styles	Edit in Selection colors, then ask the agent to write them to the CSS file	A diff limited to the selection	Not applicable	▶ 5:13

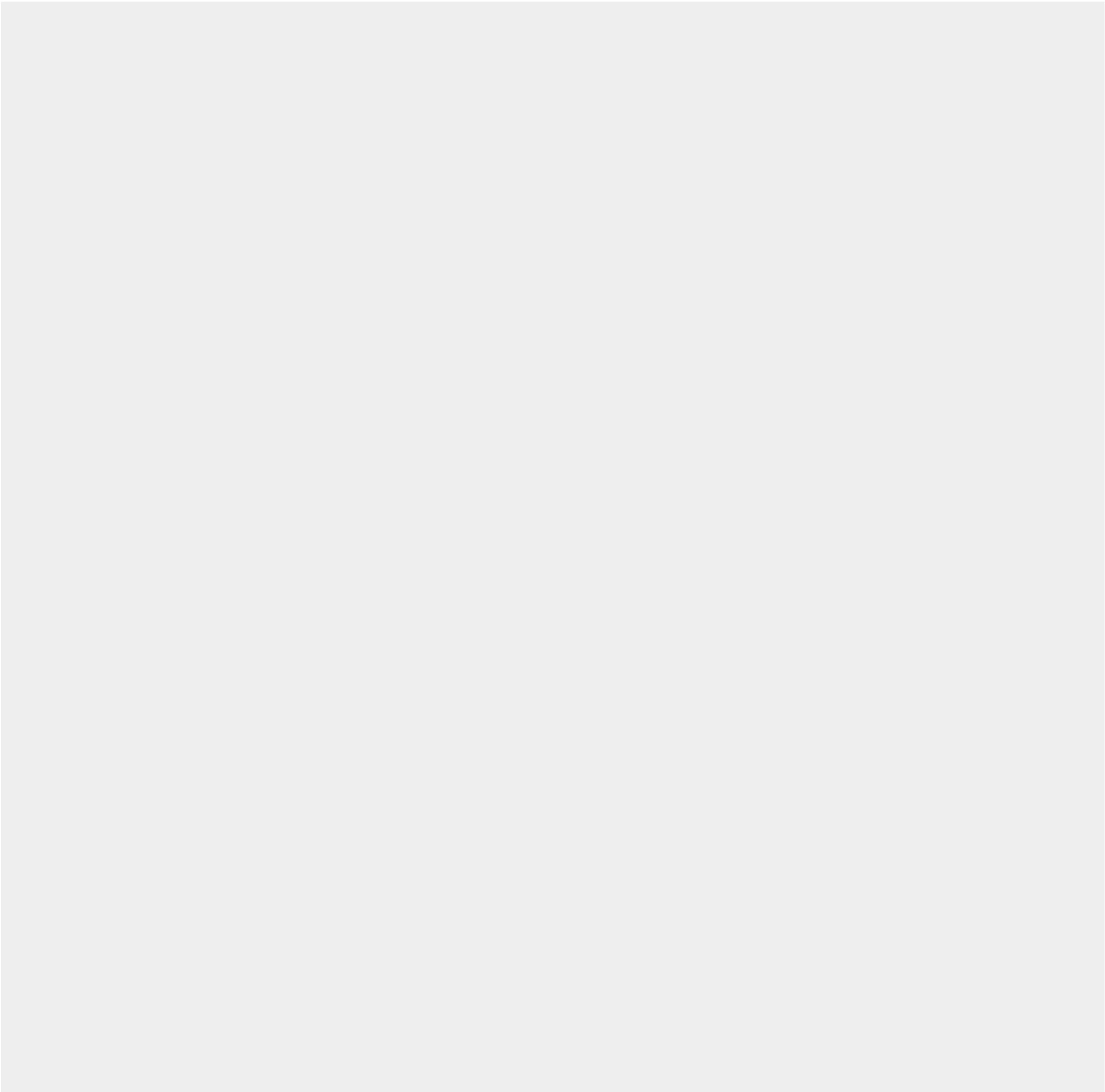
Figure 13.3

What a design-to-code agent reads, in order

Orient, look, then take exact values before writing code.

01	<code>get_selection</code>	PlaylistPage frame, 1440 × 900
02	<code>get_screenshot</code>	scale 2, visual reference
03	<code>get_tree_summary</code>	Header, TrackList (7 children), PlaybackBar
04	<code>read types/playlist.ts</code>	data shapes from the project
05	<code>get_computed_styles</code>	fontFamily Inter, gap 12px
06	<code>write components/playlist-header.tsx</code>	45 lines
07	<code>get_jsx</code>	TrackList markup

► 0:20 (Codex converting a selected Paper design)



How it works in Paper

Paper’s canvas is HTML and CSS, so there is no translation layer between a frame and code. As of September 2026 there are two routes out. An agent connected over the MCP reads the frame (get_jsx returns JSX with Tailwind classes or inline styles, get_computed_styles returns exact CSS) and edits your codebase. Or you copy a selection as Tailwind, React with inline styles or PNG from the right-click menu or the Export section. Behavior and motion are specified in prompts and built in code.

Checked against the videos and documentation as of September 2026. Paper ships often; the release timeline in the reference shows what changed when.

- 01 **Connect an agent to Paper**
- 1

Open Paper Desktop and the file you will work on; the MCP server runs inside the desktop app.
- 2

With nothing selected, click Connect your agent in the right panel’s MCP section.
- 3

Pick your harness (Cursor, Claude, Claude Code, GitHub Copilot, VS Code, OpenCode, Codex, Codex CLI, Antigravity or Other agents) and click its Add to ... button, or expand Or, install manually.
- 4

Restart the agent if needed and switch Paper on in its MCP list (the demo did both for GitHub Copilot), then run one of the dialog’s test prompts, such as creating a Hello World frame in Paper.
- 5

Approve tool calls as they come. In Claude Code, the second option, Yes, and don’t ask again, stops the prompts for that Paper tool in the current folder; the March 2026 tutorial recommends it for read tools and write tools alike.
- 6

Older route, still supported: the March 2026 tutorial ran `claude mcp add --scope user --transport http paper http://127.0.0.1:29979/mcp` in a normal terminal (not inside Claude) and confirmed the connection with `/mcp`.

► Shown at 0:56

- 02 **Lay out code values on the canvas**
- 1

In an agent session inside your repository, ask it to read a style file and arrange its values on the open Paper canvas (the demo used the repository’s colors.css).
- 2

Approve the tool calls: the agent calls get_basic_info, reads the file, checks typefaces with get_font_family_info and writes the board with write_html, which produces real HTML and CSS layers.
- 3

Check the board. The demo’s Color System board listed 22 tokens in three groups (Chromatic, Neutrals, Surfaces); each swatch showed a display name, its CSS variable, its oklch value and the usage note taken from the code comment.
- 4

Let the agent review its own screenshot. Here it found clipped bottom padding and set the artboard height to 1040px with update_styles.

► Shown at 1:11

- 03 **Edit values on the canvas and write them back**
- 1

Select the swatches to change (the demo selected five: four blues and the brand color).
- 2

In Selection colors, edit a row’s values, shown as lightness %, chroma, hue° / alpha; every selected layer using that color updates. The demo moved the hue from 258 to between 268 and 273.
- 3

Keep the selection and ask the agent to bring the colors you modified in your Paper selection back into the CSS file.
- 4

The agent reads Get Selection, Get Computed Styles and Get Node Info, maps each value to its variable and edits the file.
- 5

Review the diff (Keep File or Undo File in Cursor). The demo changed exactly the five selected variables and named two others that shared the old hue without touching them.

► Shown at 5:44

- 04 **Build a frame into an existing codebase**
- 1

Name the frames clearly and draw extra states as their own artboards (the demo: Learn — Redesign and Learn — Redesign Scrolled).
- 2

In your IDE, open a new agent and attach the file the design replaces (index.tsx in the demo).
- 3

Prompt with four parts: build the named frame from the open Paper file; update the attached file; keep the scroll behavior drawn on the canvas, which should match scrolling elsewhere in the app; and wherever the design uses an existing code component, such as the search field, use it instead of creating a new one.
- 4

Raise the effort for a real build: the demo switched Claude Opus 5 from Low to High and turned on Fast.
- 5

Answer the agent’s questions (three in the demo; the first asked what the All category should show), then read its summary of files changed, patterns reused and how the behavior works.
- 6

Run the page locally and exercise it before choosing Keep All.

► Shown at 1:32

- 05 **Copy one component as code**
- 1

Hold ⌘ and click to select the nested layer directly (the demo: a 216 × 32 Settings tab in the sidebar).
- 2

Right-click › Copy as... › Copy as Tailwind (⌘T) or Copy as React CSS (⌘R); Copy as PNG (⇧⌘C) copies an image instead.
- 3

Since September 2026 the same three options also sit behind the copy icon beside Export in the right panel.
- 4

Paste into your code. React CSS output opens with a comment (from Paper, the layer’s Paper link and the date) followed by an exported function whose elements carry inline style objects.
- 5

Prefer a unit that is already a code component; a whole frame copies too, but the useful difference gets buried.



► Shown at 3:05

- 06 **Read a selection the way an agent should**
- 1

Select the design in Paper and ask the agent to convert your selected design to code.
- 2

get_selection identifies it (the demo: a 1440 × 900 PlaylistPage frame).
- 3

get_screenshot captures it at scale 2 as a visual reference, and get_tree_summary lists its structure (Header, TrackList with 7 children, PlaybackBar).
- 4

The agent reads the project’s own types, then get_computed_styles for exact values (Inter, a 12px gap) and get_jsx for a subtree’s markup.
- 5

It writes components in the project’s conventions (components/playlist-header.tsx in the demo). Paper’s MCP instructions ask agents to take sizes and colors from these tools and to use screenshots only for checking.

► Shown at 0:22

Controls you will use

Control	Where	What it does
Copy as...	Right-click menu	Submenu with Copy as PNG (⇧⌘C), Copy as Tailwind (⌘T) and Copy as React CSS (⌘R) for the current selection.
Copy icon	Right panel › Export, beside +	The same three copy options, added to the Export section in September 2026 to make them easier to find.
Copy link	Top of the right panel (⌘L)	Copies a link to the selected frame or layer; a connected agent can open it, which makes it a precise way to hand over a frame.
Selection colors	Right panel, near the bottom	Lists each color in the selection as lightness %, chroma, hue° / alpha (or by token name) with a usage count; editing a row recolors every selected layer that uses it.
Connect your agent	Right panel › MCP, with nothing selected	One-click MCP setup per harness, with a manual install option and three test prompts.
get_jsx	Paper MCP tool	Returns a layer subtree as JSX, with Tailwind classes by default (inline styles as fallback) or pure inline styles.
get_computed_styles	Paper MCP tool	Returns the computed CSS of one or more layers in one call; the source of exact values for code.
get_screenshot	Paper MCP tool	Returns a PNG of a layer (scale 2 for small text) for visual checks. Never take sizes or colors from it.
get_tree_summary	Paper MCP tool	Lists a subtree’s layers with type, name and size; a cheap first read before get_jsx.
get_fill_image	Paper MCP tool	Returns an image fill with its original URL, so the asset can be stored with the code instead of loaded from Paper.

Shortcuts

⌘T	Copy as Tailwind ▶ 3:18	⌘R	Copy as React CSS ▶ 3:18
⬆⌘C	Copy as PNG ▶ 3:18	⌘Click	Select a nested layer directly ▶ 3:05
Esc	Select the parent layer ▶ 3:18	Enter	Select the children ▶ 3:18
⌘L	Copy a link to the selection ▶ 3:18		

Limits and what is coming

- Native Tailwind integration was listed as in progress on Paper’s roadmap as of September 2026; Tailwind output today comes from Copy as Tailwind and get_jsx.
- Paper has no prototyping or animation timeline: draw interaction states as separate artboards and specify motion in the prompt, to be built in code.
- Remote MCP was announced as coming soon on 28 September 2026; until it ships, the agent needs Paper Desktop running on the same machine with the file open.
- The March 2026 HTTP setup (127.0.0.1:29979/mcp) still works, but Paper’s documentation now recommends the stdio connection that Connect your agent installs.

What the tutorials show

Each official video, written down step by step. The dots mark the moments; they open the video there if you want to see it.

Push designs in Paper back to code

Video · 4 min 16 s · 2 September 2026

4:16

Push a redesign back into the codebase

The presenter starts from Paper’s own Learn page, running locally, and pulls it into Paper with one short Cursor prompt naming index.tsx and the dashboard sidebar. On the canvas she redesigns it with a category rail and the existing Search component, and draws the scrolled state as a second artboard. A new agent (Claude Opus 5, effort raised to High, Fast on) gets a four-part prompt: the frame, the file to update, the scroll behavior to keep and match, and reuse of existing components. It asks three questions first, such as what the All category should show. Its summary lists the rewritten page, a new content file with the 23 cards tagged by category, a category route, the app’s existing sticky-header pattern and the existing SearchField. In the browser, categories filter, the header animates on scroll and search narrows the results. Finally she ⌘-clicks the sidebar’s Settings tab and copies just that layer as Tailwind or React CSS.

- 0:24 One prompt brings the page in

1:32 Four-part build prompt

1:49 The agent’s questions

2:15 Change summary
- 2:25 Categories, scroll and search work

3:18 Copy as Tailwind or React CSS

From design to code in Paper

Short · 14 s · 3 September 2026

0:14

The round trip in fourteen seconds

A vertical teaser cut from the push-to-code tutorial; it adds no new information. It shows the short code-to-canvas prompt that rebuilt the Learn tab in Paper, the redesigned artboard with the reused Search component (200 × 28) selected, and the longer canvas-to-code prompt beside the running page. The shape to remember: a short prompt brings code in, a specific one sends the design back.

- 0:00 Code-to-canvas prompt

0:02 Reused Search component

0:04 Canvas-to-code prompt

Design first, prompt the animations after

Short · 50 s · 24 August 2026

0:50

Design the statics, prompt the motion

In a podcast clip, Paper’s co-founder describes how the company’s brand designer built the paper.design site in Paper in about a week before a launch, shaders included, then prompted the animations on top of the Paper file while an agent turned it into a Next.js codebase; this was about six months before the August 2026 clip, with older models. The footage shows Codex converting a selected Paper design: get_selection, get_screenshot at scale 2, get_tree_summary, a read of the project’s types, get_computed_styles, a new playlist-header component, then get_jsx for the track list. Interactive extras, such as a parameters panel driving floating dots, were ones the designer would not otherwise have had time to build.

- 0:01 Site designed in Paper in a week
- 0:18 Animations prompted on top
- 0:22 Codex reads the selection
- 0:30 get_computed_styles, then get_jsx
- 0:38 Interactive parameters panel

How to sync designs with your codebase using Paper MCP, Claude Code and Cursor

Video · 7 min 19 s · 5 March 2026

7:19

A color round trip with Claude Code and Cursor

As of March 2026, the presenter connects a new Paper file to Claude Code with the one-line claude mcp add command from Paper’s docs (HTTP to 127.0.0.1:29979/mcp), run in a normal terminal, and confirms the connection with /mcp. Asked to arrange the colors from colors.css on the canvas, Claude reads the file, checks fonts and writes real HTML and CSS: a Color System board with 22 tokens in three groups, each with its CSS variable, oklch value and usage note. It screenshots its work, finds clipped padding and fixes the artboard height. He then adds Paper to Cursor with the docs’ Add to Cursor button, selects five blue swatches, raises their hues in Selection colors and asks Cursor to bring the modified colors back into colors.css. Cursor reads the selection and computed styles, updates exactly those five variables, flags two others that share the old hue and offers a diff to keep or undo.

- 0:54 claude mcp add in a terminal
- 1:11 Prompt: colors onto the canvas
- 3:12 Color System board
- 3:52 Self-review fixes the height
- 5:20 Hues edited in Selection colors
- 6:48 Diff limited to the selection

Build it in Paper

- 01

Name the accepted frame and draw each behavior state that matters (scrolled, empty, selected) as its own artboard beside it.
- 02

Connect your agent (right panel › MCP › Connect your agent) and prompt with four parts: the frame’s name in the open Paper file, the file to update, the behavior to keep and where it already exists in the app, and an instruction to reuse existing components.
- 03

Answer the agent’s questions, then read its summary of files changed and components reused before you keep the changes.
- 04

For a single component, ⌘-click to reach the nested layer, then right-click › Copy as... › Copy as Tailwind (⌘T) or Copy as React CSS (⌘R).
- 05

Run the page at the frame’s width with the same content and compare; then test the task: switch categories, scroll, search for something absent and narrow the window.

Common mistakes

- ×

Handing over a still frame and letting the agent guess the behavior.

→ Draw states such as scrolled or selected as separate artboards, name the existing pattern to match, and answer the agent’s questions before it writes code.
- ×

Copying a whole page as Tailwind to change one element.

→ ⌘-click the nested layer that matches a code component and copy only that, or use the agent route for page-level changes.

Check before you move on

- ☐

Is every state the code must handle either drawn as its own artboard or written into the prompt?
- ☐

Did the values come from get_jsx and get_computed_styles rather than from a screenshot?
- ☐

Does the implementation reuse the components the design came from, instead of new look-alikes?
- ☐

Have you compared the running page with the frame at the same width, content and state?
- ☐

Have you used the feature: navigation, scrolling, a search with no result, a narrow window?

Publish an artifact that survives leaving the canvas

An export is a new rendering in a new context. Its quality depends on the frame you select, the format you choose and the result you inspect.

Choose the format for the reader’s job

An image communicates a fixed appearance. A PDF can preserve a sequence of pages and searchable text. A website can add navigation, responsive layout and interaction. These formats serve different reading tasks. Choose the deliverable according to what the reader needs to do, rather than treating every output as an interchangeable container for the same screenshot.

Paper’s outputs map onto those jobs. As of September 2026 a frame exports as PNG, JPG, AVIF, WebP or PDF at a chosen scale; a copy command puts a PNG, Tailwind or React CSS on the clipboard; video export covers moving work such as shaders on the Pro plan; and an email reaches its readers through an agent and an email service rather than as a file. Because Paper renders with HTML and CSS, its PDFs go through the browser’s print path and keep real, searchable text.

The source frame matters as much as the export format. A frame designed for a wide screen may produce tiny labels when fitted onto a page. A useful publication layout gives the reader a deliberate sequence, appropriate type size and a meaningful crop. In Paper the frame also names the file, and in a combined PDF its position on the canvas sets the page order, so exporting the right selection is the first design decision.

—

The delivered artifact is the result to judge, not the export button’s success message.

The principle of chapter 14

Export the artifact you can actually inspect

Set the size and background, then inspect the exported file.

Scenario comparison

Baseline	Scenario	Difference
100.0	112.0	+12.0%

Example values

SVG: 800 × 450 · viewBox 800 × 450 · opaque background.

SVG preserves this fixture’s shapes and text as vector content. Open the downloaded image and inspect its labels, transparency, and reading size.

The export is the thing you must inspect.

Check the final PDF at its destination, not just the source canvas.

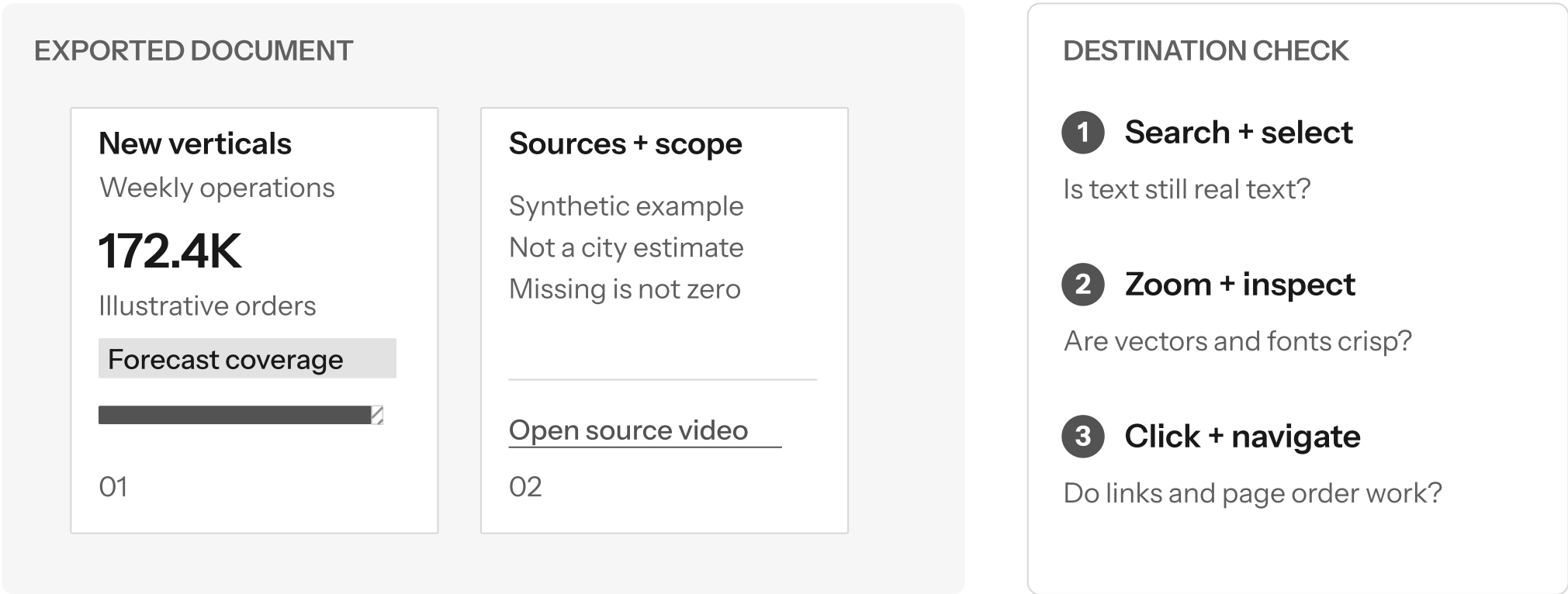


Figure 14.1 · Plate

The export is the thing you must inspect.

The channel demonstrates searchable PDF export. Validate the actual file: search and select text, inspect fonts and vectors at high zoom, follow links and confirm page order. Export and publication are separate states.

Inspect what the exported file actually contains

Paper’s PDF tutorial selects frames and demonstrates single, separate and combined exports, then searches the exported text. That final action is part of the lesson. The presence of a PDF file does not establish that the text remains useful, the page order is correct or the content fits comfortably inside the chosen dimensions.

Read the export in the environment where it will be used. Check a page at a comfortable reading size, inspect small labels and look at images whose quality may change with resampling. Search for a distinctive word and select a sentence. If the artifact includes links, test the links rather than assuming they survived because they were visible in the design.

A still file cannot show a shader’s movement or an interface’s states by itself. Export a meaningful still, explain the behavior, and provide a working reference where it matters: a video export for motion, the coded page for interaction. Renderers change too: in September 2026 Paper began rendering images and videos with HTML in Canvas, which it says makes exports faster and higher fidelity, so compare a fresh export with an older one before reusing a preset.

Keep the publication state honest

Designing a website in Paper and publishing a functioning website are separate stages. The accepted design still needs implementation, hosting and verification at the destination. The exported code or local preview may be valuable progress, but it is not evidence that the public or private published URL serves the intended build. Inspect the delivered address and its important routes.

Paper’s email tutorial makes those stages visible. The agent created the template in Resend as a draft, warned that the logo still loaded from a Paper-hosted asset that should move to the sender’s own hosting before production, and noted that raw HTML is not editable in Resend’s visual editor until converted. A test send with sample variables came before publishing, and publishing a new version changes what is sent without code changes.

Give the deliverable an identifiable date or version so feedback refers to the right edition. Paper names each export after its frame and writes it to the Downloads folder, so rename and file it deliberately. Preserve the editable source alongside the finished output. A good publication should be useful now and straightforward to correct when content or product capabilities change.

Figure 14.2

Paper's export options

Pick the route by the reader's job, not by habit.

Route	Output	Destination	Use it for	Source
Export ⌘⌘E	PNG, JPG, AVIF, WebP or PDF per element, at a scale such as 1x or 2x	Downloads, named after the frame	Images and single documents	▶ 0:20; ▶ 4:24
Export N elements	One PDF per selected frame	Downloads	Separate handouts	▶ 1:04
Export combined PDF	One PDF, a page per frame in canvas order	Downloads, Combined.pdf	Decks and reports	▶ 1:04
Copy icon or Copy as...	PNG, Tailwind or React CSS	Clipboard	Quick sharing and code snippets	▶ 3:41
Video section	Video of moving content (MP4 or WebM through the MCP)	Downloads	Shaders and motion (Pro plan)	Paper pricing and MCP export tool
Agent with an email service	Table-based email template with variables	The email service, as a draft	Transactional email	▶ 5:18

Figure 14.3

From captured page to sendable email

Every stage has a check, so each kind of error is caught before the template is published.

▶ 0:56	Capture the template with Paper Snapshot and paste it
▶ 1:24	Reskin prompt using the on-canvas style guide
▶ 3:19	Agent’s screenshot check: spacing, typography, contrast, alignment, artboard fit, repetition
▶ 3:45	Manual edits to the footer
▶ 4:22	Agent re-reads computed styles to learn the edits
▶ 5:18	Paper link handed to a second agent session
▶ 6:49	Table-based HTML, variables, draft template and warnings
▶ 7:24	Converted for visual editing; sender and subject set
▶ 7:57	Test send with JSON variables, then publish

How it works in Paper

As of September 2026 the Export section at the bottom of the right panel exports a selection as PNG, JPG, AVIF, WebP or PDF at a chosen scale, one file per frame or one combined PDF, always into the Downloads folder. PDFs use the browser’s print engine, so text stays searchable. A copy icon in the same section puts a PNG, Tailwind or React CSS on the clipboard, and images and videos now render through HTML in Canvas.

Checked against the videos and documentation as of September 2026. Paper ships often; the release timeline in the reference shows what changed when.

01

Export one frame as a PDF

1

Select the frame (the demo used a one-page resume, 612 × 792).

2

Scroll to Export at the bottom of the right panel; if the section has no export row, click + to add one.

3

Set the format dropdown to PDF. The scale field beside it read 1x.

4

Optionally click the settings (sliders) icon: File suffix (empty, with the placeholder No suffix), Color space (sRGB), Quality (High, Medium or Low) and Image resampling (Detailed or Basic).

5

Click Export ⇧⌘E. The PDF goes to Downloads, named after the frame.

⇧

⌘

E

►

Shown at 0:20

02

Export several frames, separate or combined

1

Select all the frames (the demo: four 1920 × 1080 slides in one row).

2

With PDF as the format, the Export section shows two buttons: Export 4 elements writes one PDF per frame; Export combined PDF writes one multi-page file.

3

Click Export combined PDF. A toast reads Exported 1 file to Downloads; the file is Combined.pdf, 554 KB for the four slides.

4

Pages follow the frames’ positions on the canvas, not the layer list: the title slide, last in the layers panel but first on the left, became page 1.

►

Shown at 1:08

03

Check the exported PDF

1

Open the file in Preview and check the page count and order in the sidebar.

2

Press ⌘F and search a word from the design; the demo’s search for quality found one match, on page 3, which shows the text is real text.

3

Zoom on the smallest labels and on images, whose quality depends on the Image resampling setting.

4

Optionally give the PDF to an AI assistant. Asked for a one-sentence summary, Claude correctly described single, separate and combined export and the adjustable settings.

⌘

F

►

Shown at 1:22

04 **Export an image at a chosen scale**

- 1 Select the frame or layer and go to Export in the right panel.
- 2 Set the scale field (2x in the June 2026 changelog demo) and choose PNG, JPG, AVIF or WebP in the format dropdown.
- 3 Click Export; each selected element becomes its own file in Downloads.
- 4 Use + beside Export to add an export row and – to remove one.

► Shown at 4:24

05 **Copy to the clipboard instead of exporting a file**

- 1 Select the layer.
- 2 Click the copy icon beside Export in the right panel, or right-click and open Copy as...
- 3 Choose Copy as PNG (⇧⌘C) for an image, or Copy as Tailwind (⌘T) or Copy as React CSS (⌘R) for code.
- 4 Paste it where it is needed: a message, a document or your code editor.



► Shown at 3:53

06 **Turn a frame into a sendable email template**

- 1 Finish the email frame in Paper and copy its link with Copy link (⌘L) at the top of the right panel.
- 2 In an agent session that has Paper’s plugin and the email service’s plugin (Resend’s in the demo), ask it to make the frame a Resend template and paste the link.
- 3 Approve the Paper tool calls; the agent first loads Paper’s instructions with get_guide, then the provider’s skill.
- 4 The agent rebuilds the design as table-based email HTML, turns values that change per email into variables ({{{RESET_URL}}}, {{{RESEND_UNSUBSCRIBE_URL}}}), creates the template as a draft and returns a preview link.
- 5 In Resend, convert the HTML to the visual editor if others will edit it, set the sender and subject, and send a test with Test variables as JSON, such as {"RESET_URL": "https://resend.com"}.
- 6 Move images that still load from Paper to your own hosting, then publish; a template must be published before it can be sent.

► Shown at 5:34

Controls you will use

Control	Where	What it does
Export	Right panel, last section	Holds the selection’s export rows; + adds a row and – removes one.
Scale	Export row, left field	Output size as a multiplier, such as 1x or 2x.
Format	Export row, dropdown	PNG, JPG, AVIF, WebP or PDF.
Settings	Export row, sliders icon	PDF options: File suffix, Color space, Quality (High, Medium, Low) and Image resampling (Detailed, Basic).
Export ⇧⌘E	Button under the export row	Writes one file per selected element to Downloads, named after the element.
Export N elements / Export combined PDF	Replace the Export button when several frames are selected with PDF chosen	Separate PDFs, or one PDF with a page per frame in canvas order.
Copy icon	Beside Export in the section header	Copy as PNG, Copy as Tailwind or Copy as React CSS to the clipboard (added September 2026).
Video	Right panel, just above Export	Video export settings for moving content such as shaders; video export is a Pro plan feature.
Copy link	Top of the right panel (⌘L)	A link to the selected frame, used to hand a design to a teammate or an agent.
export / export_combined_pdf	Paper MCP tools	Let an agent export: PNG, JPG, AVIF, WebP, PDF, SVG (SVG layers only), MP4 or WebM, at scales such as 2x, 720p or 512w; combined PDFs are ordered top to bottom, then left to right. Neither tool chooses an output folder.

Shortcuts



Export the selection with its export settings ▶ 0:42



Copy as PNG ▶ 3:53



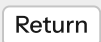
Copy as Tailwind ▶ 3:53



Copy as React CSS ▶ 3:53



Copy a link to the selected frame ▶ 4:00



Paper Snapshot: capture the highlighted element
▶ 0:56

Limits and what is coming

- Exports always go to the Downloads folder, named after the frame (Combined.pdf for a combined PDF); neither the app nor the MCP export tools choose a folder, as of September 2026.
- SVG export works only for SVG layers, per Paper’s MCP export tool; frames export as PNG, JPG, AVIF, WebP or PDF.
- Video export requires the Pro plan, per Paper’s pricing page in early October 2026.
- Email clients need table-based HTML: the agent and the email service convert Paper’s HTML and CSS, and images may still load from Paper-hosted URLs that should move to your own hosting before production.

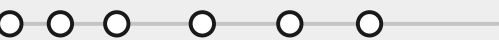
What the tutorials show

Each official video, written down step by step. The dots mark the moments; they open the video there if you want to see it.

How to create high quality, searchable PDFs in Paper

Video · 2 min 49 s · 2 June 2026

2:49



High-quality, searchable PDFs

The tutorial starts with why Paper’s PDFs differ: the canvas is HTML and CSS, so export goes through the browser’s print-to-PDF path and text stays text. A one-page resume frame (612 × 792) is selected; in Export, at the bottom of the right panel, the format is set to PDF at 1x. The sliders icon opens Settings: File suffix, Color space (sRGB), Quality (High) and Image resampling (Detailed). Export ⌘⇧E writes the file to Downloads under the frame’s name. With four 1920 × 1080 slides selected, the section offers Export 4 elements for separate files or Export combined PDF; the combined file, Combined.pdf, weighs 554 KB and orders its pages by canvas position. In Preview, ⌘F for quality jumps to page 3, and Claude, given the file, summarizes the feature correctly in one sentence. The point is to judge the file, not the button.

0:03 Why the text stays searchable

0:20 Export section set to PDF

0:39 PDF settings

1:08 Separate or combined

1:38 Search finds page 3

2:05 AI reads the PDF

Using Paper and Resend to turn a design into a ready-to-send email template

Video · 9 min 15 s · 5 June 2026



From a captured email to a published Resend template

The presenter captures a password-reset template from React Email’s demo site with Paper Snapshot (Return captures the highlighted element) and pastes it into Paper. Beside it sits a style guide: three swatches named by hex, a font note (System Sans-Serif), a logo and the reskin prompt saved as a text layer. Claude Code duplicates the email and restyles it for a fictional brand, Folio, keeping the 640 and 560 px widths, then checks a screenshot for spacing, typography, contrast, alignment, artboard fit and repetition. She restyles the footer by hand and asks the agent to confirm it understands; it reads computed styles and lists the exact new values. A developer from Resend pastes the frame’s Paper link into his own Claude Code and asks for a Resend template. With Paper’s plugin and Resend’s skill, the agent builds table-based HTML with a {{{RESET_URL}}} variable and saves a draft. He converts it for visual editing, sets the subject and sends a test with JSON variables; publishing comes last.

- 0:56 Snapshot capture
- 1:24 Style guide and reskin prompt
- 3:20 The agent’s design check
- 4:48 The agent reads the manual edits
- 6:52 Draft created, with warnings
- 8:06 Test send with variables

Build it in Paper

- 01 Lay out the frames in reading order on the canvas, left to right, at page proportions (612 × 792 for a letter page, 1920 × 1080 for a slide), and name each frame the way its file should be named.
- 02 Select them, scroll to Export at the bottom of the right panel and set the format to PDF; in the settings (sliders) icon choose Quality and Image resampling, plus an optional File suffix.
- 03 Click Export combined PDF for one document, or Export N elements for one file per frame (Export ⇧⌘E for a single frame); the file lands in Downloads.
- 04 Open the file: check page order, zoom on the smallest labels, search a distinctive word with ⌘F and test any links.
- 05 For an email, give the frame’s Paper link (Copy link, ⌘L) to an agent with the email service’s tools, then preview, send a test with sample variables and publish only after it passes.

Common mistakes

- × Ordering slides in the layers panel and expecting the combined PDF to follow it.

→ Arrange the frames on the canvas; combined PDFs follow canvas position (left to right in the demo; Paper’s MCP tool orders top to bottom, then left to right).
- × Exporting a wide screen frame as a document page.

→ Build a page-proportioned frame, such as 612 × 792 or 1920 × 1080, with type sized for reading, and export that instead.
- × Publishing an email whose images still load from Paper.

→ Ask the agent to list asset URLs, host the images yourself, and send a test with sample variables before publishing.
- × Looking for exports in the project folder.

→ Exports land in Downloads under the frame’s name; rename and move the file as part of the export step.

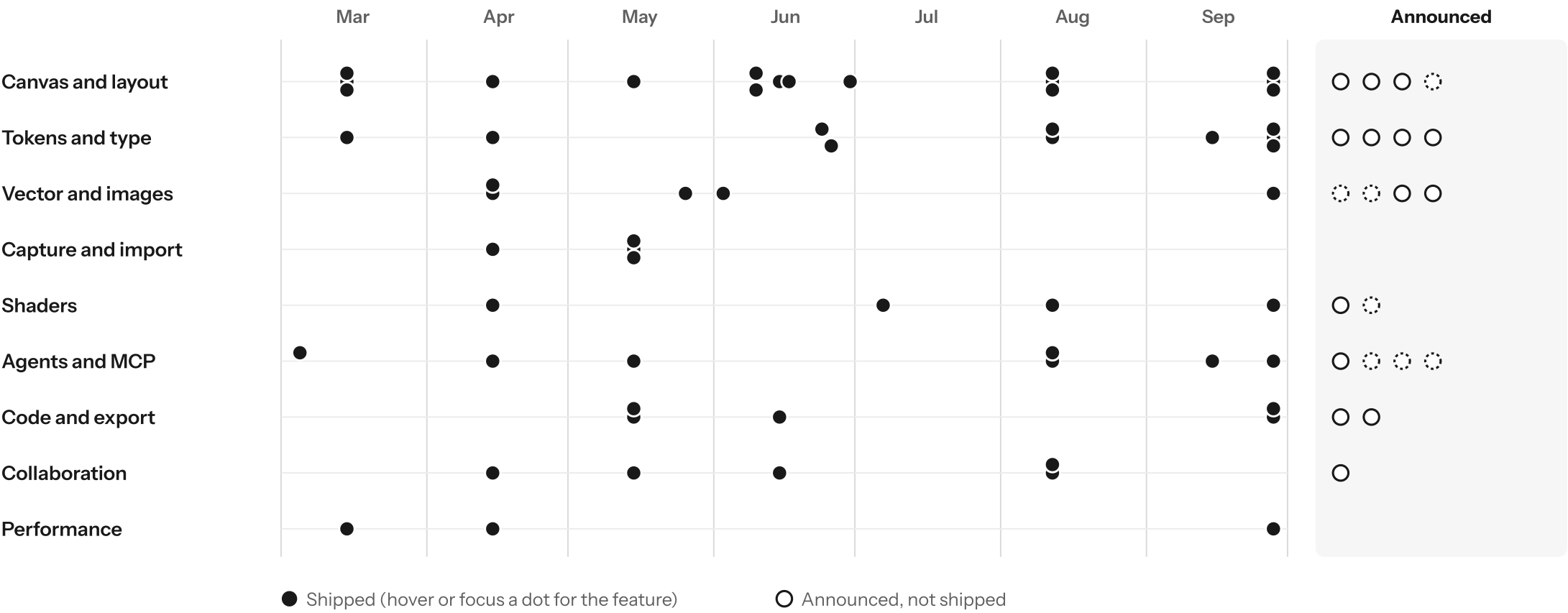
Keep this open while you work.

What shipped and when, what exists today, every shortcut, fixes for common problems, briefs you can hand to an agent, and the vocabulary.

R1	Releases, month by month
R2	What exists today
R3	Keyboard shortcuts
R4	When something goes wrong
R5	Briefs for your agent
R6	Glossary

R1• What shipped, month by month

60 releases from March to September 2026, and 37 features announced but not yet shipped.



March	6	April	9	May	9	June	10
Constraints panel — Layers outside Flex flow gained constraints that decide how they respond when their parent is resized.		Paper Snapshot — A Chrome extension copies part or all of a live web page into Paper as editable HTML and CSS layers; the first tutorial followed on 2 May.		Copy and paste from Figma — Layers copied in Figma paste into Paper as editable layers with their SVGs and images; a beta tutorial followed on 3 June.		Negative gap — Flex gap accepts negative values, so children can overlap.	
Fit and Fill by double-click — Double-clicking a frame's edge or handle resizes it to fit its content, and ⌘ + double-click makes it fill its parent.		Teams with Editor and Admin roles — Teams can invite members as Editors or Admins, and people outside the team get view-only access.		PDF export — Frames export as searchable PDFs, one per frame or combined into one file, saved to the Downloads folder.		Higher-fidelity PDFs — PDF export began keeping shadows, gradients and blend modes.	
Backdrop filters — Layers can filter whatever sits behind them, as CSS backdrop-filter does.		New image and SVG models — GPT Image 2, Nano Banana 2 and Quiver Arrow 1.1 joined the generation models.		Agents create and open files — New MCP tools let an agent create a Paper file, open existing files and change pages instead of working only in the open file.		Presence for teammates and agents — Files show who is present, including agents at work.	
OpenType features panel — Text gained a panel for turning on a font's OpenType features.		Shortcuts for generation and upload — ⌘⇧I opens image generation, ⌘⇧J SVG generation and ⌘⇧K uploads images from disk.		Paste HTML as layers — HTML with inline styles pastes onto the canvas as editable layers.		SVG editing and the pen tool — Paths inside SVGs can be edited node by node and the pen tool (P) draws new ones, released as a beta.	
Lower memory use — The March release cut Paper's memory use.		Agents can move layers — A new MCP tool lets agents reorder and reparent layers, alongside a refined system prompt for agents.		Other styles panel — CSS that Paper renders but has no dedicated control for, often added by agents, appears in an Other styles panel.		Top-layer hover selection — Hover now targets the top-level parent as in Figma; double-click or hold ⌘ to go deeper, or turn on Use deep selection.	
Paper MCP server in Paper Desktop — Paper Desktop runs a local MCP server that coding agents connect to; the first setup tutorial linked Claude Code over HTTP and confirmed it with /mcp.		Text formatting shortcuts — Shortcuts step font size, weight, letter spacing and line height, and ⌘B and ⌘I toggle bold and italic.		Guests, follow mode and cursor chat — Files support view-only access and anonymous visitors, you can follow a collaborator's view, and / opens cursor chat.		Layer picker on ⌘ + right-click — Holding ⌘ and right-clicking lists every layer under the pointer, down to the smallest shape.	
		Gem Smoke shader — A logo animation shader that reacts to the shape it is given joined the library.		Minimal interface — ⇧. collapses the panels into floating controls, leaving more of the canvas visible.		Folders and nested folders — The dashboard's Files view gained folders that nest to any depth, with Move to and rename.	
		Refreshed interface and H for panning — The interface was refreshed, and H selects the hand tool for panning the canvas.		Full-page Snapshot — ⌘Return in the Snapshot extension captures an entire page, with a progress percentage.		Design tokens through the MCP — Agents create tokens from a codebase's CSS variables or from the canvas, with types modeled on Tailwind; at first the MCP was the only way to create them.	
		Memory down 10 to 20 percent — A further release reduced memory use by 10 to 20 percent.		Grok image model from xAI — Grok joined the image models; in the tutorial it returned a 4:3 image in about two seconds.		Copy tokens between files — Copy theme and Paste theme, in the menu under the Paper logo, move a file's tokens into another file.	
						Collapse all layers and resizable panels — Right-click in the layers list to collapse all layers, and drag to resize the left panel and the pages list.	

July1	August11	September14
Paper Shaders open source and free — Paper's shader library became open source and free to use in websites, apps, templates and plugins.	Tabs in the desktop app — Several files open as tabs, and + in the tab bar opens a new or existing file. Agents across open files — Agents can work in any open file, including background tabs, so prompts should name the target file. Tokens in the Theme tab — Tokens of every type can be created and edited in the left panel's Theme tab, and edits update every layer that uses them. Favorite fonts — Starred fonts appear in a Favorites tab wherever you pick a font, including token editors. Flex wrap — Children of a Flex frame can break onto new lines, chosen with Wrap in the Flex section's menu. Repeatable duplicates — After an ⌘-drag duplicate with a rotation, ⌘D repeats both the offset and the rotation. Dashboard multi-select and file duplication — ⌘-click selects several files and folders to move, archive or copy links, and files can be duplicated. Free editors and viewer roles — Free plans allow unlimited editors, and viewer seats are free on every plan. Comments — Press C to comment anywhere, then reply, react and resolve, with a panel that searches, sorts and filters threads in comment mode. Comments as agent instructions — You can leave comments on a frame and ask an agent to implement them. Lens Distortion shader — An image filter that bends geometry and splits color channels like a camera lens, usable on any layer including text.	WebMCP in the browser — Paper on the web added WebMCP support for agents that run in the browser. CJK word-keeping — Text wrapping can keep Chinese, Japanese and Korean words intact. Faster performance — Paper uses about half its previous memory, switches tabs faster and pans and zooms more smoothly on large files. One-click agent setup — A Connect your agent button in the right panel sets up the Paper MCP for Cursor, Claude, Claude Code, GitHub Copilot, VS Code, OpenCode, Codex, Codex CLI or Antigravity, with an Other agents option for the rest. Image generation through the MCP — Agents can generate images on the canvas; asked for a variety pack, one produced four labeled images from OpenAI Image 2.5, Nano Banana 2, Flux 2 Pro and Recraft V4.1. Opacity tokens — Opacity is a token type that pairs with a color token in a fill, for example the ink color at 60%. Color tokens from a fill — The token icon beside a fill's color has a + that turns the raw value into a new color token; the build log says this works from any color field. Token picker views — Color tokens in a fill's token menu can be shown as a list, a small grid or a large grid. Dashboard search across folders — Dashboard search (⌘F) looks through every folder and opens results directly. Personal drafts — The September dashboard shows a Drafts section and a Scratchpad file labeled as your permanent draft, after drafts were announced as next on 12 August; the video does not narrate it. Reworked flex panel — Agent-built Flex layouts open in the panel, Space between sits beside the gap field, wrapped layouts get separate horizontal and vertical gaps, and grid elements support alignment and gap. Copy as code from the Export section — A copy icon beside Export offers Copy as PNG (⇧⌘C), Copy as Tailwind (⌘T) and Copy as React CSS (⌘R), the options already in the right-click menu. HTML in Canvas rendering — Images and videos are now powered by HTML in Canvas, which makes exports faster and higher fidelity. Paper Texture shader rebuilt — The shader was rebuilt for rendering quality and speed, with more controls and new textures such as wrinkles, folds and crumples.

R2· What exists today, and what is coming

- Available
- ◐ Partly available
- In progress or coming soon
- 🗓 Planned

Status as documented or announced by Paper in September 2026.

Flex layout and wrapping ● Available Rows and columns with gap, padding, alignment, Fixed / Fit / Fill sizing and wrapping. It is CSS flexbox underneath.	Native CSS Grid authoring 🕒 Planned Frames that are already grids get alignment and gap controls, but building grids from scratch is on the roadmap. Use nested Flex frames for now.
Tokens within a file ● Available Change a token and every layer that uses it follows. Agents can read, create and change tokens over MCP.	Linked token libraries across files 🕒 Not yet You can copy and paste tokens between files, but the copies are not linked. Update each file yourself.
Several theme modes 🕒 On the roadmap Switchable modes such as light and dark are described as future work. Label alternative themes clearly until then.	Components with slots ○ Coming soon Until they ship, keep one canonical frame per pattern and duplicate it deliberately.
Real code components on the canvas ○ In progress Today an agent can rebuild your code components as editable layers. They are not linked back to the code.	Copy as React CSS or Tailwind ● Available From the Export section or the context menu. Agents can also read exact JSX and computed styles over MCP.
Native Tailwind integration ○ In progress A deeper integration that is separate from today's Copy as Tailwind.	Snapshot web capture ● Available A Chrome extension that captures a region of a live page as editable layers.
SVG paths and the pen tool 🔦 Partly available Draw and edit paths. More advanced vector operations are still to come.	Inline rich text ○ Coming soon Mixed styles inside one text layer. Check imported text carefully until it ships.
MCP server in Paper Desktop ● Available Agents read and change open files through the local server that runs with the desktop app.	Remote MCP ○ Coming soon Connecting an agent without the desktop app has been announced, not shipped.
Paper Shaders ● Available Live effects on the canvas. The same effects run in code through the open-source shader packages.	Lottie, Rive and YouTube embeds 🕒 Planned On the roadmap. Do not plan a design around them yet.
PDF and combined PDF export ● Available Export frames as separate PDFs or as one combined document with searchable text.	SVG export 🔦 Available, restricted Over MCP, SVG export applies to SVG layers.
Comments and tabs ● Available Comment threads on the canvas, which agents can read, and several files open in tabs.	Permissions per file ○ Coming soon Announced for teams. Every editor of a file can currently change all of it.
Scale tool ○ Coming soon Select several layers and scale them together at the same ratio; on 12 August an agent scaled a frame 3x as a stopgap.	Rich text editing ○ Coming soon Bold, italic or a gradient fill on a word or phrase inside a larger text layer.
Theme modes for tokens ○ Coming soon Modes such as light and dark for a token set; a team reply in early September again called theme support coming soon.	Theme classes ○ Coming soon Classes that bundle several tokens or styles, such as H1, H2 or paragraph, into one choice.
Themes and tokens with calc() and color-mix() ○ In progress Listed as in development on the public roadmap.	More shaders ○ In progress Listed as in development; the July video that opened the shader library also promised more shaders.
Asset hosting ○ In progress Listed as in development on the public roadmap.	Icon packs, shadcn and kits ○ Coming soon Icon packs, shadcn and other kits, with Base UI named in the roadmap entry, listed as coming soon.
Script and prompt engine 🕒 Planned Listed as planned on the public roadmap.	Right-click Remix 🕒 Planned Listed as planned on the public roadmap.
Video generation 🕒 Planned Listed as planned on the public roadmap.	LLM search 🕒 Planned Listed as planned on the public roadmap.
Particles 🕒 Planned Listed as planned on the public roadmap.	Advanced image filters 🕒 Planned Listed as planned on the public roadmap.
three.js islands 🕒 Planned Listed as planned on the public roadmap.	Boolean operations ○ Coming soon Unite and subtract for vector shapes, on the near-term vector roadmap.

Improved snapping

Coming soon

Snapping pen points to other geometry; on 3 June points snapped only to half pixels.

Convert shapes to paths

Coming soon

Turning shapes into editable paths, on the near-term vector roadmap.

Mirroring

Planned

A mirror modifier that reflects a drawing as you edit it, shown as a prototype on the long-term roadmap.

Anchoring

Planned

Attaching an object at a position along a path so it can slide along it, shown as a prototype.

Anneal brush

Planned

Listed on the long-term vector roadmap board.

Delete a node without breaking the path

Planned

Asked for as an equivalent of Figma's ⌘Delete, the team replied that it is on the roadmap.

Aspect-ratio lock when resizing

Coming soon

A lock for the aspect ratio of an existing frame, called coming soon; an earlier reply said the same for images resized from the W and H fields, and ⌘-drag keeps proportions meanwhile.

Path editing guides

Coming soon

Guides while editing paths, on the near-term vector roadmap.

Crop, scale and resize SVGs

Coming soon

Real SVG cropping and scaling; meanwhile, resizing the SVG container is the workaround.

Repeaters

Planned

Line, grid and radial repeat modifiers, shown as a prototype with a radial repeater.

Vector networks

Planned

Listed on the long-term vector roadmap and seen in the prototype clips.

Shape builder

Planned

Listed on the long-term vector roadmap board.

Resize an image together with its frame

Coming soon

Called coming soon, with asking an agent to do the scaling as the interim workaround.

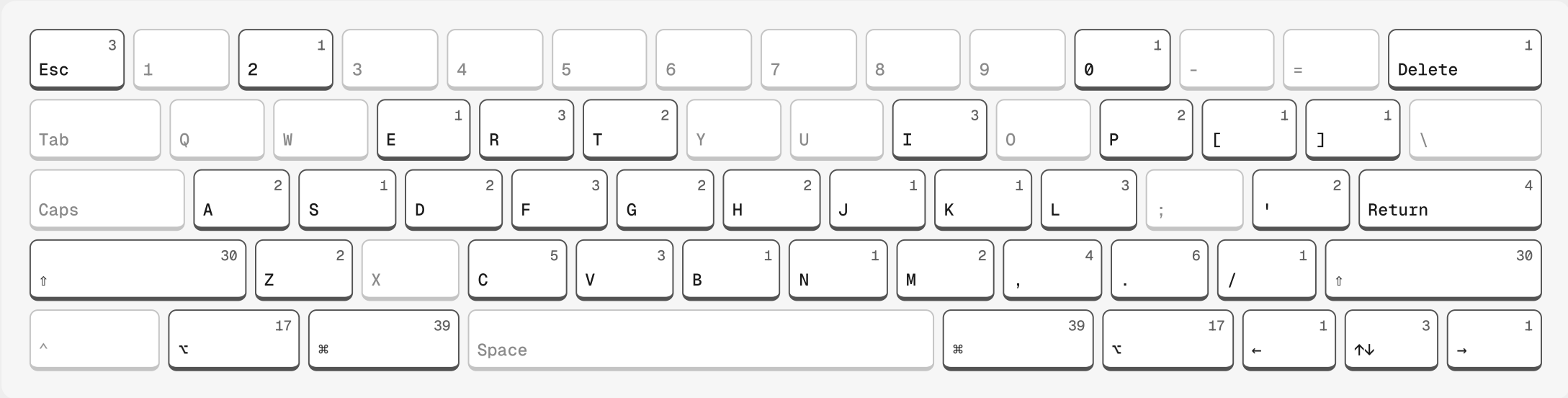
Rulers

In progress

The team said rulers are being worked on and pointed to the existing Guides section at the bottom of the right panel.

R3·Keyboard shortcuts

96 shortcuts shown or named in the videos and docs (Mac keys; use Ctrl for ⌘ on Windows). Hover a shortcut to see its keys.



Tools Pen tool P Frame tool F Text tool T Open the shaders menu S Comment: click anywhere on the canvas to leave a comment C Eyedropper for the selected layer’s fill I Hand tool, to pan the canvas H Zoom Z Create image (AI image generation) ⬆ ⌘ I Create SVG (AI vector generation) ⬆ ⌘ J Upload images from your computer ⬆ ⌘ K Cursor chat: type a message next to your cursor /	View and navigation Hide or show the UI ⌘ . Minimize the UI to floating pills, or restore it ⬆ . Zoom to and center a layer (double-click its icon in the layers list) Double-click Zoom to selection ⬆ 2 Scroll the canvas horizontally ⬆ Scroll Pan the canvas with the arrow keys (with a flex child selected, they reorder it instead) ← Show pixel grid ⌘ ' Snap to pixel ⬆ ⌘ ' Layout guides ⬆ G Multiplayer cursors ⬆ M	Selection and layers Go deeper: select a layer inside the hovered or selected one Double-click Deep-select the innermost layer under the pointer ⌘ Click List every layer under the pointer and pick one ⌘ Right-click Select parent Esc Select children Enter Add a layer to the selection ⬆ Click Rename a layer (double-click its name on the canvas or in the layers panel) Double-click Rename the selected layer ⌘ R Collapse all layers in the layers panel ⌘ L Show / hide ⬆ ⌘ H Lock / unlock ⬆ ⌘ L Bring to front] Send to back [
Layout Wrap in flex, or Add flex to a selected freeform frame ⬆ A Remove flex ⌘ ⬆ A Frame selection: wrap the selection in a freeform frame ⬆ F Ungroup ⬆ ⌘ G Clip content ⌘ C Set the height to Fit (double-click a frame’s top or bottom edge) Double-click Set Fill (⌘ + double-click an edge) ⌘ Double-click Space between (double-click the nine-point alignment grid) Double-click In a focused number field, raise the value by 1 (↓ lowers it) ⬆ In a focused number field, raise the value by the nudge amount (8 in the demo; set in Nudge settings...) ⬆ ⬆ Reorder: move the selected child one place later in a row (arrow keys reorder flex children) → Duplicate a layer while dragging it ⌘ Drag Duplicate; after an ⌘-drag with rotation, each duplicate repeats the same offset and rotation ⌘ D Hold ⌘ to show distances from the selection to the parent’s edges ⌘ Hover	Text and type Increase font size ⬆ ⌘ . Decrease font size ⬆ ⌘ , Increase font weight ⌘ ⌘ . Decrease font weight ⌘ ⌘ , Increase letter spacing ⌘ . Decrease letter spacing ⌘ , Increase line height ⬆ ⌘ . Decrease line height ⬆ ⌘ , Toggle bold ⌘ B Toggle italic ⌘ I	Vector editing Move tool, to drag path nodes without adding new ones M On an SVG, select a path; double-click again to edit its nodes Double-click Enter path-editing mode on the selected path Enter Leave path-editing mode (double-clicking empty canvas also works) Esc With the pen, place points at 45° angles ⬆ Click Drag from a node to pull out curve handles ⌘ Drag Move one curve handle on its own (breaks the mirroring) ⌘ Drag Drag a handle to mirror both handles again ⌘ Drag Curve a straight segment by dragging from its middle ⌘ Drag Delete the selected node Delete

Images, copy and export

Crop an image (hold ⌘ while dragging its edges)	⌘ Drag
Restore an image's original dimensions (double-click an edge or corner)	Double-click
Open the context menu; on an image it adds Vectorize image, Extract colors, Remove background and Open image in new tab	Right-click
With the eyedropper, add a gradient color	⇧ Click
Set the selection's opacity to 100% (number keys set opacity)	0
Copy	⌘ C
Paste, including layers copied from Figma and Snapshot captures	⌘ V
Paste on top	⇧ ⌘ V
Paste to replace	⇧ ⌘ R
Copy styles	⌘ ⌘ C
Paste styles	⌘ ⌘ V
Copy as PNG	⇧ ⌘ C
Copy as Tailwind	⌘ T
Copy as React CSS	⌘ R
Export the selection with the Export section's settings	⇧ ⌘ E
Undo	⌘ Z

Files, agents and Snapshot

Back to dashboard	⇧ ⌘ D
Open a new Paper window (it opens on the dashboard)	⇧ ⌘ N
Search files in the dashboard (across all folders since September 2026)	⌘ F
Select several files or folders in the dashboard	⌘ Click
Rename the current folder (double-click its name in the dashboard breadcrumb)	Double-click
Copy link to the selection, for example to point an agent at an exact frame	⌘ L
Start Paper Snapshot in Chrome	⇧ ⌘ P
Fine-tune the Snapshot target (⇩ also works)	⇧
Capture the targeted element (clicking also works)	Return
Capture the whole page with Snapshot	⌘ Return
Cancel a Snapshot capture	Esc

R4 · When something goes wrong

Symptoms, likely causes and the fix that works.

Problem	Likely cause	Fix
The agent says Paper is not connected, or Paper tools fail.	Paper Desktop is not running, or no design file is open in it. The MCP server lives in the desktop app; a file open in a browser, or the desktop dashboard alone, does not count. With the Paper CLI setup the tools can stay listed while the app is closed, so a listed tool does not prove Paper is reachable.	Open Paper Desktop on the target design file and try again. If the client still reports Paper as disconnected, restart the agent session.
Paper's tools are missing right after setup, or the agent calls tools that do not exist.	The server was added while the session was already running, so its tools were never loaded, or the session or its host app holds a stale tool list.	Restart the agent session; in Claude Code, close and reopen it, then check <code>/mcp</code> or <code>claude mcp list</code> . If that is not enough, restart the host app, and if the agent still invents tools, restart everything. In Cursor, reload the window or toggle the server off and on in Cursor Settings › Tools & MCP.
Paper appears twice in a client's MCP settings, or behaves differently from one session to the next.	An older HTTP entry (<code>http://127.0.0.1:29979/mcp</code> , used in spring 2026 tutorials) and a newer Paper CLI entry are both configured.	Keep one Paper entry per client. Follow the current setup in Paper's docs or the Connect your agent dialog, and migrate the old entry instead of keeping both.
The agent edits the wrong file.	Several files are open as tabs and the prompt named none, or it said “the active Paper file” and the tab in front changed before the agent started.	Name the file or paste its link (Copy link, ⌘L) in the prompt. When you rely on the active file, stay on its tab until the agent's dots appear there, and check the file name in the agent's first reply.
The agent overwrites or ignores changes you made by hand.	Agents do not track manual edits; they work from what they last read.	Ask the agent to re-read the canvas, or the specific frame, before its next change.
A recolor misses icons or changes the wrong layers.	Color searches can return false matches and miss vector details: in Paper's tabs demo, a search for <code>#000000</code> matched zero values,	Ask the agent to confirm each match by reading the subtree (<code>get_jsx</code>) or the computed styles, to check SVG strokes and fills explicitly, and to show a screenshot of the icons in the result.

Problem	Likely cause	Fix
	and SVG icon strokes did not show up in the agent's color searches.	
Agent screenshots time out, or come back too small to judge.	Screenshots are size-capped, and capturing a page with many boards, or one very large artboard, can be slow enough to time out (a viewer report answered by the Paper team).	Name the frame or section to capture rather than the whole page. Ask for scale 2 only when small text must be readable, and ask for computed styles or JSX, not pixel reading, when you need exact sizes and colors.
Agent tool calls stop working after heavy use, or limits do not reset after upgrading.	The Free plan allows 100 MCP tool calls per week (Pro: 1 million, per Paper's pricing page in early October 2026), and an outdated desktop app can keep the old limit after an upgrade.	Use Pro for sustained agent work. If limits do not reset after upgrading, update Paper Desktop (Check for Updates) and restart it.
The agent cannot reach Paper from WSL, or from behind a VPN or proxy.	Without mirrored networking, WSL may not reach the desktop app running on Windows; a proxy or VPN can block Paper's domains.	In WSL, set <code>networkingMode</code> to <code>mirrored</code> . Behind a proxy or VPN, allowlist <code>*.paper.design</code> .
A Remote Control session stops responding while you are away from the computer.	The computer went to sleep, or the session was not started from a Claude Code terminal session (a requirement as of July 2026).	Start <code>/remote-control</code> (or <code>/rc</code>) in a terminal session, keep Paper Desktop open on the file, and in System Settings › Lock Screen set both display-off timers longer than you will be away (For 3 hours in Paper's demo).
Content clips at the bottom of an agent-built artboard.	The artboard has a fixed height that is too small for its content.	Ask the agent to set the artboard height to fit-content with <code>update_styles</code> rather than guessing a taller fixed height, then take a new screenshot. By hand, double-click the frame's bottom edge to fit its height to the content.
Text renders in the wrong font or weight.	The family or weight is not installed on the computer and is not on Google Fonts, or the agent styled text without checking availability.	Ask the agent to check families with <code>get_font_family_info</code> before styling (Paper's server instructions require it once per session) and to prefer fonts already used in the file. Install missing fonts locally.
Local images are missing from a Snapshot capture of a site running on your machine.	The development server blocks cross-origin requests from Paper; Vite 6 and later also block <code>/@fs/</code> paths.	Allow the origin <code>https://app.paper.design</code> in the dev server (Paper's docs give settings for Astro, Django, Express, Fastify, Flask, Next.js, Nuxt, Rails, Vite, Remix, SvelteKit and webpack) and serve assets from <code>public/</code> .
You cannot find an exported file, or want an image without saving a file.	Paper saves exports straight to the Downloads folder (documented for PDF in the May 2026 build log), and the MCP export tool has no destination option.	Look in Downloads. To skip the file, use Copy as PNG (⌘⌘C) from the copy icon beside Export in the right panel, or from the right-click Copy as... menu.
A token changed in one file but not in another.	Token sets copied between files are independent copies, not a linked library.	Treat one file, or the codebase, as the source of truth and update the other files explicitly, by hand or by asking an agent to read the source and apply the changes.
A shader preset or code snippet no longer matches a tutorial.	The shader packages are at version 0.0.x and can change between releases; Paper Texture's parameters were reworked in 0.0.81 (September 2026).	Pin the exact package version, check the effect's current page on <code>shaders.paper.design</code> , and follow the Paper Texture migration guide when upgrading.

R5 · Briefs for your agent

Copy one into Claude Code, Cursor or any MCP client connected to Paper, then fill in the brackets.

Audit a frame before editing

Before any agent changes an existing frame, when you want a read-only diagnosis you can approve first.

Load the Paper guide if you have not this session. Work only in the Paper file "<file name or pasted link>", on the frame "<frame name>" (or my current selection). Do not change anything in this pass. First confirm which file, page and frame you are reading. Then read the frame with `get_tree_summary`, take one screenshot of that frame only, and read exact values with `get_computed_styles` and `get_tokens`. Report the fonts, sizes and weights in use; every color and whether it comes from a token; the spacing and alignment rules you can infer; layers with generic names; and any text that clips. Finish with the three problems that most hurt this screen's task, each with the change you propose and why. Wait for my approval before writing.

Explore N directions on separate artboards

When you want distinct alternatives to compare side by side without touching the original.

In the Paper file "<file>", explore <3> directions for the frame "<frame name>" and leave the original untouched. Before writing, list every action and content block on the original, then give each direction a short name, a one-line idea, its palette as hex values and its layout change, and wait for my go-ahead. Keep fixed in every direction: the logo, the navigation items, all listed actions, the real content and these fonts: <fonts>. Vary hierarchy and layout, not only color. Create one artboard per direction beside the original, at its size, named "<frame name> – 01 <direction>" and so on. Build breadth-first: the same section on every artboard before the next one. Screenshot each artboard and check it against the action list. If a tool call fails, tell me which artboard is incomplete. Call `finish_working_on_nodes` at the end.

Import code tokens into Paper

When the codebase already defines CSS variables and the Paper file should use the same values.

Create tokens in the open Paper file "<file>" from the CSS variables in this codebase, starting with <path, for example src/styles/>. Run get_tokens first and reuse any token that already exists; never duplicate one. Keep each variable's name where possible, keep aliases as var(--...) references to their base tokens, and create the tokens in batches with create_tokens: colors first, then typography, radius and spacing. If a value cannot be stored as written, such as light-dark() or calc(), do not guess: tell me which value you stored. End with a table of source file, variable, token, value and any conflict. Then build a "Style Guide" artboard that shows every token through var() so I can check the result on the canvas.

Implement an accepted frame in an existing codebase

When one frame is approved and should become production code that follows the repository's conventions.

Build the frame "<exact frame name>" from the open Paper file "<file>" into <path/to/file>. It is the accepted design; ignore every other variant. Read it with get_screenshot, get_jsx and get_computed_styles, and use get_fill_image for images: take sizes and colors from those values, never from the screenshot. Follow this repository's conventions. Reuse existing components such as <component> instead of creating new ones, use the existing tokens or theme, and keep <behavior, for example the sticky header> consistent with <where it already exists in the app>. Ask me before deciding any behavior the design does not show. When you are done, run the app, screenshot the page at <width>, compare it with the Paper frame, and list the remaining differences and every file you changed.

Align a comparison

When a table, plan comparison or set of metric rows looks tidy but values do not line up across rows.

In the Paper file "<file>", make the comparison in the frame "<frame name>" readable across rows. Read it first with get_tree_summary and one screenshot. Put every row on shared lanes: a label lane and one lane per compared value, each with a fixed width and flexShrink 0, so columns line up whatever the content. Left-align labels, right-align numbers, keep units attached to their values and use tabular figures if the font has them. Edit the existing nodes; do not regenerate the frame. Then stress-test a duplicate named "<frame name> – stress" with one very long label, one missing value shown explicitly rather than as zero, and one number with more digits than the rest. Screenshot after every three rows and at the end, and report any lane that still drifts.

Generate a token theme

When a file needs a token system built from an existing design before more screens are added.

In the Paper file "<file>", build a token theme from the frame "<reference frame>". Run get_tokens first and reuse anything that exists. Propose the set before creating it, in Tailwind v4 namespaces: --color-* (semantic roles such as background, surface, text, muted, border and accent before the raw palette), --font-*, --text-*, --leading-*, --tracking-* in em, --radius-*, --spacing-* and --breakpoint-*. Take every value from the reference with get_computed_styles; invent nothing. After I approve, create the tokens with create_tokens, aliasing with var(--...) where one token derives from another. Build a "Style Guide" artboard beside the reference that uses only var() references, then rebind the reference frame's colors and type to the tokens. Paper has no theme modes yet, so put any dark values in a separately named set and apply them to a duplicate of the frame.

Prepare an export

Before handing finished frames to someone outside Paper as PDF, PNG, SVG or video.

In the Paper file "<file>", prepare <frame names> for <destination, for example a client PDF>. Do not change the designs. Check each frame first for clipped text, missing images, images still generating (exports do not wait for them) and the smallest text size, and tell me what you find. For a multi-page PDF, list the frames in the order export_combined_pdf will use, which is canvas position from top to bottom, then left to right, and tell me if any frame needs moving. Then export with the default settings unless I give a format and scale, for example png at 2x. SVG export works only for SVG nodes. Report the file names and where they were saved. I will open the output to check page order, image quality and searchable text.

Review drift across parallel agents' artboards

After several agents have worked in parallel, before you choose, combine or port any of their frames.

Do not edit anything. In the Paper file "<file>", review the artboards <list them, or all artboards on page "<page>"> against the reference "<reference frame or style guide>" and these brand rules: fonts , theme <light or dark>, colors only from the file's tokens. Use find_nodes and get_computed_styles to collect every font family, text size, color and radius in use. find_nodes can match substrings, so confirm each hit with get_computed_styles, and check SVG fills and strokes separately. Produce one table per artboard: property, expected value, value found, layers affected. Flag raw hex values that should be tokens, fonts that fell back to a default and any action present on the reference but missing here. Propose fixes grouped so that one agent can apply them, then wait for my approval.

R6·Glossary

75 terms, each linked to the chapter that explains it.

Absolute position

A setting that takes a child out of its Flex frame's flow so it floats over the content and follows constraints. 04

Alias

A token whose value is another token, written var(--color-blue-500), so a semantic role follows its palette color. 05

Behavior state

An extra artboard that shows a non-default state, such as a scrolled header, so the implementer knows how the page should respond. 13

Brand drift

Output that departs from a brand's fonts, colors or theme, usually because the agent filled a gap in the brief with a default. 11

Breadcrumb

The folder path at the top of the dashboard (for example Files / Marketing / Social Media), used to move up a level or rename the current folder. 01

Breadth-first build

Building the same section on every variant before starting the next section, so all variants reach the same stage together. 11

Canonical frame

The single named frame that defines a pattern; specimens and variants are duplicated from it and judged against it. 12

Capture target

The element inside Snapshot’s blue box, which the ⬆ and ⬇ keys adjust before you capture. 08

Center

A constraint that keeps a layer's size and its centered position as the parent is resized. 04

Code component

A component defined in the codebase; an agent can rebuild it on Paper’s canvas as editable layers, but the copy is not linked to the code. 12

Combined PDF

One multi-page PDF built from several selected frames, one page per frame, ordered by position on the canvas. 14

Comment thread

A comment pinned to a layer, with its replies, reactions and an open or resolved status that people and agents can change. 10

Constraint

A rule that decides how a layer outside Flex flow keeps its position or size when its parent is resized. 04

Context image

An image selected while the Create image panel is open, which the next prompt edits instead of starting from scratch. 08

Contour

On logo animations, how far inside the shape pixels respond to its edge, from 0% (a plain mask) to 100% (edge-following bands throughout). 09

Copy as React CSS

Copies the selection as a React function whose elements carry inline style objects, headed by a comment with the layer’s Paper link and the date. 13

Deep selection

An optional hover mode, ticked in the file-name menu, in which the pointer targets the frame closest to the object under it. 01

Detach

Replacing a token on a property with its current literal value, which later token edits do not change. 05

Detached component

A Figma component or instance that arrives in Paper as plain layers, without its link to the main component. 08

Draft template

A Resend template that can be previewed and test-sent but must be published before it can be sent. 14

Encoding

The visual property that carries a value, such as position along a shared axis, length or color. 06

Fill

A size that takes the space the parent has left along that axis. 02

Fit

A size that follows the frame's content, like CSS fit-content; the tutorial describes it as hugging the contents, and some tools call it Hug. 02

Fixed

A size you type or drag that does not respond to the content or to the parent. 02

Fixed, Fit and Fill

Paper's size modes: a set value, a size that hugs the contents, or a share of the parent's unused space. 03

Flex frame

A frame whose children are arranged by direction, gap, padding and alignment (CSS display: flex) instead of by coordinates. 02

Frame

Paper’s container layer, the equivalent of an HTML div; the layers inside it are its children. 01

Freeform frame

A frame without flex, whose children keep the positions you give them; the layers panel marks it with dashed corners. 01

Gap

The fixed space between neighboring children of a flex frame; with wrap on, the space between items and between lines is set separately. 02

get_computed_styles

The Paper MCP tool that returns the computed CSS of layers, the source of exact values for code. 13

get_jsx

The Paper MCP tool that returns a layer subtree as JSX, styled with Tailwind classes by default or with inline styles. 13

Harness

The agent application a model runs in, such as Claude Code, Cursor or GitHub Copilot, which you choose when connecting Paper. 10

HTML in Canvas

The rendering method Paper adopted in September 2026 for images and videos, which it says makes exports faster and higher fidelity. 14

Image filter

A shader that transforms an input image, which can be any layer captured with the eyedropper, including text or another shader. 09

Image resampling

The PDF setting that processes images as Detailed (finer detail; Lanczos, per the MCP tool) or Basic (smaller files; Hamming). 14

Inline style

CSS written in an element’s style attribute, the only styling Paper’s HTML paste reads. 08

Lane

A position that the same kind of content occupies in every row, such as a label lane or a value lane of one fixed width. 03

Left + Right

A horizontal constraint that keeps fixed distances to both side edges, so the layer's width changes with its parent. 04

Logo animation

A shader (Heatmap, Liquid Metal or Gem Smoke) whose animated pattern is shaped by the edges of a supplied logo or built-in shape. 09

Magic Swatches

The freeform frame that Extract colors creates from an image, holding 100 × 60 rectangles named by their hex values. 01

MCP

Model Context Protocol: the interface through which an agent reads and writes the open Paper file with tools such as get_jsx and write_html. 13

MCP server

A connector that lets an agent read or write another system, such as Notion or a Paper file. 06

Mirrored handles

A node’s two curve handles moving symmetrically; hold ⌘ to move one alone and ⌘ to mirror them again. 07

Opacity token

A percentage token, added in September 2026, that can be paired with a color token in a fill’s percentage field. 05

Paper CLI

A command-line program, run with the argument `mcp`, that relays a client’s stdio connection to Paper Desktop; Paper’s docs name it as the current way to connect most clients. 10

Paper Snapshot

Paper’s Chrome extension that captures a page region or a whole page and pastes it into Paper as editable HTML/CSS layers. 08

Path

One editable vector shape inside an SVG, with its own nodes, fill and stroke. 07

Preset

A named set of parameter values, such as Warp’s Cauldron Pot, that replaces all current values when clicked. 09

Reference mark

A target, baseline or threshold drawn so that it cannot be mistaken for a measured value. 06

Remote Control

A Claude Code feature, started with `/remote-control` or `/rc`, that lets the Claude mobile app continue a session running on your computer. 10

Searchable PDF

A PDF whose text is stored as text rather than pixels, so people and tools can search, select and read it. 14

Shader

A small GPU program that computes the color of every pixel; in Paper it is a layer whose look comes from a preset, parameters and colors, rendered live with WebGL2. 09

Shared scale

One mapping from value to length that every comparable mark uses, so equal lengths always mean equal values. 06

Sibling

A layer that shares its parent with another layer; siblings move with the parent and follow its layout rules. 01

Silhouette

An asset’s overall outline, which must stay recognizable at its smallest display size. 07

Slot

In component systems, a named region that accepts different content in each use; Paper’s roadmap listed components with slots as coming soon in September 2026. 12

Space between

An alignment that pushes the first and last children to the ends and spreads the rest evenly (CSS justify-content: space-between). 02

Specimen

A copy of the canonical frame filled with a demanding case, such as long text, missing data or a selected state, to test the pattern’s limits. 12

Speed

A shader’s animation rate, shown in percent in Paper and as a multiplier in code; 0 stops the animation loop and leaves a still frame. 09

Steer

Sending a message into an agent’s current run so it changes course immediately, as opposed to queuing it until the run ends. 11

Stretch

A constraint that scales a layer’s size and its distances to the edges in proportion, shown as a percentage width. 04

Stroke position

Whether a path’s stroke paints above its fill, centered on the edge, or below it, so that it reads as an outside outline. 07

SVG container

The outer SVG layer in Paper, which behaves like a frame, holds Path layers and clips them at its edges. 07

Tabular numerals

Digits that all share one width, so numbers with the same count of digits line up in a column. 03

Template variable

A placeholder such as {{{RESET_URL}}} that the email service fills with a real value when it sends. 14

Theme

Paper’s name for a file’s set of tokens, as in the Copy theme and Paste theme commands. 05

Token

A named value stored in a Paper file as a CSS custom property, such as --color-accent, that layers reference with var(). 05

Translation layer

A conversion step between a design format and code; Paper has none because its canvas is HTML and CSS. 13

Variable font

A font with continuous axes such as weight and optical size, so any value in its range can be set. 05

Variant

A duplicate that changes one rule of the pattern, named after that change and kept beside the original for comparison. 12

Variety pack

Create image’s default option, which sends one prompt to four models and returns four images. 08

Vector editing mode

The state, entered by double-clicking a selected path, in which its nodes and curve handles can be moved. 07

Working indicator

The animated dots Paper shows on a tab and above an artboard while an agent is working there. 10

Workspace (Conductor)

An isolated copy of the repository on its own branch, so agents in different workspaces cannot overwrite each other’s code. 11

Wrap

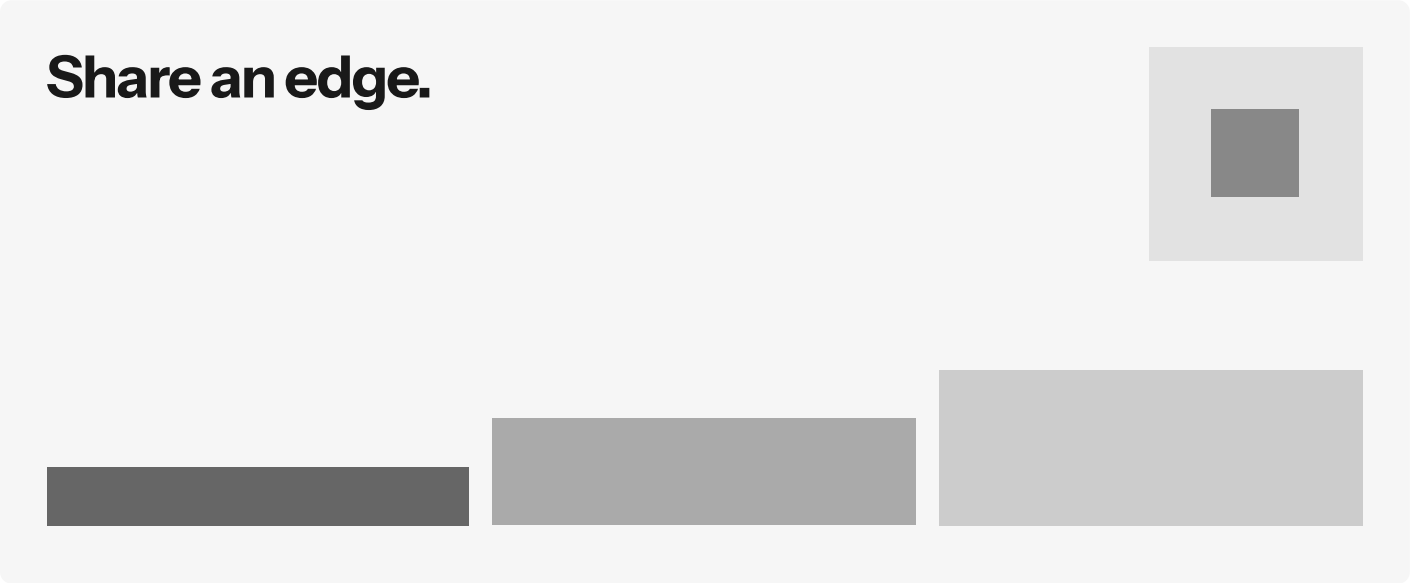
A Flex setting that lets children break onto new lines instead of shrinking or overflowing, like CSS flex-wrap. 04

Further reading

Seven authors to read next.

Seven books offer seven ways to question a design: its rules, alignment, encoding, evidence, color, reading rhythm and purpose. Their ideas are summarized here in our own words; read the books for the real thing.

One composition, three lessons



- Grid
- Type
- Tone

A common edge turns separate pieces into one composition. Change the rule and watch the relationships move.

Karl Gerstner

Designing Programmes · 1964

Swiss graphic designer and artist who developed systematic approaches to typography, identity and composition.

Describing the problem precisely is already part of solving it.

A reusable design starts with a precise problem: which content varies, which relationships stay fixed, and how the reader will use them. Treat a layout as a programme of explicit choices (where the label, value, unit and comparison go) before you make the second card. Variation then becomes a controlled response to content rather than improvisation.

Try it in Paper

Make one comparison row with a label lane and two equal value lanes. Duplicate it into compact and spacious versions, changing only padding and gap. Add a long label to both and keep the rules that survive.

Related: chapter 12, Reuse

Josef Müller-Brockmann

Grid Systems in Graphic Design · 1981

Swiss graphic designer whose teaching and books made the grid a practical method for organizing visual communication.

Choosing a grid is choosing to systematize, and so to clarify.

A grid's value is the reading path it creates. A common left edge makes labels easy to scan; a common right edge makes quantities easy to compare. A row should take its positions from the shared structure, not from the length of its current label, and an orderly screen still needs exceptions when the content or task changes.

Try it in Paper

Rebuild three rows as a vertical Flex stack with identical label, bar and value widths. Change 56.2K to 156.2K and lengthen one label, then check whether the invisible structure still holds.

Related: chapter 03, Alignment

Jacques Bertin

Semiology of Graphics · 1967

French cartographer and theorist who studied how graphic variables carry different kinds of information.

Every component of data is qualitative, ordered or quantitative, and each kind needs different visual variables.

A name identifies a category, a rank gives an order, a share gives an amount. A visual treatment should preserve the relationship it claims to show: equal identity treatment for names can sit beside bars whose lengths encode different shares, but a change of shade alone cannot communicate an exact ratio.

Try it in Paper

Take three options A, B and C at 42%, 35% and 23%. Give them equal labelled marks for identity, then add horizontal bars on one scale for quantity, sorted by share. Say what each visual variable now means.

Related: chapter 06, Information

Edward R. Tufte

The Visual Display of Quantitative Information · 1983

Statistician and author whose books examine how graphics support reasoning about evidence.

Above everything else, a display should show the data.

A display earns its space by helping someone judge evidence. Removing a heavy border may help; removing the unit or an uncertainty note harms the same task. Motion should explain a change or preserve continuity, not compete with the reading.

Try it in Paper

Duplicate a comparison card. In the copy, remove the shadow, the enclosure and redundant labels, but keep the unit, the baseline and the comparison. Compare both at phone width and restore any cue whose removal made the meaning harder to recover.

Related: chapter 06, Information

Josef Albers

Interaction of Color · 1963

Artist and teacher whose color exercises grew out of direct observation at Black Mountain College and Yale.

Color is relative: the same color reads differently with every change of surrounding.

A grey token has no fixed visual strength. The same mark looks lighter, darker or more prominent depending on its surface, which matters in a monochrome interface: a selected state must stay clear on the page ground and on a white panel. This is perception, not a contrast check; text still needs a measured contrast ratio.

Try it in Paper

Place two identical #888888 squares, one on #EEEEEE and one on #535353. Keep size and fill identical and compare them. Then judge a selected row in its real context before you change its token.

Related: chapter 05, Tokens and type

Robert Bringhurst

The Elements of Typographic Style · 1992

Poet, translator and typographer whose work connects the craft of language with the form of the page.

Rhythm, proportion, structure and the choice of type have to be judged together.

A pleasant typeface cannot rescue an exhausting line length, and a bigger heading cannot repair a confused section structure. Adjust the measure and the line spacing before you add another container or label.

Try it in Paper

Set the same 120-word passage in two frames at the same size, one narrow and one wide, and adjust the line spacing in each. Read both aloud, keep the one where the next line is easiest to find, and check it again at phone width.

Related: chapter 05, Tokens and type

Stephen Few

Show Me the Numbers · 2012

Author and educator focused on the practical design of tables, graphs and analytical displays.

Different quantitative relationships call for different kinds of graphs.

The same values answer several questions. Someone looking up an exact quantity needs a different emphasis from someone looking for the biggest gap against a target. Name the decision first, then choose the arrangement that reveals it, with targets and actuals on one scale.

Try it in Paper

Show stock as an aligned row: 64 on hand, 24 incoming, target 100. Duplicate it as a bar ending at 88 with a target marker at 100. Compare which view supports exact lookup and which makes the 12-unit gap visible.

Related: chapter 06, Information

The 52 videos, and how they were used.

Paper published 33 videos and 19 Shorts between 5 March and 28 September 2026, about 3 hours 15 minutes in all. Every one is written down in a chapter; this is the index.

Chart S1

Uploads per month

Hover a column for minutes. Launch months bring tutorials; later months bring Shorts.

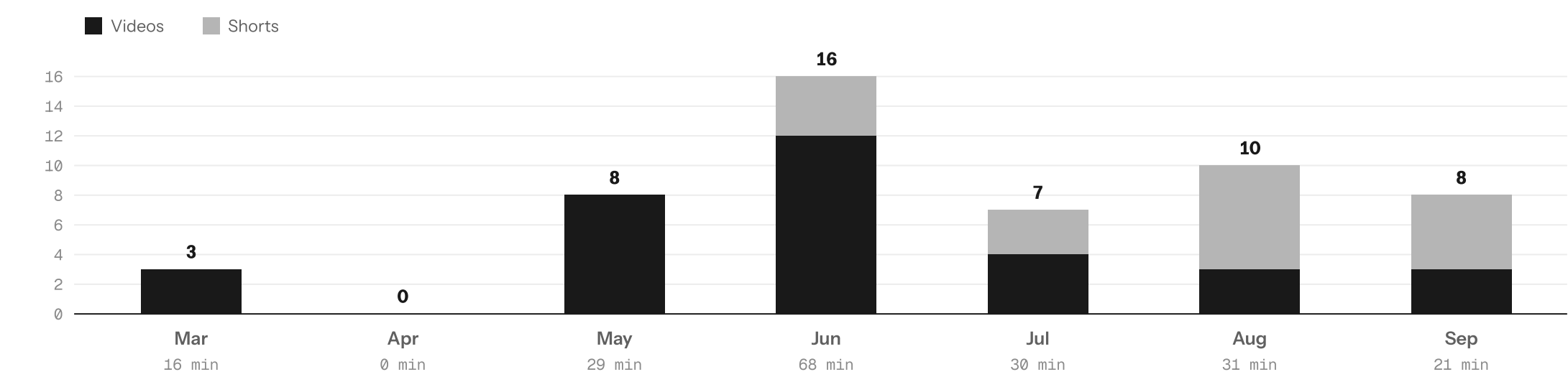
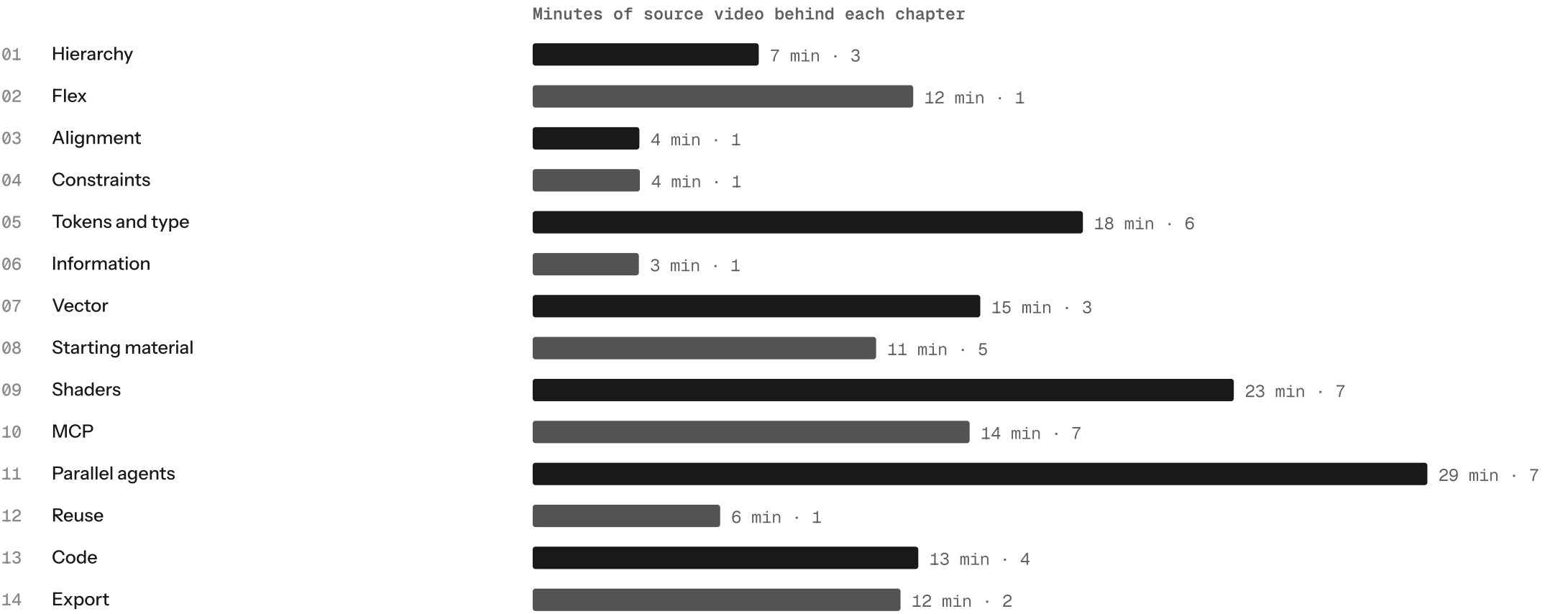


Chart S2

Where each video went

Minutes of source video written into each chapter.



28 September 2026 Video · 5 min 40 s	New in Paper: faster performance, agent reliability, opacity tokens, and more September 2026 changelog: half the memory and faster tabs; a 'Connect your agent' dialog for one-click MCP setup across ten harnesses; image generation via MCP (a four-model 'variety pack'); opacity tokens paired with color tokens, color tokens created from fills, token picker views; dashboard search across folders; a reworked flex panel; Copy as PNG/Tailwind/React CSS from the Export section; HTML in Canvas exports; a rebuilt Paper Texture shader. Coming soon: remote MCP, components on the canvas, scale tool, rich text editing, per-file permissions.	Written down in Release notes
24 September 2026 Short · 55 s	What can Opus 5.5 design in five minutes? After Anthropic's benchmark table (Opus 5.5 ahead of Fable 5.1 on every row shown), a dictated prompt in the Claude desktop app (Opus 5.5, 'Extra' effort) asks for as many good meditation-app home screens as possible in five minutes. The agent names six mood directions (Tidal, Candlelit, Alpine Swiss, Chapel, Editorial pop, Mineral), creates six 402×874 artboards ('01 Tidal — Device (402×874)' etc.), then builds breadth-first (headers on all, then heroes on all). Log shows 'Used 18 tools (6 failed)'. All six finished in just under six minutes.	Written down in 11 · Parallel agents
23 September 2026 Short · 42 s	Use Mobbin and Paper to design flows from proven patterns An agent pulls reference flows from Mobbin's MCP onto the canvas as labelled boards ('Ref · <product> - <flow>'); the designer culls them and pins comments, then prompts (Opus 5, High) with Paper links to the references and the design-system boards to create five variations. The agent writes rows such as 'V1 · Drawer', 'V2 · Toast', 'V3 · Popover', and the loop repeats with comments before manual edits.	Written down in 08 · Starting material
21 September 2026 Short · 2 min 09 s	How the Skydive team ships with their agents in Paper Skydive designers create a design agent (Tallulah, 'UX Design Expert'), train it on their Paper design system via a file link, and ask it in Slack to draft a partnership page artboard and a dark version; they polish it by hand in Paper. A second agent (Moon) receives the design from Tallulah in a Slack group DM and publishes it to Webflow, where the live page matches the Paper design, animation included. How Skydive connects to Paper is not shown.	Written down in 11 · Parallel agents
18 September 2026 Short · 1 min 12 s	Paper is the design tool for teams building with agents Pitch that doubles as the read/write model: the canvas is rendered with code so agents read it (reference images, pasted drafts, your manual tweaks as visual context) and write it (first drafts, many variations, boilerplate like applying styles and renaming layers). B-roll prompts: list best practices from reference images; check what I pasted (a frame 'pasted from figma'); apply my changes to the rest of the flow; make a login screen from my style guide; explore 10 variations of my selection. Agent log shows reading the user's selection and a 12,108-node, 56-artboard file; layers renamed into a semantic tree. Vertical 1080×1920, filed by YouTube as a regular video.	Written down in 11 · Parallel agents
18 September 2026 Video · 6 min 09 s	How to bring your components from code onto the Paper canvas Harcourt shows two pre-native-components routes to get code components into Paper: an agent connected to the repo rebuilds a labelled component sheet and, unasked, imports and binds the codebase's tokens; or Paper Snapshot copies elements or whole pages from the live site as editable layers with no agent or tokens. She composes a For You section with real API data, tries a smaller-card variant on a duplicate, and has the agent add the final frame to the Explore page, where the reused components keep their behaviour.	Written down in 12 · Reuse
3 September 2026 Short · 14 s	From design to code in Paper A 14-second vertical teaser cut from 'Push designs in Paper back to code': the code-to-canvas prompt, the redesigned Learn artboard with the reused Search component, the canvas-to-code prompt beside the running page, and the end card. No information beyond the full video.	Written down in 13 · Code
2 September 2026 Video · 4 min 16 s	Push designs in Paper back to code Harcourt pulls Paper's Learn page into Paper with a one-line Cursor prompt, redesigns it with categories, a reused Search component and a scrolled-state artboard, then prompts Claude Opus 5 (High, Fast) to build the redesign frame into index.tsx, keep the designed scroll behaviour and reuse existing components. The agent asks questions, reuses the app's sticky-header pattern and SearchField, and the page works locally. She also shows Copy as Tailwind (⌘T) and Copy as React CSS (⌘R) on a ⌘-selected nested layer and explains the no-translation-layer advantage.	Written down in 13 · Code
29 August 2026 Short · 26 s	Remove image backgrounds in Paper Right-click an image > Remove background: after a couple of seconds the subject is transparent inside the same 1731 x 2181 frame, then shown over other backgrounds and cropped inside a YouTube thumbnail frame. The captured right-click menu lists every image action and shortcut, including Vectorize image, Extract colors and Open image in new tab (not demonstrated).	Written down in 08 · Starting material
26 August 2026 Short · 35 s	Paper: the connected canvas for teams shipping with agents YC Startup Podcast clip: Paper is agent-native because HTML and CSS are its rendering engine, which agents know well; custom rendering engines mean higher token spend, slower runs and lower accuracy. Team of about twelve designers and engineers. Animated b-roll log: get_selection, a code-file read (labelled read_file), a live API fetch (10 tracks), create_artboard, one write_html per row, set_text_content, then 'now generate the mobile screens' creating 'Mobile / Home', 'Mobile / Playlist', 'Mobile / Library' at 390×844.	Written down in 11 · Parallel agents

24 August 2026 Short · 50 s	Design first, prompt the animations after Podcast clip in which Stephen Haney explains that Paper's brand designer built the whole paper.design site in Paper in about a week, shaders and Easter eggs included, then prompted the animations on top of the Paper file while an agent turned it into a Next.js codebase. On-screen footage shows Codex reading a Paper selection through get_selection, get_screenshot, get_tree_summary and get_computed_styles before writing a component.	Written down in 13 · Code
21 August 2026 Short · 35 s	Paper shader of the month: Lens Distortion A recut of the August 2026 changelog's shader of the month: Lens Distortion, a new image filter that bends geometry and splits color channels like a camera lens. Content is pulled in with the eyedropper, and the short shows it works on any layer, including text, with five variants of one headline.	Written down in 09 · Shaders
20 August 2026 Short · 54 s	Comment, reply, and resolve in Paper Cut-down of the August comments segment: press C or use the toolbar button to comment anywhere, reply, react with emoji, resolve with the check icon, and use the right-panel list (comment mode only) with search, Newest first / Oldest first / By page sorting, and filters by person, page and resolved status.	Written down in 10 · MCP
19 August 2026 Short · 27 s	Leave comments for your agent in Paper Comments work for agents too: pin comments on layers, then tell the agent you left comments on the frame and to implement them. A comment pinned on a 'Research Agent' card asks for another card; the agent adds an 'Ops Agent' card in matching style and, per narration, fixes the other commented issues in the same pass. Claude, Cursor and OpenAI logos shown. Whether threads are resolved afterwards is not shown.	Written down in 10 · MCP
19 August 2026 Video · 13 min 53 s	Paper and Conductor: design to code with parallel AI agents Harcourt and Conductor's CEO capture reference pages with Paper Snapshot, ask an Opus 5 agent for three iterations of a subway-map roadmap, then run several agents at once: one restyling the same Paper file to Geist and dark mode while the first still works, another creating and naming a new Paper file for a musical version. A fresh workspace recreates the chosen frame as a standalone HTML page served locally (get_screenshot, get_jsx, headless screenshot comparison), and they show feedback via Conductor diff comments and Paper canvas comments.	Written down in 11 · Parallel agents
12 August 2026 Video · 9 min 23 s	New in Paper: tabs, comments, free editors, and more August 2026 changelog: Series A; tabs in the desktop app with two Cursor agents (Opus 5 High) working in two files while presence shows on tabs, frames and avatars; Theme-tab token creation and editing with live propagation; flex wrap; Option-drag plus ⌘D repeating rotation; dashboard multi-select, file Duplicate and higher Pro limits; unlimited free editors and free viewer roles; comments for teammates and agents with a filterable panel; Lens Distortion shader; personal drafts announced next.	Written down in Release notes
11 August 2026 Short · 33 s	Tabs, now in Paper Silent 33-second motion promo for tabs: a fixed Dashboard tab plus file tabs, '+' for new tabs, animated dots on tabs and dotted markers on frames while agents work, a Claude Code panel holding a two-option dashboard spec beside wireframe cards, an agent dock, mixed human and agent presence avatars, and A/B dashboard variants.	Written down in 10 · MCP
10 August 2026 Video · 2 min 57 s	Using tabs to run agents across multiple Paper files at once Tabs (August 2026) let agents work in any open file. In Conductor (Opus 4.8 1M, Medium), three agents target three files: by file name, by pasted file link (Copy link ⌘L; the agent loads the guide and calls open_file), or 'in the active Paper file' (resolved when the agent starts). Per-tab dot indicator and robot avatar show where agents work; an agent reads tokens on a Tokens page and applies them on a Design page of the same file. Agent reasoning reveals find_nodes matches substrings (#000000 false positives on 0 values) and SVG strokes need separate checks. Tab menu: Reload, Copy Link ⌘L, Rename, Open in Browser, Move to New Window, Close Tab, Close Other Tabs, Close Tabs to the Right, Close All Tabs.	Written down in 10 · MCP
31 July 2026 Video · 3 min 00 s	Use Claude's Remote Control to design in Paper from your phone Start Remote Control from a terminal Claude Code session (not the desktop app) with /remote-control or /rc; keep the Mac awake via System Settings > Lock Screen display-off timers 'For 3 hours'; open the Connected session in the Claude mobile app's Code tab. From a car, a dictated prompt duplicates an email frame in Paper and fills it with copy from a private Slack canvas and images matched by layer name; review happens through screenshots the agent returns. A follow-up makes a dark mode on a duplicated frame using the codebase's CSS variables. On-screen email lists Paper's tips: prompt in small chunks, use the codebase as token source of truth, have the agent re-read the canvas after manual edits. Session: Claude Code v2.1.220, Opus 5 (1M context), Claude Max.	Written down in 10 · MCP
21 July 2026 Short · 34 s	Create magic swatches in Paper Magic swatches: paste a photo into Paper (Cmd+V), right-click > Extract colors, and Paper adds a 'Magic Swatches' frame under the image with six swatch layers named 'Color #HEX', ready to use in the UI. The same context menu also offers Vectorize image, Remove background and Open image in new tab.	Written down in 05 · Tokens and type

20 July 2026 Video · 12 min 29 s	Flex layout in Paper Harcourt Allen teaches flex layout from zero (part one of a two-part layout series). Paper uses the CSS term flex because it is built on HTML/CSS and agents and developers share that vocabulary; frames are divs. She builds a menu button and a sidebar step by step: Wrap in flex (⇧A), four ways to set gap, token fill, axis or individual padding, direction, nine-point alignment, Fit via edge double-click, Space between (sliders menu or grid double-click), nested frames to split alignment, Fixed/Fit/Fill sizing (Relative listed but unexplained), Option + double-click edge for Fill, a parent sized by its child, a responsive split-view dashboard, and Absolute position for a close button that still needs constraints.	Written down in 02 · Flex
20 July 2026 Video · 3 min 31 s	Constraints in Paper Part two of the layout series. Constraints govern non-flex items (children of freeform frames and absolute children of flex frames) when the parent is resized. The UI is a horizontal dropdown (Left, Right, Left + Right, Center, Stretch), a vertical dropdown and a pin diagram. The close button defaults to Left/Top and is pinned Right to hold its corner; a modal centered with the Layout header's align buttons gets Center/Center automatically; Left + Right keeps fixed side margins (500 px) while the width grows, and Stretch scales both and shows the width as a percentage (36%).	Written down in 04 · Constraints
20 July 2026 Short · 55 s	Apply a Halftone Shader to Text Halftone Dots applied to a letter: add a shader (tooltip Shaders S), pick Halftone Dots, use the Image eyedropper on a text layer, set Type to Gooley, scale to fit, explore Size/Radius/Contrast on a Hex grid, then recolor Foreground (yellow) and Fill (olive).	Written down in 09 · Shaders
9 July 2026 Short · 56 s	Understanding contour in Paper's logo animation shaders Vertical recut of the Contour explanation from the logo shaders video: each pixel knows its distance to the shape's edge, and Contour sets how far in pixels react. 18% hugs the rim, 0% is a plain mask with no edge reaction, 100% makes deep pixels follow the edges too (shown with Liquid Metal on a triangle).	Written down in 09 · Shaders
7 July 2026 Video · 9 min 05 s	How to use logo animation shaders in Paper Harcourt Allen announces Paper Shaders are open source and free, then covers the three logo animation shaders (Heatmap, Liquid Metal, Gem Smoke), which react to a shape picked with the eyedropper, uploaded, or chosen from presets. She tunes Heatmap on the Paper logo, explains Contour with Liquid Metal on the Vercel logo, and places a faint Gem Smoke inside Resend's real footer captured with Paper Snapshot (Cmd+V, Shift+A, Absolute position, Clip content).	Written down in 09 · Shaders
30 June 2026 Video · 8 min 18 s	New in Paper: tokens, vector editing, Figma import, and more Paper's June 2026 changelog. On tokens, a Cursor agent imports color and typography CSS variables, and its log shows two rules: palette tokens must exist before semantic aliases, and Paper rewrites Tailwind's double-dash names (--text-xs--line-height becomes --text-xs-line-height). Token types mirror Tailwind, and themes and modes were in progress. Also covered: SVG path editing, the Cmd + right-click layer picker, Collapse all, panel resizing and page reordering, cursor chat and follow mode, shaders, PDF export (separate or combined, saved to Downloads), Hide UI (Cmd+.) and minimize UI (Shift+.), Figma copy and paste, nested folders, MCP tools to create and open files, and full-page Paper Snapshot (Cmd+Return).	Written down in Release notes
26 June 2026 Short · 52 s	Create Design Tokens From Your Code Base A vertical cut of the tokens tutorial. One Cursor prompt creates 96 tokens from the codebase's CSS variables through Get Guide, Get Basic Info, Get Tokens and Create Tokens, with the approval card visible. In Paper, the Fill row's 'Color tokens' button lists them, and blue-1 then blue-3 are applied.	Written down in 05 · Tokens and type
26 June 2026 Short · 25 s	Copy and paste tokens between Paper files How to move a file's tokens to another file while libraries are in progress: Paper logo > Theme > Copy theme in the source file, Cmd+Shift+N for a new file, then Paper logo > Theme > Paste theme; the tokens are available at once. Clear theme is visible in the submenu but not demonstrated.	Written down in 05 · Tokens and type
24 June 2026 Video · 4 min 29 s	From CSS variables in your codebase to Paper design tokens with MCP Harcourt uses Cursor with the Paper MCP to turn Paper's own codebase CSS variables into 96 Paper tokens with one prompt (Get Guide, Get Basic Info, Get Tokens, then batched Create Tokens). She tests a color token through the Fill row's four-circle button, has the agent draw a style guide board with Write Html, inspects typography token chips (size, line height, hover name, dropdown with Detach), and copies the whole set into a new file with Paper logo > Theme > Copy theme / Paste theme. As of 24 June 2026 the MCP was the only way to create tokens, and light-dark() values were reduced to their light value.	Written down in 05 · Tokens and type
22 June 2026 Video · 10 min 16 s	Explore Paper's MCP-generated tokens with the Paper team Harcourt and Dom have Claude (Opus 4.8, Claude desktop app with the paper-desktop plugin) build a style guide from an existing landing page. Unprompted, the agent creates 60 Tailwind v4-named tokens and binds the board through var(). They tour the color and type token controls and the spacing and container tokens in the W dropdown, then build a four-screen onboarding flow on the tokens. Next they detach and recolor two elements teal and have the agent update and rename the tokens with set_tokens, so every bound element changes; the agent warns that the detached elements now hold literal values. Roadmap as of June: in-app creation, theme classes, modes.	Written down in 05 · Tokens and type

18 June 2026 Video · 4 min 36 s	Generate and refine SVG illustrations with Quiver AI inside Paper Create SVG (⇧⌘J) with QuiverAI Arrow 1.1 Max (vs the default Arrow 1.1): prompt anatomy (subject, style, specific colours, restraint), a live build behind a 'Generating with QuiverAI...' placeholder, and an auto-named 256 x 256 SVG made of 3 polygons and about 20 paths. The cleanup pass uses the SVG tools: hide unwanted polygons, normalise uneven strokes (1.28 and 0.99 set to 1), delete stray paths, trim path sections in vector editing with Command+Z as the safety net. About five minutes of edits were skipped.	Written down in 07·Vector
17 June 2026 Video · 1 min 54 s	How to organize your Paper files with folders and nested folders In the team dashboard's Files view, + New folder creates a folder (name prefilled Untitled project). Files move via right-click > Move to, with the current location greyed out. Folders nest to any depth (Marketing > Social Media > Instagram, TikTok), move the same way with a confirmation toast, and are renamed via right-click > Rename or by double-clicking the current folder name in the breadcrumb.	Written down in 01·Hierarchy
15 June 2026 Video · 3 min 46 s	8 Paper tips and tricks every designer should know Eight canvas tips with on-screen key overlays: ⌘. hides the UI and ⇧. minimizes it into floating pills; double-clicking a layer icon zooms and centers it; ⇧F makes a freeform frame; ⇧A adds flex to a frame (Add flex ⇧A) or wraps loose layers (Wrap in flex ⇧A); ⌘-drag crops an image; double-clicking an edge or clicking the size chip in Fill > Image restores the original size; right-click Extract colors builds a Magic Swatches frame of rectangles named by hex; I opens the eyedropper. Also shows the full right-click menu and the color picker (sRGB / Display P3, LCH, HSL, RGB).	Written down in 01·Hierarchy
10 June 2026 Video · 1 min 45 s	We updated hover selection in Paper based on your feedback Paper's default hover selection now targets the top-level parent under the cursor, like Figma; double-click goes deeper and ⌘ deep-selects. The old deep selection (hover picks the closest frame) is re-enabled with Use deep selection in the menu under the file name (narration calls it Project Settings), which also holds Back to dashboard ⇧⌘D, Show pixel grid ⌘', Snap to pixel ⇧⌘', Layout guides ⇧G, Multiplayer cursors ⇧M, Open links in desktop app and Nudge settings. ⌘ + right-click lists every layer under the cursor.	Written down in 01·Hierarchy
10 June 2026 Video · 4 min 22 s	Claude's Fable 5 vs Opus 4.8: which model designs better in Paper? Two Claude desktop Code sessions (Fable 5 and Opus 4.8, both High effort) get the same prompt for one expense-entry screen, each titling its artboard with the model name; both write into one Paper file simultaneously. Opus 4.8 finished clearly faster despite starting later; the presenter calls Fable 5 slower and significantly more expensive, possibly better at long independent work. Both loaded the guide, checked fonts (Archivo; Inter Tight), built in chunks, reviewed and released nodes; Fable 5 explained more, Opus 4.8 ended by proposing a variant. June model menu: Fable 5, Opus 4.8, Sonnet 4.6, Haiku 4.5, Opus 4.7/4.6 Legacy, fast-mode toggle. Comments report MCP screenshot timeouts on large artboards; Paper team says naming the frame limits the capture.	Written down in 11·Parallel agents
9 June 2026 Short · 1 min 27 s	If your font supports variable weights, your design tool should too! Variable font controls: with text selected, the Text section's more button (sliders icon) opens a popover with Formatting, Features and Variable font tabs. The Variable font tab sets any weight (100 to 800 shown, including 364 and 560) through a slider and number field, plus optical size with 'Automatic optical sizing'. The presenter explains the benefits: exact weights, smooth weight animation, and one font file instead of one per weight.	Written down in 05·Tokens and type
5 June 2026 Short · 48 s	SVG editing just launched in Paper 48-second recap of the SVG editing launch: select an SVG, double-click to select one path and move/resize/rotate it alone, double-click again to drag its nodes; a node dragged past the container is clipped at the container edge unless the container is enlarged; recolour the fill, add a stroke (1, Above fill), set its colour and width.	Written down in 07·Vector
5 June 2026 Video · 9 min 15 s	Using Paper and Resend to turn a design into a ready-to-send email template Harcourt (Paper) and Chris (Resend) go from a Snapshot capture of a password-reset email to a published Resend template. Claude Code (Sonnet 4.6) duplicates and reskins the email from a style guide on the canvas, self-reviews, and later re-reads Harcourt's manual edits through computed styles. Chris pastes the Paper frame link into his own Claude Code (Opus 4.8), which loads Paper's guide and Resend's skill, builds table-based HTML with {{{RESET_URL}}}, creates a Draft template, and he tests it with JSON variables before publishing.	Written down in 14·Export
3 June 2026 Video · 9 min 17 s	SVG editing in Paper: pen tool, path editing, and what's coming next First release of SVG editing (beta). An SVG is a clipping container (default 256 x 256, Clip content ticked) holding Path layers; double-click a path for vector editing (Escape to leave); per-path Fill and Stroke with an Above fill / Below fill position; pen tool (P) with Shift for 45 degrees, Command to pull curve handles, Option to break handle mirroring, click an edge to add a point; drawing outside a container creates a new SVG (Cmd+C/Cmd+V to merge); resizing the container is the crop workaround. Ends with a roadmap board (booleans, improved snapping, path guides, convert shapes, crop/resize near-term) and prototype clips of mirroring, a radial repeater, vector networks and anchoring.	Written down in 07·Vector
3 June 2026 Video · 4 min 01 s	Copy designs from Figma and paste them into Paper Figma copy/paste (beta): Cmd+C in Figma, Cmd+V in Paper; a toast fetches images and a one-time 'Images from Figma' dialog connects the Figma account; frames arrive 1:1 with their Figma names. A Claude Code agent on Sonnet 4.6 duplicates a frame named in the prompt and translates it to Spanish through the Paper MCP (guide, find, duplicate, set text content, screenshot, finish), revealing new line breaks. Right-click > Copy as offers PNG ⇧⌘C, Tailwind `T, React CSS `R. Components arrive detached; tokens and components were not in Paper yet.	Written down in 08·Starting material

2 June 2026 Video · 2 min 49 s	How to create high quality, searchable PDFs in Paper Paper prints PDFs through the browser because it renders with HTML/CSS, keeping real searchable text and small files. Select a frame, set Export to PDF, optionally adjust Settings (File suffix, Color space sRGB, Quality High/Medium/Low, Image resampling Detailed/Basic) and click Export ⌘⌘E; files land in Downloads named after the frame. With several frames, Export N elements writes separate PDFs and Export combined PDF writes one file (Combined.pdf, 554 KB for four 1980 × 1080 slides) ordered by canvas position. Preview search finds text, and Claude (Opus 4.8 in the model menu) summarizes the PDF correctly.	Written down in 14 · Export
28 May 2026 Video · 4 min 30 s	Remix Resend's action button with Paper Snapshot and Paper Shaders Harcourt captures Resend's pill button with Paper Snapshot (structure intact: outer frame, hover rectangle, content frame), duplicates it, and builds an animated border with Pulsing Border: black page background, shader fill removed, colors from the original via the Foreground eyedropper, Scale and Roundness 100%, Radius Full. She nests the shader in the button frame with Absolute position, Fill W/H, X/Y 0, and moves it behind the opaque content frame for hard inner and outer edges.	Written down in 09 · Shaders
26 May 2026 Video · 3 min 37 s	Generate images in Paper using the xAI model Create image (⌘⌘I): model menu as of May 2026 (Variety pack default = OpenAI Image 2, Grok, Imagen 4, Seedream 4.5; plus Flux 2 Pro, Recraft V3, Ideogram v3, Nano Banana Pro), aspect ratios (1:1, 3:2, 4:3, 16:9, 2:1, 2:3, 3:4, 9:16, 1:2). Grok returns a 4:3 photoreal image in about two seconds, placed at 592 x 444 from a 1184x888 source. Selecting the image makes it context; one change per prompt (sunset, terracotta pot, healthier plant) keeps the scene consistent, with one artifact; versions are auto-named and placed beside the original. A banner shows a weekly rate limit with Upgrade.	Written down in 08 · Starting material
11 May 2026 Video · 1 min 39 s	Agent set up: How to connect Cursor to Paper using the Paper MCP server Two ways to connect Cursor: Cursor Settings (Command+Shift+J) > plugin marketplace > Paper > Get, or paste ` /add-plugin paper-desktop ` from the Paper docs into a New Agent chat and choose Add for Myself > Add Plugin. Cursor's agent then lists what the paper-desktop plugin bundles: the `paper` MCP server at http://127.0.0.1:29979/mcp, the skills code-to-design and design-to-code, and an auto-applied rule ensure-paper-started. The one hard requirement: Paper Desktop running with a design file open. Docs add: check Cursor Settings > Tools & MCP, reload or toggle the server if the agent can't see it.	Written down in 10 · MCP
2 May 2026 Video · 4 min 50 s	Agent set up: How to connect Claude Code to Paper using the Paper MCP server From zero: install Claude Code (curl installer), run `claude`, paste `claude mcp add paper --transport http http://127.0.0.1:29979/mcp --scope user` (stored in ~/.claude.json). The first prompt fails because tools added mid-session need a Claude Code restart (Claude suggests `claude mcp list`). After restart, 'In Paper, design a basic social media home page' produces a design brief (hex palette, Inter weights, 24/16/8 spacing, direction), a 'Social Home' artboard built in chunks under a dotted working indicator, a screenshot review checklist (spacing, typography, contrast, alignment, clipping, repetition) and a summary. UI hidden with ⌘\; manual edits afterwards (sizing badge 'Fill 191 × Fit 38'). Requires Paper Desktop with a design file open, not Paper web or the dashboard.	Written down in 10 · MCP
2 May 2026 Video · 6 min 15 s	Use an AI agent to explore different design directions in Paper In the Claude desktop app (Code tab, Sonnet 4.6 · Medium), a pre-flight 'Are we connected to the Paper MCP?' returns file, page, artboard 'Social Home' (1440×900), 189 nodes, font Inter. Prompt for two iterations with different colors and layouts: the agent reads the frame structure, proposes named directions 'Ink' (dark, icon-only nav) and 'Slate' (cool, top nav, DM Sans), builds each as a new artboard beside the untouched original, screenshots and critiques each. One variant lost the add button. A combine prompt (Ink's nav + Slate's palette + dark purple accent) yields a planned 'Violet' version (#5B21B6 accent, #1A0F2E rail) with a checkpoint review and finish_working_on_nodes; final colour fix done by hand.	Written down in 11 · Parallel agents
2 May 2026 Video · 2 min 11 s	Recreate Cloudflare's loading shader from scratch with Paper Shaders Harcourt rebuilds Cloudflare's teased loading animation: screenshot pasted as reference, Warp added with S and made square, Cauldron Pot chosen as the nearest preset, colors matched first with the color picker's eyedropper (extra blue deleted), then Speed, Scale, Softness and Swirl iterations adjusted. The method is nearest preset, then colors, then motion.	Written down in 09 · Shaders
2 May 2026 Video · 2 min 30 s	Design using your live site as a starting point with Paper Snapshot Harcourt installs the Paper Snapshot Chrome extension, captures the 'Go beyond editing' section of resend.com with a hover-and-highlight selector (click or Return to capture, arrow keys to fine-tune, Esc to cancel) and pastes it into Paper with ⌘V. The capture lands as editable layers (a 1280 × 900 frame with text layers, a flex row, per-side borders with alpha, radii, Selection colors), which she rearranges with an arrow key and edits in place. Shown on the May 2026 'Paper Alpha' build.	Written down in 08 · Starting material
2 May 2026 Video · 3 min 30 s	How to use an AI agent to iterate on a design you captured with Paper Snapshot On the Snapshot capture, Harcourt prompts Claude Code in the Claude desktop app's Code tab (Sonnet 4.6, Auto accept edits) to add a third column using copy parked in a side frame. The agent reads the canvas (Get Screenshot, Get Tree Summary, JSX), changes a repeat(2, 1fr) grid to three columns, duplicates a column, updates its text and icon, and posts a Checkpoint review. She approves 'Always allow for project (local)' and finishes by hand, double-clicking media frames' edges to set fit height.	Written down in 03 · Alignment

5 March 2026 Video · 3 min 29 s	Design with real content: How to pull Notion data into Paper with Claude Code Earliest agent video (Paper Alpha). Stephen adds the Notion MCP to Claude Code next to the Paper MCP, checks both with /mcp (Notion: connected, authenticated, https://mcp.notion.com/mcp, 12 tools; actions View tools, Re-authenticate, Clear authentication, Reconnect, Disable), and asks Claude to design a Notion 'Core Values' page in Paper. The agent runs notion-search and Paper get_basic_info in parallel, notion-fetch, get_font_family_info for four families, writes a brief, creates an artboard and builds it with semantically named sections (Header, Core Values Section, Tactics Section, Reading Section, Footer). Works with any MCP source, web search, and data viz.	Written down in 06 · Information
5 March 2026 Video · 7 min 19 s	How to sync designs with your codebase using Paper MCP, Claude Code and Cursor Stephen Haney connects a fresh Paper file to Claude Code (one-line claude mcp add over HTTP to 127.0.0.1:29979/mcp, verified with /mcp) and has Claude lay out colors.css as a Color System board with variables, oklch values and usage notes, self-reviewing by screenshot. He then installs the server in Cursor via Add to Cursor, retunes five blues by editing hues in the Selection colors list, and has Cursor write only the selected colours back to colors.css, with an accept/reject diff. As of March 2026.	Written down in 13 · Code
5 March 2026 Video · 4 min 48 s	How to use Paper Shaders to elevate your designs Stephen Haney tours Paper Shaders (March 2026): Warp presets as quick starts, Speed 0% for still textures, and right-click Copy as React CSS, which outputs an exact @paper-design/shaders-react component with a link back to the Paper node. He feeds generated images into Fluted Glass with the image eyedropper, chains Halftone CMYK (Vintage) on top of the glass output, and closes with the AI Vectorize command on a flat image.	Written down in 09 · Shaders

How this guide was made

All 52 entries on youtube.com/@paperdesign were downloaded with their captions. 51 have spoken captions; one Short is music only and was read from its frames. Every video was analysed from its transcript and from frames sampled every few seconds, and the procedures, control names and shortcuts in this guide were checked against those frames.

Paper's own documentation (agent setup, Snapshot, pasting from Figma and HTML, tokens, vector editing, the shader library and the public roadmap) supplied current facts. The principle chapters, studies and plates began as an illustrated book designed in Paper; this edition makes them general and adds the Paper detail.

Everything is dated September 2026. Paper changes quickly, so treat statuses and limits as a snapshot and check the build log for anything newer.

Learn Paper with me. An illustrated guide to Paper, September 2026 edition. Written by Enzo Simier with AI assistance, from Paper's public videos and documentation. Not affiliated with Paper. Paper Shaders © Paper, Apache License 2.0.

Web edition and PDF: enzosimier.com/projects/paper-guide/
Set on a twelve-column programme: 64 px columns, 32 px gutters.